

MODULE HANDBOOK

Full Stack Strategies and Development

Module Code: COM661

Programme(s): BSc (Hons) Software Engineering,
BSc (Hons) Computing Science,
BSc (Hons) Computing Technologies,
BSc (Hons) Interactive Computing,
BSc (Hons) Computing Systems,
BEng (Hons) Computer Science.

School of Computing
Faculty of Computing, Engineering and the Built Environment

COM661 Full Stack Strategies and Development

BE01. Python Text, Files and Data Structures

Aims

- To introduce simple string processing via the Python interpreter
- To demonstrate splitting and joining strings
- To demonstrate case manipulation of character strings
- To introduce the basic options for opening text files
- To demonstrate interating across each line of a text file
- To demonstrate writing values to a text file
- To introduce complex Python data structures
- To measuse the time performance of Python code
- To present the serialization of complex Python data structures

Table of Contents

1.1 TEXT PROCESSING 2

1.2 FILE HANDLING 8

1.3 CASE STUDY – A HANGMAN GAME 11

1.4 PYTHON DATA STRUCTURES 15

1.4.1 LISTS 15
1.4.2 DICTIONARIES 19
1.4.3 MEASURING PERFORMANCE 22
1.4.4 SERIALIZING DATA STRUCTURES 28

1.5 FURTHER INFORMATION 30

1.1 Text Processing

Python is an object-oriented programming language that is suitable for a wide range of software development tasks. In recent years, it has become popular for web applications development, due to its strong support for integration with other languages and tools, and its extensive libraries.

Python is a simple language to learn, with relatively few keywords and data types – but it is extremely powerful and flexible. Python code is also among the most readable of all high-level languages.

Note:	If you are installing Python on your own computer, please use the latest version (currently **3.12**) to ensure compatibility with the material in this section. There is still a large legacy community that uses the older Python 2.7, but official support for this will soon be withdrawn. Version 3.x has some important differences from earlier versions.

String values in Python are enclosed in either single or double quote characters and are concatenated by the **+** operator. Python also supports a powerful **slice** operator that can be applied to strings. The slice operator is invoked by **`string[start:end]`** which extracts all characters from position **`start`** to position **`end-1`**. If we omit either **`start`** or **`end`**, then the slice is assumed to be either from the beginning or to the end of the string. A negative value denotes character positions counting from the end of the string so that **`string[-1]`** refers to the last character, **`string[-2]`** refers to the next-to-last character, and so on.

Do it now!	Try the string slice manipulations presented in Figure 1.1 by entering them directly to the Python interpreter. Experiment with other examples of your own until you are comfortable with the operation of the string slice operator.

Note:	To run Python code from the command prompt, you need to open a terminal window and launch the Python interpreter via the command **`python`** (or **`python3`** depending on your installation). A terminal window is most easily obtained by opening it from the Visual Studio Code menu. Alternatively, on a Windows machine, open a terminal window (command prompt) by clicking the Windows "Start" button, entering "cmd" in the search box, and clicking "cmd.exe' from the list of options provided. On a MacOS machine, you open the application called **Terminal** that will be pre-installed on the computer. You can find this by opening the Launchpad on the Dock (**Terminal** is most often in a group of applications called "Other"), or by searching for it in the Finder.

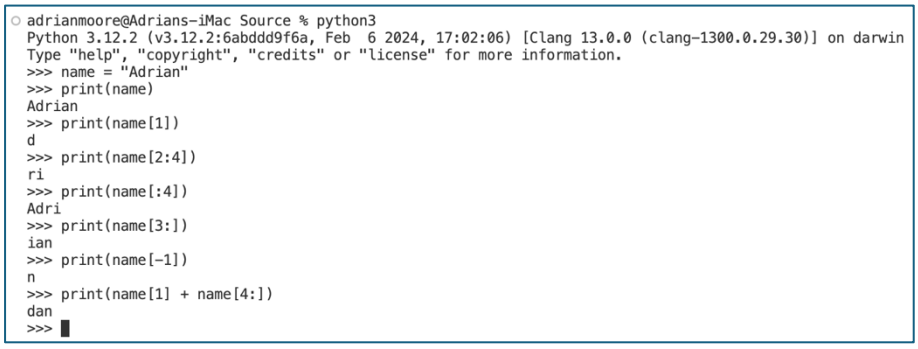


Figure 1.1 String Slice Operator

One very important concept in Python strings is that they are immutable – i.e., once created, the value of a string cannot be modified. We can prove this by attempting to change the first character of our **`name`** variable, as shown in Figure 1.2 below.

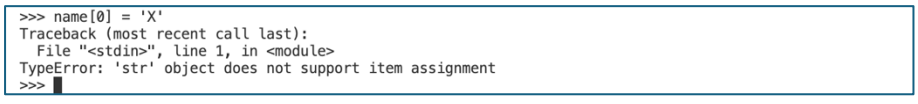


Figure 1.2 Immutable Strings

Do it now!	Try modifying the value of the name variable, by issuing the command **`name[0]='X'`**. Observe the error message as displayed in Figure 1.2.

Try it now!	Given that strings are immutable, try and devise a command that has the same effect that you tried to achieve in the previous example. i.e., the value of **`name`** should be 'X' followed by the remaining characters from the original value of **`name`**.

Python provides a rich set of string manipulation facilities, some of the most useful of which are illustrated by the following set of examples.

Do it now!	Open the Python interpreter and enter the following commands shown in Figure 1.3. Verify the operation of the string manipulation functions described

```
>>> test = "This is a simple string to practice on"
>>> len(test)
38
>>> test.count("s")
4
>>> test.count("is")
2
>>> test = test.replace("string", "character sequence")
>>> test
'This is a simple character sequence to practice on'
>>> test[10:16]
'simple'
>>> test.find("a")
8
>>> test.find("a", 10)
19
>>>
```

Figure 1.3 String Operations

Note that some operations are functions (e.g. **`len()`**) while others are methods (e.g. **`count()`**, **`replace()`**) – Typically, functions can be applied to other collections such as lists, while methods apply only to strings.

The **`len()`** function returns the number of characters (i.e. the length) in the string. Characters are numbered from 0 to length-1, with individual characters accessed by **`stringName[position]`**.

The **`count()`** method returns the number of non-overlapping occurrences of the sub-string that is passed as a parameter,

The **`find()`** method returns the first instance of the specified substring. Where a second parameter is supplied, it denotes the character position from which the search should begin.

Earlier, we noted that strings are **immutable** – once created, their value cannot be changed. This has a consequence for methods such as **`replace()`** where we have to return the

method result to a new string object. By giving the new string the same variable name as the original, we cause the old string called **`test`** to be replaced by the new one.

Do it now!	Enter the commands shown in Figure 1.4 below into the Python interpreter and verify the operation of the string **`split()`** method.

```
>>> test.split()
['This', 'is', 'a', 'simple', 'character', 'sequence', 'to', 'practice', 'on']
>>> test.split("character")
['This is a simple ', ' sequence to practice on']
>>> test.split("s")
['Thi', ' i', ' a ', 'imple character ', 'equence to practice on']
>>> test.split("s", 3)
['Thi', ' i', ' a ', 'imple character sequence to practice on']
>>>
```

Figure 1.4 Splitting Strings

The **`split()`** method optionally takes 2 parameters

- the *delimiter* to be used as the split character
- the *maximum number* of split operations to be performed

and returns a list of substrings of the original value.

The delimiter defaults to any sequence of whitespace characters. In the first example above, we demonstrate this by splitting the string into individual words, each retuned as a separate element in a list. We will examine list structures in more detail later in this Practical.

Note in the second example how the delimiter value does not form part of the returned list. Splitting on the instance of the word 'character' results in a list consisting of 2 elements – all text up to the delimiter word, and all text following the delimiter.

The third and fourth examples demonstrate the effect of the maximum-number-of-splits parameter. By default, the **`split()`** method will generate as many substrings as possible, but when a second parameter is provided, we stop splitting after the requisite number of splits have taken place. Note that the number of returned elements will be one greater than the parameter value – in the example above we can see how 3 split operations result in 4 elements being generated.

Do it now!	Enter the commands shown in Figure 1.5 below into the Python interpreter and verify the operation of the string **`join()`** method.

```
>>> words = test.split()
>>> words
['This', 'is', 'a', 'simple', 'character', 'sequence', 'to', 'practice', 'on']
>>> " ".join(words)
'This is a simple character sequence to practice on'
>>> "...".join(words)
'This...is...a...simple...character...sequence...to...practice...on'
>>>
```

Figure 1.5 Joining strings

The **`join()`** method takes as a parameter a list of string elements, and is applied to another string element that will be used as a divider in the joined string. For example, in the first **`join()`** operation above, we apply the **`split()`** method to a string consisting of a space character, resulting in a string object containing all of the words in the list separated by a space. In the second example, we illustrate a different string being used as the divider.

Do it now!	Enter the commands shown in Figure 1.6 below into the Python interpreter and verify the operation of the case manipulation methods.

```
>>> test.upper()
'THIS IS A SIMPLE CHARACTER SEQUENCE TO PRACTICE ON'
>>> test.lower()
'this is a simple character sequence to practice on'
>>> test.lower().capitalize()
'This is a simple character sequence to practice on'
>>> test.title()
'This Is A Simple Character Sequence To Practice On'
>>> "UPPER".isupper()
True
>>> "UpPeR".isupper()
False
>>> "lower".islower()
True
>>> "lOwEr".islower()
False
>>>
```

Figure 1.6 Case Manipulation

Python provides a wide range of methods that manipulate the case of string objects. Here we demonstrate **`upper()`** and **`lower()`** which return uppercase and lowercase versions of strings respectively, as well as **`capitalize()`** and **`title()`**, which capitalize the first letter of the string and the first letter of each word respectively.

In addition, there is a set of methods that test the case of string objects. Here, we demonstrate **`isupper()`** and **`islower()`**, which test for uppercase and lowercase strings respectively.

Note also the format of the Python Boolean constants **`True`** and **`False`**.

Do it now!	Enter the final set of string manipulation commands shown in Figure 1.7 below into the Python interpreter and verify their operation.

```
>>> "qwerty123".isalnum()
True
>>> "qwerty$%^123".isalnum()
False
>>> "letters".isalpha()
True
>>> "letters123".isalpha()
False
>>> "let ters".isalpha()
False
>>> "12345".isdigit()
True
>>> "123abc".isdigit()
False
>>> "     ".isspace()
True
>>> "a   b".isspace()
False
>>> "A string".ljust(15)
'A string       '
>>> "A string".rjust(15)
'       A string'
>>> "A string".center(15)
'    A string   '
>>> "   A string   ".strip()
'A string'
>>>
```

Figure A1.7 Testing String Composition

Python provides functions that allow us to test the composition of a string. This can be very useful if we want to ensure that (for example) only digits are accepted for a particular input. Here we demonstrate tests for alpha-numeric (**`isalnum()`**), letters only (**`isalpha()`**), numeric only (**`isdigit()`**) and whitespace (**`isspace()`**).

Finally, we demonstrate methods that pad out a string to a specified number of characters. **`ljust()`** pads the string by adding space characters to the end of the string, **`rjust()`** pads by adding spaces to the beginning of the string, while **`center()`** adds extra spaces equally to both left and right. These are useful for formatting output in neat columns. The **`strip()`** method preforms the opposite function and removes extraneous whitespace from a string.

Try it now!	Write the Python script *parse_url.py* that reads from the keyboard a string value representing a full URL including a querystring and extracts & prints its component parts. For example, the URL http://www.example.com?name=Adrian&module=COM661&weeks=12 should produce output such as that illustrated in Figure 1.8 below. **Note:** The querystring is the part of the URL that follows the **?** character and contains a number of **`name=value`** pairs separated by the ampersand (**&**) and representing additional data to be passed to the webpage. **Hint** – your code should be generalized to work for any URL and any number of querystring parameters

```
adrianmoore@Adrians-iMac Source % python3 parse_url.py
Please enter a URL: http://www.example.com?name=Adrian&module=COM661&weeks=12
Host is www.example.com
Name is name, value is Adrian
Name is module, value is COM661
Name is weeks, value is 12
adrianmoore@Adrians-iMac Source %
```

Figure 1.8 URL Parser

1.2 File Handling

Up to now, application input and output has been to/from the standard console using the **`input()`** and **`print()`** functions. In this section we will examine the use of text files to store application data.

Files are opened the by **`open()`** function, that takes parameters specifying the location and name of the file and the type of access required, and returns a file object which is used as the subject for further file activity.

The most common access modes are

Mode	Example	Description
`r`	`f = open("data.txt", "r")`	Open file for reading only. The file pointer is placed at the beginning of the file. This is the default mode
`w`	`f = open("data.txt", "w")`	Open file for writing only. If the file already exists, overwrite it with the new version.
`a`	`f = open("data.txt", "a")`	Open file for appending. File pointer is placed at the end of the file. Create file if not already existing

Examine the application *longest_word.py*, that opens the text file *words.txt*, which contains the full set of allowable English language words in crossword puzzles, organised one word per line in the file. The application reads each word from the file, and determines the length of the longest word. It also creates an output file *biggest.txt* and writes each word with the longest length to this file.

Do it now!	Run the application *longest_word.py* and verify its operation. Examine the output file *biggest.txt* that is generated.

File: longest_word.py

```
fin = open("words.txt", "r")
biggest, num_of_words = 0, 0
for line in fin:
   word = line.strip()
   num_of_words = num_of_words+1
   if len(word) > biggest:
      biggest = len(word)

print("The longest of the {} words contains {} \
       characters".format(num_of_words, biggest))

fin.seek(0)
fout = open("biggest.txt", "w")
for line in fin:
      word = line.strip()
      if len(word) == biggest:
            output = word + "\n"
            fout.write(output)
            print(word)
fin.close()
fout.close()
```

The application is built around two file objects **`fin`** (file in) and **`fout`** (file out). **`fin`** is created by opening *words.txt* for reading in the first line of the code fragment.

Note in the second line, how we can create and initialise multiple variables on the same line. Here we will use **`biggest`** to store the length of the longest word, and **`num_of_words`** to count the total number of words in the file.

The **`for…in`** loop demonstrates a very useful technique for reading text files, by iterating across each line of the file. In the body of the loop, we use **`strip()`** to remove the newline character at the end of the line and compare the length of the word to the greatest length discovered so far – updating the value of **`biggest`** if required.

Once all words have been read, the **`for`** loop terminates and we report the total number of words read and the length of the longest word.

Now we need to traverse the input file again, this time comparing the length of each word to the longest length previously discovered and adding the word it to the output file if it is one of the group of longest words.

To re-read the input file from the beginning, we could simply close and re-open it, but instead we move the file pointer by invoking the **`seek()`** method. The parameter of **`seek()`** is an offset in bytes from the beginning of the file, hence **`seek(0)`** will indicate the top (beginning) of the file. The output file is opened for writing – causing the file to be created if it does not already exist.

The second **`for…in`** loop again iterates across each line in the input file. This time we compare the length of the current word against the biggest length found, and if they match, we **`write()`** the word (plus newline character) to the output file. Once the **for** loop terminates, we **`close()`** both files.

Running the application should result in output such as that illustrated in Figure 1.9 below.

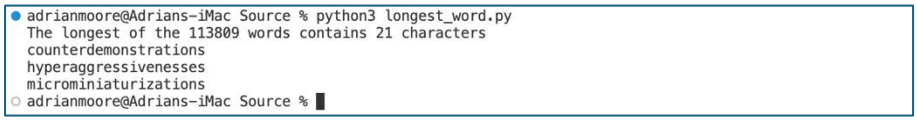


Figure 1.9 File Processing

Try it now!	Write the Python application *check_for_letters.py* that prompts the user to enter 3 letters from the keyboard. The application should then generate an output file *letters.txt* that contains all words from *words.txt* that contain the 3 letters in the order in which they were supplied by the user. The letters do not have to appear consecutively in the word – for example letters 'p', 'c' and 'o' would result in the word 'application' being accepted.

1.3 Case Study – A Hangman Game

As a slightly more complex example of an interactive text processing application, we will develop a version of the game Hangman, where the object is to guess a word by suggesting letters that appear in the word. (Source: https://en.wikipedia.org/wiki/Hangman_(game))

The first task for the application is to have the computer supply a word for the user to guess. In our version, we will prompt the user to supply the number of letters that should be in the word and the application selects a random word from *words.txt* with that number of letters.

File: hangman.py

```
import random

number_of_letters = int(input("Enter word length: "))
words = get_words(number_of_letters)
print("There are {} words with {} letters". \
            format(len(words), number_of_letters))

guess_word = words[random.randint(0, len(words)-1)]

print("Guessing the word: {}".format(guess_word))
```

Here, we call the function **`get_words()`** (which we will develop in the next stage), passing to it the number of letters required. This function returns a list of words from which we select one at random. The selected word is assigned to the variable **`guess_word`**.

File: hangman.py

```
import random

def get_words(number_of_letters):
   words_found = []
   fin = open("words.txt")
   for line in fin:
      word = line.strip()
      if len(word) == number_of_letters:
         words_found.append(word)
   fin.close()
   return words_found
...
```

The function **get_words()** opens the file *words.txt* and reads each word in turn. If the length of a word matches the number supplied by the user, then that word is added to the **words_found** list by the **append()** method. Once all words have been tested, the **words_found** list is returned by the function.

Now that we have the word for the user to guess, we can move to the main game loop. In this version of the game, we give the user 6 lives (corresponding to head, torso, 2 arms and 2 legs in the classic pencil-and-paper version). For as long as the user has any lives remaining, they suggest a letter that might be found in the word. If the letter is present, the computer reveals all instances of the letter – otherwise the user is informed that the letter is not present, and he loses a life.

File: hangman.py

```
...

lives = 6
guess_string = "_" * number_of_letters
while lives > 0:
    this_letter = input("Guess a letter: ")
    if this_letter in guess_word:
        guess_string = \
             replace_all(guess_string, guess_word, this_letter)
        if guess_string == guess_word:
            print("You guessed the word!")
            break
    else:
        lives = lives - 1
        print("Letter not found lives remaining: {}". \
               format(lives))
    print(guess_string)
```

Note the **if…in** operator that provides an easy test for the presence of a character in a string. This avoids the need to manually iterate across the string in a loop.

The variable **guess_string** is used as the visual representation of the user's progress. Initially it is set to a string of "_ _ _ _ _ _", with one underscore character for each letter in the word. If the letter suggested by the user is contained in the word, the function **replace_all()** replaces the appropriate underscores with the actual letter. For example if the word is "apple", and the user's first guess is 'p', the value of **guess_string** will change from "_ _ _ _ _" to "_ p p _ _". The function **replace_all()** is presented below.

File: hangman.py

```
def replace_all(guess, word, letter):
   for pos in range(len(guess)):
      if word[pos] == letter:
         guess = guess[:pos] + letter + guess[pos+1:]
   return guess

...
```

The function **`replace_all()`** takes 3 parameters

- **`guess`** – the current state of the **`guess_string`** (e.g. "_ p p _ _")
- **`word`** – the word being guessed by the user
- **`letter`** – the letter suggested by the user

and returns the updated version of the **`guess_string`**. The function operates by iterating across each letter in the word, testing for the letter being the same as that provided by the user. Where a match is found, a new version of the **`guess_string`** is created by taking the characters before and after the current position and appending them either side of the suggested letter. For example, with a word "apple", a **`guess_string`** value '_ p p _ _' and letter "l", the function would concatenate the strings "_ p p", "l" and "_".

The final stage of the application adds the name and score of a successful player to the roll of honour maintained in the file *hangmanScores.txt*.

File: hangman.py

```
...

if lives > 0:
   player = input("Enter player name: ")
   fout = open("hangman_scores.txt", "a")
   score = lives * number_of_letters
   new_score_text = "{}, {}\n".format(player, score)
   fout.write(new_score_text)
   fout.close()
```

If the player reaches the end of the game with some lives intact (i.e., they have successfully guessed the word), the application prompts them to enter their name. The results file

hangman_scores.txt is opened in "append" mode, and the player's name and score are added to the file.

Note that we provide an incentive for users to attempt longer words by defining the score as the number of lives remaining multiplied by the number of letters in the word.

All of the code presented in this section is available in the file *hangman.py*.

Do it now!	Execute the application *hangman.py* and see its operation. Comment out the line that prints the word to be guessed to get a "true" game. Perform multiple runs and see how the *hangman_scores.txt* file records details of all successful games.

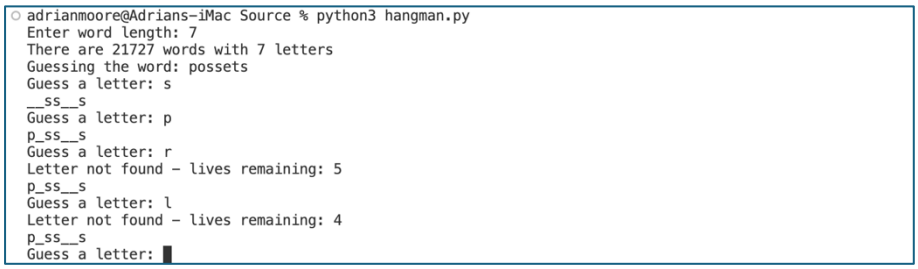


Figure 1.10 Hangman

Try it now!	Further develop the hangman game to make the following improvements: 1) The game should keep track of which letters are still available for selection and should display this information to the user before each turn. See Figure 1.11 below for an example of the output required. 2) When a player runs out of lives, the game should tell them the word they failed to guess 3) The scores in the file should be cumulative – i.e., if a player's name is already in the file, then the new score should be added to his existing score. The file should maintain one line per player – rather than one line per game played.

```
s_rm_s_
Letters available: ___defghijkl_nopq__tuvwxyz
Guess a letter: e

s_rm_se
Letters available: ___d_fghijkl_nopq__tuvwxyz
Guess a letter: y
Letter not found - Lives remaining: 2

s_rm_se
Letters available: ___d_fghijkl_nopq__tuvwx_z
Guess a letter: u

surm_se
Letters available: ___d_fghijkl_nopq__t_vwx_z
Guess a letter: i

You guessed the word!

Game over!

Enter player name: Adrian
adrianmoore@Adrians-iMac Source %
```

Figure 1.11 Hangman Enhanced

1.4 Python Data Structures

1.4.1 Lists

We have previously seen the Python list structure as the output from the string **`split()`** method, where the substrings are presented as a collection enclosed within square brackets and separated by commas such as

`[ 'this', 'is', 'a', 'list' ]`

In general, lists are ordered collections where each element has an index position beginning from zero, elements can be of mixed types including lists, and duplicates are allowed. Figure 1.12 below illustrates the definition and use of lists in Python. Note that the slice operator previously used in string manipulation can also be applied to lists.

```
>>> l1 = [0, 'one', 2.0, [1, 2, 3], False]
>>> l1
[0, 'one', 2.0, [1, 2, 3], False]
>>> l1[0]
0
>>> l1[3]
[1, 2, 3]
>>> l1[-1]
False
>>> l1[-2]
[1, 2, 3]
>>> l1[1:3]
['one', 2.0]
```

Figure 1.12 Defining and Using Lists

Note also that assigning a list to a new variable simply creates a new reference to the original list. If the copy is intended to create a new list that can be manipulated separately, then the list **`copy()`** method should be used, as shown in Figure 1.13.

```
>>> l2 = l1
>>> l2
[0, 'one', 2.0, [1, 2, 3], False]
>>> l2[0] = 'zero'
>>> l1
['zero', 'one', 2.0, [1, 2, 3], False]
>>> l3 = l1.copy()
>>> l3
['zero', 'one', 2.0, [1, 2, 3], False]
>>> l3[0] = 0
>>> l1
['zero', 'one', 2.0, [1, 2, 3], False]
>>> l3
[0, 'one', 2.0, [1, 2, 3], False]
```

Figure 1.13 Copying Lists

Do it now!	Enter the commands shown in Figure 1.12 and 1.13 into the Python interpreter to ensure that you understand their operation. Try different values of your own to check your understanding of the manipulation of lists and their individual elements.

Python also provides a pair of methods that allow us to manipulate lists without using their index values. The **`append()`** and **`pop()`** methods operate at the end of the list (i.e. the highest index position), such that **`append()`** adds an element to the end of the list and **`pop()`** removes and returns the element at the end of the list. These methods are illustrated by Figure 1.14 below which defines an empty list **`my_list`** before appending three items. We then see that the first item popped is the last to be added, before popping a further item and appending a final element.

```
>>> my_list = []
>>> my_list.append(1)
>>> my_list.append(2)
>>> my_list.append(3)
>>> my_list
[1, 2, 3]
>>> print(my_list.pop())
3
>>> my_list
[1, 2]
>>> print(my_list.pop())
2
>>> my_list
[1]
>>> my_list.append(4)
>>> my_list
[1, 4]
>>>
```

*Figure 1.14 Using **`append()`** and **`pop()`***

As a slightly more complex example, we will consider a situation where we want to maintain multiple lists in a single structure. Here, we create a word storage structure as a list of lists where each sub-list will be used to store words that begin with successive letters of the alphabet; such that element [0] is a list of words that begin with the letter ‘a’, element [1] is a list of words that begin with the letter ‘b’, and so on, as illustrated below.

```
[
  [ 'apple', 'archer', …, ],
  ['ball', 'boy', 'butterfly', …, ]
  …
  ['zebra', 'zoo', …]
]
```

The words to be stored in the structure will be a set of 50 words randomly chosen from the file of allowable words in crossword puzzles and the contents of the structure will be displayed when the application runs.

The application is presented in Figure 1.15 below. First, we open the file containing the full set of allowable words and read them into a list **`all_words`**. Next, we initialize **`words_list`** to an empty list and then use a **`for`** loop to append 26 empty lists into **`words_list`**. This will result in **`words_list`** being a list of empty lists in the form

```
[ [], [], [], [], …, [] ]
```

Note that there is no way in Python to declare an empty list to store a specified number of elements in a single operation. Note also the use of the dummy variable _ in the definition of the **`for`** loop

```
for _ in range(26):
```

This is used when we deploy a **`for`** loop in such a way that we have no need to access the loop counter variable. Had we used a regular loop counter variable here, such as

```
for i in range(26):
```

then some Python environments will generate a warning that the variable **`i`** is unused and so the dummy variable is a way of avoiding this.

Having initialized the **`words_list`**, we then choose 50 random entries from **`all_words`**. For each word selected, we store it in the **`words_list`** entry corresponding to its initial letter, such that all selected words beginning with 'a' will be in **`words_list[0]`**, all selected words beginning with 'b' will be in **`words_list[1]`**, and so on.

Finally, we print each letter of the alphabet together with its corresponding element from **`words_list`**.

File: store_words_list.py

```
import random

all_words = []
fin = open("words.txt")
for line in fin:
    word = line.strip()
    all_words.append(word)
fin.close()

words_list = []
for _ in range(26):
    words_list.append([])

for _ in range(50):
    random_word = all_words[random.randint(0,len(all_words)-1)]
    words_list[ord(random_word[0]) -  \
               ord('a') ].append(random_word)

letters = "abcdefghijklmnopqrstuvwxyz"
for i in range(26):
    print(letters[i], words_list[i], '\n')
```

```
a ['alienor', 'autogyro', 'anthills']

b ['biked', 'brattiest', 'bagnios', 'bemadams', 'bounciest']

c ['compassion', 'cowhides', 'counties', 'contrails', 'cannot', 'caesurae', 'crunodal']

d ['decomposing', 'denoting', 'dorsa']

e ['exosmose', 'entertains', 'enfaced']

f []

g []

h ['halloaed', 'heisting']
```

Figure 1.15 Lists of Randomly Selected Words

Do it now!	Run the file *store_words_list.py* and verify that you receive output similar to that shown in Figure 1.15 above. You should be able to observe that multiple runs of the code generate varying output and that each of the randomly selected words is stored in the appropriate element of the list.

Try it now!	Add code to *store_words_list.py* so that the user is given 3 opportunities to enter a search word from the console. For each word entered, the list is searched and the result “FOUND” or “NOT FOUND” is displayed as shown in Figure 1.16 below.

```
s ['swoopers', 'slimsy', 'sowar', 'subahdar', 'shaving']

t ['trampolinist', 'tripos', 'totems', 'throws', 'towie']

u ['unbelt']

v ['vesicular', 'vaccinas', 'visualizes']

w ['wisses', 'whirried']

x []

y []

z []

Enter a word to search for >>> throws
FOUND
Enter a word to search for >>> torch
NOT FOUND
Enter a word to search for >>> visualizes
FOUND
```

Figure 1.16 Searching the Lists of Words

1.4.2 Dictionaries

A dictionary is a data structure that can store any number of Python objects. Dictionaries consist of pairs of keys and their corresponding values.

Do it now!	Open the Python command line interpreter and verify the following sequence of Python dictionary manipulation commands.

First, we will create a new dictionary. Note that dictionaries are enclosed by { } brackets.

```
>>> modules = {“COM668”: “Project”, “COM661”: “Full Stack Strategies and Development”, “COM682”: “Cloud Native Development”}
```

This creates a dictionary containing 3 items, each consisting of a module code as a key and a module title as a value. To retrieve the value associated with any key, we can refer to is as…

```
>>> modules[“COM668”]
‘Project’
```

If we wish to add a new item to the dictionary, we simply assign a value to its key, e.g.

```
>>> modules[“COM662”] = “Data Analytics”
>>> modules
{‘COM668’: ‘Project’, ‘COM661’: ’Full Stack Strategies and Development’, ‘COM682’: ‘Cloud Native Development’, ‘COM662’: ‘Data Analytics’}
```

Note:	In versions of Python before 3.7, the order of elements in a dictionary was entirely arbitrary and not governed by the order in which they are added. From v3.7 on, dictionary order is preserved, but it is not considered good practice to depend on it.

Each key in a dictionary can correspond to only one value. If a second value for any key is presented, then it over-writes the previous entry.

```
>>> modules["COM682"] = "Cloud Development"
>>> modules
{'COM668': 'Project', 'COM661': 'Full Stack Strategies and
Development', 'COM682': 'Cloud Development', 'COM662': 'Data
Analytics'}
```

Elements can be removed from a dictionary by the **`del`** command.

```
>>> del modules["COM662"]
>>> modules
{'COM668': 'Project', 'COM661': 'Full Stack Strategies and
Development', 'COM682': 'Cloud Development' }
```

We can also test for the presence of a key using the powerful **`in`** operator.

```
>>> 'COM662' in modules
False
>>> 'COM661' in modules
True
```

Dictionaries are often a preferred solution to lists for complex data structures given their flexibility. Just as a list element can be of any type including a list, so the value of a dictionary can be of any type, including lists and dictionaries. We will demonstrate this be revisiting our previous word storage application and re-factoring it to use a dictionary instead of a list of lists.

Note in the code box below, the … notation at the top of the listing refers to code that is unchanged from the previous version. In this case, we import the **`random`** module and generate the **`all_words`** list as before. In the remaining code, the modifications from the previous version are highlighted in bold text.

This time, we define **word_dict** as an empty dictionary and use the **for** loop to add 26 elements, where the keys are successive letters of the alphabet (from 'a' to 'z') and the values are empty lists.

Then, for each of the 50 randomly selected words, we append that word to the appropriate dictionary entry according to its initial letter, so that words beginning in 'a' are appended to the list at dictionary entry 'a' and so on.

Finally, we traverse the characters of the **letters** string, printing out for each the letter and the corresponding dictionary value.

File: store_words_dictionary.py

```
...

letters = "abcdefghijklmnopqrstuvwxyz"
words_dict = {}
for i in range(26):
    words_dict[letters[i]] = []

for _ in range(50):
    random_word=all_words[random.randint(0,len(all_words)-1) ]
    words_dict[random_word[0]].append(random_word)

for letter in letters:
    print(letter, words_dict[letter], '\n')
```

```
a ['aboded', 'alias', 'anti', 'areolate', 'annal', 'anuran', 'austere']

b ['butternut', 'bowfins', 'benzylic']

c ['charquid', 'commutations', 'copecks', 'chants', 'cooing', 'credo', 'cobwebbier', 'conjure']

d []

e ['eightieths', 'endarchy']

f ['foresees', 'faders', 'fertilizers']
```

Figure 1.17 Dictionary of Randomly Selected Words

Do it now!	Run the file *store_words_dictionary.py* and verify that you receive output similar to that shown in Figure 1.17 above. You should be able to observe that multiple runs of the code generate varying output and that each of the randomly selected words is stored in the appropriate dictionary entry.

Try it now!	Repeat the previous exercise by adding code to *store_words_dictionary.py* so that the user is given 3 opportunities to enter a search word from the console. For each word entered, the dictionary is searched and the result "FOUND" or "NOT FOUND" is displayed as shown in Figure 1.18 below.

```
t ['thrice', 'trotlines', 'theorbo']

u []

v []

w ['weakling', 'whimbrels', 'whilst']

x []

y []

z []

Enter a word to search for >>> wobble
NOT FOUND
Enter a word to search for >>> whilst
FOUND
Enter a word to search for >>> zebra
NOT FOUND
```

Figure 1.18 Searching the Dictionary of Words

1.4.3 Measuring Performance

The main performance advantage offered by dictionaries over lists is the direct access to each individual element via the dictionary key. Although the **`in`** keyword can be used in the same way on list and dictionary structures, the fact that it operates on array **elements** but on dictionary **keys** makes a significant difference when using complex data structures.

For example, if a list of modules is organised as

```
modules = [ 'COM661', 'COM668', 'COM682']
```

then the statement

```
if 'COM668' in modules:
```

can check the entire list in a single operation. Likewise, if the **`modules`** structure is organised as a dictionary with the module code as the key and the module coordinator as the value, such as

```
modules = { 'COM661' : 'Adrian',
            'COM668' : 'George',
            'COM682' : 'Zeeshan' }
```

then the same **if** statement can confirm the presence or otherwise of a given module in the collection. However, if the list structure is to contain all of information as the dictionary (i.e. both the module code and coordinator), then the list definition would be such as

```
modules = [  ['COM661', 'Adrian'],
             ['COM668', 'George'],
             ['COM682', 'Zeeshan']
          ]
```

Now the **if** statement would fail to pick up a matching module code since no list entry is an exact match for a module code and every element would need to be checked to verify whether the desired module code is present, such as

```
for module in modules:
   if module[0] == 'COM668']:
      return True
return False
```

The effect of this can be clearly seen by an example that models the storage of keywords and URLs in a search engine. Following a web scraping exercise, a search engine maintains a collection of keywords that have been scraped from webpages along with the URLs on which the keywords can be found. If a user enters a keyword in a search operation, the search engine responds with the list of URLs where the word is found. When stored as a list, the structure might be

```
contents = [
              [ 'word1', [ 'url1', 'url2', 'url3' ] ],
              [ 'word2, [ 'url1', 'url4', 'url5' ] ],
              [ 'word3', [ 'url2', 'url5', 'url6' ] ],
              ...
           ]
```

representing that **word1** is found on pages at **url1**, **url2** and **url3**; **word2** is found on pages at **url1**, **url4** and **url5**, and so on. Now, to check whether the structure contains a given **search_term**, the entire structure would need to be checked one element at a time, such as

```
for entry in contents:
   if entry[0] == search_term:
      return entry[1]
return []
```

In the worst case, where the search term is not stored in the collection, then every element would need to be compared. This is demonstrated by the example ***timed_list_search.py*** which constructs a collection of between 1000 and 32000 unique keywords and measures the time required to search the collection for a term that is not present.

First, we provide the function **`time_execution()`** that takes a reference to a piece of Python code as a parameter and returns the time required to execute this code. Execution time is calculated by taking a system clock reading by the **`time.perf_counter()`** function both before and after executing the code, and subtracting the start time to the finish time to obtain the execution time in nanoseconds.

The function **`add_word()`** accepts the current state of the collection as a parameter, together with the word to be added to the collection and the URL at which the word has been found. The function checks whether the collection already contains an entry for the word and if so, adds the URL to the list for that entry. If the word is one that has not previously been seen, then a new collection entry is added for that word, with the list of URLs initialised to the one passed as a parameter.

File: timed_list_search.py

```
import time

def time_execution(code):
      start = time.perf_counter()
      eval(code)
      run_time = time.perf_counter() - start
      return run_time

def add_word(this_collection, keyword, url):
      for entry in this_collection:
            if entry[0] == keyword:
                  entry[1].append(url)
                  return
      this_collection.append([keyword,[url]])
```

Next, we provide functions to construct and search the collection. The function **`make_collection()`** accepts a large list of words as a parameter as well as an integer that states the size of collection required. The function creates a new collection entry for each word, specifying that the word has been found at a dummy URL. The collection of the specified size is then returned from the function. The function **`lookup()`** takes as

parameters the word to be searched for and the current collection being used. The function checks each collection entry in turn, returning the corresponding list of URLs if a match is found or an empty list if there is no match.

File: timed_list_search.py

```
...

def make_collection(all_words, size):
      this_collection = []
      for i in range(size):
            add_word(this_collection, all_words[i], "dummyURL")
      return this_collection

def lookup(keyword, collection):
      for e in collection:
            if e[0] == keyword:
                  return e[1]
      return []
```

Finally, we read the complete list of words from the previous example into a list and compare execution times for a range of collection sizes from 1000 to 32,000 elements. For each size, we create a collection of that size before measuring the execution time of the worst-case scenario search – that when the word being searched does not exist within the collection meaning that every collection element must be examined.

File: timed_list_search.py

```
...

all_words = []
fin = open("words.txt")
for line in fin:
    word = line.strip()
    all_words.append(word)
fin.close()

collection = make_collection(all_words, 1000)
print("Lookup for 1000 words")
print(time_execution("lookup('xxx', collection)"))

# and also for 4000, 8000, 16000 and 32000
```

Running the program demonstrates that the execution time grows with the size of the collection, and it can be seen from the results in Figure 1.19 that execution time is roughly doubled as the collection size is doubled.

```
Lookup for 1000 words
5.251402035355568e-05
Lookup for 2000 words
0.00010238203685730696
Lookup for 4000 words
0.00013192999176681042
Lookup for 8000 words
0.0002537780674174428
Lookup for 16000 words
0.0004331059753894806
Lookup for 32000 words
0.0007295239483937621
```

Figure 1.19 Search time increases proportionally

Having observed the performance of a list-based approach, we can then re-factor the code to use a dictionary instead. This time, the collection structure will be as

```
contents = {
              'word1' : [ 'url1', 'url2', 'url3' ],
              'word2' : [ 'url1', 'url4', 'url5' ],
              'word3' : [ 'url2', 'url5', 'url6' ],
              ...
           }
```

where each keyword in the collection is a dictionary key and the corresponding value is the previous list of URLs where that word has been observed.

The first update is in the **`add_word()`** function, where rather than check each element for a matching keyword, we can now use the **`in`** operator to check all keys in a single operation. If a match is found, then the URL is added to the corresponding list, otherwise a new collection element is created with the keyword as the key and the list initialized to the URL.

`make_collection()`only requires that the collection be initialized to an empty dictionary rather than the previous list, while **`lookup()`** also makes use of the **`in`** operator to return the corresponding list if a match is found, or the empty list otherwise.

File: timed_dict_search.py

```
...

def add_word(this_collection, keyword, url):
      if keyword in this_collection:
            this_collection[keyword].append(url)
      else:
            this_collection[keyword] = [url]

def make_collection(all_words, size):
      this_collection = {}
      for i in range(size):
            add_word(this_collection, all_words[i], "dummyURL")
      return this_collection

def lookup(keyword, collection):
      if keyword in collection:
            return collection[keyword]
      else:
            return []

...
```

Now when we run the application, we see not only that overall execution time is much faster, but also that the individual search times are relatively constant – in other words the time required to search a dictionary is unaffected by an increase in dictionary size.

```
Lookup for 1000 words
2.826494164764881e-05
Lookup for 2000 words
2.1233921870589256e-05
Lookup for 4000 words
1.962995156645775e-05
Lookup for 8000 words
2.8433045372366905e-05
Lookup for 16000 words
9.231793228536844e-05
Lookup for 32000 words
4.938605707138777e-05
```

Figure 1.20 Search time remains constant

Do it now!	Run the files *timed_list_search.py and timed_dict_search.py* and verify that you receive output similar to that shown in Figures 1.19 and 1.20 above. Your execution times may not correspond exactly with those in the Figures, but you should be able to see that while the search time for a list implementation roughly doubled as the list size doubles, the search time for a dictionary is unaffected by dictionary size.

1.4.4 Serializing Data Structures

We have seen how lists and dictionaries can be combined to create complex data structures, but this effort is pointless without some way of storing our structure so that we don't have to generate it from scratch every time we want to execute the application. In the search engine example, we could traverse our collection, writing the data to a file item by item – but this would lose the structure that our code has worked so hard to generate.

Fortunately, Python provides **Pickle** – a powerful module that serializes and de-serializes data structures so that they can be written to/read from a text file in a single operation. Consider the code below, which extends the store_words_dictionary.py example to use Pickle's **`dump()`** method to write the data structure to a text file in a single operation. Note that we use "wb" as the file mode when opening the file for writing, respectively, rather than the more usual "w". The additional "b" flag denotes that the information being written and read is a byte stream, rather than simple text.

File: store_words_dictionary_pickled.py

```
import random, pickle

...

fout = open("words_dictionary.txt", "wb")
pickle.dump(words_dict, fout)
fout.close()
```

Do it now!	Run the file *store_words_dictionary_pickled.py* and see how it creates the new file *words_dictionary.txt* that contains a representation of the dictionary structure.

We can demonstrate that the structure has been saved by reading it from the file into a new Python application. Examine the source of retrieve_words_dictionary_pickled.py which uses the Pickle **`load()`** method to read the list back into a Python variable. The contents of the variable can then be parsed and printed in the usual way, verifying that the dictionary has been successfully saved and then loaded.

File: retrieve_words_dictionary_pickled.py

```
import pickle

fin = open("words_dictionary.txt", "rb")
words_dict = pickle.load(fin)
fin.close()

letters = "abcdefghijklmnopqrstuvwxyz"
for letter in letters:
    print(letter, words_dict[letter], '\n')
```

Do it now!	Run the file *store_words_dictionary_pickled.py* and verify that the dictionary contents printed to the console are the same as those originally written.

Try it now!	Revisit your Hangman application from Section 1.3 so that the high score table is maintained as a dictionary rather than a text file. Modify the application so that the dictionary of scores is automatically pickled to a file after each update.

1.5 Further Information

- http://en.wikibooks.org/wiki/Python_Programming/Variables_and_Strings
  Python variables and strings

- http://docs.python.org/library/stdtypes.html#string-methods
  Python manual – string methods

- https://docs.python.org/3/library/stdtypes.html - printf-style-string-formatting
  Python manual – string formatting operations

- http://www.tutorialspoint.com/python3/python_strings.htm
  Python strings from Tutorialspoint

- https://www.tutorialspoint.com/python3/file_methods
  Python files I/O from Tutorialspoint

- https://www.thoughtco.com/analyze-a-file-with-python-2813717
  Ways to work with files in Python

- https://www.geeksforgeeks.org/python-program-to-read-file-word-by-word/
  Processing every word in a file

- http://docs.python.org/tutorial/inputoutput.html#reading-and-writing-files
  Python manual – reading and writing of files

- http://docs.python.org/library/pickle.html
  Python Pickle module

- http://wiki.python.org/moin/UsingPickle
  Using Pickle

- https://www.w3schools.com/python/python_lists.asp
  Python Lists

- http://www.tutorialspoint.com/python/python_dictionary.htm
  Python dictionary tutorial

- http://www.i-programmer.info/programming/python/3990-the-python-dictionary.html
  The Python dictionary: i-programmer tutorial

- http://www.greenteapress.com/thinkpython/html/book012.html
  Think Python! Dictionaries

- https://builtin.com/articles/timing-functions-python
  Timing Python Functions

COM661 Full Stack Strategies and Development

BE02. Introducing APIs and Flask

Aims

- To introduce the concept of an API as a software interface
- To demonstrate retrieval of data from a commercial API
- To introduce JSON data structures
- To maniulate data returned from an API in your own Python code
- To introduce Flask as a framework for development of Python web applications
- To examine the structure of a Flask application
- To develop API endpoints to return JSON data

Table of Contents

2.1 INTRODUCING APIS 2

2.1.1 USING A COMMERCIAL API 3
2.1.2 RETRIEVING CURRENT WEATHER DATA FROM THE OPENWEATHERMAP API 4
2.1.3 RETRIEVING WEATHER FORECAST DATA 8

2.2 INTRODUCING FLASK 11

2.3 BUILDING A FIRST FLASK API 15

2.3.1 RETRIEVING THE ENTIRE COLLECTION 17
2.3.2 RETRIEVING A SINGLE ELEMENT 18

2.4 FURTHER INFORMATION 21

2.1 Introducing APIs

An Application Programming Interface (API) is a software library specification designed to be used as a means of enabling software components to communicate with each other. Essentially, an API is an interface to some service or collection of data – except that rather than an interface designed to be used by a human operator, an API is an interface designed to be consumed by software.

In the Web environment, APIs are provided by many online platforms to allow content from an application or a service to be accessed over the HTTP protocol and embedded in 3rd party software. APIs have recently become a major growth area in web applications development, as seen by the Rapid API Hub (https://rapidapi.com/hub), which lists over 40,000 APIs in its collection. Many of these are freely available to use, some require a free user account to be created and others are made available on a subscription basis.

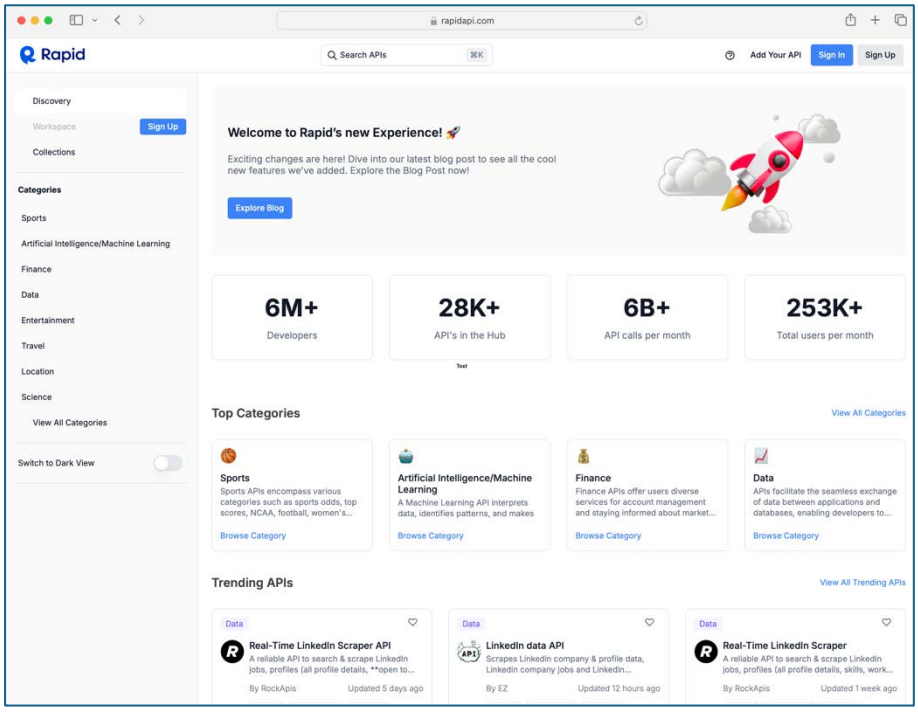


Figure B1.1 Rapid API Hub

Most Web APIs are characterized by the term REST (Representational State Transfer). This is a simple stateless architecture for the provision of web services that run over HTTP. Each client request to a REST service contains all information required to service the request and no client context information is maintained by the service provider. REST services are defined by an endpoint (URL), a media type to be returned (XML, JSON, etc.) and the HTTP method to be used (GET, POST, etc.). REST services are consumed by making an HTTP request to the endpoint, and accepting the data returned. We will examine the structure

and use of REST (sometimes referred to as RESTful) APIs over the course of this section of the module.

In this practical, we will first examine the facilities provided by a typical commercial API and see how we can retrieve real-time data for use in our own applications. However, our focus will be on designing and building our own APIs, so we will also introduce Flask – a Python package that makes it easy to provide an API that can manage access to a data source of our own.

2.1.1 Using a Commercial API

The ***OpenWeatherMap*** REST API is a collection of URLs (known as **endpoints** in API terminology) that enable us to write code to search an online weather reporting resource and manipulate the results for presentation in our own applications. The API provides very powerful retrieval functionality and allows us to manipulate the weather data that is returned within our own code – but is only available to registered developers who have signed up using the following process.

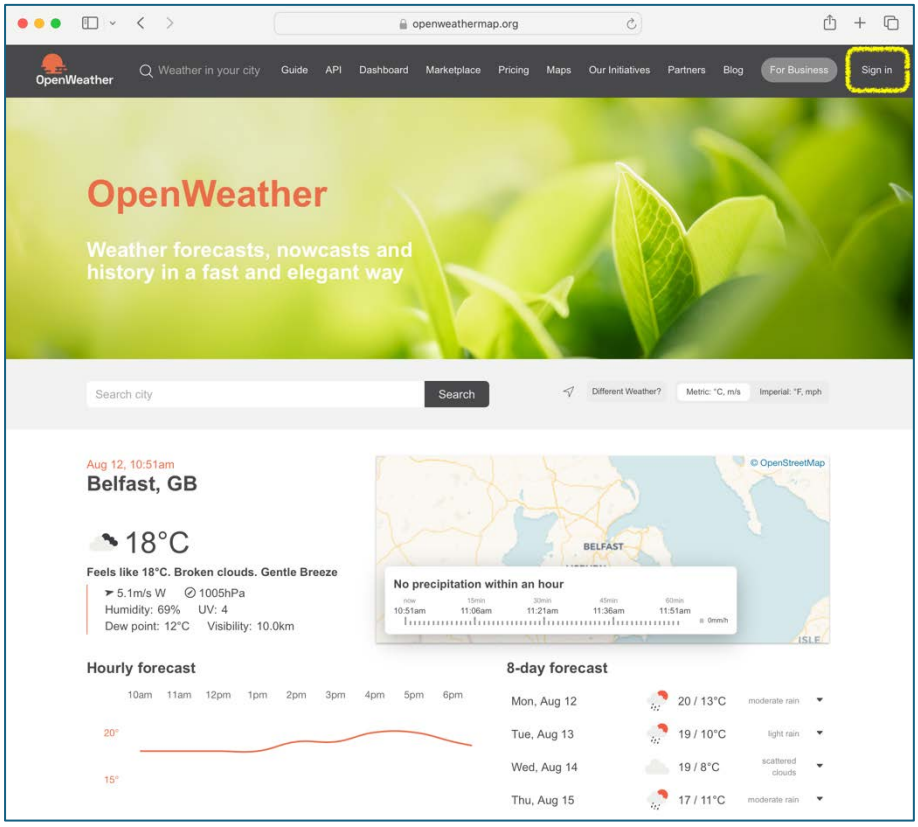


Figure 2.2 OpenWeatherMap

1. Go to https://openweathermap.org/ and click the "Sign In" icon at the top of the browser display (highlighted in Figure 2.2 above)..

2. Complete the application process until you are logged in and see the main menu near the top of the window as illustrated in Figure 2.3.

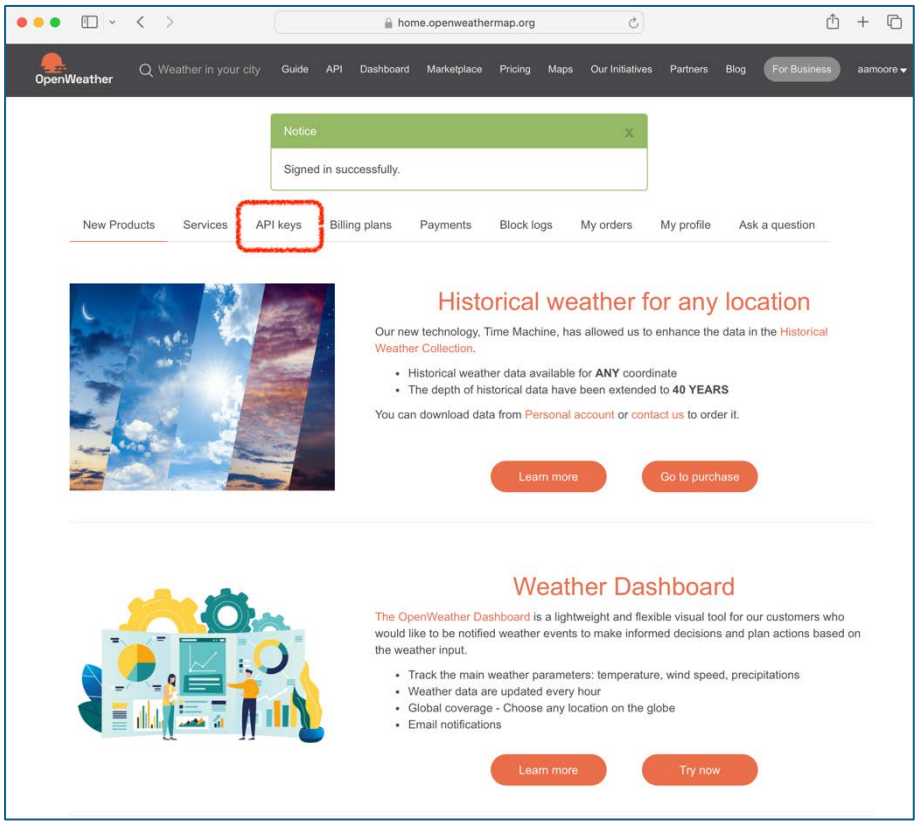


Figure 2.3 OpenWeatherMap Main Menu

3. Click on the **API Keys** option in the main menu (highlighted in Figure 2.3 above) and take a note of your API key. You will need this when making calls to the API from your applications.

Do it now!	Follow the steps above to sign up to OpenWeatherMap and generate your Access Key.

2.1.2 Retrieving Current Weather Data from the OpenWeatherMap API

As a first example, we will create a simple application that calls the **/data/2.5/weather** endpoint to return the current weather conditions. The sample program *currentWeather.py* provides a first demonstration of an application that queries the ***OpenWeatherMap*** API to harvest the most recently reported weather conditions at the location specified by the latitude and longitude values that correspond to Belfast.

The key functions here are **`url_builder()`** and **`fetch_data()`** that respectively compose the URL to be accessed and retrieve the JSON data from that location. The function **`url_builder()`** takes the stem of the API endpoint (http://api.openweathermap.org/data/2.5/weather) and appends parameters as a querystring as follows:

Parameter	Values
`mode`	Determines the format of the data to be returned. Options are **`json`** (the default), **`xml`** or **`html`**
`unit`	**`imperial`** for temperature in Fahrenheit, **`metric`** for temperature in Celsius, no value (default) for temperature in Kelvin
`APPID`	the user's API key
`lat`	The latitude value of the forecast location
`lon`	The longitude value of the forecast location

File: current_weather.py

```
import datetime
import json
import urllib.request

def url_builder(lat, lon):
    user_api = 'add_your_api_key_here'
    unit = 'metric'
    return 'http://api.openweathermap.org/' + \
           'data/2.5/weather'+ \
           '?units=' + unit + \
           '&APPID=' + user_api + \
           '&lat=' + str(lat) + \
           '&lon=' + str(lon)

def fetch_data(full_api_url):
    url = urllib.request.urlopen(full_api_url)
    output = url.read().decode('utf-8')
    return json.loads(output)

def time_converter(timestamp):
    return datetime.datetime.fromtimestamp(timestamp). \
           strftime('%d %b %I:%M %p')

lat = 55.147872
lon = -6.675298
json_data = fetch_data(url_builder(lat, lon))

temperature = str(json_data['main']['temp']);
timestamp = time_converter(json_data['dt'])
description = json_data['weather'][0]['description']

print("Current weather")
print(timestamp + " : " + temperature +  " : " + \
      description)
```

Note: As well as latitude and longitude, we can also search OpenWeatherMap by place name, country, zip-code, city ID or within a region specified as a bounding box. See the API endpoint documentation at https://openweathermap.org/current for full details

The endpoint URL produced by **`url_builder()`** is passed to **`fetch_data()`** which opens the url and retrieves the data as a string. The string is then deserialized to a Python object by the **`loads()`** method provided by Python's **`json`** package.

The data returned from the API call is a complex JSON object that contains many fields. You can see the full list in the API documentation, but some of the main elements are described below.

```
{
 "name"       : "name of location",
 "dt"         : "time of data calculation as a Unix UTC value"
 "weather"    :[
                    {
                      "main"          : "headline description",
                      "description"   : "more detailed description",
                      "icon"          : "weather symbol icon ID"
                    }
                ],
 "main"       :    {
                      "temp"          : "temperature reading",
                      "pressure"      : "air pressure reading",
                      "humidity"      : "air humidity reading"
                    },
 "sys"        :    {
                      "sunrise"       : "sunrise time as a Unix UTC value",
                      "sunset"        : "sunset time as a Unix UTC value"
                    }
}
```

Note:	JSON (JavaScript Object Notation) is an open standard that uses human-readable text to transmit data consisting of attribute/value pairs. It is a very convenient structure for Python developers as the format of JSON maps very closely to the structure of a Python dictionary. It is most often used in web applications as an alternative to XML. Further information on JSON can be found in Section 2.4

Finally, we retrieve the information to be displayed from the data structure, converting the UTC timestamp to a more readable form by the **`time_converter()`** function. You can find more information on the codes used in the Python **`strftime()`** function to convert Unix UTC values to readable form in Section 2.5.

Do it now!	Add your API key to the code (line 6). Now, run *current_weather.py* and verify that it displays the current weather information as illustrated in Figure 2.4 below.

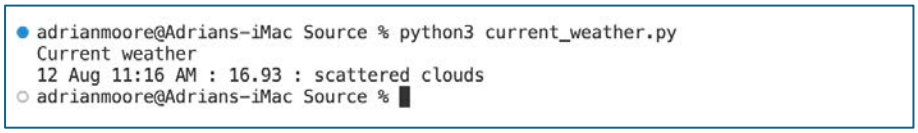


Figure 2.4 Using the Current Weather API Endpoint

Note:	It may take a few minutes for your API key to be activated after registration. If this is the case, use my API key that I have included within the code for *current_weather.py*.

Try it now!	Modify the code so that (i) the name of the location is retrieved from the data structure and added to the title message so that it reads "Current weather in Belfast"; and (ii) the forecasted sunset time is displayed

Try it now!	Write the Python program *compare_temperatures.py* that prompts the user for two latitude and longitude values representing a pair of locations and uses the ***OpenWeatherMap*** API to retrieve the location name and current temperature for each. The program should output the result in the form (e.g.) "Belfast is hotter than Berlin". You can obtain latitude and longitude values for any location from Google Maps or by searching for the location on Wikipedia.

2.1.3 Retrieving Weather Forecast Data

The sample program *forecast.py* illustrates the API endpoint that retrieves weather forecast information for a specified location. This time, the API returns a data structure that contains 3-hourly forecasts for a 5-day period (i.e. up to 40 forecasts in total)

File: forecast.py

```
...

def url_builder(lat, lon):

    ...
    return 'http://api.openweathermap.org/' + \
           'data/2.5/forecast'+ \
           '?mode=' + mode + \
           '&units=' + unit + \
           '&APPID=' + user_api + \
           '&lat=' + str(lat) + \
           '&lon=' + str(lon)

def fetch_data(full_api_url):
    ...

def time_converter(timestamp):
    ...

...

print("Weather forecast for " + \
      json_data['city']['name'])
print(str(json_data['cnt']) + " forecast(s) retrieved")
for forecast in json_data['list']:
    timestamp = time_converter(forecast['dt'])
    temperature = str(forecast['main']['temp'])
    description = forecast['weather'][0]['description']
    print(timestamp + " : " + temperature +  \
          " : " + description)
```

This time, the code that generates the output is in a loop that iterates across of the members of the list structure, retrieving a new forecast in each iteration. The fields available in the returned data are similar to the **`weather`** endpoint – except that the location information is presented only once, and the individual forecasts are in a list.

The complete specification of the data set returned can be seen in the online API documentation, but all of the main elements (including all those used in the code presented here) are shown in the following JSON structure.

```
{
  "city": {
            "id" : " location ID",
            "name" : "name of location",
            "coord" : {
                        "lon" : " longitude value",
                        "lat" : " latitude value"
                      }
          }
  "cnt" : "number of forecasts returned"
  "list" : [
             {
               "dt" : "time of data calculation as a Unix UTC value"
               "weather": [
                           {
                             "main": "headline description",
                             "description": "more detail",
                             "icon": "weather symbol icon ID"
                           }
                         ],
               "main" : {
                           "temp" : "temperature reading",
                           "pressure" : "air pressure reading",
                           "humidity" : "air humidity reading"
                         },

             }
           ]
}
```

Do it now!	Add your API key to the source code (line 6). Now, run *forecast.py* and verify that it displays the forecast information for Coleraine as illustrated in Figure 2.5 below.

```
adrianmoore@Adrians-iMac Source % python3 forecast.py
Weather forecast for Belfast
40 forecast(s) retrieved
12 Aug 01:00 PM : 17.96 : broken clouds
12 Aug 04:00 PM : 18.41 : broken clouds
12 Aug 07:00 PM : 17.85 : overcast clouds
12 Aug 10:00 PM : 12.97 : scattered clouds
13 Aug 01:00 AM : 12 : few clouds
13 Aug 04:00 AM : 13.14 : scattered clouds
13 Aug 07:00 AM : 15.01 : broken clouds
13 Aug 10:00 AM : 16.2 : light rain
13 Aug 01:00 PM : 17.36 : light rain
13 Aug 04:00 PM : 18.17 : broken clouds
13 Aug 07:00 PM : 16.01 : scattered clouds
13 Aug 10:00 PM : 10.29 : clear sky
14 Aug 01:00 AM : 8.88 : few clouds
14 Aug 04:00 AM : 8.77 : overcast clouds
14 Aug 07:00 AM : 9.44 : overcast clouds
14 Aug 10:00 AM : 14.56 : broken clouds
14 Aug 01:00 PM : 18.54 : scattered clouds
14 Aug 04:00 PM : 19.15 : overcast clouds
14 Aug 07:00 PM : 15.96 : overcast clouds
14 Aug 10:00 PM : 16.3 : overcast clouds
15 Aug 01:00 AM : 15.85 : light rain
15 Aug 04:00 AM : 15.63 : light rain
15 Aug 07:00 AM : 14.32 : light rain
15 Aug 10:00 AM : 13.33 : light rain
15 Aug 01:00 PM : 11.61 : moderate rain
15 Aug 04:00 PM : 14.75 : light rain
15 Aug 07:00 PM : 17.2 : light rain
15 Aug 10:00 PM : 11.03 : moderate rain
16 Aug 01:00 AM : 9.56 : broken clouds
16 Aug 04:00 AM : 8.92 : few clouds
16 Aug 07:00 AM : 8.94 : scattered clouds
16 Aug 10:00 AM : 13.16 : few clouds
16 Aug 01:00 PM : 16.28 : scattered clouds
16 Aug 04:00 PM : 15.49 : light rain
16 Aug 07:00 PM : 15.4 : light rain
16 Aug 10:00 PM : 12.34 : light rain
17 Aug 01:00 AM : 11.96 : light rain
17 Aug 04:00 AM : 12.4 : light rain
17 Aug 07:00 AM : 12.86 : light rain
17 Aug 10:00 AM : 14.42 : light rain
adrianmoore@Adrians-iMac Source %
```

Figure 2.5 Using the Forecast API Endpoint

Try it now!	Modify the code so that (i) the name of the location is retrieved from the data structure and added to the title message so that it reads “Current weather in Belfast”; and (ii) the forecasted sunset time is displayed

2.2 Introducing Flask

Flask is a framework for the development of web applications written in Python. It is classified as a micro-framework as it does not require any particular tools or libraries but makes it very easy to quickly create working useful applications. A Web Application Framework is “a software architecture designed to support the development of dynamic websites, web applications and web services” (Ref: https://en.wikipedia.org/wiki/Web_framework). In effect, it is a library that hides the basic functions of the underlying gateway, by providing a range of functions or class libraries that promote rapid development and code reuse.

Using any external library in an application always raises a potential issue if that library is updated during the lifetime of our program. It might be the case that an update to a library creates a breaking change that prevents our program from running, in which case we would refuse the update – but we might also want to install the update in order to take advantage of new features in future applications that we will develop.

The answer in Python is to use a virtual environment to contain each application that uses external libraries. By housing each application within its own container, each can have the versions of the external libraries that are applicable to it – without compromising the run-time environment of any other applications.

This is such a common situation with Python that the language is packaged with its own virtual environment module called **venv**. Creating a new Flask application is therefore a 4-stage process as illustrated below by the creation of a new application called "first":

Note:	On the lab machines, Virtual Studio Code may default to a **PowerShell** terminal (identified by "PS" at the beginning of the prompt). You should make sure that you are using a "basic" **cmd** shell for Virtual Environment work.

1. **Create the folder** in which your application will reside and navigate into it

2. **Create a virtual environment**

Windows	MacOS
`python -m venv venv`	`python -m venv venv`

   In this command we are using the python interpreter to make (-m) a new **venv** instance called **venv**. You are not required to call the environment **venv**, but it is the most common approach.

3. **Activate** the virtual environment

Windows	MacOS
`venv\Scripts\activate`	`. venv/bin/activate`

Once the virtual environment has been launched, you should see its name (**venv**) at the beginning of the command prompt.

4. **Install the Flask module**. This is most easily done using the pip Package Manager that comes bundled with Python

Windows	MacOS
`pip install flask`	`pip install flask`

Do it now!	Follow steps 1-4 above to create the environment for our first Flask application.

To prove that everything is correctly installed, we can create a basic "Hello World" application. Create a new file within the *first* folder called *app.py* as follows.

Note:	The version of ***app.py*** provided in the ***BE02 Practical Files*** is that at the end of this practical. You should begin with a blank file at this point and build up the code as you work through this material. You are strongly encouraged to enter the code at the keyboard rather than use copy and paste.

File: app.py

```
from flask import Flask, make_response

app = Flask(__name__)

@app.route("/", methods=["GET"])
def index():
      return make_response("<h1>Hello world</h1>", 200)

if __name__ == "__main__":
      app.run(debug=True)
```

There are four distinct phases to this code as follows

1.

```
from flask import Flask, make_response
```

Import the elements of the Flask package that will be needed in this application. We will see how these are used in the later stages.

2.

```
app = Flask(__name__)
```

Create the Flask application. This line of code simply creates the application object as an instance of class **`Flask`** imported from the flask package. The **`__name__`** variable passed to the class (note the double underscore at the beginning and the end) is a Python predefined variable, which is set to the name of the module in which it is used.

3.

```
@app.route("/", methods=["GET"])
def index():
      return make_response("<h1>Hello world</h1>", 200)
```

This is effectively the main program body, where we specify an **`@app.route`** decorator and a function. We will look more closely at Python decorators later, but here we are stating that if a GET request is received to the root address of the website, then the function provided should be run. Note that the name of the function is completely arbitrary – it is not called by name but is run automatically when a request to the specified URL is received.

The body of the function returns a value by the Flask **`make_response()`** function that was imported from the **`flask`** package in the first line of code. Here, this takes two parameters – the HTML to be returned in response to the request and the HTTP status code returned to the browser. Here we return code 200 denoting a successful call – but we will see other codes and when to use them later.

4.

```
if __name__ == "__main__":
      app.run(debug=True
```

Finally, we have the standard Python statement that runs the application. The role of the **`if`** statement here is to make sure that this application can only be invoked directly (e.g. from the command line). This is useful when your application contains multiple source files, but your code has a single entry point that should be used.

Do it now!	Follow steps 1-4 above to code and run the first Flask application. When the application is running, visit http://localhost:5000/ in a web browser and ensure that you obtain output such as that shown in Figure 2.6

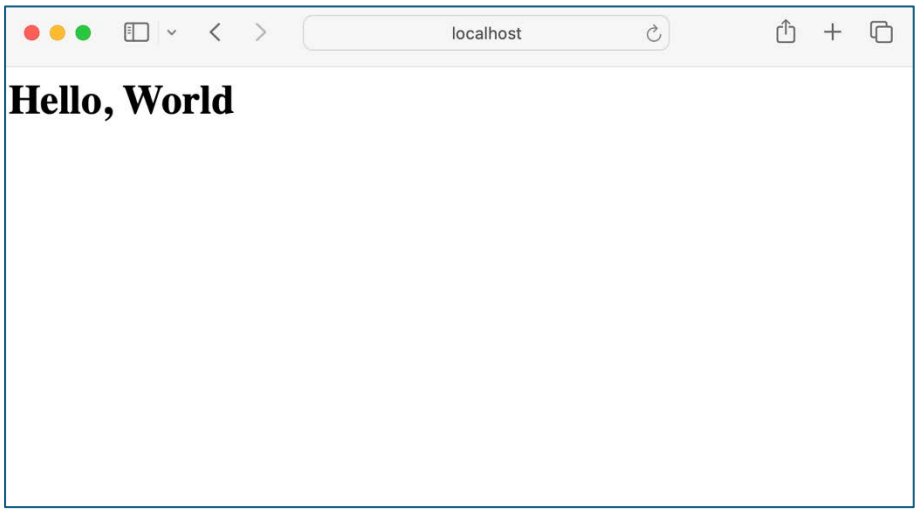


Figure 2.6 "Hello world" Flask application

2.3 Building a First Flask API

Now that our first Flask application is running, we will modify it to act as an API that provides access to a collection of data about businesses. Each business is described by a unique id value, a name, a town, a star rating out of 5 and a collection of reviews (initially empty). In time, we will see how to manipulate this data from a live database, but initially we will hard-code that data as a JSON object stored in a list of Python dictionaries.

The initial state of our API is presented in the code box below. Note that we have added the **`jsonify`** function to the list of elements to be imported from the Flask module (line 1) and we have also removed the **`@app.route()`** that supports the "Hello World" functionality.

File: app.py

```
from flask import Flask, make_response, jsonify

app = Flask(__name__)

businesses =  [
    {
        "id" : 1,
        "name" : "Pizza Mountain",
        "town" : "Coleraine",
        "rating" : 5,
        "reviews" : []
    },
    {
        "id" : 2,
        "name" : "Wine Lake",
        "town" : "Ballymoney",
        "rating" : 3,
        "reviews" : []
    },
    {
        "id" : 3,
        "name" : "Sweet Desert",
        "town" : "Ballymena",
        "rating" : 4,
        "reviews" : []
    }
]

if __name__ == "__main__":
      app.run(debug=True)
```

Do it now!	Modify *app.py* as shown above. You can replace the values in the **`businesses`** list with any data of your own and add additional business entries to increase the size of the data set.

2.3.1 Retrieving the entire collection

Now, we add the route and function that provide the initial functionality that retrieves details of all businesses in the collection. As you will have seen in the ***OpenWeatherMap*** API, it is good practice to include the API name and version number in the URL (so that your host can support multiple versions of multiple APIs if required), so we adopt that practice here by specifying that the URL to retrieve the collection of businesses is **/api/v1.0/businesses**.

The function to return all businesses is very straightforward and simply requires us to return the entire **`businesses`** object, passing it through the Flask **`jsonify()`** function so that it is returned in JSON form.

File: app.py

```
...

@app.route("/api/v1.0/businesses", methods=["GET"])
def show_all_businesses():
    return make_response( jsonify( businesses ), 200 )

if __name__ == "__main__":
      app.run(debug=True)
```

Do it now!	Add the code shown above to *app.py* and run the application (the server should automatically re-load when you change the source code, but if not, you can use CTRL-C to quit the previous server if it is still running). Now visit the URL http://localhost:5000/api/v1.0/businesses and make sure that you are presented with the entire businesses collection as shown in Figure 2.7 below

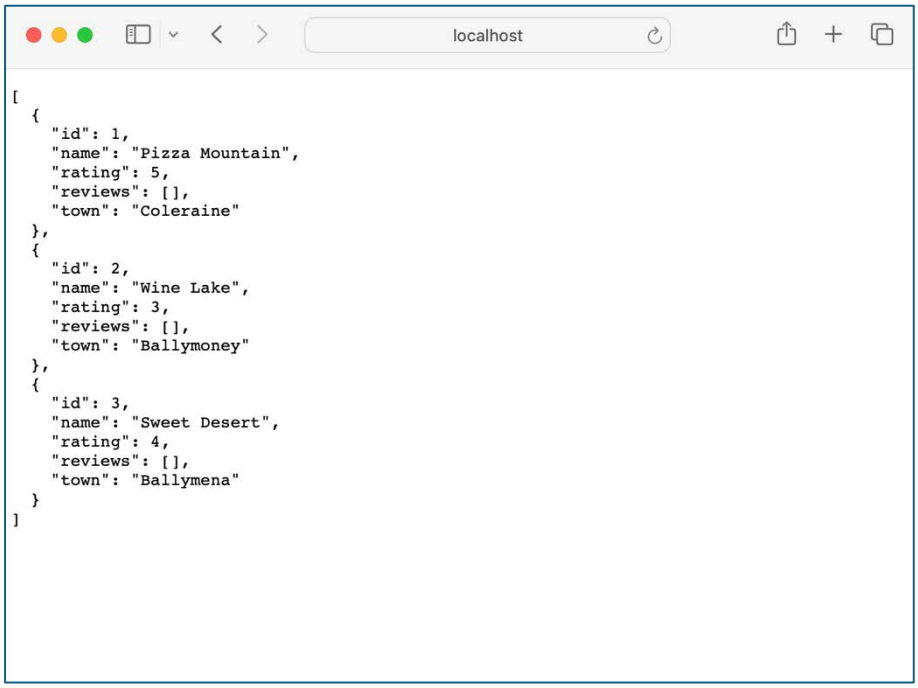


Figure 2.7 Retrieving all businesses

2.3.2 Retrieving a single element

Next, we will add the ability to retrieve a single element from the collection by specifying it by id in the URL. Therefore, to request the business with **`id`** value 1, the URL is **/api/v1.0/businesses/1**, and so on.

This requires that we add a new **`@app.route()`** decorator and corresponding function to the application as provided below.

File: app.py

```
...

@app.route("/api/v1.0/businesses/<int:id>", methods=["GET"])
def show_one_business(id):
    data_to_return =  \
       [ business for business in businesses  \
         if business["id"] == id ]
    return make_response( jsonify(  \
                          data_to_return[0] ), 200 )

if __name__ == "__main__":
      app.run(debug=True)
```

The first thing to notice in this new code is the specification of a variable parameter in the URL. Here, we know that the URL will contain an integer **`id`** value, but we don't know

exactly what the value will be, so we specify it in the URL as **`<int:id>`**. The variable element (**`id`** in this case) will then be specified as a parameter to the function that is run in response to a request to that URL.

There are many ways in which we could extract the element to be returned, but here we show a construction that is very typical of Python. Essentially, the task is to generate a list of all businesses where the **`id`** value matches that provided as a parameter, so we build up the expression as follows

1. We want to return a list

```
[       ]
```

2. We want to return a list of elements called **`business`**

```
[ business ]
```

3. We want to return a list of elements where each element is a member of the **`businesses`** collection

```
[ business for business in businesses  ]
```

4. We are only interested in the elements where the **`id`** value matches that passed as the parameter to the function

```
[ business for business in businesses if business[“id”] == id ]
```

Finally, we only **`return`** element **`[0]`** of the list retrieved, since only one business should match the **`id`** value in the URL.

Do it now!	Add the code shown above to app.py and run the application. Now visit the URL http://localhost:5000/api/v1.0/businesses/1 and make sure that you are presented with the single business requested as shown in Figure 2.8 below.

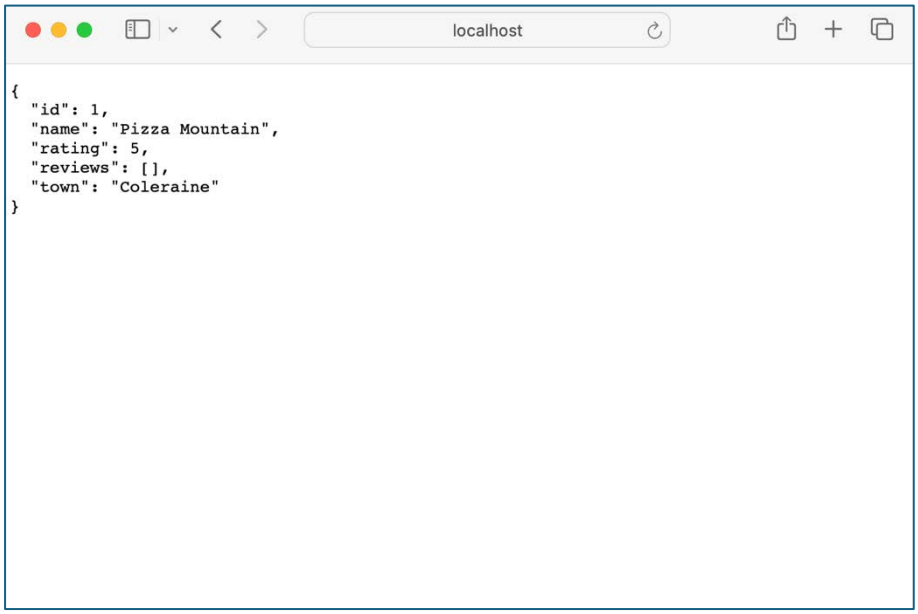


Figure 2.8 Retrieve details of a single business

2.4 Further Information

- https://rapidapi.com/hub
  The RapidAPI Hub API Directory

- https://medium.com/@perrysetgo/what-exactly-is-an-api-69f36968a41f
  What exactly IS an API?

- https://techterms.com/definition/api
  API definition from TechTerms

- http://vrici.lojban.org/~cowan/restws.pdf
  RESTful Web Services

- https://flask.palletsprojects.com/en/2.2.x/
  Flask – the documentation

- https://pythonspot.com/flask-hello-world/
  Getting started with Flask – Hello World

- https://openweathermap.org
  OpenWeatherMap home page

- http://www.infoq.com/articles/rest-introduction
  A brief introduction to REST

- http://code.tutsplus.com/tutorials/a-beginners-guide-to-http-and-rest--net-16340
  A beginner's Guide to HTTP and REST

- http://www.youtube.com/watch?v=7YcW25PHnAA
  REST API: Concepts and Examples (YouTube)

- http://www.json.org
  Specification of the JSON notation

- http://www.w3schools.com/js/js_json_intro.asp
  W3Schools JSON tutorial

- http://strftime.org
  Codes for converting Unix RTC time to readable form

COM661 Full Stack Strategies and Development

BE03. API Design and Implementation

Aims

- To introduce the concept of RESTful APIs
- To identify HTTP verbs and response codes
- To present a framework for API design
- To implement API endpoints to modify a data set
- To introduce Postman as a tool for testing of API endpoints
- To present sample code to respond to POST, PUT and DELETE requests
- To demonstrate sub-document manipulation in a dataset

Table of Contents

3.1 API DESIGN 2

3.1.1 What is a RESTful API? 2
3.1.2 HTTP Methods and Responses 3
3.1.3 API Design 5

3.2 API ENDPOINTS TO UPDATE THE DATASET 6

3.2.1 Adding a New Element 6
3.2.2 Introducing Postman 8
3.2.3 Editing an Element 11
3.2.4 Deleting an Element 12

3.3 WORKING WITH SUB-DOCUMENT COLLECTIONS 12

3.4 FURTHER INFORMATION 17

3.1 API Design

In the previous practical we created a first Flask API to return information from a collection of businesses by using a GET request via a web browser. In this session we will consider more formally the construction of a RESTful API and identify principles of good practice in API design, before continuing with our first example to enhance its functionality.

3.1.1 What is a RESTful API?

REST is an abbreviated form of "**RE**presentational **S**tate **T**ransfer" and describes a software model in which a resource-based architecture communicates representational state information between client and server. By resource-based, we mean that the architecture is designed around resources or objects (nouns) rather than operations (verbs). Hence, we implement URLs for businesses, students, cars, or whatever other objects our application is dealing with – rather than "new", "edit", "delete" or whatever other operations are being implemented.

Typical URL patterns in a RESTful architecture (for example in our sample list of businesses application at http://www.bizlist.com) might therefore be (e.g.)

- http://www.bizlist.com/api/v1.0/businesses
  when referring to the collection of businesses
- http://www.bizlist.com/api/v1.0/businesses/101
  when referring to a specific business by its identifier
- http://www.bizlist.com/api/v1.0/businesses/101/reviews
  when referring to the collection of reviews for a specific business
- http://www.bizlist.com/api/v1.0/users
  when referring to the collection of users
- http://www.bizlist.com/api/v1.0/users/101
  when referring to a specific user by identifier

Note:	Remember that the **/api** path is to separate our API functionality from other public-facing content of the business website at http://www.bizlist.com, while the **/v1.0** path is to support the possibility of multiple versions of the API in simultaneous use

The REST architecture describes six constraints as follows.

- **Uniform interface between client and server**
  This is fundamental to RESTful design and has three main elements; (1) the client indicates a request to the server by using one of the HTTP verbs such as GET, POST, PUT or DELETE; (2) The client request is in the form of a URL; (3) The server response consists of an HTTP status code and a body represented in JSON or XML

- **Statelessness**
  This is the concept that the server contains no state information about the client – each client request is self-contained and provides enough information for it to be processed. Any state information that is required (for example cookies, session variables, etc.) are held on the client.

- **Cacheable**
  All server responses are cacheable

- **Client-server**
  The server and client are disconnected with no shared resources. The uniform interface (above) is the link between the two.

- **Layered system**
  Connected to cacheability and client-server organization, the client cannot assume any direct connection with the server as there may be software or hardware intermediaries. For example, a request could be satisfied by a cache rather than by the server, or the request may be re-directed to an alternative server.

- **Code on demand**
  The server can temporarily extend the client by providing executable code as part of the response, which is then run by the client. For example, the response from the server may contain JavaScript to be run on the browser.

3.1.2 HTTP Methods and Responses

We have seen how the URL pattern in a RESTful application identifies the data object with which we want to perform some operation, but how do we denote the actual operation that we want to execute? For example, a call to the URL

http://www.bizlist.com/api/v1.0/businesses/101/reviews

might mean that we want to see the collection of user reviews for the business identified by the **id** value **101**. However, it might equally mean that we want to add a review to the collection of reviews for that business.

The way in which we communicate our intent to the server is by selection of the appropriate HTTP method (verb). You may already be familiar with the **GET** and **POST** methods from previous work with HTML forms, but there is a wider selection of methods available, each with their own specific purpose, corresponding to the standard CRUD (**C**reate, **R**ead, **U**pdate, **D**elete) operations on databases and described in Table 3.1 below.

HTTP Method	Action requested
GET	Return a resource
POST	Create a new resource
PUT	Update a resource
DELETE	Delete a resource

Table 3.1 HTTP Methods

Therefore, a **GET** request to the previous URL http://www.bizlist.com/api/v1.0/businesses/101/reviews will indicate that we want to fetch the list of reviews for the business with **id** 101, while a **POST** request to the same URL will indicate that we want to add a new review to the collection for that business.

In addition, the RESTful model suggests recommended HTTP Status Codes to be used for each combination of URL and method. Again, you will have already used some of these (e.g. 200 OK, 404 Page not found, etc.), but the codes that we will use are are presented in full in Table 3.2, below.

Method	Collection (e.g. /api/v1.0/businesses)	Specific Item (e.g. /api/v1.0/businesses/101)
GET	**200** (OK); List of businesses. Use pagination, sorting and filtering to navigate large collections	**200** (OK) or **404** (Not found) if **id** not found or invalid
POST	**201** (Created); Location header with a link to the newly created business	**404** (Not found) or **409** (Conflict) if the resource already exists
PUT	**404** (Not found) – unless you provide functionality to modify every element in the collection	**200** (OK) or **404** (Not found) if **id** not found or invalid
DELETE	**404** (Not found) – unless you intend functionality to delete the entire collection	**200** (OK) or **204** (No content) if no data returned or **404** (Not found) if **id** not found or invalid.

Table 3.2 HTTP Methods and Status Codes

3.1.3 API Design

The previous discussions on RESTful services and HTTP methods and responses lead us towards a design pattern for the endpoints (URLs) in our API. We can illustrate this by compiling a list of the functionality we intend to provide in our demonstration application, specifying the HTTP method and URL for each. This list is presented in Table 3.3 below – note how a single URL can be used to indicate multiple actions, with the HTTP Method used to distinguish between them.

Method	URL	Action
GET	/api/v1.0/businesses	Get all (or multiple) businesses
POST	/api/v1.0/businesses	Create a new business
GET	/api/v1.0/businesses/123	Get a specific business (with ID=123)
PUT	/api/ v1.0/businesses/123	Update a specific business
DELETE	/api/ v1.0/businesses/123	Delete a specific business
GET	/api/ v1.0/businesses/123/reviews	Get all (or multiple) reviews for a specific business
POST	/api/ v1.0/businesses/123/reviews	Add a review for a specific business
GET	/api/ v1.0/businesses/123/reviews/321	Get a specific review (with ID=321) for a specific business
PUT	/api/ v1.0/businesses/123/reviews/321	Update a specific review for a specific business
DELETE	/api/ v1.0/businesses/123/reviews/321	Delete a specific review

Table 3.3 API Design for the Sample Application

Note:	If the GET requests on collections of businesses or reviews are intended to fetch a subset of the data (i.e. multiple businesses/reviews but not all), then the request will also need additionality information to specify which subset to return. We will examine this case later in this practical.

3.2 API Endpoints to Update the Dataset

In the previous practical we built a pair of API endpoints (routes) to retrieve the entire set of businesses and to retrieve a single business. However, from the discussion above, we now know that we can also implement endpoints to add an item to the data set, modify an item and delete individual items. We will add these endpoints in this section.

3.2.1 Adding a New Element

To add a new business, we receive values POSTed from an HTML **`<form>`** element and create a new object that can be appended to the list as follows.

- First, we determine the **`id`** for the new business by finding the **`id`** of the business currently at the end of the list and incrementing that value.

- Next, we create the new dictionary object by providing values for each of the fields. Note the use of the **`flask request`** object to retrieve the POSTed values as (e.g.) **`request.form["name"]`** which in turn means that we need to add the **`request`** object to the list of **`flask`** elements **`import`**ed in the first line of the application. The **`id`** is set to the newly calculated value, while **`name`**, **`town`** and **`rating`** are retrieved from the form. Finally, we initialize the **`reviews`** to an empty list.

- Once the object has been created, we simply append it to the list of businesses

- Finally, we return the **201** status code for a successful create operation, along with a copy of the newly created element. Note that it is common practice to also return the URL from which the new object can be retrieved, but we will simply return the object for now.

File: app.py

```
from flask import Flask, jsonify, make_response, request

...

@app.route("/api/v1.0/businesses", methods=["POST"])
def add_business():
    next_id = businesses[-1]["id"] + 1
    new_business = { "id": next_id,
                     "name" : request.form["name"],
                     "town" : request.form["town"],
                     "rating" : request.form["rating"],
                     "reviews" : []
                    }
    businesses.append(new_business)
    return make_response( jsonify( new_business ), 201 )

if __name__ == "__main__":
   app.run(debug=True)
```

Do it now!	Add the new route and function to your **first** application that you created in Practical BE02. Remember to also add the **`request`** object to the **`import`** list on the first line.

3.2.2 Introducing Postman

It is easy to test **GET** requests in the browser, but other HTTP methods are not so convenient. For example, to test a **POST** request, we would have to create an HTML **`<form>`** to send the data to the server in order to check if it is being received and interpreted correctly.

Fortunately, there is a free app called **Postman** that will allow us to easily check all HTTP request types. Postman can be downloaded from https://www.postman.com/downloads/ and is available as a standalone app for Windows, MacOS, Linux, or as an extension for the Chrome browser. Postman is also available for you on the lab machines.

When launched, Postman appears as illustrated in Figure 3.1 below.

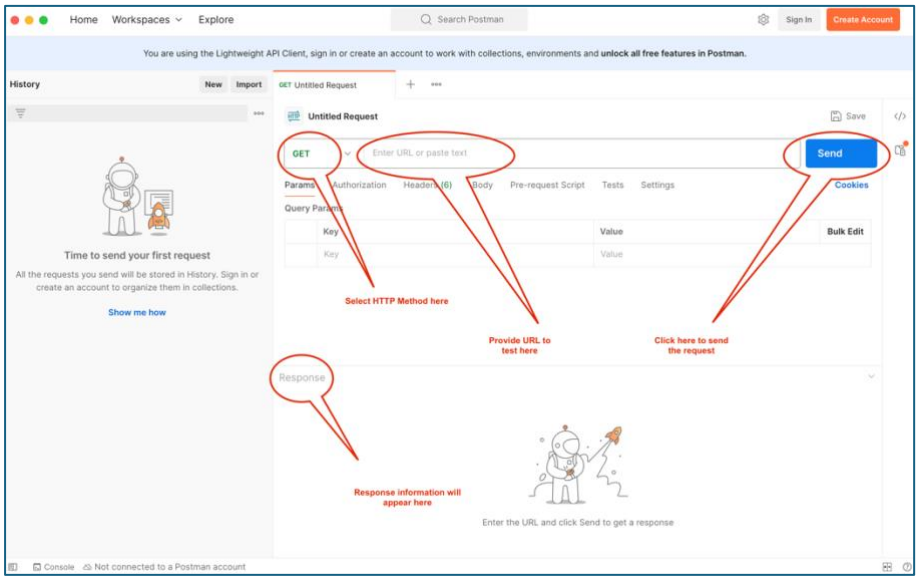


Figure 3.1 Postman

We can see the operation of Postman by sending a simple GET request to http://localhost:5000/api/v1.0/businesses (make sure that your Flask application is running). Choose GET from the list of HTTP methods and provide the URL in the "*Enter request URL*" box and click "*Send*". You should see output as illustrated in Figure B2.2 that verifies that the request has been fulfilled and that Postman is operating correctly.

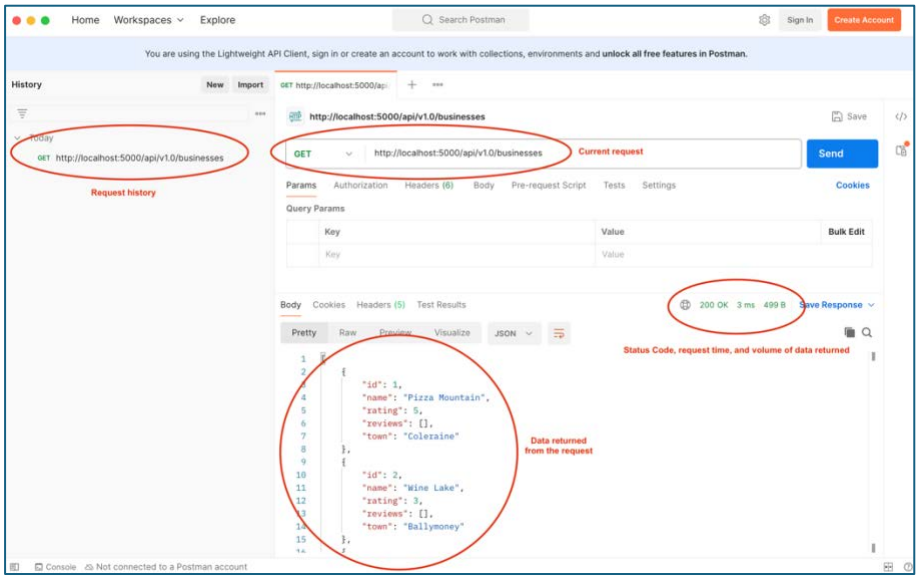


Figure 3.2 Using Postman

The real value in a tool such as Postman is in how it helps us easily check HTTP methods other than GET. For example, we can validate our **/api/v1.0/businesses** POST route by selecting **POST** from the list of methods, choosing **x-www-form-urlencoded** from the options in the ***body*** menu and providing name and value entries for each of the fields that we want to submit as illustrated in Figure 3.3 below.

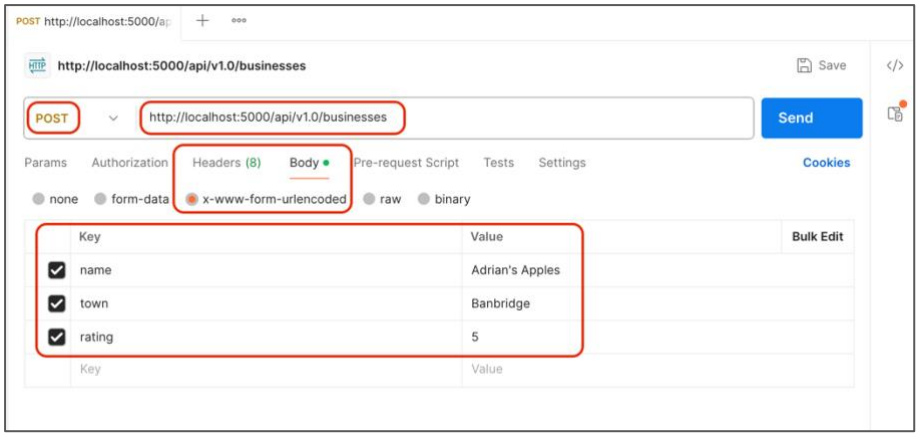


Figure 3.3 POSTing Values to the API

Clicking "*Send*" generates output, as shown in Figure 3.4, that verifies the operation of our POST route for **/api/v1.0/businesses**. We can see that the status code **201** has been provided and that the newly specified object has been returned.



Figure 3.4 Response from POST Request

Finally, we re-issue the **GET** request to return all businesses to verify that the new element has been added to the collection.

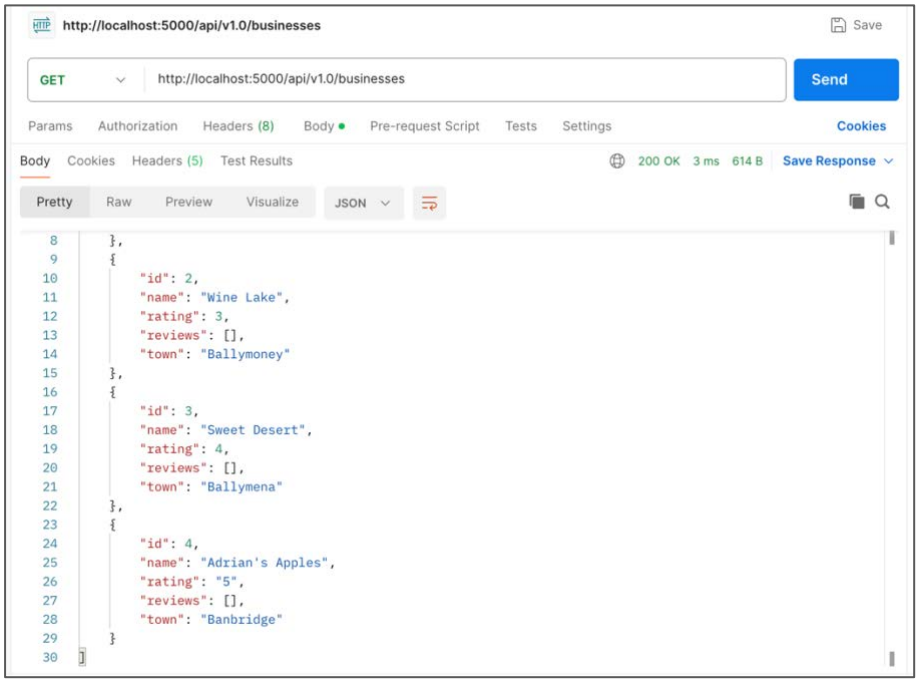


Figure 3.5 New Business Added

Do it now!	Follow the sequence of operations illustrated above to add a new business of your choice to the collection.

Try it now!	Obtain the **`id`** value of a newly created business (copy it from the data returned by the **GET all businesses** request) and use Postman to send the **GET** command that retrieves just that business.

3.2.3 Editing an Element

To edit an element, we need to pass its **`id`** value in the URL and then use that value to locate the business to be edited. Later, we will see how to do this in conjunction with a back-end database, but for now we simply use a **`for`** loop to iterate across the collection testing the **`id`** value of each until a match is found.

Once the business to be amended is found, we simply assign the values passed from the client to the corresponding fields of that business. Note that we assume that all field values are provided – this is the typical case, but we could add additional code to check for the presence of a form value and only update those fields where a new value is provided. The code for the new route and function are provided below.

File: app.py

```
...

@app.route("/api/v1.0/businesses/<int:id>", methods=["PUT"])
def edit_business(id):
    for business in businesses:
        if business["id"] == id:
            business["name"] = request.form["name"]
            business["town"] = request.form["town"]
            business["rating"] = request.form["rating"]
            break
    return make_response( jsonify( business ), 200 )

if __name__ == "__main__":
   app.run(debug=True)
```

Do it now!	Add the new route and function and test it in Postman by making a **PUT** request to **/api/v1.0/businesses/1** to update the first business in the collection.

Note:	Remember that each time the application is re-loaded, the collection of businesses is reset. Hence, the new business that you added in the previous operation will not be available for editing.

3.2.4 Deleting an Element

Removing an element is a similar process to editing. Again, we accept the **`id`** value passed as the URL variable parameter and loop through the business objects until a match is found. Once the business to be deleted is located, we delete it by using the **`remove()`** method that can be applied to lists. Once done, we return status code **200** with an empty response body.

File: app.py

```
...

@app.route("/api/v1.0/businesses/<int:id>", methods=["DELETE"])
def delete_business(id):
    for business in businesses:
        if business["id"] == id:
            businesses.remove(business)
            break
    return make_response( jsonify( {} ), 200)

if __name__ == "__main__":
   app.run(debug=True)
```

Do it now!	Add the new route and function and test it in Postman by making a **DELETE** request to **/api/v1.0/businesses/2** to remove the second business in the collection.

3.3 Working with Sub-document Collections

Sub-document collections arise when one of the fields of our top-level object (i.e. a business) is itself a collection of documents. In our example we specified reviews as one of the fields of a business and we can further develop this now.

To this point, we have set all reviews fields to empty lists, but we will now create a definition of a review object and produce the endpoints to retrieve the collection of reviews from a specified business and to add a review to a business's collection.

First, we provide the function to serve a GET request to **/api/v1.0/businesses/<id>/reviews** which returns the collection of reviews for the business with the specified **`id`**. This is a very straightforward operation which involves finding the specified business and returning its **`reviews`** element with a status code of **200**.

File: app.py

```
...

@app.route("/api/v1.0/businesses/<int:id>/reviews",
            methods=["GET"])
def fetch_all_reviews(id):
    for business in businesses:
        if business["id"] == id:
            break
    return make_response( jsonify( business["reviews"] ), 200 )

if __name__ == "__main__":
   app.run(debug=True)
```

Adding a new review is slightly more complex. First, we loop through the **businesses** collection as usual, but when we locate the one to which we want to add a review, we need to know whether any reviews already exist in that collection. If the **reviews** element is currently empty, we set the **id** value for the new review to 1, otherwise we retrieve the **id** of the last review in the collection and increment that value to set the new review **id**.

Now, we can create the new review object as a Python dictionary, retrieving the values for **username**, **comment** and **stars** from the **request.form** object, and **append** it to the collection for that business. Once complete, we return the new review with a status code of 201.

File: app.py

```
...

@app.route("/api/v1.0/businesses/<int:b_id>/reviews",
           methods=["POST"])
def add_new_review(b_id):
    for business in businesses:
        if business["id"] == b_id:
            if len(business["reviews"]) == 0:
                new_review_id = 1
            else:
                new_review_id =
                    business["reviews"][-1]["id"] + 1
            new_review = {
                "id": new_review_id,
                "username" : request.form["username"],
                "comment" : request.form["comment"],
                "stars" : request.form["stars"]
            }
            business["reviews"].append(new_review)
            break
    return make_response( jsonify( new_review ), 201 )

if __name__ == "__main__":
   app.run(debug=True)
```

Do it now!	Add the new routes and functions and test in Postman by making **GET** and **POST** requests to **/api/v1.0/businesses/1/reviews** to manipulate the set of reviews for the first business. N.B. Remember to be very careful with Python indentation so that the **`return`** statement in **`add_new_review()`** is outside the scope of the **`for`** loop.

Now that we have implemented the ability to add a collection of reviews to a business, we can begin to provide routes to manipulate individual reviews. First, we will add functionality to retrieve a specific review for a specific business.

This time, the URL requires two variable parameters – one for the business **`id`** and another for the review **`id`**. Both of these are then made available to the code through the parameter list of the function. In the function, we first loop through the collection of

businesses and once a match is found, we loop through the collection of reviews for that business. When the matching review is located, we **break** out of both **for** loops and return the review object with a status code of **200**.

File: app.py

```
...

@app.route( \
  "/api/v1.0/businesses/<int:b_id>/reviews/<int:r_id>",
  methods=["GET"])
def fetch_one_review(b_id, r_id):
    for business in businesses:
        if business["id"] == b_id:
            for review in business["reviews"]:
                if review["id"] == r_id:
                    break
            break
    return make_response( jsonify( review ), 200)

if __name__ == "__main__":
   app.run(debug=True)
```

Do it now!	Add the new route and function and test in Postman by making **POST** requests to **/api/v1.0/businesses/1/reviews** to add a set of reviews for the first business and then a series of **GET** requests to **/api/v1.0/businesses/1/reviews/<id>** with a range of **<id>** values matching the **id** properties of the reviews.

Finally, in this section, we present the endpoints to edit or delete a specified review. These are quite similar in structure and can be presented together.

In both cases, we loop through the businesses looking for the entry that matches the business **id** value provided. Then, we loop through the collection of reviews for that business, looking for the matching review **id**. Once the review **id** is found, the **edit_review()** function updates the fields with the values provided by the user, while the **delete_review()** function uses the **remove()** method to strike the selected entry from the list. When the action is complete, we break out of both **for** loops and generate the appropriate response and status code.

File: app.py

```
...

@app.route(
  "/api/v1.0/businesses/<int:b_id>/reviews/<int:r_id>",
  methods=["PUT"])
def edit_review(b_id, r_id):
    for business in businesses:
        if business["id"] == b_id:
            for review in business["reviews"]:
               if review["id"] == r_id:
                  review["username"] = request.form["username"]
                  review["comment"] = request.form["comment"]
                  review["stars"] = request.form["stars"]
                  break
            break
    return make_response( jsonify( review ), 200)

@app.route(
        "/api/v1.0/businesses/<int:b_id>/reviews/<int:r_id>",
        methods=["DELETE"])
def delete_review(b_id, r_id):
    for business in businesses:
        if business["id"] == b_id:
            for review in business["reviews"]:
                if review["id"] == r_id:
                    business["reviews"].remove(review)
                    break
            break
    return make_response( jsonify( {} ), 200)

if __name__ == "__main__":
   app.run(debug=True)
```

Do it now!	Add the new routes and functions and test in Postman by making **POST** requests to **/api/v1.0/businesses/1/reviews** to add a set of reviews for the first business and then a series of **PUT** and **DELETE** requests to **/api/v1.0/businesses/1/reviews/<id>** with a range of **<id>** values matching the **`id`** properties of the reviews.

Note:	Ultimately, we will work with information stored in a back-end database rather than a hard-coded data structure. This will result in much of the internal code in the functions changing, but the general structure of the API will remain unchanged.

3.4 Further Information

- https://en.wikipedia.org/wiki/Representational_state_transfer
  Wikipedia – Representational State Transfer

- https://hackernoon.com/restful-api-designing-guidelines-the-best-practices-60e1d954e7c9
  RESTful API Design Guidelines – the best practice

- https://restfulapi.net/rest-api-design-tutorial-with-example/
  How to design a REST API

- https://blog.miguelgrinberg.com/post/designing-a-restful-api-with-python-and-flask
  Designing a RESTful API with Python and Flask

- https://kite.com/blog/python/flask-restful-api-tutorial/
  Tutorial – Building a RESTful API with Flask

- https://www.postman.com/
  Postman home page

- https://www.youtube.com/watch?v=juldrxDrSH0
  YouTube – Postman Beginner Tutorial

- https://www.youtube.com/watch?v=CjYKrbq8BCw
  YouTube – Creating a RESTful API with Flask (series of 4 videos)

- https://www.youtube.com/watch?v=s_ht4AKnWZg
  YouTube – Build a REST API with Python and Flask

COM661 Full Stack Strategies and Development

BE04. Improving the Basic API

Aims

- To recognise the limitations of the basic API produced previously
- To introduce the Python **`uuid`** (**U**niversal **U**nique **ID**entifier) library
- To demonstrate the generation of a randomised data set
- To appreciate the performance improvements associated with dictionaries
- To demonstrate the detection of errors in API calls and the generation of appropriate responses
- To implement pagination in the return of a large data set

Table of Contents

4.1 MORE FLEXIBLE DATA RETRIEVAL 2

4.1.1 GENERATING UNIQUE KEY VALUES 2
4.1.2 A MORE REALISTIC DATASET 4
4.1.3 UPDATING THE APPLICATION LOGIC 5

4.2 VALIDATION AND ERROR HANDLING 8

4.3 PAGINATION OF LARGE DATASETS 12

4.4 FURTHER INFORMATION 16

4.1 More Flexible Data Retrieval

In the previous practical, we built a fully functional API that supported CRUD operations on data relating to a set of businesses, where each business object also includes a nested sub-document consisting of a set of reviews contributed by users. We also saw how Postman provides a very useful tool for testing API requests such as POST, DELETE and PUT without having to build an HTML front-end.

However, the API is quite basic in some ways and before proceeding to replace the hard-coded data set with a live database, we will identify the major weaknesses on our provision and identify ways in which we can address them.

4.1.1 Generating Unique Key Values

Our basic provision used a simple incremental count to assign unique id values to businesses and reviews. This is a useful starting point, but is not a good long-term solution for two reasons:

i) Without implementing an **`id`** management scheme, it is possible for previously used **`id`** values to be assigned to new businesses. For example, say we have 10 businesses with **`id`**s numbered from 1 to 10. If we then delete the business with **`id`** value 10 and then issue a POST request to create a new business, the value 10 will be assigned to that new business. This is not necessarily a problem, but it is better practice that an **`id`** is assigned to an object for life – and if that object should be deleted, the **`id`** value is never re-assigned.

ii) More seriously, our update and delete operations require we loop across all of the businesses (and reviews) in order to find the one to which we want to apply the operation. For a small collection of businesses, this might be acceptable, but if the collection grows to 100s (or even more), then this will be a significant overhead.

We will therefore make two changes to our object structure – we can use the Python **`uuid`** (**U**niversal **U**nique **ID**entifier) package to generate a unique value to be used as the **`id`** and we will structure the collection of businesses as a dictionary rather than a list, to give direct access to a business if the **`id`** value is known.

The Python method **`uuid.uuid1()`** generates a 128-bit unique value as a combination of the MAC address of the host computer and the current time. The combination of MAC

address and time guarantees that values will be unique across different applications and different machines.

As a first step, we import **`uuid`** into our application and use it to generate the **`id`** values in our data set. Note that we convert the 128-bit value to a string so that we can use it as part of a URL, so we also need to change all of the instances of business ID within **`@app.route()`** decorators to denote that the variable parameter is now a **`string`** rather than an **`int`**.

File: app.py

```
from flask import Flask, jsonify, make_response, request
import uuid

...

businesses =  [
    {
        "id" : str(uuid.uuid1()),
        "name" : "Pizza Mountain",
        "town" : "Coleraine",
        "rating" : 5,
        "reviews" : []
    },

    ...
]

...

@app.route("/api/v1.0/businesses/<string:id>", \
           methods=["GET"])
def show_one_business(id):

...
```

Do it now!	Make the modifications shown above to incorporate **`uuid`** values as identifiers. Be sure to change all of the business ID parameters in all **`@app.route()`** decorators. Use Postman to ensure that the GET, POST, PUT and DELETE functionality on businesses still works as previously.

Try it now!	Make the same modification to the **review ID** values. Once again, use Postman to make sure that all functionality still works as previously.

4.1.2 A More Realistic Dataset

Our hard-coded data set was sufficient to test the basic operation of the API, but for a more realistic simulation we require more data. At the same time, we will re-structure the collection of businesses so that it is a dictionary rather than a list with the **`uuid`** value used as the key field for each business. The function **`generate_dummy_data()`** generates a dictionary of 100 business objects, where each object has a key field of a **`uuid`** string and a value of a dictionary of *name*, *town*, *rating* and *reviews*

File: app.py

```
from flask import Flask, jsonify, make_response, request
import uuid, random

...

businesses = {}

def generate_dummy_data():
    towns = ['Coleraine', 'Banbridge', 'Belfast',
             'Lisburn', 'Ballymena', 'Derry', 'Newry',
             'Enniskillen', 'Omagh', 'Ballymoney']
    business_dict = {}

    for i in range(100):
        id = str(uuid.uuid1())
        name = "Biz " + str(i)
        town = towns[random.randint(0, len(towns)-1)]
        rating = random.randint(1, 5)
        business_dict[id] = {
           "name" : name,
           "town" : town,
           "rating" : rating,
           "reviews" : []
        }
    return business_dict

...

if __name__ == "__main__":
    businesses = generate_dummy_data()
```

To randomize the data set, the *town* is selected at random from a list of 10 locations and the *rating* is set to a random value in the range 1-5. The *name* is a concatenation of the string "Biz " and the loop counter so that each of the 100 businesses has a different name. To generate the data, we call the new function from the main section at the bottom of the code.

Do it now!	Add the new code shown above to the application and verify its effect by loading the URL http://localhost:5000/api/v1.0/businesses to a web browser to see the complete list of business objects. Verify that you receive output similar to that shown in Figure 4.1 below.

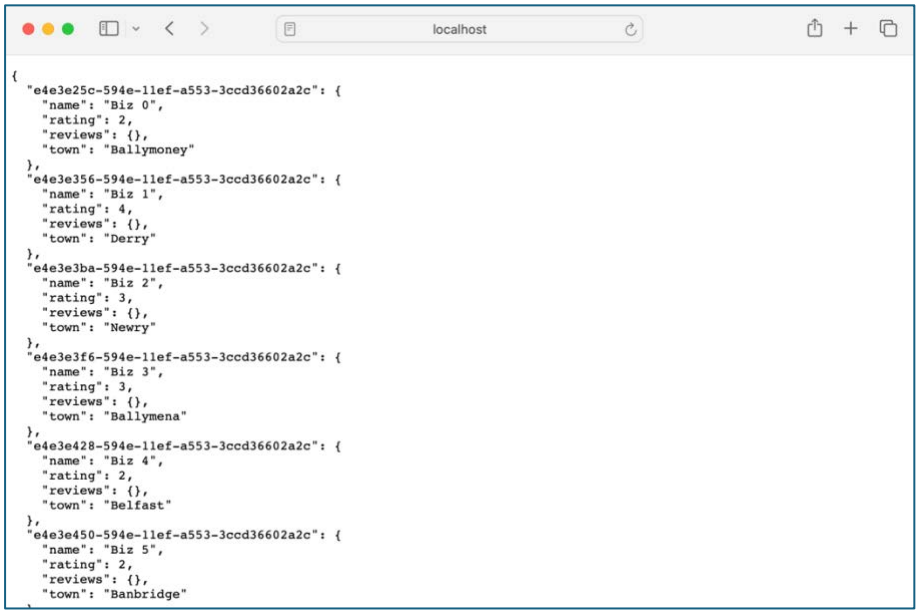


Figure 4.1 Random Dataset Organised as a Dictionary

Note how the data set above is now a dictionary, where the key field of each entry is the random **`uuid`** value. This allows us to significantly simplify the code in the functions that support each route – while also benefitting from an even more significant performance improvement.

4.1.3 Updating the Application Logic

Previously, when returning a single business, we had to loop through the collection to find the entry where the **`id`** value matched that passed as a parameter. With a dictionary, access it much more efficient since we can simply directly access the required dictionary element by using the key value as the index.

File: app.py

```
...

def show_one_business(id):
    return make_response( jsonify(  businesses[id] ), \
                          200 )


...
```

Do it now!	Make the modification to the **`show_one_business()`** function shown above and check that the function still works as expected.

The code to add a new business only has minor changes from the previous version. Rather than calculate the new **`id`** value to be used, we generate it with a call to **`uuid.uuid1()`** and create a new dictionary entry with the new **`id`** value as the key. Finally, we return a JSON object consisting of the new **`id`** as the key and the body of the new business object as the value.

File: app.py

```
...

def add_business():
    next_id = str(uuid.uuid1())
    new_business = { "name" : request.form["name"],
                     "town" : request.form["town"],
                     "rating" : request.form["rating"],
                     "reviews" : []
                    }
    businesses[next_id] = new_business
    return make_response( \
           jsonify( { next_id : new_business } ), 201


...
```

Do it now!	Make the modification to the **`add_business()`** function shown above and check using Postman that the function still works as expected.

Editing a business is another example of the performance efficiency provided by using dictionaries. Previously we had to loop across the business objects comparing the **`id`** of

each, while we can now directly access the element, changing the values to those provided by the user.

File: app.py

```
...

def edit_business(id):
    businesses[id]["name"] = request.form["name"]
    businesses[id]["town"] = request.form["town"]
    businesses[id]["rating"] = request.form["rating"]
    return make_response( \
           jsonify( { id : businesses[id] } ), 200 )
...
```

Do it now!	Make the modification to the **`edit_business()`** function shown above and check using Postman that the function still works as expected.

Note:	Remember that each time the application is run, the set of businesses will be generated anew. This means that **`id`** values from a previous run will no longer be valid, so you will need to display all businesses to get an **`id`** to use when testing the edit function.

Finally, the code to delete a business is reduced to two simple statements – one to remove the dictionary entry with the specified key (**`id`** value) and another to send the empty JSON response.

File: app.py

```
...

def delete_business(id):
    del businesses[id]
    return make_response( jsonify( {} ), 204)

...
```

Do it now!	Make the modification to the **`delete_business()`** function shown above and check using Postman that the function still works as expected.

Try it now!	Update the functions to retrieve, add, edit and delete **reviews** in the same way. Make sure that reviews are structured as a dictionary with each review identified by a **`uuid`** value in the same way as businesses. **Hint:** Remember that you will need to change the definition of the reviews element in **`generate_dummy_data()`** and **`add_business()`** to a dictionary rather than a list.

4.2 Validation and Error Handling

So far, we have assumed that all values provided to API calls are correct. For example, see what happens when you provide an invalid business ID to http://localhost:5000/api/v1.0/businesses/id

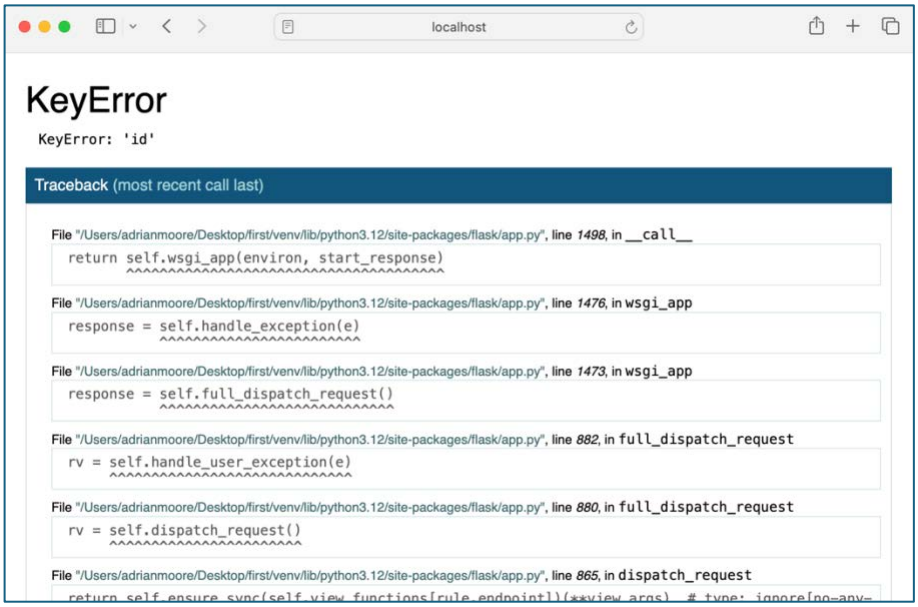


Figure 4.2 Invalid ID Value in URL

What we see in the browser in Figure 4.2 is the standard Flask/Python error message generated when we attempt to look up a dictionary using a key value that does not match any of the dictionary entries. We want to improve our API by generating a more meaningful response when an invalid operation is attempted.

As a first step, we will trap the *bad id* error in the GET request for an individual business. Before generating the response, we first test to see if the specified key value is present in the dictionary. If it exists, then we can generate the response as before, but if not, we

generate a JSON error message that can be returned along with the **404** status code that denotes *page not found*.

File: app.py

```
...

def show_one_business(id):
    if id in businesses:
        return make_response( jsonify(  \
                                businesses[id] ), 200 )
    else:
        return make_response( jsonify( \
          { "error" : "Invalid business ID" } ), 404 )

...
```

Do it now!	Make the modification to the **`show_one_business()`** function shown above and check using Postman that the function still works as expected.

With the application running, repeat the previous experiment to retrieve a business with an invalid **`id`** value. Check that you can see the error message and status code returned as illustrated in Figure 4.3 below.

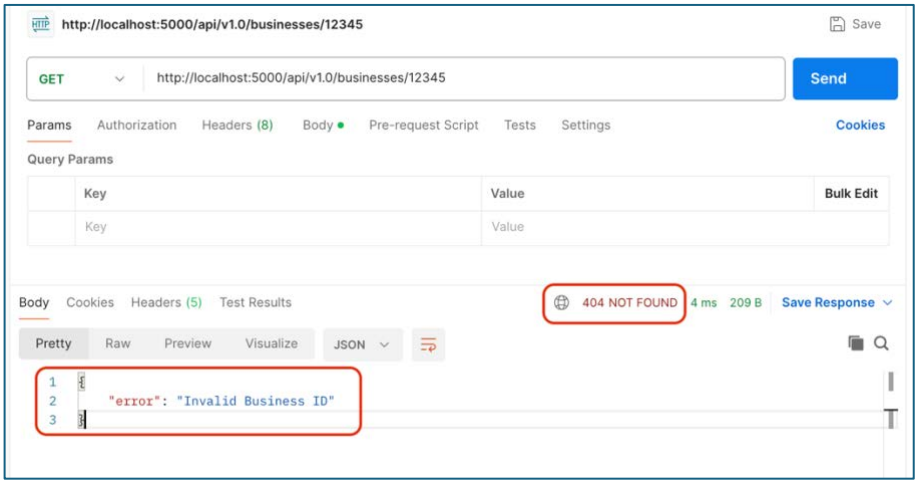


Figure 4.3 Error Handling

When adding a new business, the main potential for error is when the user does not provide all of the information that is required. Obviously, in a real application some fields may be optional, but in our example, we will assume that all of *name*, *town* and *rating* are needed before the business can be added.

In order to implement this, we check that the required values are present in the **`request.form`** object, only generating the new business if this is the case. Where any of the parameters are missing, we generate an error message.

File: app.py

```
...

def add_business():
    if "name" in request.form and
       "town" in request.form and
       "rating" in request.form:
        next_id = str(uuid.uuid1())
        new_business = {
            "name" : request.form["name"],
            "town" : request.form["town"],
            "rating" : request.form["rating"],
            "reviews" : {}
        }
        businesses[next_id] = new_business
        return make_response( jsonify(
               { next_id : new_business } ), 201 )
    else:
        return make_response( jsonify(
               { "error" : "Missing form data" } ), 404 )

...
```

Do it now!	Test the new functionality by adding a new business with all values provided and with some missing to ensure that the error trapping is working properly.

Note:	A further level of error trapping might be to test the type and range of each parameter to check that (for example) the rating is an integer between 1 and 5. This level of checking is left for you to complete as an exercise.

When editing an existing business entry, there are two different errors that we want to trap (i) when the **`id`** provided does not match a business in the collection and (ii) when all of the required values for the edit operation are not provided. This makes for a slightly more complex scenario than the previous examples, where a pair of nested **`if`** statements are used to check for the potential errors.

File: app.py

```
...

def edit_business(id):
    if id not in businesses:
        return make_response( jsonify(
           { "error" : "Invalid business ID" } ), 404 )
    else:
        if "name" in request.form and
           "town" in request.form and
           "rating" in request.form:
            businesses[id]["name"] = request.form["name"]
            businesses[id]["town"] = request.form["town"]
            businesses[id]["rating"] =
                                    request.form["rating"]
            return make_response( jsonify(
                   { id : businesses[id] } ), 200 )
        else:
            return make_response( jsonify(
               { "error" : "Missing form data" } ), 404 )

...
```

Do it now!	Make the changes shown above and test the new functionality by providing input such that all three potential paths are followed.

Try it now!	Update **`edit_business()`** so that only the fields provided by the user are modified. For example, if a PUT request were received and only the rating value was provided, then the operation would be successful with the rating updated to the new value and the name and town fields left at their existing values. Only if no field values were provided would an error message be generated.

Finally, the delete operation simply needs to check that the specified dictionary key exists before attempting to delete it.

File: app.py

```
...

def delete_business(id):
    if id in businesses:
        del businesses[id]
        return make_response( jsonify( {} ), 204)
    else:
        return make_response( jsonify(
             { "error" : "Invalid business ID" } ), 404 )

...
```

Do it now!	Test the operation of the delete operation by trying it with both invalid and valid business **`id`** values.

Try it now!	Update the functions to retrieve, add, edit and delete reviews in the same way **Hint:** Remember that you will need to check for the presence of a valid business **`id`** as well as for a review **`id`**.

4.3 Pagination of Large Datasets

Now that we have a larger collection of sample data to work with, we can resolve another weakness of the API – the delivery of the full set of results in response to a **GET** request to http://localhost:5000/api/v1.0/businesses. When only a small quantity of data is stored, it is probably OK to return the full set to the user in one go, but when 100s, 1000s or even more results are available, it is more usual to paginate them so that only a sub-section of the results are returned at once.

Pagination depends on two parameters – the page number that is requested (i.e. page 1, page 2, etc) and the number of results to be returned on each page. The most flexible approach is to allow these to be determined by the application requesting the resource, so that (for example) a **GET** request to http://localhost:5000/api/v1.0/businesses?pn=2&ps=10 where **pn** is the page number variable and **ps** is the page size variable denotes that we are being asked for the 2nd page of results (**pn=2**) and that there are 10 results required per page (**ps=10**).

File: app.py

```
...

def show_all_businesses():
    page_num, page_size = 1, 10
    if request.args.get('pn'):
        page_num = int(request.args.get('pn'))
    if request.args.get('ps'):
        page_size = int(request.args.get('ps'))
    page_start = (page_size * (page_num - 1))
    businesses_list = [ {k : v} for k,v in businesses.items() ]
    data_to_return = businesses_list[
                     page_start:page_start + page_size]
    return make_response( jsonify(
                          data_to_return ), 200 )

...
```

In the code presented above, we first set default values for page number and page size and then overwrite these with values in the querystring, if they have been provided. Note the Flask method **`request.args.get()`** to test for the presence of a value in the querystring and to retrieve it. Once the values for page number and page size are known, we can calculate the elements required by the formula

first element = page size * (page number – 1)

For example, if page size is 10, the first element on each page can be calculated as in the following table (remembering that the first page begins with the element at position zero).

Page number	First element
1	10 * (1-1) = 0
2	10 * (2-1) = 10
3	10 * (3-1) = 20
4	10 * (4-1) = 30
5	10 * (5-1) = 40

Therefore, page 1 will consist of elements 0-9, page 2 of elements 10-19, page 3 comprises elements 20-29 and so on.

The final stage is to convert the dictionary into a list so that the required elements can be extracted from it using the slice operation. This is performed by the complex Python **list comprehension** statement

```
businesses_list = [ { k : v } for k, v in businesses.items() ]
```

which can be explained as follows.

Stage 1: We want to return a list called **`businesses_list`**

```
businesses_list = [ ]
```

Stage 2: the list will be a collection of dictionary key:value pairs

```
businesses_list = [ { k : v } ]
```

Stage 3: Each **`{k:v}`** entry will be obtained by looping across the dictionary of businesses

```
businesses_list = [ { k : v } for k, v in businesses.items() ]
```

Finally, the Python slice operation is used to extract the requested elements from the **`businesses_list`** and return those as the response from the request.

Do it now!	Modify the function **`show_all_businesses()`** as illustrated above and test it by specifying different values for **pn** (page number) and **ps** (page size) in a GET request to API endpoint http://localhost:5000/api/v1.0/businesses?pn=1&ps=10. Note that the order of querystring parameters is insignificant. Check that you receive output such as that shown in Figure 4.4, below.



Figure 4.4 Page 1 and Page 2 of Paginated Output

Try it now!	Implement a similar pagination scheme when retrieving all of the reviews of a business.

Note:	It may feel like a backward step to convert the dictionary of businesses to a list so that the slice operation can be used to extract the required subset of data. This is purely to demonstrate the use of page size and page number variables passed to the API in the querystring. In the coming practicals we will replace the hard-coded data with a database which will enable us to implement the pagination logic within the database query.

4.4 Further Information

- https://docs.python.org/3/library/uuid.html
  UUID Objects – Python manual

- https://www.listendata.com/2019/04/create-dummy-data-in-python.html
  Create dummy data in Python

- https://realpython.com/python-random/
  Generating random data in Python

- https://docs.python.org/3/tutorial/datastructures.html
  Python data structures

- https://www.programiz.com/python-programming/nested-dictionary
  Nested dictionaries in Python

- https://www.datacamp.com/community/tutorials/python-dictionary-comprehension
  Python dictionary comprehension tutorial

- https://documentation.commvault.com/commvault/v11/article?p=45599.htm
  REST API – Response codes and statuses

- https://nordicapis.com/best-practices-api-error-handling/
  Best practices for API error handling

- https://www.pythonforbeginners.com/basics/list-comprehensions-in-python
  List comprehensions in Python

- https://medium.com/better-programming/list-comprehension-in-python-8895a785550b
  List comprehension in Python – creating list from other iterables

COM661 Full Stack Strategies and Development

BE05. MongoDB

Aims

- To compare the relational and document store models for databases
- To provide guidelines for the design of document store databases
- To introduce the MongoDB server and shell
- To explain the purpose and structure of MongoDB **`_id`** values
- To introduce the **`find()`** method to search a MongoDB collections
- To demonstrate the **`update()`** and **`delete()`** methods for modifying documents and collections
- To demonstrate the **`drop()`** method for deleting collections
- To compare BSON and JSON as alternatives for data transfer

Table of Contents

5.1 NOSQL DATABASES 2

5.1.1 Relations vs Documents 2
5.1.2 Document-based Design 4

5.2 INTRODUCING MONGODB 6

5.2.1 MongoDB ID Values 7
5.2.2 The MongoDB Shell 7
5.2.3 Launching a MongoDB Server 8
5.2.4 Working with Databases, Collections and Documents 9

5.3 BASIC MONGODB COMMANDS 12

5.3.1 Basic Searching 12
5.3.2 Sorting Query Results 13
5.3.3 Updating Documents 14
5.3.4 Deleting Documents and Collections 15

5.4 EXPORTING AND IMPORTING DATA 16

5.4.1 Exporting and Importing BSON Data 17
5.4.2 Exporting and Importing JSON Data 19

5.5 FURTHER INFORMATION 21

5.1 NoSQL Databases

NoSQL refers to a collection of database organisation models that offer alternatives to the tabular relations used in relational databases. Among the most popular NoSQL models are Key-value stores, Document Stores and Graph databases.

MongoDB is an example of a document store database, so we will begin by examining the main differences between this and the relational architecture that you are familiar with.

5.1.1 Relations vs Documents

To illustrate the difference between relational and document store databases, consider a potential relational structure for a database to support a simple blog application, consisting of blog posts and comments.

The relational approach is to maintain separate data items in separate tables, with relationships between them identified by foreign key fields. For example, consider the structure illustrated in Figure C1.1 where the first two entries in the COMMENTS table can be seen to refer to the first entry in the POSTS table by use of the foreign key field **PostID**

POSTS

PostID	Title	Content
1	My first post	Some content
2	My second post	Some content
3	My third post	Some content

COMMENTS

CommentID	PostID	Comment
1	1	Commenting on your first post
2	1	Also on your first post
3	2	Commenting on your second post

Figure 5.1 Relational Table Structure

In order to display a post and its comments, the application would need to query both tables, retrieving a row of data from the POSTS table as well as all rows in the COMMENTS table where the **PostID** value matches that in the row being retrieved from the POSTS table.

Relational databases are designed for **efficiency of storage** and a primary aim when designing a relational structure is to eliminate duplication of data. Each piece of data should only be stored once, so that when any information is updated, it only needs to be changed in one place. The rules that enforce this make up the process known as **normalisation**.

A document store database, on the other hand, is designed for **efficiency of retrieval**. The design is governed by the application in which the database will be used, so that (as far as

possible) all of the information required should be retrieved in a single operation (i.e. one query, with no joins between different collections).

In the blog example, above, the most commonly performed operation will be the retrieval and display of posts and their comments, so the same dataset might be represented by the following JSON-style structure.

```
[
  {
    "PostID" : 1,
    "Title" : "My first post",
    "Content" : "Some content",
    "Comments" : [
       {
         "CommentID" : 1,
         "Content" : "Commenting on your first post"
       },
       {
         "CommentID" : 2,
         "Content" : "Also on your first post"
       }
    ]
  },
  {
    "PostID" : 2,
    "Title" : "My second post",
    "Content" : "Some content",
    "Comments" : [
       {
         "CommentID" : 3,
         "Content" : "Commenting on your second post"
       }
    ]
  },
  {
    "PostID" : 3,
    "Title" : "My third post",
    "Content" : "Some content"
  }
]
```

Here, the entire structure is represented as an array (enclosed within **[** … **]**), with each post defined as a JSON object (enclosed within **{** … **}**). Each object (post) is specified as a key/value pair, where the key is the name of the element (i.e. the column name in the relational table) and the value is the data element that is stored. Note that comments on

posts are embedded into the structure, with the **Comments** item defined as an array of comment objects. Note also, that when a field in an object is not required (e.g. the third post has no comments), then it can be simply left out. This is in contrast to the relational model which demands that each field has a value (even if that value is NULL).

5.1.2 Document-based Design

The principle of efficiency of retrieval has a significant effect on the way in which we approach database design. In a relational architecture, there is typically one "correct" normalised structure for any given set of data, and all applications that make use of this database will interact with it according to this structure. However, in a document store database we must consider the application when designing the database so that we produce the most efficient data access platform. This may result in two applications that use the same raw data set with different preferred database architectures – because of the way in which they consume the data.

Consider the addition of a 'users' collection to our blog structure. We may want to store a set of various information about the user such as a name, email address, short bio, star sign, favourite colour, etc., – so an obvious approach would be to have a separate structure for users such as that shown below.

```
[
  {
    "UserID" : 1,
    "UserName" : "Adrian",
    "DisplayName" : "Adrian Moore",
    "Bio" : "Senior Lecturer"
  },
  {
    "UserID" : 2,
    "UserName" : "Joe",
    "DisplayName" : "Joe Rafferty",
    "Bio" : "Course Director"
  }
]
```

An application that needs to display all details pertaining to an individual user will simply read the appropriate object from this collection and retrieve each field.

However, it is obvious that we may want to display the name of a user alongside any posts that they have made, but maintaining this information in a separate collection goes against

the principle of efficiency of retrieval – as the information on a post and the user who contributed it would be spread across two collections.

The solution is one that is alien to a relational database designer – we simply store the information in each place where it is needed – giving the following structure for the collection of posts and comments.

```
[
  {
    "PostID" : 1,
    "Title" : "My first post",
    "Content" : "Some content",
    "Comments" : [
       {
         "CommentID" : 1,
         "Content" : "Commenting on your first post"
       },
       {
         "CommentID" : 2,
         "Content" : "Also on your first post"
       }
    ],
    "Author" : {
      "UserId" : 1,
      "DisplayName" : "Adrian Moore"
    }
  },
  {
    "PostID" : 2,
    "Title" : "My second post",
    "Content" : "Some content",
    "Comments" : [
       {
         "CommentID" : 3,
         "Content" : "Commenting on your second post"
       }
    ] ,
    "Author" : {
      "UserId" : 1,
      "DisplayName" : "Adrian Moore"
    }
  },
  {
    "PostID" : 3,
    "Title" : "My third post",
    "Content" : "Some content" ,
```

```
      "Author" : {
        "UserId" : 2,
        "DisplayName" : "Joe Rafferty"
      }
    }
]
```

Under this scheme, we are storing some of the user information twice – all user details are held in the "Users" collection so that the page that shows a user's information can retrieve it all from one place, but some details are duplicated in the "Posts" collection so that we can satisfy the 'view posts' operation with a single database query.

The obvious drawback of this is that if a user decided to change their *DisplayName*, we would need to open the database and change every instance of it. This would be a very expensive operation, but its cost is outweighed by the benefit of the simple single-shot data retrieval for the most commonly performed operation.

In general, there are three main principles for design of document store database structures:

1. Keep the number of collections (tables) to a minimum
2. Keep each page or view to a single database query (no joins)
3. Optimise for the most common operations – even at the expense of less common tasks

5.2 Introducing MongoDB

MongoDB (derived from Hu**mongo**us **D**ata**b**ase) is a document store database that organises information as **collections**. In our Posts example from the previous section, the data represents a single collection, which is made up of a number of **documents** – each document being a specification of a single post. Therefore, our **Posts** collection comprised three documents, while our **Users** collection was made up of two documents.

Although we have used JSON as the means of describing the collections, MongoDB actually uses a notation called **BSON** (pronounced ***bi-son***) – a binary encoding of JSON that maintains the flexibility and ease of use of JSON, while adding the speed advantages of a binary format. However, MongoDB accepts JSON as input and produces JSON as output, so we as developers do not need to be concerned with the internal representation.

5.2.1 MongoDB ID Values

In our example, we included **ID** fields for **Posts**, **Comments**, **Authors** and **Users**. In fact, MongoDB will generate **ID** values automatically for each new document (or sub-document) that is created – so we never need to provide these ourselves.

In MongoDB, the ID field is always called **_id** and has a value that is defined as an **ObjectId()** represented as a long alphanumeric string derived from the date, time, machine identifier, process identifier and a counter – so no two **_id** values will be identical even across different installations. For example, a MongoDB implementation of our Posts collection might have auto-generated **_id** values as follows.

```
[
  {
    "_id" : ObjectId("165de4208b234b2140"),
    "Title" : "My first post",
    "Content" : "Some content",
    "Comments" : [
       {
         "_id" : ObjectId("567be1208b234b5778"),
         "Content" : "Commenting on your first post"
       },
       {
         "_id" : ObjectId("bc009208b2738f6e5"),
         "Content" : "Also on your first post"
       }
    ],
    "Author" : {
      "_id" : ObjectId("5e53c4208b234012b4"),
      "DisplayName" : "Adrian Moore"
    }
  },
  ...
]
```

5.2.2 The MongoDB Shell

MongoDB has been provided for you on the lab machines, but if you want to install it on your own machine, you can obtain it free from https://www.mongodb.com/try/download/community.

Note:	Installation notes for MongoDB on all platforms can be found at https://www.mongodb.com/docs/v6.0/installation/. When installing your own copy, make sure that you choose the **Community Edition** as this is the freeware version.

The only piece of setup we need to do is to provide a data folder in which MongoDB will store the databases. On your personal machine, this will be a folder **C:\data\db** – but the lab installation should be set to **P:\data\db** so that your work will be available to you regardless of which lab machine you use.

Check your **P:** drive for the presence of a folder **P:\data\db** and, if it does not exist, create it now by opening a Command window, navigating to the **P:** drive and issuing the commands

```
P:\> mkdir data
P:\> cd data
P:\data> mkdir db
```

5.2.3 Launching a MongoDB Server

Note:	If, during installation, you opt to "Run MongoDB as a service", you will not need to launch the server – it will automatically run when your machine boots up.

In order to use MongoDB, we need to launch a MongoDB server that will service requests that are made either from the Mongo Shell or from other applications. You can launch the MongoDB server by issuing the command

```
P:\> mongod --dbpath=p:\data\db
```

which should result in output as illustrated in Figure 5.2 below.

Note:	If you are using your personal (Windows) machine with the default data folder at **c:\data\db** then the command is simply **`mongod`** without the **`--dbpath`** command line argument.

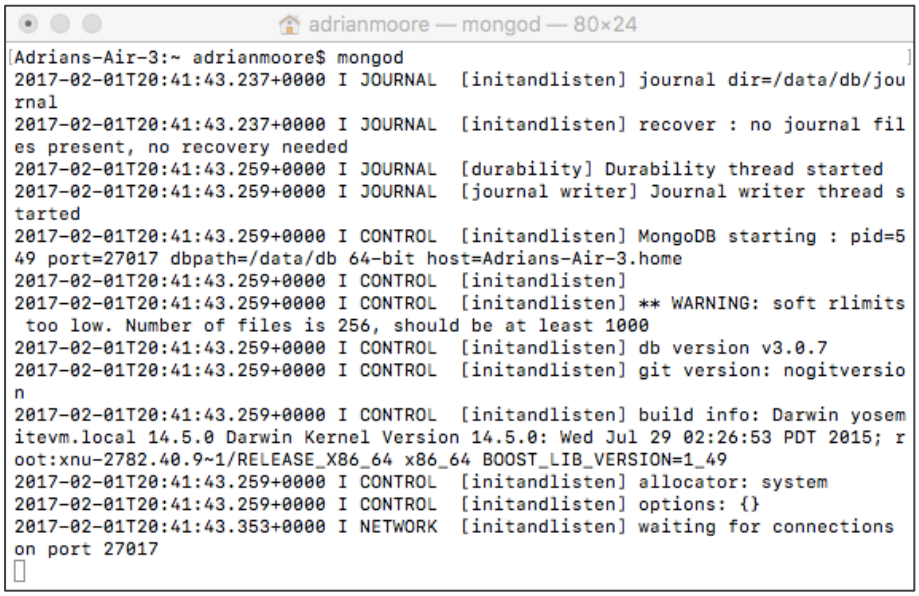


Figure 5.2 MongoDB Server

We leave this Command window open with the server application running while we interact with the database.

Note:	On a MacOS machine, the process for running the MongoDB Server may differ depending on how you have installed the software. If you have used the **Homebrew Package Manager**, then the command to start the server is **`brew services start mongodb-community`**

5.2.4 Working with Databases, Collections and Documents

We can verify that MongoDB is installed and that a server is running by opening a new Command window (remember to leave the previous one open and running) and issuing the command to enter the Mongo Shell as follows.

```
P:\> mongosh
```

Note:	On Windows installations, the MongoDB Shell needs to be downloaded as a separate tool from https://www.mongodb.com/try/download/shell. MacOS installations will automatically include the Shell with the main MongoDB download.

After some initial start-up and status messages, you should see the MongoDB prompt **`name>`**, where **`name`** is the name of the currently selected database, at which we can enter MongoDB Shell commands.

First, we will ask MongoDB to list the databases available to us by the command

```
test> show dbs
```

By default, databases called **admin**, **config**, **local** are available on most installations (though this may vary). We can switch to our desired database by the command **use**, for example

```
test> use local
```

To add a new database, we simply **use** one that is not already present. For example, to create a new database called **first**, we issue the command

```
local> use first
```

and create a collection within this database by the command

```
first> db.createCollection("fullStack")
```

where the object **db** refers to the currently selected database (the subject of the most recent **use** command) and the parameter **fullStack** is the name to be given to the new collection.

Note:	To list the collections in a database, you can issue the command `> show collections`

Note:	From this point, we will stop prefacing the **>** prompt with the name of the database, since the commands are applicable to any database.

To add a document to a collection, we need to **insert** it into a collection on the currently active database by the command

```
> db.fullStack.insertOne(  {
      name:"MongoDB", role:"Database" }  )
```

We should now see confirmation that have inserted one document into the collection.

To add multiple documents at once, we can simply pass an a JSON array as the parameter to the instruction **insertMany()**

```
> db.fullStack.insertMany( [

    { name: "Flask", role: "Web application framework" },
    { name: "Angular", role: "Front-end framework" },
    { name: "Python", role: "Back-end language" }
] )
```

Figure 5.3 illustrates the output generated by the above sequence. Note that you can enter JSON data over multiple lines by using the ***<return>*** key.

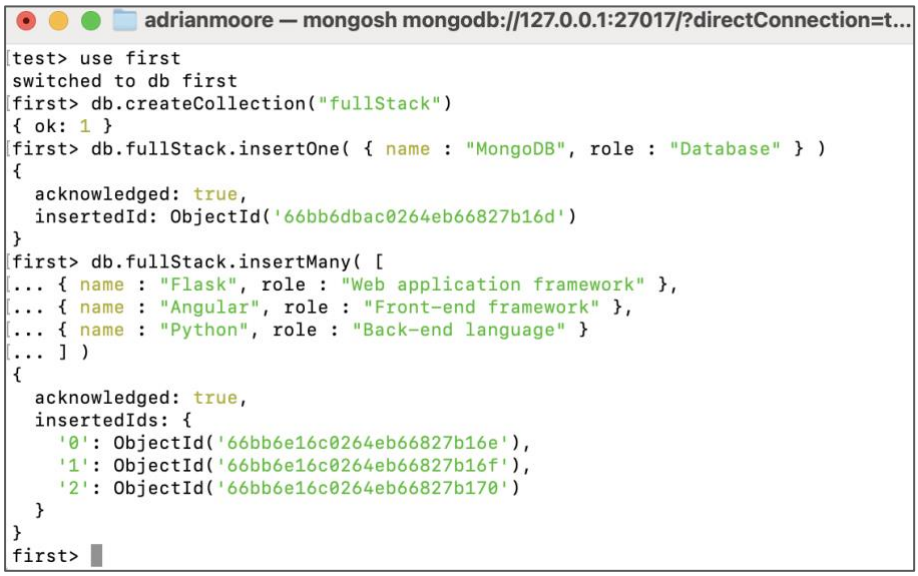


Figure 5.3 Using the MongoDB Shell

Do it now!	Follow the sequence of instructions above to enter the MongoDB shell, create the database and collection and add the documents to the collection. Verify that you receive output similar to that shown in Figure 5.3 above

Try it now!	Add a second collection of your choice to the first database and insert at least three documents to it. Each document should contain at least two fields – but remember that the fields do not have to be consistent across the documents in the collection.

To retrieve documents from a collection, we chain the **`find()`** method to the database and collection

```
> db.fullStack.find()
```

Figure 5.4 illustrates the output when retrieving the collection Note the format of the **`_id`** field that was automatically assigned by MongoDB as each document was generated.

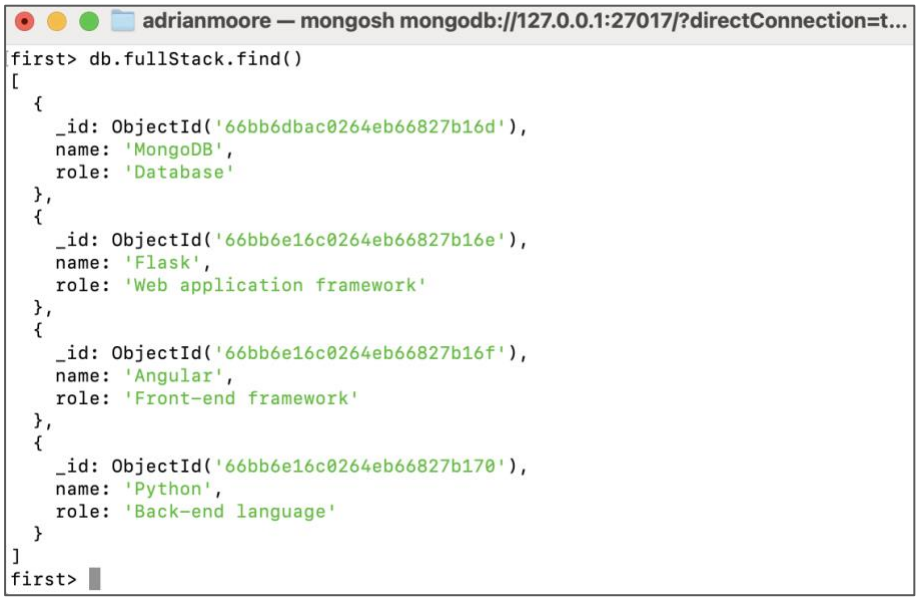


Figure 5.4 Retrieving the Collection

To leave the Mongo Shell, either enter **CTRL-C** or issue the command **exit**. The **mongod** server can also be stopped by **CTRL-C.**

Do it now!	Issue the **`find()`** on your ***fullStack*** collection to generate output similar to that shown in Figure 5.4 above.

5.3 Basic MongoDB Commands

Now that we can create databases and collections and provide documents for them, we can begin to manipulate the data with some of the basic MongoDB commands.

5.3.1 Basic Searching

We have already seen how the **`find()`** method can be used to retrieve the documents within a collection. Where we want to retrieve a subset of the documents, we pass a JSON object to **`find()`** which contains the fields and values that we want to match.

For example, to return the document where the **name** field matches the value “Flask”, we can use the command

```
> db.fullStack.find ( { name : "Flask" } )
```

If we want to match multiple values, we simply add them to the object passed as a parameter to **`find()`**. For example, to retrieve documents where the **name** field is “MongoDB” and the **role** field is “Database”, we would issue the command

```
> db.fullStack.find ( { name:"MongoDB", role:"Database" } )
```

By default, the **`find()`** method returns whole documents – i.e. the object returned contains every field in all matching documents. Where we only want specific values to be retrieved, we specify those fields in a second parameter (known in MongoDB terminology as a ***projection***) to **`find()`** as shown in the following command which requests that only the **role** values should be returned.

```
> db.fullStack.find ( {name:"MongoDB"}, {role:true }  )
```

Note that by default, the **`_id`** field is ALWAYS returned – even when we do not specify it. If we want the **`_id`** to be excluded, we need to explicitly state that in the command as shown in the following

```
> db.fullStack.find( {name:"MongoDB"},{role:true,_id:false } )
```

If we want to return specific fields from **all documents**, we can simply leave the first parameter of **`find()`** as an empty object.

```
> db.fullStack.find ( {}, {name:true,_id:false } )
```



Figure 5.5 Searching the Collection

5.3.2 Sorting Query Results

We can sort the results of MongoDB queries by chaining the **`sort()`** method to the result of a **`find()`** operation. We specify the field to sort by and the direction of sort by providing the information as a Javascript object, passed as a parameter to **`sort()`**. For example, to return the collection of **name** values sorted in ascending order, we would use the command

```
> db.fullStack.find ( { }, { name : true, _id : false } )
  .sort( { name : 1 } )
```

To perform the sort in descending order, we change the sort key value from 1 to -1.

```
> db.fullStack.find ( { }, { name : true, _id : false } )
  .sort( { name : -1 } )
```

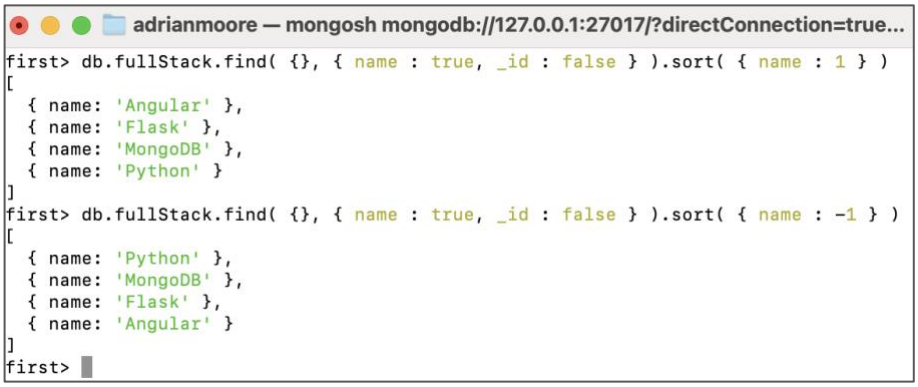


Figure 5.6 Sorting the Collection

Do it now!	Try the searching and sorting commands shown in Sections 5.3.1 and 5.3.2 above and ensure that you receive output such as that shown in Figures 5.5 and 5.6

Try it now!	Repeat the **searching** and **sorting** exercise with your own collection created earlier. Try different combinations of search fields, projections and sort options to ensure that you understand how the queries are constructed.

5.3.3 Updating Documents

Updating a document is a two-stage command that consists of a **find** to locate the document(s) to be modified and a **set** to write the new values. These are passed as two parameters to the **`updateOne()`** method as illustrated in the following command. Note the use of the special MongoDB command **`$set`** in the second parameter.

```
> db.fullStack.updateOne ( { name : "MongoDB" },
                           { $set : { name : "Mongo"} }
                         )
```

This command locates all documents with a **name** value of "MongoDB" and updates those **name** fields to the new value "Mongo". By default, the **`updateOne()`** method will only

update a single document – i.e. the first document that matches the search term. If we want to update multiple documents, we need to use the **`updateMany()`** method. We can illustrate this in our database by the following command that adds a new field to every document with a key of "module" and a value of "COM661"

```
> db.fullStack.updateMany ( { } ,
                            { $set : { module : "COM661" } }
                          )
```

Do it now!	Try the update commands shown above and ensure that you receive output such as that shown in Figures 5.7 below. Add a **`find()`** command to verify that the updates have been completed successfully

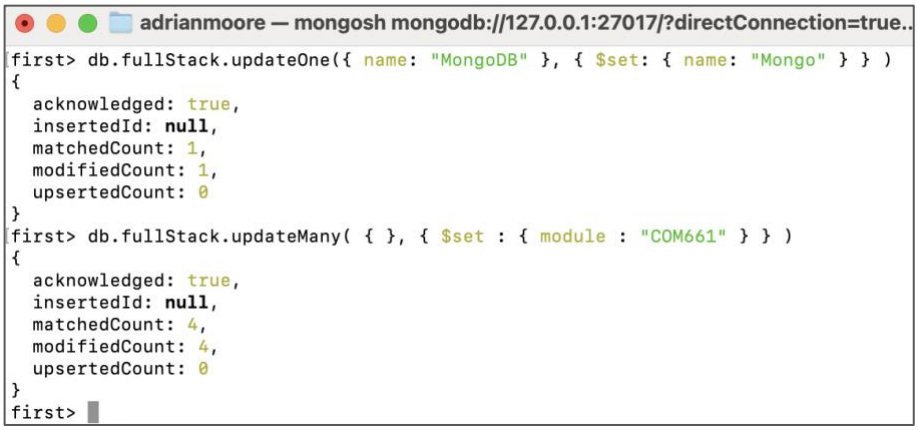


Figure 5.7 Updating a Collection

Try it now!	Repeat the update exercise with your own collection to ensure that you understand how it works.

5.3.4 Deleting Documents and Collections

In order to delete a document from a collection, MongoDB provides a **`deleteOne()`** method. This method takes a JSON object as a parameter that specifies the documents to be deleted. For example, we could remove the "Angular" document from our example database by the command

```
> db.fullStack.deleteOne ( { "name" : "Angular" } )
```

`deleteOne()` will remove the first matching element but if multiple deletions are required the **`deleteMany()`** method can be used.

Note that the format of the parameter in **`deleteMany()`** is exactly that already seen in the **`find()`** method – hence an empty object will match ALL documents and remove everything from the collection.

```
> db.fullStack.deleteMany ( { } )
```

Deleting an entire collection is performed by the **`drop()`** method as illustrated by the command

```
> db.fullStack.drop ( )
```

Figure 5.8 demonstrates the **`deleteOne()`**, **`deleteMany()`** and **`drop()`** methods

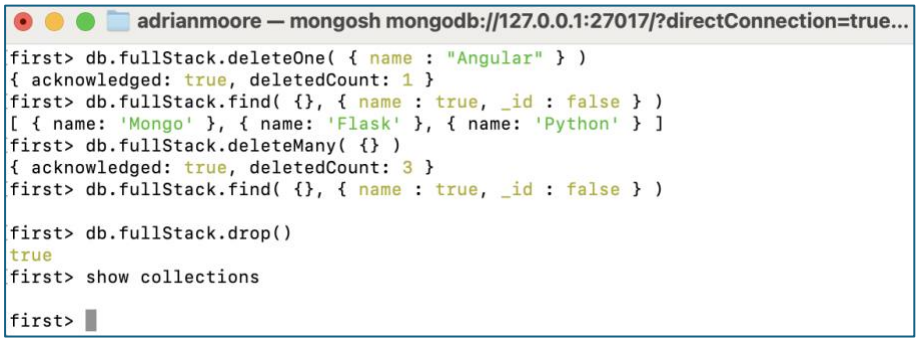


Figure 5.8 Delete and Drop Methods

Do it now!	Try the **delete** and **drop** commands shown above and ensure that you receive output such as that shown in Figure 5.8 above.

Try it now!	Repeat the **deleteexit()** and **drop** exercise with your own collection to ensure that you understand how the commands are formed

5.4 Exporting and Importing Data

A common requirement in database-driven applications is the need to be able to move data in and out of the database. This might be to create backups or to populate an application with data before launch.

Do it now!	In preparation for this next section, re-create the **fullStack** collection and add the original four documents.

5.4.1 Exporting and Importing BSON Data

Mongo provides options for export and import as both BSON (Binary JSON) and standard JSON. The option we choose to use depends on our requirement – if we are exporting the data for archive purposes, we will most likely use the binary format, while if we will want to read the data ourselves, then the JSON option is the most appropriate choice.

The Mongo tool for exporting database contents is **mongodump** which is run from the command prompt (not the MongoDB shell) as

```
P:\> mongodump --db first
```

where the name of the database to be dumped is passed as the value of the **--db** flag.

Note:	On Windows installations, the **MongoDB Database Tools** (which includes **mongodump** and the other applications we will use in this section) needs to be downloaded separately from https://www.mongodb.com/try/download/database-tools. MacOS installations will automatically include the Database Tools with the main MongoDB download.

The **mongodump** tool saves the exported data in a folder called **dump** within the current working directory. If you explore the **dump** folder you should find another sub-folder called **first** (the name of our database) and inside this folder you should see the BSON files that have been created.

The opposite action to **mongodump** is **mongorestore**, which allows us to create a database by importing a previously dumped dataset. The **mongorestore** tool is run as

```
P:\> mongorestore dump\first
```

Which will restore the contents of the **first** database from file. In order to demonstrate it, however, we will restore the data to a new database called **second**, which requires a slightly more complex form of the command.

```
P:\> mongorestore --nsInclude="first.*"
        --nsFrom="first.*" --nsTo="second.*" dump\
```

This will create a new database called **second** by importing the data from our previous **mongodump** action. Figure 5.9 illustrates the dump and restore process used to make a copy of our database **first** in a new database called **second**.

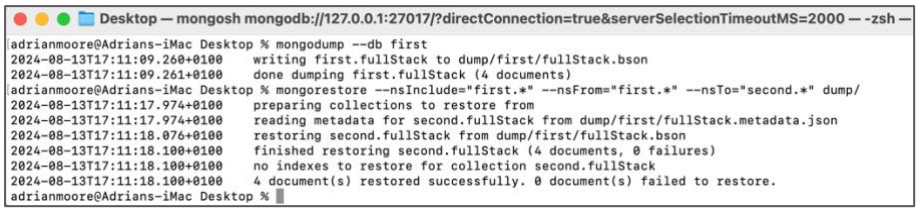


Figure 5.9 Dump and Restore Operations

Having restored the database, we can show that the operation has worked by entering the MongoDB shell, specify that we want to use the new database and display the data. Figure 5.10 illustrates this process.

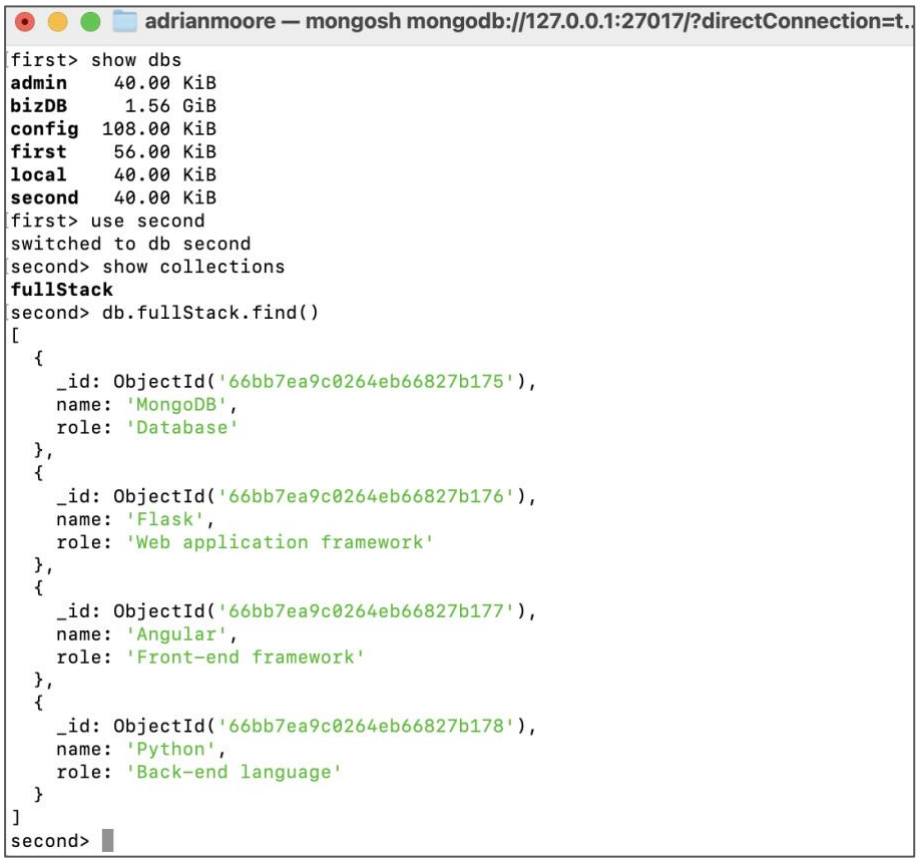


Figure 5.10 Using the Copied Database

Note that **mongorestore** only performs **insert** operations – it does not do updates. If we are restoring to a database that already has content, only documents with **`_id`** values that are NOT already present will be added. It will not replace current values with new data from the file to be restored.

Do it now!	Try the **dump** and **restore** sequence shown above to create a copy of your database called ***first*** as a new database called ***second***. Display the contents of the ***fullStack*** collection in ***second*** to verify that it has worked as expected.

5.4.2 Exporting and Importing JSON Data

When we export JSON from a MongoDB database, we create a JSON representation of the data. We can see this by using the **mongoexport** tool and examining the JSON that is returned. The **mongoexport** tool is initiated as

```
P:\> mongoexport --db first --collection fullStack
```

Note that we need to specify the collection to be exported as well as the database and that, as illustrated in Figure 5.11, the default behavior is to output the JSON data to the console.

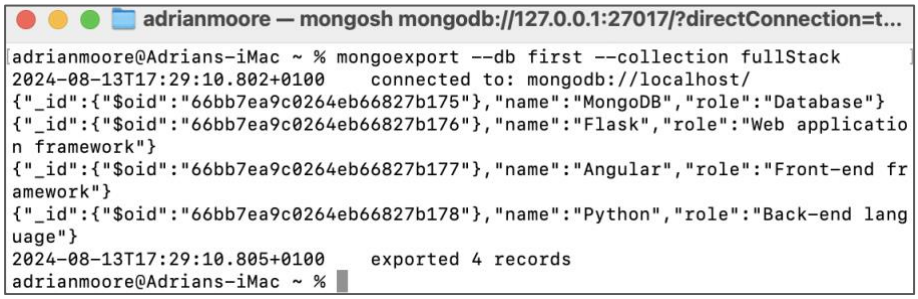


Figure 5.11 Using mongoexport

In order to save the data to a file we need to specify an additional flag to the **mongoexport** command as follows

```
P:\> mongoexport --db first --collection fullStack
      --out data.json
```

If we examine the data file generated, we can see that it is still not quite usable as it does not represent a valid JSON structure as each document in the collection is represented as a separate JSON object, but it has failed to present the objects as an array.

Fortunately, there are another pair of flags we can specify, that will structure the data as an array (**`--jsonArray`**) and format it in a more human-readable form (**`--pretty`**)

```
P:\> mongoexport --db first --collection fullStack
     --out data.json --jsonArray --pretty
```

Now, if we examine the JSON file generated, we see that is has a more useful appearance

```
[{
      "_id": {
            "$oid": "5898e88e6f3eb4a7babc4682"
      },
      "name": "MongoDB",
      "role": "Database"
 },
...
]
```

Now, we can see the JSON representation of the data, and in particular how it treats the **_id** value. When we retrieved documents earlier using the **find()** method, we saw how the **_id** was represented as an **ObjectId()** value. However, as this is not valid JSON, **mongoexport** creates a valid representation by defining **_id** as a JSON object, with the unique identifier as the value of an **$oid** property.

The opposite command to **mongoexport** is **mongoimport**, which allows us to create a database and import a collection from a JSON file. The format of the **mongoimport** command is

```
P:\> mongoimport --db third --collection fullStack
     --jsonArray data.json
```

where we specify the database to be used or created (**third**), the collection to be populated with the database (**fullStack**), that the file is a JSON array (**--jsonArray**) and finally the JSON file containing the data (**data.json**).

Figure 5.12 illustrates the **mongoimport** process and demonstrates that the database called ***third*** has been created and populated with the data.

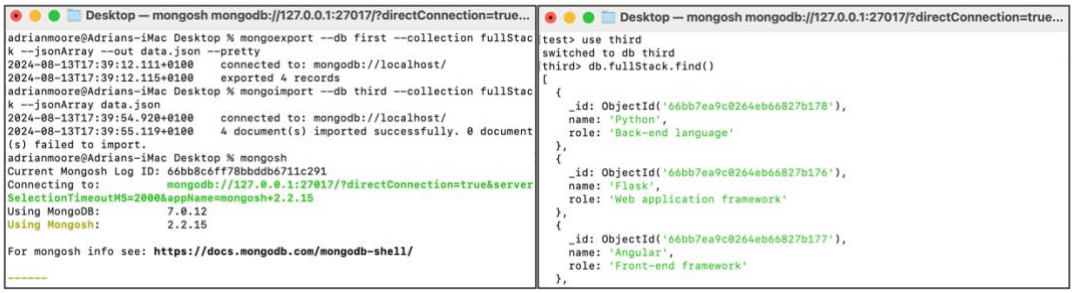


Figure 5.12 Using mongoimport

Do it now!	Try the **export** and **import** sequence shown above to export your database called ***first*** into a new database called ***third***. Display the contents of the ***fullStack*** collection in ***third*** to verify that it has worked as expected.

5.5 Further Information

- https://www.youtube.com/watch?v=FwMwO8pXfq0
  How to install MongoDB on Windows 10 - YouTube
- https://zellwk.com/blog/install-mongodb/
  Installing MongoDB on a Mac (Catalina and non-Catalina)
- https://www.youtube.com/watch?v=2yQ9TGFpDuM
  NoSQL Databases for Beginners - YouTube
- https://www.youtube.com/watch?v=ZS_kXvOeQ5Y
  SQL vs NoSQL - YouTube
- https://www.w3resource.com/mongodb/nosql.php
  Introduction to NoSQL - HotJar
- https://www.mongodb.com/
  MongoDB home page
- https://www.tutorialspoint.com/mongodb/index.htm
  MongoDB Tutorial from TutorialsPoint
- https://www.guru99.com/mongodb-tutorials.html
  MongoDB Tutorial for Beginners
- https://docs.mongodb.com/manual/tutorial/query-documents/
  Searching MongoDB Documents
- https://beginnersbook.com/2017/09/mongodb-sort-method/
  Sorting MongoDB documents

- https://www.youtube.com/watch?v=KmzjcRRHD5s
  Importing and Exporting Data with MongoDB - YouTube

COM661 Full Stack Strategies and Development

BE06. MongoDB and Python

Aims

- To present an algorithm for the generation of dummy data for an application
- To introduce the PyMongo package for integration of a MongoDB database with a Python application
- To demonstrate the specification of a database client and selection of a database and collection
- To implement the CRUD operations on MongoDB databases in Python
- To demonstrate the provision of CRUD operations on sub-documents
- To identify opportunities for error trapping within functions.

Table of Contents

6.1 BUILDING A DATASET 2

6.1.1 GENERATING DUMMY DATA 2
6.1.2 PREPARING PYMONGO 3

6.2 BUILDING A DATABASE-DRIVEN API 5

6.2.1 GET ALL BUSINESSES 5
6.2.2 GET ONE BUSINESSES 7
6.2.3 ADD A NEW BUSINESSES 8
6.2.3 UPDATE A BUSINESSES 9
6.2.3 DELETE A BUSINESSES 10

6.3 WORKING WITH SUB-DOCUMENTS 10

6.3.1 ADD A REVIEW 11
6.3.2 GET ALL REVIEWS 12
6.3.3 GET ONE REVIEW 12
6.3.4 UPDATE A REVIEW 14
6.3.5 DELETE A REVIEW 15

6.4 FURTHER INFORMATION 16

6.1 Building a Dataset

So far, we have been working with an API application where the data is hard-coded. Every time the application is re-started, the data is refreshed and all of the changes we have previously made are lost. In this practical, we will see how we can integrate a MongoDB database into the application, creating a persistent data store that is maintained over time.

6.1.1 Generating Dummy Data

In a later practical, we will see how to identify and integrate a real-world data set from an external source, but initially, we will create a set of dummy data related to businesses and reviews to work with.

As a starting point, we will create a new Flask application called **biz** and code a new file ***make_json.py*** to generate the dummy data. This program generates a list of 100 businesses, where each business has a unique *name*, a *town* chosen at random from a set of locations and a random *rating* in the range 1-5. All businesses also have a set of *reviews* that is initialised to an empty list.

File: make_json.py

```
import random, json

def generate_dummy_data():
    towns = ['Coleraine', 'Banbridge', 'Belfast',
             'Lisburn', 'Ballymena', 'Derry', 'Newry',
             'Enniskillen', 'Omagh', 'Ballymoney']
    business_list = []

    for i in range(100):
        name = "Biz " + str(i)
        town = towns[random.randint(0, len(towns)-1)]
        rating = random.randint(1, 5)
        business_list.append(  /
           { "name" : name, "town" : town,
             "rating" : rating, "reviews" : [] }  )
    return business_list

businesses = generate_dummy_data()
fout = open("data.json", "w")
fout.write(json.dumps(businesses))
fout.close()
```

Do it now!	Create a new Flask application ***biz*** containing the Python file ***make_json.py*** as specified above and supplied in the ***BE06 Practical Files***. Run the code by **`python3 make_json.py`** and check that the data generated in the file ***data.json*** represents a valid data set for the application.

Note:	This new application will evolve into the ***Biz Directory*** that is our demonstrator application that we will work with for the remainder of this module.

Now that we have our data set specified as an array of JSON objects in a text file, we can import it into a MongoDB database. Create a new database called ***bizDB*** and import the data into a collection called ***biz*** by issuing the command

```
P:\biz> mongoimport --db bizDB --collection biz --jsonArray
        data.json
```

Do it now!	Import the data file into a collection called ***biz*** in a database called ***bizDB***. Use the Mongo shell to verify that the data has been loaded by generating output such as that illustrated in Figure 6.1, below.

```
test> use bizDB
switched to db bizDB
bizDB> db.biz.find()
[
  {
    _id: ObjectId('66bc7e83964374e2126146c4'),
    name: 'Biz 1',
    town: 'Banbridge',
    rating: 4,
    reviews: []
  },
  {
    _id: ObjectId('66bc7e83964374e2126146c5'),
    name: 'Biz 15',
    town: 'Belfast',
    rating: 5,
    reviews: []
  },
  {
    _id: ObjectId('66bc7e83964374e2126146c6'),
    name: 'Biz 0',
    town: 'Omagh',
    rating: 2,
    reviews: []
  },
  {
```

Figure 6.1 Importing the Dataset into MongoDB

6.1.2 Preparing PyMongo

With our database created, we can now begin to develop the code that manipulates it. This is most commonly done in Python through a package called **PyMongo** that acts as a driver for a MongoDB database. PyMongo is not native to Python and needs to be installed using Pip by the following command (first ensuring that your virtual environment is running):

```
(venv) P:\biz> pip install pymongo
```

Now, with ***PyMongo*** installed and the database in place, we can create the shell of the database-driven API application. As well as the usual Flask **`import`**s, we also **`import`** the **`pymongo.MongoClient`** class that manages the connection to the database as well as the **`bson.ObjectId`** class that manipulates MongoDB **`objectID`** values.

At the top of the main program body, we create a new database client by calling the **`MongoClient`** class with the URL of the ***mongod*** database server (remember that the IP address 127.0.0.1 is the localhost and that the **mongod** servers listens on port 27017). Next, we select the database to work with and finally identify the collection to which we want to address queries.

Note:	If your application needs to address multiple collections (or even multiple databases), you can repeat these commands as often as you need, specifying different variable names, databases and collections as appropriate.

File: app.py

```
from flask import Flask, request, jsonify, make_response
from pymongo import MongoClient
from bson import ObjectId

app = Flask(__name__)

client = MongoClient("mongodb://127.0.0.1:27017")
db = client.bizDB      # select the database
businesses = db.biz    # select the collection

# application functionality will go here

if __name__ == "__main__":
      app.run(debug=True)
```

Do it now!	Create the shell of ***app.py*** (within your new ***biz*** project) as specified above. You can check that everything is installed correctly by running the application by the command **`python3 app.py`**, though it does not yet have any functionality.

6.2 Building a Database-driven API

With the shell of the application in place, we can now begin to provide the functionality of the API. The best approach to adopt is to copy and paste each route decorator and function from the final version of the ***first*** API (i.e. the state of the application at the end of Practical BE04) – one at a time as we meet them in this practical. The main differences between the BE04 version and the database-driven implementation will be highlighted in bold text in the code that follows.

6.2.1 Get All Businesses

The first API endpoint to develop is that which returns the complete list of businesses, including the pagination introduced in Practical BE04. This code introduces the PyMongo **`find()`** method that returns selected documents from the collection. This method has the form

```
businesses.find().skip(page_start).limit(page_size)
```

where the **`skip()`** and **`limit()`** methods specify the number of documents to be ignored (from the start of the collection) and the maximum number of documents to be returned, respectively. If all documents are required, then the **`skip()`** and **`limit()`** methods can be omitted.

In this example, the **`find()`** method returns all documents (the **`skip`** and **`limit`** specifications notwithstanding), but this can be tailored to search for specific documents and return only specific fields as seen in Practical BE05. For example, to limit the search to businesses from Belfast the command would be

```
businesses.find({town:"Belfast"})
```

while if only certain fields are required, a projection could be added as a second parameter such as

```
businesses.find( {town:"Belfast"}, {name:1} )
```

Note:	In PyMongo **`find()`** projections, **1** and **0** are used to denote **true** and **false** when specifying if a field is to be included or omitted. **1** and **0** are also valid in the MongoDB shell, but are mandatory in PyMongo.

It is important to note that the **`find()`** method returns a cursor (pointer) rather than a set of data values. In the code below, you can see how we iterate across the results returned

by **`find()`** to retrieve each business object and append it to the list of **`data_to_return`**. Note also how we need to convert the **`_id`** value to a string to be able to return it within a JSON structure. We also need to add code to repeat this conversion for the **`_id`** value of every review of the business.

File: app.py

```
...

@app.route("/api/v1.0/businesses", methods=["GET"])
def show_all_businesses():
    page_num, page_size = 1, 10
    if request.args.get('pn'):
        page_num = int(request.args.get('pn'))
    if request.args.get('ps'):
        page_size = int(request.args.get('ps'))
    page_start = (page_size * (page_num - 1))

    data_to_return = []
    for business in businesses.find() \
                    .skip(page_start).limit(page_size):
        business['_id'] = str(business['_id'])
        for review in business['reviews']:
            review['_id'] = str(review['_id'])
        data_to_return.append(business)

    return make_response( jsonify(data_to_return), 200 )

...
```

Do it now!	Create the **`show_all_businesses()`** function and test it in the browser or Postman. Try adding values for the **`pn`** and **`ps`** querystring parameters and verify that it works as expected.

Do it now!	Comment out the line that converts the **`_id`** value to a string and re-run the application. Verify that when making a GET request you now obtain an "**Object of type ObjectId is not JSON serializable**" error. Restore the commented line of code to remove the error.

6.2.2 Get One Businesses

To retrieve a single document from a collection, we use the ***PyMongo*** **`find_one()`** method. This takes the same parameters as the **`find()`** method discussed above, but returns only the first document to match the search criteria, or the special Python object **`None`** if no match is found.

Note how we need to use the **`ObjectId()`** method to convert the string parameter to the format required by MongoDB. Also, once again, we need to convert the document's **`_id`** value to a string and repeat the conversion for the **`_id`** field of any reviews.

File: app.py

```
...

@app.route("/api/v1.0/businesses/<string:id>", \
           methods=["GET"])
def show_one_business(id):
    business = businesses.find_one({'_id':ObjectId(id)})
    if business is not None:
        business['_id'] = str(business['_id'])
        for review in business['reviews']:
            review['_id'] = str(review['_id'])
        return make_response( jsonify( business ), 200 )
    else:
        return make_response( jsonify( \
               {"error" : "Invalid business ID"} ), 404 )
...
```

Do it now!	Add the **`show_one_business()`** functionality to the API and test it in the browser or Postman.

Try it now!	Providing a random string for the business ID value in the URL will generate an error that is not caught by our error trapping code, as the call to **`ObjectId()`** expects a valid value to be passed as a parameter. Fix this by adding additional code to ensure that the value passed as the business ID is a 24-character hexadecimal string and return an appropriate response if this is not the case.

6.2.3 Add a new Businesses

Adding a document to a collection is achieved by the **`insert_one()`** method, which takes the document to be added as a JSON formatted parameter. Here, we create the JSON object as a Python dictionary by retrieving the values from the **`request.form`** object as before.

The most interesting thing of note here is the use of the value returned from the ***PyMongo*** **`insert_one()`** method. All PyMongo methods return a value which is an instance of an object related to the method. In the case of **`insert_one()`**, it returns an instance of **`InsertOneResult`** which contains a property **`inserted_id`** which is the **`_id`** key value of the newly-created object.

The guidelines of RESTful API design say that the reply to a POST request should be either a copy of the new resource or a link to it, so here, we use the **`inserted_id`** value to return a link to the new business entry.

File: app.py

```
...

@app.route("/api/v1.0/businesses", methods=["POST"])
def add_business():
    if "name" in request.form and \
       "town" in request.form and
       "rating" in request.form:
        new_business = {
            "name" : request.form["name"],
            "town" : request.form["town"],
            "rating" : request.form["rating"],
            "reviews" : []
        }
        new_business_id = businesses.insert_one(new_business)
        new_business_link = \
           "http://localhost:5000/api/v1.0/businesses/" \
           + str(new_business_id.inserted_id)
        return make_response( jsonify(
                  {"url": new_business_link} ), 201)
    else:
        return make_response( jsonify(
                  {"error":"Missing form data"} ), 404)

...
```

Do it now!	Add the **`add_business()`** functionality to the API and test it in Postman.

6.2.3 Update a Businesses

Document updates are performed by the **`update_one()`** method which takes two parameters – a search object that specifies the object to be updated and a **`$set`** object that describes the fields to be amended. Any fields not specified in the **`$set`** object will retain their current values, hence in this case, the *reviews* for the business are left unchanged.

Again, in this example, we make use of the value returned by the ***PyMongo*** method. This time, the return value is an instance of **`UpdateResult`** which includes a value for **`matched_count`** that is the number of documents in the collection that match the search criteria. Here, we use it to determine whether the business ID value was found and if not, a "bad business ID" response is returned.

File: app.py

```
...

@app.route("/api/v1.0/businesses/<string:id>", methods=["PUT"])
def edit_business(id):
    if "name" in request.form and "town" in request.form and \
       "rating" in request.form:
        result = businesses.update_one( \
          { "_id" : ObjectId(id) }, {
            "$set" : { "name" : request.form["name"],
                       "town" : request.form["town"],
                       "rating" : request.form["rating"]
                     }
          } )
        if result.matched_count == 1:
           edited_business_link = \
           "http://localhost:5000/api/v1.0/businesses/" + id
            return make_response( jsonify(
               { "url":edited_business_link } ), 200)
        else:
            return make_response( jsonify(
               { "error":"Invalid business ID" } ), 404)
    else:
        return make_response( jsonify(
           { "error" : "Missing form data" } ), 404)

...
```

Do it now!	Add the **`edit_business()`** functionality to the API and test it in Postman.

6.2.3 Delete a Businesses

To delete a document from a collection, we use the ***PyMongo*** **`delete_one()`** method, which takes a single search object parameter specifying the document to be deleted. Again, we make use of the value returned from the method and check its **`deleted_count`** property to determine if the search object actually matched a document in the collection. Note that the return code 204 is used to denote that the operation has been successfully completed, but no data is returned.

File: app.py

```
...

@app.route("/api/v1.0/businesses/<string:id>", \
           methods=["DELETE"])
def delete_business(id):
    result = businesses.delete_one( { "_id" : ObjectId(id) } )
    if result.deleted_count == 1:
        return make_response( jsonify( {} ), 204)
    else:
        return make_response( jsonify( \
              { "error" : "Invalid business ID" } ), 404)

...
```

Do it now!	Add the **`edit_business()`** functionality to the API and test it in Postman.

6.3 Working with Sub-documents

The code in the previous section presents implementations for all of the CRUD operations on databases. However, the presence of a collection of review objects for each business gives an extra level of complexity that we must deal with. In this section, we see how to provide add, edit, delete and retrieve functionality for sub-documents in a MongoDB database.

6.3.1 Add a Review

As our database initially contains no reviews, the first action we will consider is the addition of a new review. This is achieved by the ***PyMongo*** **`update_one()`** method, which takes the **`_id`** of the business to which the review belongs as the first parameter and specifies a **`$push`** action on the *reviews* property in the second parameter to add the new review to the collection. As for adding a new business, we return a link to the new review object by constructing a URL with the **`_id`** of the business and the newly-generated **`_id`** of the new review.

File: app.py

```
...

@app.route("/api/v1.0/businesses/<string:id>/reviews", \
           methods=["POST"])
def add_new_review(id):
    new_review = {
        "_id" : ObjectId(),
        "username" : request.form["username"],
        "comment" : request.form["comment"],
        "stars" : request.form["stars"]
    }
    businesses.update_one( { "_id" : ObjectId(id) }, \
                { "$push": { "reviews" : new_review } } )
    new_review_link =
           "http://localhost:5000/api/v1.0/businesses/" \
           + id +"/reviews/" + str(new_review['_id'])
    return make_response( jsonify( \
                { "url" : new_review_link } ), 201 )

...
```

Do it now!	Add the **`add_new_reviews()`** functionality to the API and test it in the Postman.

Try it now!	This code does not check that the business **`_id`** is valid or that all of the required values are provided in **`request.form`** before adding the new review. Implement the error trapping code that checks for this and returns an appropriate error message.

6.3.2 Get All Reviews

The implementation of the function to fetch all the reviews of a business is very similar to that which fetches the details of an individual business. First, the **`find_one()`** method retrieves the business, with a projection that requests that only the reviews element is requested, then a **`for`** loop iterates across the reviews, adding each to the list to be returned after converting its **`_id`** value to a string so that it can be expressed in JSON.

File: app.py

```
...

@app.route("/api/v1.0/businesses/<string:id>/reviews", \
           methods=["GET"])
def fetch_all_reviews(id):
    data_to_return = []
    business = businesses.find_one( \
          { "_id" : ObjectId(id) }, \
          { "reviews" : 1, "_id" : 0 } )
    for review in business["reviews"]:
        review["_id"] = str(review["_id"])
        data_to_return.append(review)
    return make_response( jsonify( \
           data_to_return ), 200 )

...
```

Do it now!	Add the **`fetch_all_reviews()`** functionality to the application and test it in Postman.

Try it now!	This code does not check for a valid business **`_id`** before attempting to return the reviews. Implement the error trapping code that checks for this and returns an appropriate error message.

6.3.3 Get One Review

Fetching a single review is a good example of the power offered by providing each review with its own **`_id`** attribute. As all **`_id`** values are guaranteed to be unique, we can address the review specifically without first fetching the business that the review relates to.

The format of the query is

```
businesses.find_one(
            { "reviews._id" : ObjectId(rid) },
            { "_id" : 0, "reviews.$" : 1 }
)
```

This can be read as "*find the business that contains a review with an _id value matching the review ID passed as a parameter and only return that review*". Note the use of the positional operator **$** which identifies the review that matches the search expression.

When returning the review, we need to remember that the **reviews** element is a list, so even though only a single review is requested, it is returned as a list comprising a single review. Hence, we return it by specifying element **[0]** of the reviews object that is fetched.

File: app.py

```
...

@app.route("/api/v1.0/businesses/<bid>/reviews/<rid>",
           methods=["GET"])
def fetch_one_review(bid, rid):
    business = businesses.find_one(
                { "reviews._id" : ObjectId(rid) },
                { "_id" : 0, "reviews.$" : 1 } )
    if business is None:
        return make_response(
          jsonify(
          {"error":"Invalid business ID or review ID"}),404)
    business['reviews'][0]['_id'] =
                    str(business['reviews'][0]['_id'])

    return make_response( jsonify( \
                        business['reviews'][0]), 200)


...
```

Do it now!	Add the **fetch_one_review()** functionality to the application and test it in Postman.

Try it now!	The error trapping code does not distinguish between invalid business **_id** and invalid review **_id**. Rectify this to report a "bad business ID" message if the business **_id** is invalid or a "bad review ID" message if the business **_id** is good but the review _id is invalid.

6.3.4 Update a Review

Updating a review provides another example of the use of the **$** positional operator. Just as when retrieving a review, the review is located by a *find* expression that searches for a business containing a review with an **_id** matching that provided as a parameter. Then, the $set operator is used to update the review with the fields provided in the **edited_review** parameter. Once again, we construct a link to the edited review by using the **_id** values of the business and review and return it as the result of the operation.

File: app.py

```
...

@app.route("/api/v1.0/businesses/<bid>/reviews/<rid>", \
           methods=["PUT"])
def edit_review(bid, rid):
    edited_review = {
        "reviews.$.username" : request.form["username"],
        "reviews.$.comment" : request.form["comment"],
        "reviews.$.stars" : request.form['stars']
    }
    businesses.update_one( \
          { "reviews._id" : ObjectId(rid) }, \
          { "$set" : edited_review } )
    edit_review_url = \
         "http://localhost:5000/api/v1.0/businesses/" + \
         bid  + "/reviews/" + rid
    return make_response( jsonify( \
                  {"url":edit_review_url} ), 200)

...
```

Do it now!	Add the **edit_review()** functionality to the application and test it in Postman.

Try it now!	The code does not contain any error trapping. Add whatever code is required to report on invalid business **_id** and invalid review **_id** values.

6.3.5 Delete a Review

The final element of functionality in the API is that which allows a user to delete a review from a collection. Note that this action uses the **`update_one()`** method (as for adding a new review) as we are effectively modifying the business to which the review relates. Once the business has been located by the *find* expression, we use the **`$pull`** operation to remove the entry with the specified review ID from the *reviews* element.

File: app.py

```
...

@app.route("/api/v1.0/businesses/<bid>/reviews/<id>", \
           methods=["DELETE"])
def delete_review(bid, rid):
    businesses.update_one( \
        { "_id" : ObjectId(bid) }, \
        { "$pull" : { "reviews" : \
                    { "_id" : ObjectId(rid) } } } )
    return make_response( jsonify( {} ), 204)

...
```

Do it now!	Add the **`delete_review()`** functionality to the application and test it in Postman.

Try it now!	The code does not contain any error trapping. Add whatever code is required to report on invalid business **`_id`** and invalid review **`_id`** values.

6.4 Further Information

- https://pymongo.readthedocs.io/en/stable/
  PyMongo documentation

- https://www.w3schools.com/python/python_mongodb_getstarted.asp
  Python MongoDB tutorial

- https://realpython.com/introduction-to-mongodb-and-python/
  Introduction to MongoDB and Python

- https://www.youtube.com/watch?v=3ZS7LEH_XBg
  Connecting to a MongoDB in Flask - YouTube

- https://stackabuse.com/integrating-mongodb-with-flask-using-flask-pymongo
  Integrating MongoDB with Flask using Flask-PyMongo

- https://www.youtube.com/watch?v=4o7C4JMGLe4
  MongoDB operations using Flask-PyMongo - YouTube

COM661 Full Stack Strategies and Development

BE07. Complex MongoDB Queries

Aims

- To demonstrate how the structure of a MongoDB collection can be modified in code
- To introduce the MongoDB Aggregation Pipeline
- To implement complex database queries through a pipeline of stages
- To investigate options for storage and summary of pipeline output
- To introduce GeoJSON to support location-based queries
- To implement location-based queries using the Aggregation Pipeline

Table of Contents

7.1 ENHANCING THE DATASET 2

7.1.1 ADDING ADDITIONAL FIELDS 2
7.1.2 REMOVING FIELDS 4

7.2 THE AGGREGATION PIPELINE 6

7.2.1 BASIC RETRIEVAL 7
7.2.2 IN-QUERY PROCESSING 9
7.2.3 HANDLING PIPELINE OUTPUT 14

7.3 LOCATION-BASED QUERIES 18

7.3.1 PREPARING THE DATA 19
7.3.2 GEOSPATIAL QUERIES 22

7.4 FURTHER INFORMATION 26

7.1 Enhancing the Dataset

In the previous practical, we introduced the **PyMongo** library which enables us to embed MongoDB queries into Python applications and process the results obtained. Sometimes, however, we prefer (at least part of) the data processing to be performed by the Database engine rather than the Python application logic so that we can minimize the volume of data returned and return the data in the precise form in which it is required by the application. In such circumstances, the **`find()`** and **`find_one()`** methods do not support sufficiently complex queries, but MongoDB provides a much more flexible query structure known as the **Aggregation Pipeline**. In this practical, we will introduce and examine the Aggregation Pipeline and demonstrate how we can use it to specify more complex retrieval queries.

7.1.1 Adding Additional Fields

Initially, we want to provide a more complex dataset on which to operate, so we will revisit the **biz** collection within the **bizDB** database generated last time to introduce and populate additional fields. Currently, each business is described by a **`name`**, a **`town`** and a **`rating`** – each set to a random value, as well as a **`reviews`** object that is initialised to an empty list. We will extend the definition by adding a field to hold the number of employees in the range 1-100 as well as random gross profit figures for the years 2019, 2020 and 2021. A business will therefore be defined as follows, with the newly added fields highlighted.

```
{
   "name" : "Biz 12",
   "town" : "Coleraine",
   "rating" : 3,
   "reviews" : [],
   "num_employees" : 35,
   "profit" : [
      {
         "year" : "2022",
         "gross" : 50000
      },
      {
         "year" : "2023",
         "gross" : -10000,
      },
      {
         "year" : "2024",
         "gross" : 15000
      }
   ]
}
```

We can make this modification to the structure of each business object by retrieving all businesses with the **`find()`** method and then invoking **`update_one()`** on each in turn, using the **`$set`** command to identify the new fields to be added and the random value for each.

Note:	The combination of **`update_one()`** and **`$set`** is exactly the same as already shown when editing the value of a field. If a field already exists, then its current value is changed, otherwise the field is created and initialised to the value provided.

File: add_fields.py

```
from pymongo import MongoClient
import random

client = MongoClient("mongodb://127.0.0.1:27017")
db = client.bizDB
businesses = db.biz

for business in businesses.find():
    businesses.update_one(
        { "_id" : business['_id'] },
        {
          "$set" : {
            "num_employees" : random.randint(1, 100),
            "profit" : [
               { "year": "2022",
                 "gross": random.randint(-500000, 500000)
               },
               { "year": "2023",
                 "gross": random.randint(-500000, 500000)
               },
               { "year": "2024",
                 "gross": random.randint(-500000, 500000)
               },
            ]
          }
        }
    )
```

Do it now!	Add the program ***add_fields.py*** (supplied in the ***BE07 Practical Files***) to the Biz application and run it by the command **`python3 add_fields.py`**. Remember that you will also need to invoke your virtual environment and make sure that the MongoDB server is running. Once the program has run, check that it has had the desired effect by using the MongoDB shell to retrieve a document from the **biz** collection and ensure that the new fields have been added as shown in Figure 7.1, below.

```
bizDB> db.biz.find()
[
  {
    _id: ObjectId('66bc7e83964374e2126146c4'),
    name: 'Biz 1',
    town: 'Banbridge',
    rating: 4,
    reviews: [],
    num_employees: 41,
    profit: [
      { year: '2022', gross: 323427 },
      { year: '2023', gross: 461211 },
      { year: '2024', gross: -274827 }
    ]
  },
  {
    _id: ObjectId('66bc7e83964374e2126146c5'),
    name: 'Biz 15',
    town: 'Belfast',
    rating: 5,
    reviews: [],
    num_employees: 39,
    profit: [
      { year: '2022', gross: -200567 },
      { year: '2023', gross: -358667 },
      { year: '2024', gross: 72646 }
    ]
  },
```

Figure 7.1 New Fields Added

7.1.2 Removing Fields

In the same way as we can modify the structure of a document by adding additional fields, so we can also remove fields from a document. To demonstrate this, let's first add an additional field called **`"dummy"`** initialised to the string value **`"Test"`**.

Note:	Leaving the **`num_employees`** and **`profit`** fields in the code for this example will result in new values being generated for these fields.

File: add_fields.py

```
...

for business in businesses.find():
    businesses.update_one(
        { "_id" : business['_id'] },
        {
          "$set" : {
            "num_employees" : random.randint(1, 100),
            "profit" : [
                …
            ],
            "dummy" : "Test"
          }
        }
    )
```

Do it now!	Change ***add_fields.py*** as shown above to add the additional "dummy" field. Now, run the program again and use the MongoDB shell to repeat the previous retrieval query and verify that the field has been added.

```
bizDB> db.biz.find()
[
  {
    _id: ObjectId('66bc7e83964374e2126146c4'),
    name: 'Biz 1',
    town: 'Banbridge',
    rating: 4,
    reviews: [],
    num_employees: 41,
    profit: [
      { year: '2022', gross: 323427 },
      { year: '2023', gross: 461211 },
      { year: '2024', gross: -274827 }
    ],
    dummy: 'Test'
  },
  {
    _id: ObjectId('66bc7e83964374e2126146c5'),
    name: 'Biz 15',
    town: 'Belfast',
    rating: 5,
    reviews: [],
    num_employees: 39,
    profit: [
      { year: '2022', gross: -200567 },
      { year: '2023', gross: -358667 },
      { year: '2024', gross: 72646 }
    ],
    dummy: 'Test'
  },
```

Figure 7.2 Adding an Extra Field to Demonstrate Removal

Now that the new field has been added to each document in the collection, we can remove it by specifying the **`$unset`** command in the **`update_one()`** method. Note how the name

of the field to be removed is provided as the key field of a JSON object, while the value of the field is left as an empty string.

File: add_fields.py

```
...

for business in businesses.find():
    businesses.update_one(
        { "_id" : business['_id'] },
        { "$unset" : { "dummy" : "" } }
    )
```

Do it now!	Change ***add_fields.py*** as shown above to remove the additional "dummy" field. Now, run the program again and use the MongoDB shell to repeat the previous retrieval query and verify that the field has been removed and that a MongoDB query once again produces output such as that shown in Figure 7.1.

7.2 The Aggregation Pipeline

Now that our dataset has the required complexity, we can introduce the power and flexibility of the Aggregation Pipeline through the development of some sample queries. In general, the Pipeline is a multi-stage operation, by which the query is divided into a series of individual steps, each of which performs a selection, filtering or transformation operation and passes the result on to the next stage. It can be represented graphically by the illustration in Figure 7.3 which presents the transformation of a source collection into some output by a 3-stage process. In reality, the pipeline can consist of as many stages as are required for the specific query.

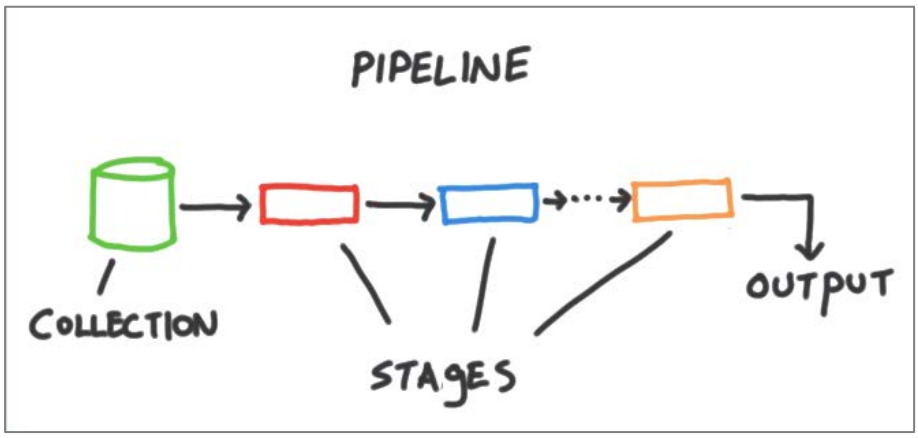


Figure 7.3 The Aggregation Pipeline

7.2.1 Basic Retrieval

An Aggregation Pipeline is constructed by the **`aggregate`** method shown below applied to the **`biz`** collection, which takes a single parameter which is a list of stages.

```
db.biz.aggregate( [

])
```

At the most basic level, the Pipeline can replicate the functionality of the ***PyMongo*** **`find()`** method. For example, to return the collection of businesses where the **`"town"`** field has the value **`"Banbridge"`**, we could apply the **`$match`** operator as shown below.

```
db.biz.aggregate( [
     { $match : { "town" : "Banbridge" } }
])
.pretty()
```

This command will have exactly the same effect as the standard MongoDB command

```
db.biz.find( { town : "Banbridge" } )
```

Note:	We have chained a **`pretty()`** operation to the query purely in order to format the result in a readable way within the MongoDB console. **`pretty()`** is not part of the Aggregation Pipeline.

Just as for the **`find()`** query, we can add a projection that specifies the fields to be returned from the query. For example, if the only fields required are **`"town"`** and **`"profit"`**, we can specify this by adding a **`$project`** stage to the pipeline as shown below. Note how each stage in the Pipeline is added as a new object in the list of stages, with stages separated by a comma.

```
db.biz.aggregate( [
     { $match : { "town" : "Banbridge" } },
     { $project : { "town" : 1, "profit" : 1  } }
])
.pretty()
```

Again, this could be equally be accomplished by the standard **`find()`** query as

```
db.biz.find( { town : "Banbridge" },
             { town : 1, profit : 1 }
           )
```

Do it now!	Test the Aggregation Pipeline by opening a MongoDB console and specifying an **`aggregate()`** query using the **`$match`** and **`$project`** operations. Ensure that you receive a result similar to that shown in Figure 7.4 below.

```
bizDB> db.biz.aggregate( [
... { "$match" : { "town" : "Banbridge" } },
... { "$project" : { "town" : 1, "profit" : 1 } }
... ] ).pretty()
[
  {
    _id: ObjectId('66bc7e83964374e2126146c4'),
    town: 'Banbridge',
    profit: [
      { year: '2022', gross: 323427 },
      { year: '2023', gross: 461211 },
      { year: '2024', gross: -274827 }
    ]
  },
  {
    _id: ObjectId('66bc7e83964374e2126146ca'),
    town: 'Banbridge',
    profit: [
      { year: '2022', gross: 462304 },
      { year: '2023', gross: -112136 },
      { year: '2024', gross: -267675 }
    ]
  },
  {
    _id: ObjectId('66bc7e83964374e2126146ce'),
    town: 'Banbridge',
    profit: [
      { year: '2022', gross: -10723 },
```

Figure 7.4 The $match and $project Stages

As well as issuing aggregation queries from the MongoDB console prompt, we can also embed the queries in our Python programs. Consider the program ***aggregation.py*** which repeats the above query and prints out the town name and gross profit for 2024 for each business returned by the query.

File: aggregation.py

```
from pymongo import MongoClient

client = MongoClient("mongodb://127.0.0.1:27017")
db = client.bizDB
businesses = db.biz

pipeline = [
    { "$match" : { "town" : "Banbridge"} },
    { "$project" : {"town" : 1, "profit" : 1 } }
]

for business in businesses.aggregate(pipeline):
    print( business["town"], \
           str(business["profit"][2]["gross"]) )
```

Here, we specify the **`pipeline`** as a list of objects before passing it as a parameter to the **`aggregate()`** method. This is purely for reasons of code readability – we could equally specify the pipeline directly as the parameter to **`aggregate()`**. Note also how we gain access to the gross profit value for 2024. As the 2024 figure is the 3rd element in the **`profit`** list, we specify element **`[2]`** in the list before obtaining the **`gross`** attribute from that element. Finally, as this data item is an integer, we convert it to string by the **`str()`** function before outputting it using the **`print()`** statement.

Do it now!	Verify the operation of ***aggregation.py*** by adding the file supplied in the ***BE07 Practical Files*** to the ***biz*** folder and running it. Remember that you will need to activate your virtual environment before running the program.

Try it now!	Modify ***aggregation.py*** to output the name and number of employees of all businesses in Belfast.

7.2.2 In-query Processing

So far, all examples of **`aggregate()`** have been equally achievable by the standard **`find()`** command. However, **`find()`** has significant limitations that require a more advanced solution – especially when working with sub-documents.

Often, sub-document processing requires a pair of nested loops where the outer loop iterates across the collection of top-level objects and the inner loop iterates across the

collection of sub-documents in each. We have already seen this organization in the API endpoint to retrieve the collection of reviews for each business in Practical BE03. The Aggregation Pipeline supports an **unwind** operation that generates separate documents by looping over each sub-document in turn. For example, a collection of

```
[
   {
     "person" : "Adrian"
     "offspring" : [
                     { "name" : "Adam" },
                     { "name" : "Caitlin" }
                   ]
   }
]
```

subjected to an **unwind** operation on the **"offspring"** field would result in the collection

```
[
   {
     "person" : "Adrian",
     "offspring" : { "name" : "Adam" }
   },
   {
     "person" : "Adrian",
     "offspring" : { "name" : "Caitlin" }
   },
]
```

with a separate document generated for each of the sub-documents in the original collection and all other fields in the top-level document repeated with their original values. We can demonstrate this in the context of our **biz** collection by unwinding the collection on the **profit** field. Note how the field name **profit** is prefixed by **$** in the **$unwind** command. This is standard in MongoDB to distinguish field names from literal string values.

Do it now!	Test the Aggregation Pipeline by opening a MongoDB console and add an **$unwind** operation as shown. Ensure that you receive a result similar to that illustrated in Figure 7.5 below.

```
bizDB> db.biz.aggregate( [
... { "$match" : { "town" : "Banbridge" } },
... { "$project" : { "town" : 1, "profit" : 1 } },
... { "$unwind" : "$profit" }
... ] ).pretty()
[
  {
    _id: ObjectId('66bc7e83964374e2126146c4'),
    town: 'Banbridge',
    profit: { year: '2022', gross: 323427 }
  },
  {
    _id: ObjectId('66bc7e83964374e2126146c4'),
    town: 'Banbridge',
    profit: { year: '2023', gross: 461211 }
  },
  {
    _id: ObjectId('66bc7e83964374e2126146c4'),
    town: 'Banbridge',
    profit: { year: '2024', gross: -274827 }
  },
  {
    _id: ObjectId('66bc7e83964374e2126146ca'),
    town: 'Banbridge',
    profit: { year: '2022', gross: 462304 }
  },
  {
    _id: ObjectId('66bc7e83964374e2126146ca'),
```

*Figure 7.5 $**unwind** on the **profit** Sub-document Collection*

Having generated individual documents by the **$unwind** operation, we may want to filter them based on some criteria – in the same way in which we filtered the initial collection by only selecting those based in a specific town. We can achieve this by adding another **$match** operation as the next stage of the pipeline, to select only those documents for which the number of employees is at least 50 and where the business broke even or turned a profit.

```
db.biz.aggregate([
   { $match : { "town" : "Banbridge" }},
   { $project : {
      "town" : 1, "profit" : 1, "num_employees" : 1 } },
   { $unwind : "$profit" },
   { $match : {
        "num_employees" : { $gte : 50 },
        "profit.gross" : { $gte : 0 } } }
])
.pretty()
```

As the **$match** operation refers to **num_employees**, so we need to update the **$project** object to include this field. Note the **$gte** "greater than or equal to" operators applied to the "**num_employees**" and "**profit.gross**" fields. Other comparison operators such as **$gt** (greater than), **$lt** (less than), **$lte** (less than or equal to), **$eq** (equal to) and **$ne** (not equal to) are also available and are used in the same way.

Do it now!	Add the new **`$match`** stage to the pipeline as shown above and verify its effect by running the query and confirming that you obtain a result similar to that of Figure 7.6 below.

```
bizDB> db.biz.aggregate( [
...     { "$match" : { "town" : "Banbridge" } },
...     { "$project" : { "town" : 1, "profit" : 1, "num_employees" : 1 } },
...     { "$unwind" : "$profit" },
...     { "$match" : { "num_employees" : { "$gte" : 50 }, "profit.gross" : { "$gte" : 0 } } }
... ] ).pretty()
[
  {
    _id: ObjectId('66bc7e83964374e2126146ca'),
    town: 'Banbridge',
    num_employees: 92,
    profit: { year: '2022', gross: 462304 }
  },
  {
    _id: ObjectId('66bc7e83964374e2126146cf'),
    town: 'Banbridge',
    num_employees: 92,
    profit: { year: '2023', gross: 192745 }
  },
  {
    _id: ObjectId('66bc7e83964374e2126146dc'),
    town: 'Banbridge',
    num_employees: 59,
    profit: { year: '2022', gross: 279636 }
  },
  {
    _id: ObjectId('66bc7e83964374e212614713'),
    town: 'Banbridge',
    num_employees: 79,
    profit: { year: '2022', gross: 366627 }
  },
```

Figure 7.6 Adding a $match to the Unwound Dataset

Sometimes, we prefer that the data returned from a query is sorted in a particular order so that it can be easily displayed in a readable fashion. The next stage of the pipeline is to add a **`$sort`** operation to present the data sorted by descending order of gross profit, so that the most profitable business is displayed first.

```
db.biz.aggregate([
   { $match : { "town" : "Banbridge" }},
   { $project : {
      "town" : 1, "profit" : 1, "num_employees" : 1 } },
   { $unwind : "$profit" },
   { $match : {
        "num_employees" : { $gte : 50 },
        "profit.gross" : { $gte : 0 } } },
   { $sort : { "profit.gross" : -1 } }
])
.pretty()
```

Do it now!	Add the new **`$sort`** stage to the pipeline as shown above and verify its effect by running the query and confirming that you obtain a result similar to that of Figure 7.7 below. Try changing the direction of the sort from descending to ascending by replacing the **`-1`** in the **`$sort`** command with **`1`**.

```
bizDB> db.biz.aggregate( [
...     { "$match" : { "town" : "Banbridge" } },
...     { "$project" : { "town" : 1, "profit" : 1, "num_employees" : 1 } },
...     { "$unwind" : "$profit" },
...     { "$match" : { "num_employees" : { "$gte" : 50 }, "profit.gross" : { "$gte" : 0 } } },
...     { "$sort" : { "profit.gross" : -1 } }
... ] ).pretty()
[
  {
    _id: ObjectId('66bc7e83964374e2126146ca'),
    town: 'Banbridge',
    num_employees: 92,
    profit: { year: '2022', gross: 462304 }
  },
  {
    _id: ObjectId('66bc7e83964374e21261471d'),
    town: 'Banbridge',
    num_employees: 62,
    profit: { year: '2023', gross: 420999 }
  },
  {
    _id: ObjectId('66bc7e83964374e212614713'),
    town: 'Banbridge',
    num_employees: 79,
    profit: { year: '2023', gross: 367312 }
  },
```

*Figure 7.7 Adding a **`$sort`** to the Pipeline*

The final processing options that we will examine (though by no means all that the Aggregation Pipeline has to offer) are operations to limit the number of documents that are returned and to select which slice of the document list to return. These are the equivalent of the **`skip()`** and **`limit()`** methods that we have seen chained to the end of the **`find()`** method in Practical C2 and are specified in a similar way. Both accept a single non-negative integer as a parameter which specifies the number of documents to be ignored (in the case of **`$skip`**) and the number of documents to be returned (in the case of **`$limit`**). The following code fragment illustrates **`$skip`** and **`$limit`** stages added to our pipeline.

```
db.biz.aggregate([
   { $match : { "town" : "Banbridge" }},
   { $project : {
      "town" : 1, "profit" : 1, "num_employees" : 1 } },
   { $unwind : "$profit" },
   { $match : {
        "num_employees" : { $gte : 50 },
        "profit.gross" : { $gte : 0 } } },
   { $sort : { "profit.gross" : -1 } },
   { $skip : 2 },
   { $limit : 3 }
])
.pretty()
```

Do it now!	Add the **`$skip`** and **`$limit`** stages to the pipeline as shown above and verify their effect by running the query and confirming that you obtain a result similar to that of Figure 7.8 below. Choose parameter values for **`$skip`** and **`$limit`** that enable you to clearly demonstrate that the stages are having the desired effect.

```
bizDB> db.biz.aggregate( [
...     { "$match" : { "town" : "Banbridge" } },
...     { "$project" : { "town" : 1, "profit" : 1, "num_employees" : 1 } },
...     { "$unwind" : "$profit" },
...     { "$match" : { "num_employees" : { "$gte" : 50 }, "profit.gross" : { "$gte" : 0 } } },
...     { "$sort" : { "profit.gross" : -1 } },
...     { "$skip" : 2 },
...     { "$limit" : 3 }
... ] ).pretty()
[
  {
    _id: ObjectId('66bc7e83964374e212614713'),
    town: 'Banbridge',
    num_employees: 79,
    profit: { year: '2023', gross: 367312 }
  },
  {
    _id: ObjectId('66bc7e83964374e212614713'),
    town: 'Banbridge',
    num_employees: 79,
    profit: { year: '2022', gross: 366627 }
  },
  {
    _id: ObjectId('66bc7e83964374e2126146dc'),
    town: 'Banbridge',
    num_employees: 59,
    profit: { year: '2022', gross: 279636 }
  }
]
bizDB>
```

*Figure 7.8 Adding **`$skip`** and **`$limit`** Stages*

7.2.3 Handling Pipeline Output

The previous section demonstrated a range of pipeline stages which enable highly flexible retrieval of information from a MongoDB database, and we have seen how the pipeline generates a collection of documents according to the structure specified in the pipeline.

However, there are two other situations in which we may wish to post-process the documents returned – (i) we may want to save the output to a new collection so that we can revisit it for later analysis or share it with another application without having to re-generate it from scratch, and (ii) we may want to generate a summary or other analysis of the results. The Aggregation Pipeline supports both activities, and we will introduce them in this section.

First, we will add an **`$out`** stage to the end of the pipeline to write the results of the query to a new collection called **`"profitable_big_biz"`**. As we would like all matching documents to be recorded and we are not concerned with the order of the documents, we also take the opportunity to remove the **`$sort`**, **`$skip`** and **`$limit`** stages. Note also that we need to add an element to the **`$project`** object to explicitly specify that the **`_id`** field is not returned. The **`_id`** is returned by default from every MongoDB query and if we were to allow **`_id`** values to be returned, the database would generate an error when trying to create the new collection since **`_id`** values must be unique and cannot be duplicated across collections.

```
db.biz.aggregate([
   { $match : { "town" : "Banbridge" }},
   { $project : {
      "town" : 1, "profit" : 1, "num_employees" : 1, "_id" : 0 } },
   { $unwind : "$profit" },
   { $match : {
        "num_employees" : { $gte : 50 },
        "profit.gross" : { $gte : 0 } } },
   { $sort : { "profit.gross" : -1 } },
   { $out : "profitable_big_biz" }
])
.pretty()
```

Do it now!	Modify the pipeline as shown to remove the **`$sort`**, **`$skip`** and **`$limit`** stages and add the **`$out`** stage. Run the query and verify that the new "profitable_big_biz" collection is added to the database. Then, issue a **`find()`** query on the new collection to verify that it contains the result set from the aggregation query, as shown in Figure 7.9.

```
bizDB> db.profitable_big_biz.find()
[
  {
    _id: ObjectId('66bcca48a1bfd37a1aa7fd95'),
    town: 'Banbridge',
    num_employees: 92,
    profit: { year: '2022', gross: 462304 }
  },
  {
    _id: ObjectId('66bcca48a1bfd37a1aa7fd96'),
    town: 'Banbridge',
    num_employees: 62,
    profit: { year: '2023', gross: 420999 }
  },
  {
    _id: ObjectId('66bcca48a1bfd37a1aa7fd97'),
    town: 'Banbridge',
    num_employees: 79,
    profit: { year: '2023', gross: 367312 }
  },
  {
    _id: ObjectId('66bcca48a1bfd37a1aa7fd98'),
    town: 'Banbridge',
    num_employees: 79,
    profit: { year: '2022', gross: 366627 }
  },
```

*Figure 7.9 Creating a New Collection with the **$out** Stage*

The other possibility is that we wish to generate some summary information on the results generated by the pipeline. This is achieved by the **$group** operation which combines the documents in a collection according to some criteria and outputs a single document for each group that contains the fields specified in the **$group** syntax. As a first example, consider the following query that reports the number of documents output by the pipeline along with the total, average, maximum and minimum profit values.

```
db.biz.aggregate([
   { $match : { "town" : "Banbridge" }},
   { $project : {
      "town" : 1, "profit" : 1, "num_employees" : 1, "_id" : 0 } },
   { $unwind : "$profit" },
   { $match : {
        "num_employees" : { $gte : 50 },
        "profit.gross" : { $gte : 0 } } },
   { $group : {
        _id : null,
        count : { $sum : 1 },
        total : { $sum : "$profit.gross" },
        average : { $avg : "$profit.gross" },
        max : { $max : "$profit.gross" },
        min : { $min : "$profit.gross" } } }
])
.pretty()
```

Note that, since the output of the first 4 stages in the pipeline is exactly that which we have stored in the **"profitable_big_biz"** collection, we could equally write this query making use of this as shown here.

```
db.profitable_big_biz.aggregate([
   { $group : {
        _id : null,
        count : { $sum : 1 },
        total : { $sum : "$profit.gross" },
        average : { $avg : "$profit.gross" },
        max : { $max : "$profit.gross" },
        min : { $min : "$profit.gross" } } }
])
.pretty()
```

Do it now!	Use the $group option to generate a document of summary information as described above. Try both methods and ensure that they generate identical results in the form demonstrated by Figure 7.10 below.

```
bizDB> db.profitable_big_biz.aggregate([
...     { "$group" : {
...          "_id" : null,
...          "count" : { "$sum" : 1 },
...          "total" : { "$sum" : "$profit.gross" },
...          "average" : { "$avg" : "$profit.gross" },
...          "max" : { "$max" : "$profit.gross" },
...          "min" : { "$min" : "$profit.gross" } }
...     }
... ]).pretty()
[
  {
    _id: null,
    count: 11,
    total: 2746576,
    average: 249688.72727272726,
    max: 462304,
    min: 1807
  }
]
bizDB>
```

Figure 7.10 Generating Summary Information

There are two particularly notable elements to the **`$group`** stage that can be explored further. First, note how the number of documents is calculated by the expression **`{ $sum : 1 }`**. The effect of this is to create a "running total" by adding 1 for each document that is examined. This can be easily seen by changing the value in the **`$sum`** object from 1 to 2, re-running the query, and examining the result obtained.

Secondly, the **`_id`** field in the **`$group`** output is used to control the grouping. Where the **`_id`** is set to **`null`** (as in the example here), then all documents are bundled into a single group and hence a single document is returned as a result. Alternatively, we can specify one of the field names as the value of the **`_id`** element, which will cause the collection to be grouped into unique values for that field and a separate summary document is produced for each group. We can see this in the example below, which specifies the **`"town"`** field as the

value of the **_id** and uses **$sum** to return a collection of documents reporting the number of businesses in each town.

```
db.biz.aggregate([
   { $group : {
        _id : "$town",
        count : { $sum : 1 } } }
])
.pretty()
```

Do it now!	Use the **$group** option to generate a document specifying each town and the number of businesses as shown above. Run the query and verify that you obtain output such as that shown in Figure 7.11 below.

```
bizDB> db.biz.aggregate([
...     { "$group" : {
...         "_id" : "$town",
...         "count" : { "$sum" : 1 } } }
... ]).pretty()
[
  { _id: 'Omagh', count: 9 },
  { _id: 'Banbridge', count: 12 },
  { _id: 'Belfast', count: 12 },
  { _id: 'Derry', count: 11 },
  { _id: 'Enniskillen', count: 11 },
  { _id: 'Ballymena', count: 9 },
  { _id: 'Lisburn', count: 7 },
  { _id: 'Coleraine', count: 9 },
  { _id: 'Ballymoney', count: 13 },
  { _id: 'Newry', count: 8 }
]
bizDB>
```

Figure 7.11 Summary Report by Town

Try it now!	Produce the application ***profitable_by_town.py*** that first produces the number of businesses in each town as shown above and then displays the most profitable business in each town for each of the years for which data is available.

7.3 Location-based Queries

A useful feature of MongoDB is the support for queries on GeoJSON data. GeoJSON is an open standard format designed for representing simple geographical features such as points (addresses or locations), lines (streets, roads, etc.) and polygons (areas, counties, provinces countries, etc.). For example, the GeoJSON coordinate location for the Belfast campus of Ulster University might be expressed as

```
{ "location" :
      {
        "type" : "Point”,
        "coordinates" : [54.60388, -5.92937]
      }
}
```

where the **`coordinates`** list specifies the longitude and latitude values for the location as seen in Figure 7.12 below.

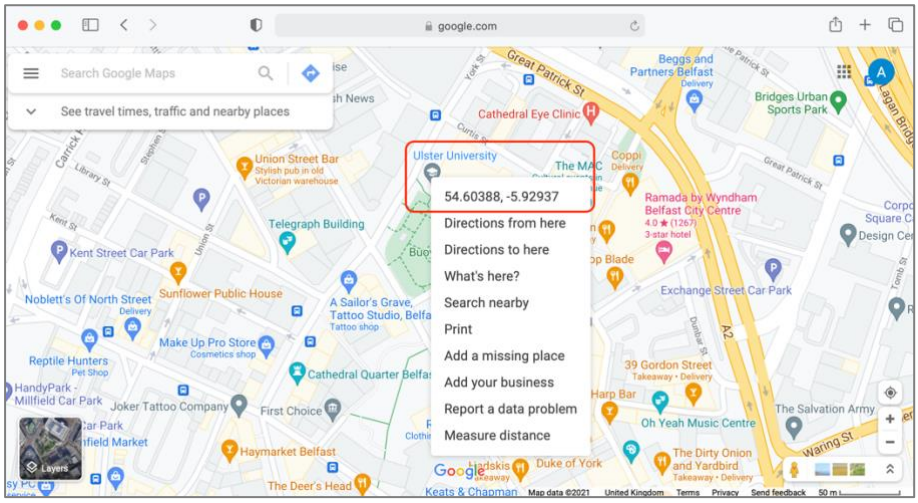


Figure 7.12 Longitude and Latitude

Where GeoJSON data is available, MongoDB provides a range of geospatial queries that allow us to query elements based on their physical location. As an example of this, we will provide a realistic location for each of our business objects and implement a query that retrieves the closest neighbours for any given business.

7.3.1 Preparing the Data

First, we need to generate coordinate data for each business. As our businesses are randomly assigned a location from one of a collection of 10 towns, we can specify a bounding box for each town and generate random longitude and latitude values within the relevant box. For example, the box for Coleraine might be specified as shown in Figure 7.13 with the lower left boundary at the point (55.10653864221481, -6.703013870894064) and the upper right boundary at the point (55.16083114339611, -6.640640630380869).

Note:	The order of coordinate values is always longitude followed by latitude. This can be confusing since it corresponds to y-direction (i.e. north-south) followed by x-direction (i.e. east-west) in a traditional coordinate system.

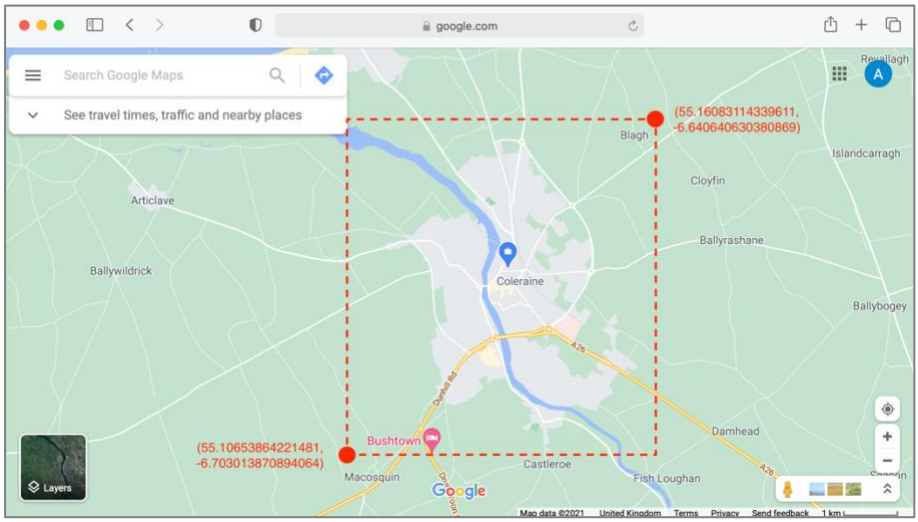


Figure 7.13 Bounding Box for Coleraine

For each of the businesses located in Coleraine, we can then generate a value between the lower and upper boundaries of the longitude and latitude range for the Coleraine area and use a PyMongo **`update_one()`** query to **`$set`** a **`location`** field for the business to the randomly generated value. Repeating this for each of the towns represented in the dataset results in the code presented overleaf.

In ***add_location.py***, the bounding box for each town is represented as a list of 4 values (x1, y1, x2, y2) corresponding to the lower longitude value, lower latitude value, upper longitude value and upper latitude value. The random location for each business is calculated as the lower coordinate value (i.e. x1 or y2) plus a random percentage of the distance between the x and y values. The result of running ***add_location.py*** should be to add a **`location`** field to each business, in GeoJSON format, where the coordinate values are within the bounding box corresponding to that business's **`town`** field.

Do it now!	Run ***add_location.py*** (supplied in the ***BE07 Practical Files***) to add a **`location`** field to each business object. Verify that it has worked by using the MongoDB shell to retrieve the details of a business and make sure that you receive output such as that shown in Figure 7.14 below.

File: add_location.py

```
from pymongo import MongoClient
import random

locations = {
    "Coleraine" : [55.10653864221481, -6.703013870894064,
                   55.16083114339611, -6.640640630380869],
    "Banbridge" : [54.32805966474902, -6.29894073802459,
                   54.36914017698541, -6.238287009221747],
    # ... for every location used in the application
}

client = MongoClient("mongodb://127.0.0.1:27017")
db = client.bizDB
businesses = db.biz

for location in locations:
    for business in businesses.find( { "town" : location } ):
        rand_x = locations[location][0] +
                 ( (locations[location][2] –
                    locations[location][0]) *
                    (random.randint(0,100) / 100) )
        rand_y = locations[location][1] +
                 ( (locations[location][3] –
                    locations[location][1]) *
                   (random.randint(0,100) / 100) )
        businesses.update_one(
            { "_id" : business["_id"] },
            { "$set" :
                {
                    "location" :
                        {
                            "type" : "Point",
                            "coordinates" : [rand_x, rand_y]
                        }
                }
            }
        )
```

```
bizDB> db.biz.find().limit(1)
[
  {
    _id: ObjectId('66bc7e83964374e2126146c4'),
    name: 'Biz 1',
    town: 'Banbridge',
    rating: 4,
    reviews: [],
    num_employees: 41,
    profit: [
      { year: '2022', gross: 323427 },
      { year: '2023', gross: 461211 },
      { year: '2024', gross: -274827 }
    ],
    location: {
      type: 'Point',
      coordinates: [ 54.34490267476594, -6.273466171927396 ]
    }
  }
]
bizDB>
```

Figure 7.14 Location Data Generated

The final stage before we can use the GeoJSON data is to construct an index that allows the data to be used in a geospatial query. MongoDB provides a **`2dsphere`** index that supports the calculation of geometries on an earth-like sphere. The index only needs to be created once and can be built by issuing the following command from the MongoDB console prompt.

```
db.biz.createIndex( { "location" : "2dsphere" } )
```

Do it now!	Create the **`2dsphere`** index on the **biz** collection by issuing the **`createIndex()`** command as shown above. You can verify that the index has been created by issuing the **`getIndexes()`** query before and after creating the index as shown by Figure 7.15 below.

```
bizDB> db.biz.createIndex( { location : "2dsphere" } )
location_2dsphere
bizDB> db.biz.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  {
    v: 2,
    key: { location: '2dsphere' },
    name: 'location_2dsphere',
    '2dsphereIndexVersion': 3
  }
]
bizDB>
```

Figure 7.15 Create the 2dsphere Index

7.3.2 Geospatial Queries

With the GeoJSON location data provided for each business and the 2dsphere index created, we can now query businesses based on their location. We will demonstrate this through an application ***neighbours.py*** that selects a business at random and then retrieves

the 10 businesses that are located closest to the randomly selected one, ordered by increasing distance.

The MongoDB **`$geoNear`** operation is provided as a stage in the Aggregation Pipeline in the form

```
{ $geoNear :
      {
        { geoNear option 1,
          geoNear option 2,
          ...
          geoNear option n
      }
}
```

The demonstration in ***neighbours.py*** uses the following options:

`near`	The point on which to centre the search. Here, we assign it the GeoJSON location of the randomly selected business.
`maxDistance`	An optional parameter specifying the maximum distance in metres for which to return a result.
`minDistance`	An optional parameter specifying the minimum distance in metres for which to return a result. Here, we set it to 1 to avoid returning a business as a neighbour of itself.
`distanceField`	A string value that will be used as the field name for the element that returns the distance in metres from the search location.
`spherical`	Set to **`True`** to specify that the search should be performed using spherical geometry.

File: neighbours.py

```
from pymongo import MongoClient

client = MongoClient("mongodb://127.0.0.1:27017")
db = client.bizDB
businesses = db.biz

for business in businesses.aggregate( [
                { "$sample" : { "size": 1 } } ] ):
    print("Random business is " + business["name"] + \
          " from " + business["town"])

    for neighbour in businesses.aggregate( [
           { "$geoNear" :
               { "near" :
                  { "type" : "Point",
                     "coordinates" :
                        business["location"]["coordinates"] },
                 "maxDistance": 50000,
                 "minDistance" : 1,
                 "distanceField" : "distance",
                 "spherical" : True
               }
           },
           { "$project" : { "name" : 1, "town" : 1,
                            "distance" : 1 } },
           { "$limit" : 10 }
        ]):
        distance_km = str(round(neighbour["distance"] / 1000))
        print(neighbour["name"] + " from " +
              neighbour["town"] + " is at a distance of " +
              distance_km + "km")
```

Note:	The list of **`$geoNear`** options shown here is not exhaustive. Other options are available and are described in the MongoDB manual linked from Section 7.4.

neighbours.py begins by selecting a random business from the **`biz`** collection. This is achieved by using the **`$sample`** stage of the Aggregation Pipeline, providing a **`size`** parameter that dictates the size of the sample to be randomly selected. Once the **`name`** and **`town`** values for the selected business have been output, the application then uses a 2nd **`aggregate()`** query to issue a **`$geoNear`** command to return businesses close to that which was randomly selected. The pipeline also includes a **`$project`** stage to determine

that the fields required are the **`name`**, **`town`** and **`distance`** (the name of which was specified in the "**`distanceField`**" parameter of the **`$geoNear`** stage), and a **`$limit`** stage to retrieve only the 10 closest businesses. Finally, we convert the **`distance`** value returned to kilometres and print out the **`name`**, **`town`** and **`distance`** of each business returned.

Do it now!	Add the program ***neighbours.py*** (in the ***BE07 Practical Files***) to your ***biz*** project and run it. Verify that it returns (up to) 10 businesses which are listed in increasing order of distance from the initial random selection. Make sure that you receive output such as that shown in Figure 7.16 below, where the initial selection of a business located in Newry is followed by the remaining Newry-based businesses before offering the 3 closest businesses in the nearest town, Banbridge.

```
(venv) adrianmoore@Adrians-iMac biz % python3 neighbours.py
Random business is Biz 38 from Newry
Biz 37 from Newry is at a distance of 1 km
Biz 40 from Newry is at a distance of 2 km
Biz 17 from Newry is at a distance of 2 km
Biz 69 from Newry is at a distance of 4 km
Biz 84 from Newry is at a distance of 5 km
Biz 77 from Newry is at a distance of 5 km
Biz 35 from Newry is at a distance of 7 km
Biz 74 from Banbridge is at a distance of 16 km
Biz 7 from Banbridge is at a distance of 17 km
Biz 34 from Banbridge is at a distance of 17 km
(venv) adrianmoore@Adrians-iMac biz %
```

Figure 7.16 Geospacial Query Output

Try it now!	Modify ***neighbours.py*** so that it returns the closest business from a town that is not the same as that selected at random. For example, for the selection of Biz 35 in Newry shown in Figure 7.16 above, the result returned should be Biz 74 from Banbridge. Make sure that the modification works even if there are 10 or more businesses located in the same town.

7.4 Further Information

- https://docs.mongodb.com/manual/core/aggregation-pipeline/
  MongoDB Aggregation Pipeline manual

- https://docs.mongodb.com/manual/reference/operator/aggregation-pipeline/
  Aggregation Pipeline stages

- https://docs.mongodb.com/manual/reference/operator/aggregation/
  Aggregation Pipeline operators

- https://www.analyticsvidhya.com/blog/2021/04/how-to-create-an-aggregation-pipeline-in-mongodb/
  How to create an Aggregation Pipeline in MongoDB

- https://studio3t.com/knowledge-base/articles/mongodb-aggregation-framework/
  The Beginner's Guide to MongoDB Aggregation (with exercise)

- https://www.tutorialspoint.com/mongodb/mongodb_aggregation.htm
  MongoDB Aggregation – TutorialsPoint

- https://geojson.org/
  The GeoJSON Specification

- https://docs.mongodb.com/manual/reference/operator/aggregation/geoNear/
  $geoNear Manual

- https://thecodebarbarian.com/80-20-guide-to-mongodb-geospatial-queries
  The 80/20 Guide to MongoDB Geospatial Queries

COM661 Full Stack Strategies and Development

BE08. Authentication and User Management

Aims

- To introduce Basic Authentication and JSON Web Tokens
- To introduce the **`pyjwt`** package
- To demonstrate the concept of decorators in Python
- To implement a Python decorator that protects functions from unauthorised use
- To demonstrate the presentation of JWT tokens to the request header.
- To present the implementation of a set of user accounts
- To introduce the Python **`bcrypt`** package and **`bytes`** object
- To implement a multi-level access scheme

Table of Contents

8.1 JSON WEB TOKENS 2

8.1.1 Basic Authentication and Preparation for JWT 2
8.1.2 Providing a Login Route 3

8.2 PYTHON DECORATORS 6

8.2.1 Building a First Decorator 6
8.2.2 Using Decorators 9

8.3 PROTECTING SELECTED ROUTES 13

8.3.1 Building the Decorator 13
8.3.2 Applying the Protection 14
8.3.3 Passing the Token in the Request Header 15

8.4 USER MANAGEMENT 17

8.4.1 Creating User Accounts 17
8.4.2 User Login 20
8.4.3 User Logout 22
8.4.4 Multi-level Access 25

8.5 FURTHER INFORMATION 29

8.1 JSON Web Tokens

So far, we have developed an API that allows users to view, add, edit and delete information about a collection of businesses and reviews on those businesses. All information is stored in a back-end MongoDB database, so that changes made by a user in one session are maintained for all users in later sessions. The API is currently fully-functional, but in some respects, it is too functional – anyone is able to perform any operation regardless of their status and even without identifying themselves.

In this practical, we will see how to protect certain operations so that they are only available to logged-in users who have permission to perform those operations. We will use a combination of techniques known as Basic Authentication and JSON Web Tokens (JWT) to implement this.

8.1.1 Basic Authentication and Preparation for JWT

Basic Authentication is where we require the user to provide some specific piece of identification information before allowing them to access a resource. Normally, this is a username and some secret such as a password. JWT is a scheme by which a unique identification string (token) is created which can then be passed by the client along with each request to be made. The token can contain any information that is required by the application (usually a username and other information such as that user's status/permissions/etc.) and is decoded and checked by the server before the requested operation is fulfilled.

Before we can make use of JWT in our Python applications, we need to install the **`pyjwt`** library that provides the set of methods to create and manipulate tokens. The easiest way to do this is by using the **pip** package manager as follows:

```
P:\biz> pip install pyjwt
```

Once **`pyjwt`** is installed, we can prepare our application by importing the additional elements that we will use in this practical (we will explain each as we meet it) and create the secret key that will be used to encrypt our data and generate the JWT token.

Note:	**`mysecret`** is obviously not the most secure of choices for a secret key value but has been chosen here for illustration. Feel free to choose any text string of your choice.

File: app.py

```
from flask import Flask, request, jsonify, make_response
from pymongo import MongoClient
from bson import ObjectId
import jwt
import datetime
from functools import wraps

app = Flask(__name__)

app.config['SECRET_KEY'] = 'mysecret'

...
```

Do it now!	Make the additions specified above to your copy of ***app.py*** in your ***Biz Directory*** project

8.1.2 Providing a Login Route

To generate a JWT token, we need to provide a login route. This enables users to identify themselves to the application before proceeding to make use of its functionality. The **`login()`** method first attempts to retrieve any existing token by the ***Flask*** **`request.authorization`** method. In a browser, this generates a Basic Authorization pop-up which prompts the user to enter values for a username and password.

In this initial example, we will consider the user to be logged in if the value for password is equal to "password", so if this is the case, we create a new token using the **`jwt.encode()`** method, setting values for '***user***' (the value entered as the username) and '***exp***' (the date on which the token expires, set as 30 minutes in the future). The second parameter to **`jwt_encode()`** is the string used to encode the token, so we pass the secret key value defined earlier. Finally, we return the token in a JSON object. If the username and password provided were invalid, we generate a response that causes the browser to re-prompt the user for the username and password.

The code for the initial version of the **`login()`** function is defined below.

File: app.py

```
...

@app.route('/api/v1.0/login', methods=['GET'])
def login():
    auth = request.authorization
    if auth and auth.password == 'password':
        token = jwt.encode( {
            'user' : auth.username,
            'exp' : datetime.datetime.now(datetime.UTC) +
                    datetime.timedelta(minutes=30) },
            app.config['SECRET_KEY'],
            algorithm="HS256")
        return make_response(jsonify({'token' : token}), 200)
    return make_response('Could not verify', 401, \
           {'WWW-Authenticate' : \
            'Basic realm = "Login required"'})

if __name__ == "__main__":
      app.run(debug=True)
```

Do it now!	Add the login route presented above to the application and test it by visiting http://localhost:5000/api/v1.0/login in a web browser. Check that the pop-up window shown in Figure 8.1 is generated and that when any username and the password "password" are provided, a token is generated and displayed as shown in Figure 8.2.

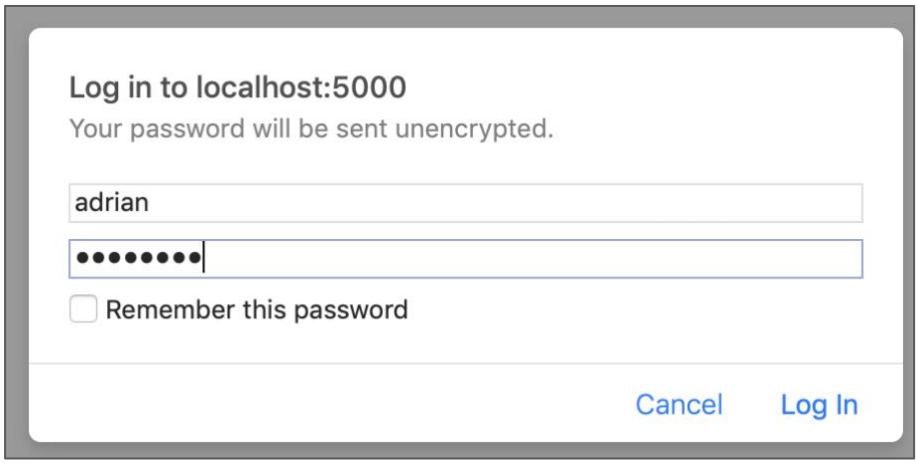


Figure 8.1 Username and Password Prompt



Figure 8.2 Token Generated

In order to better understand the structure of the token, it is useful to use the online tool at http://jwt.io to analyse it. If you visit http://jwt.io and scroll past the initial text, you will see the tool as shown in Figure 8.3. Paste your newly generated token into the pane on the left-hand side of the window and provide your secret key value in the area highlighted in Figure B4.3 below. You should be able to see that the token is verified as valid and that the token **payload** contains an encrypted representation of the '***user***' and '***exp***' values defined in the **`login()`** function.

Note:	The JSON object specified as the payload can contain any values that we deem appropriate or useful for the application. This might include the user's status (i.e. 'admin' or not), their full name, their **`_id`** value to be used in building personalized URLs, or any other information.

Do it now!	Follow the steps illustrated above to generate a JWT token and verify it in the online tool at http://jwt.io.

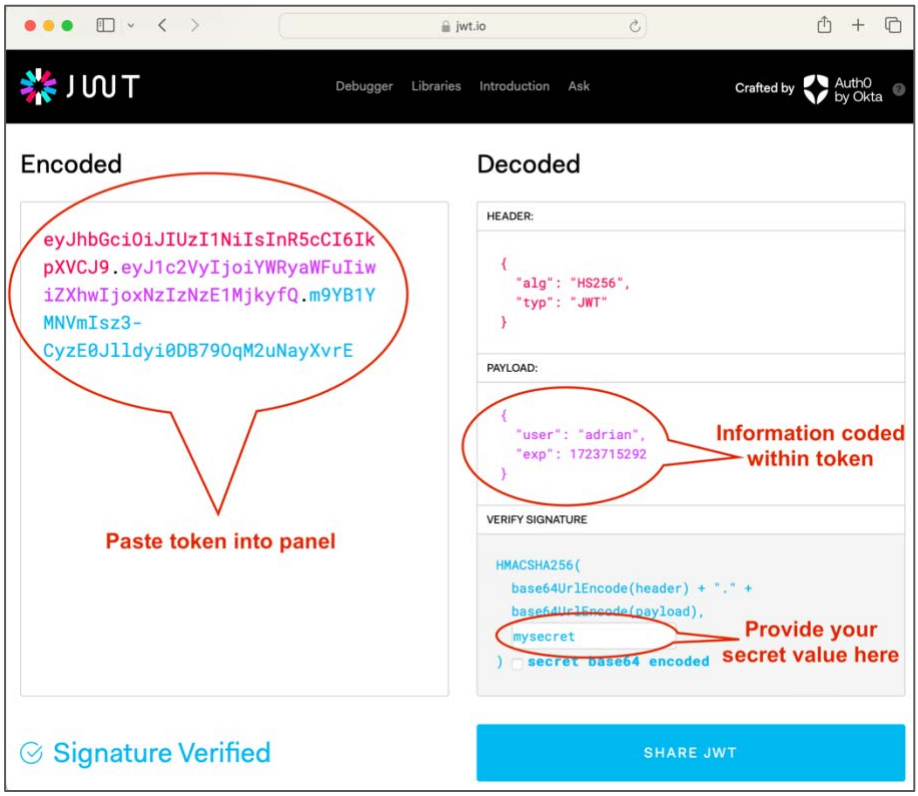


Figure 8.3 Analysing the Token

8.2 Python Decorators

Now that the token is created, verifying that the user is logged in, we can use it to determine whether they should be allowed to access any of the functionality. We could do this by adding code to each of the routes, but we quickly realise that we need to add the same check to any route for which the token is required. A much better approach is to build a Python decorator - a function that can amend the behavior of another function, but without changing the internal code of that function. Decorators are defined once but can be applied to multiple other functions – allowing us to write a single code description that can be used wherever we need it.

8.2.1 Building a First Decorator

To demonstrate, we will use the Python Command prompt to demonstrate applying decoration to a function. First, we enter the Python command prompt and define a simple function.

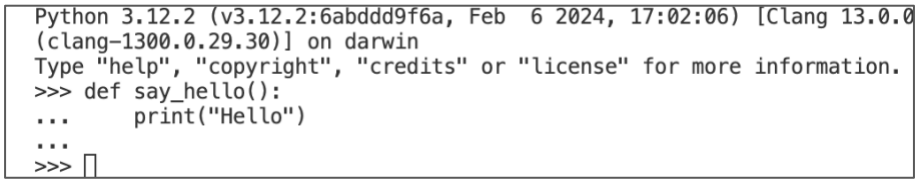


Figure 8.4 Define a Simple Function

Now we can ask the Command prompt to report the value of the function, before running it. Note that stating the name of the function (without parentheses) returns confirmation that it is indeed a function, while specifying the function with parentheses causes the function to be called and executed and its result displayed.

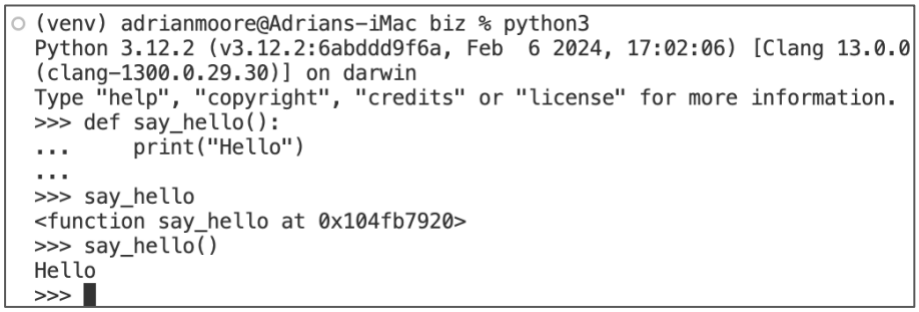


Figure 8.5 Inspect and Run the Function

Now that the function is in place, we create a simple decorator that defines some additional operations that we would like to add to selected functions

A decorator is a function (in this case **`my_decorator()`**) that accepts a function (defined as **`func`**) as a parameter. The decorator function contains an internal function (here called **`wrapper()`**) that describes the environment in which **`func()`** will operate. In this case, we specify that a **`print()`** command generates some output, before the original **`func()`** is run and a second **`print()`** generates more output. Finally, the output of the wrapper is returned to where the decorator was called.

In effect, the function passed to the decorator will be run (the call to **`func()`**) but only after and before **`print()`** statements generate additional output. The code for the decorator can be seen in Figure 8.6 below.

```
(venv) adrianmoore@Adrians-iMac biz % python3
Python 3.12.2 (v3.12.2:6abddd9f6a, Feb  6 2024, 17:02:06) [Clang 13.0.0
(clang-1300.0.29.30)] on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>> def say_hello():
...     print("Hello")
...
>>> say_hello
<function say_hello at 0x104fb7920>
>>> say_hello()
Hello
>>> def my_decorator(func):
...     def wrapper():
...             print("Before the function")
...             func()
...             print("After the function")
...     return wrapper
...
>>>
```

Figure 8.6 Define a Decorator

Finally, for now, we specify the decoration by re-defining **`say_hello`** to be a call to the decorator, with itself as the parameter. Viewing the updated value of **`say_hello`** reveals that it is now effectively a reference to the wrapper function defined within the decorator.

We can now run **`say_hello()`** to see the effect we have created.

```
(venv) adrianmoore@Adrians-iMac biz % python3
Python 3.12.2 (v3.12.2:6abddd9f6a, Feb  6 2024, 17:02:06) [Clang 13.0.0
(clang-1300.0.29.30)] on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>> def say_hello():
...     print("Hello")
...
>>> say_hello
<function say_hello at 0x104fb7920>
>>> say_hello()
Hello
>>> def my_decorator(func):
...     def wrapper():
...             print("Before the function")
...             func()
...             print("After the function")
...     return wrapper
...
>>> say_hello = my_decorator(say_hello)
>>> say_hello
<function my_decorator.<locals>.wrapper at 0x104fb7a60>
>>> say_hello()
Before the function
Hello
After the function
>>>
```

Figure 8.7 Demonstrating the Effect of the Decorator

Do it now!	Follow the steps outlined above to specify a decorator and apply it to the **`say_hello()`** function. Verify that running **`say_hello()`** calls the decorator by observing the messages that are printed before and after the output from the function.

8.2.2 Using Decorators

The example demonstrated is seen to work well, but the re-definition of the **`say_hello()`** function is a slightly "clunky" technique that will become a burden if we have multiple functions to which we want to apply the same decorator. Fortunately, Python provides a syntax that applies the decorator in a much more readable way.

Examine the code for ***decorators_1.py*** (provided in the ***Practical BE8 Files***) below, that repeats the previous experiment, but introduces the @ syntax to apply a decorator to a function. Note also how we use the **`@wraps`** decorator imported from the **`functools`** library. This preserves the state of the function being wrapped by the decorator and allows us to apply the decorator to multiple functions.

File: decorators_1.py

```
from functools import wraps

def my_decorator(func):
    @wraps(func)
    def wrapper():
        print("Before the function")
        func()
        print("After the function")
    return wrapper

@my_decorator
def say_hello():
    print("hello")

say_hello()
```

Do it now!	Add the file ***decorators_1.py*** to your application (copy it into the ***biz*** folder) and run it. Verify that the application still works as described above

There are two other issues around decorators that we need to address before we can apply them to multiple functions.

The first concerns parameters – examine the code for ***decorators_2.py*** (also provided in the ***BE08 Practical Files***) that demonstrates the decorator applied to two functions, **`shout_out()`** and **`whisper_it()`**.

File: decorators_2.py

```
...

@my_decorator
def shout_out():
    print("HELLO")

@my_decorator
def whisper_it():
    print("goodbye")

shout_out()
whisper_it()
```

Note how the decorator is defined exactly as before, but this time it has been applied to both functions. When we run the application, we see that the output of each function is wrapped in the result of the **`print()`** statements, verifying that the decorator has been applied to both.

Do it now!	Run ***decorators_2.py*** and verify that the effect of the decorator can be seen on both functions.

Now, we modify the code so that the strings to be printed in each function are passed as parameters. In addition, we add the **`upper()`** method to the string passed to **`shout_out()`** and add the **`lower()`** method to the string passed to **`whisper_it()`** so that the output from each is in upper and lowercase, respectively.

File: decorators_2.py

```
...

@my_decorator
def shout_out(shout_value):
    print(shout_value.upper())

@my_decorator
def whisper_it(whisper_value):
    print(whisper_value.lower())

shout_out("Hello")
whisper_it("Goodbye")
```

Do it now!	Make the changes shown in the code box above and try to run the application.

This time, running the application generates an error message that the **`wrapper()`** function is not expecting any arguments, but one was provided. We can easily fix this by adding parameters to the definition of **`wrapper()`**, but remember that we want our decorator to be available to multiple functions – regardless of the number of parameters that may be expected.

The solution to this is to specify the generic Python parameter list **`(*args, *kwargs)`** to the **`wrapper()`** function. This is used when we don't know in advance how many arguments will be passed to a function and so is ideal for this situation. We therefore update the **`my_decorator()`** code in ***decorators_2.py*** as shown below.

File: decorators_2.py

```
def my_decorator(func):
    def wrapper(*args, **kwargs):
        @wraps(func)
        print("Before the function")
        func(*args, **kwargs)
        print("After the function")
    return wrapper

...
```

Do it now!	Make the changes above and verify that the application runs as expected.

Try it now!	Verify that the wrapper will accept a variable number of parameters by changing the definition and call for **`whisper_it()`** so that it requires two string values to be provided as parameters. The revised version of the function should output both strings, converted to lowercase and separated by a space.

The final modification to the decorator is to deal with values returned from functions. Examine the modification to the **`shout_out()`** function below such that the lowercase string is not **`print`**ed by the function, but instead is returned from it.

File: decorators_2.py

```
...

def shout_out(shout_value):
    return shout_value.upper()

...

print(shout_out("Hello"))

...
```

Do it now!	Make the changes above and try to run the application.

This time, running the application reveals that the value **None** is printed as the output from the **`shout_out()`** function. To fix this, we need to explicitly return the value from the **`wrapper()`**, so we make the update shown below.

File: decorators_2.py

```
def my_decorator(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        print("Before the function")
        return_value = func(*args, **kwargs)
        print("After the function")
        return return_value
    return wrapper
...
```

Do it now!	Make the changes above and verify that the application runs as expected. Note that the **`whisper_it()`** function that does not expect a return value still works as before

Now that our decorator is sufficiently flexible to cope with variable parameter lists and correctly handles return values, we can return to our API application and see how to use a decorator to protect selected routes against unauthorized access.

8.3 Protecting Selected Routes

Now that we understand the concept of Python decorators and have seen how a single decorator function can be applied to multiple other functions to change their behavior, we can build a decorator that checks for the presence of a valid JWT token before allowing the user to proceed to the requested operation

8.3.1 Building the Decorator

The structure of the decorator **`jwt_required()`** mirrors that of the **`my_decorator()`** definition from the previous section. The decorator contains an embedded function (here called **`jwt_required_wrapper()`**) that checks for the presence of a token as a querystring parameter by examining the **`request.args`** object. If a token is found, it is decoded using the secret key value defined earlier and, only if the **`decode()`** is successful (i.e. the token is valid), does execution proceed to the function that has been decorated. If either the check for a token or the **`decode()`** operation fails, then an appropriate error message is returned and the decorated function is not executed.

File: app.py

```
...

client = MongoClient("mongodb://127.0.0.1:27017")
db = client.bizDB            # select the database
businesses = db.biz          # select the collection name

def jwt_required(func):
    @wraps(func)
    def jwt_required_wrapper(*args, **kwargs):
        token = request.args.get('token')
        if not token:
            return make_response(jsonify(
                  {'message' : 'Token is missing'}), 401)
        try:
            data = jwt.decode(token,
                              app.config['SECRET_KEY'],
                              algorithms="HS256")
        except:
            return make_response(jsonify(
                  {'message' : 'Token is invalid'}), 401)
        return func(*args, **kwargs)

    return jwt_required_wrapper

...
```

8.3.2 Applying the Protection

We can test the effect of the decorator by applying it to routes that we want to protect. As a first test, we apply the decorator to the **`show_one_business()`** function. This should result in the usual output when we attempt to display all businesses, but a "token required" message when we add one of the business ID values to the URL in order to display only that business. We apply the decorator to the function by the **`@decorator_name`** syntax as demonstrated earlier.

File: app.py

```
...

@jwt_required
def show_one_business(id):

...
```

Now, when the application is run, the URL http://localhost:5000/api/v1.0/businesses returns the full list of business objects as usual, but when we take one of the business ID values and add it to the URL, we see the "Token is missing" message as illustrated in Figure 8.8 below.

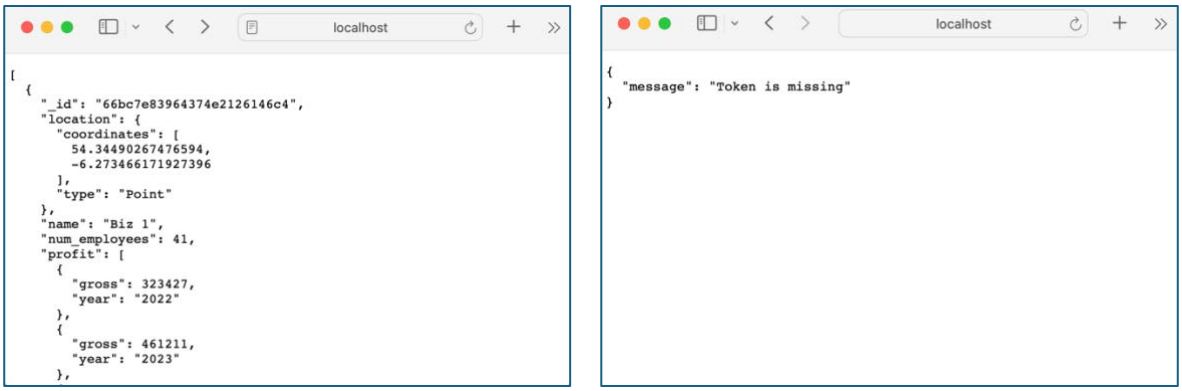


Figure 8.8 Access Allowed and Denied

Do it now!	Make the changes to ***app.py*** described above and make sure that the application behaves as illustrated in Figure 8.8.

To generate the token required to see a single business, we need to visit the ***login*** route and specify a valid password as described in Section 8.1. If we copy the token and add it to the URL as a querystring parameter, the decorator will detect the presence of the token, verify

that it is valid and allow the application to proceed to the **`show_one_business()`** function. This situation is illustrated in Figure 8.9, below.

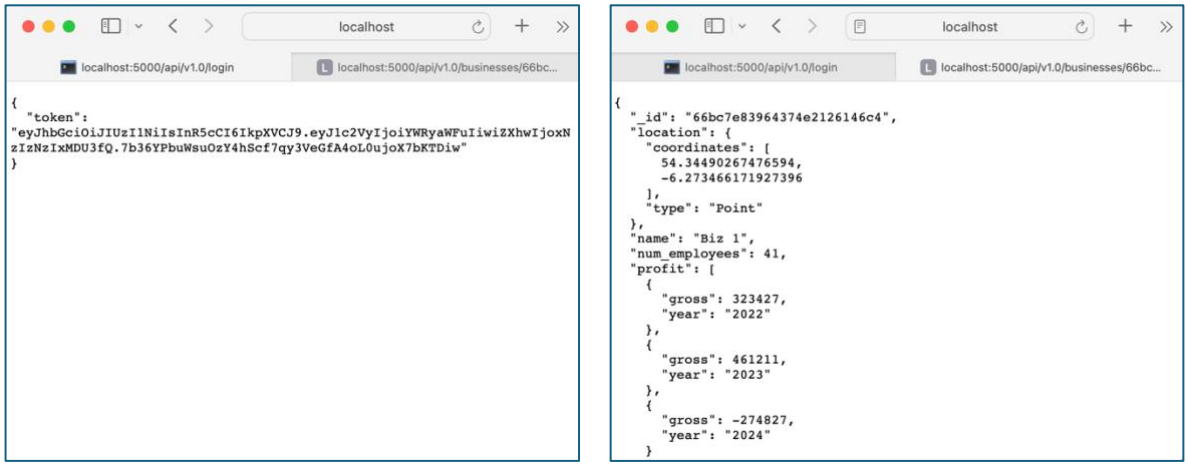


Figure 8.9 Obtaining and Using a Token

Do it now!	Visit http://localhost:5000/api/v1.0/login and present any username and the password value "password". Copy this value and add it as a querystring parameter to the URL to view details of a single business (http://localhost:5000/api/v1.0/businesses/<business_id>?token=<token>). Verify that output such as that shown in Figure 8.9 is achieved.

Try it now!	Modify the application so that all GET requests are available to all users (i.e. anyone can view business or reviews without a token), but all POST, PUT and DELETE requests are only available when a valid token is presented.

8.3.3 Passing the Token in the Request Header

Passing the token in the querystring has been seen to work well, but it is more convenient (and more secure) to pass it as part of the request header. In order to achieve this, we need to make a small modification to the decorator to retrieve the token from the request header rather than the querystring. Comment out (or delete) the existing line and replace it with the code highlighted below.

File: app.py

```
...

def jwt_required(func):
    @wraps(func)
    def jwt_required_wrapper(*args, **kwargs):
        #token = request.args.get('token') – REMOVE THIS LINE
        token = None
        if 'x-access-token' in request.headers:
            token = request.headers['x-access-token']

...
```

We can test this in Postman by making a GET request to the URL http://localhost:5000/api/v1.0/login, selecting the ***Basic Auth*** type from the ***Authorization*** menu and entering the username and password in the text boxes provided. Clicking the "Send" button passes the values to the server code, where the password is checked, and the token returned as shown in Figure 8.10 below.

Note:	Alternatively, you can generate a new token by using a web browser to visit the URL http://localhost:5000/api/v1.0/login as seen earlier.

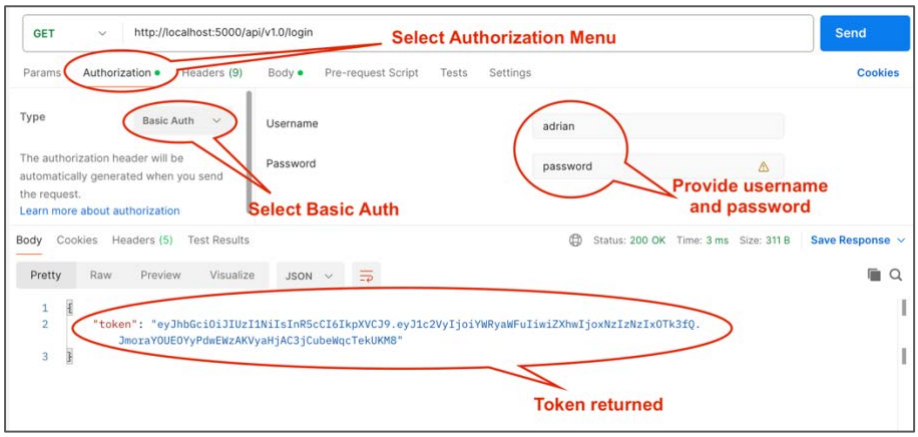


Figure 8.10 Basic Auth in Postman

Now, we will use this token to make a POST request to the reviews collection of a business. First, we make a GET request to http://localhost:5000/api/v1.0/businesses to generate a list of businesses and copy one of the business ID values. Add this value to the URL to set up a POST request to http://localhost:5000/api/v1.0/businesses/<business_id>/reviews and add values for the ***username***, ***text*** and ***stars*** fields of the review in the ***Body*** as usual. Next, click

on the link to the ***Headers*** menu and add a header with name 'x-access-token' and the new token as the value. Clicking 'Send' should result in the new review being accepted, as illustrated in Figure 8.11, below.

Note:	Make sure that you have applied the **`@jwt_required`** decorator to the **`add_new_review()`** endpoint as decribed in the previous ***"Try it now!"*** exercise.

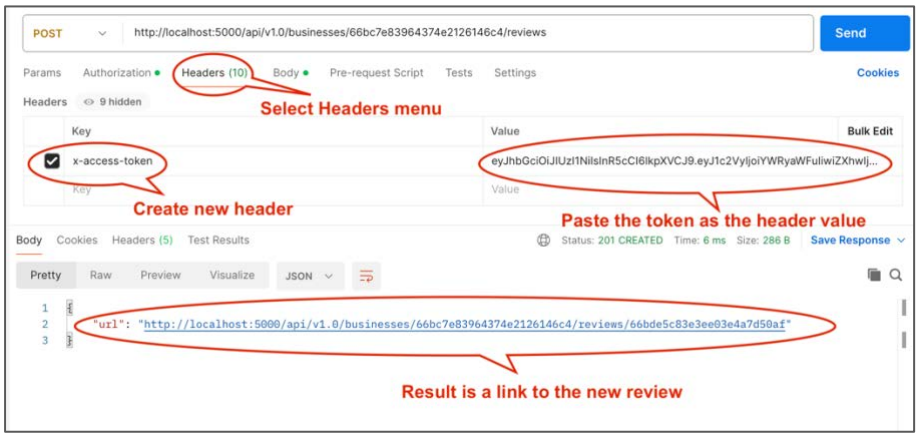


Figure 8.11 Passing the Token with a POST Request

8.4 User Management

In the previous sections, we implemented back-end authentication with JSON Web Tokens and provided a login route that required a user to provide a username and password before access to selected functionality was granted. This implementation, however, used a dummy account which only required a hard-coded password value to be entered and did not distinguish between different users.

In this section, we generate "real" user accounts and see how to manage them. A 3-tiered access scheme is developed with some operations open to all, some only to logged in users and some only available to logged in users with "admin" status.

8.4.1 Creating User Accounts

In this application, we use five fields to describe a staff user as follows:

- The user's real name
- A username to use when logging on to the application
- An encrypted password
- An email address

- The user's administrator status recorded as ***true*** or ***false***.

Examine the code (provided in the ***BE08 Practical Files*** and shown below) for the file ***create_users.py***, that specifies a data structure containing details of five users and generates the collection in the MongoDB database that will be used to store them.

File: create_users.py

```
from pymongo import MongoClient
import bcrypt

client = MongoClient("mongodb://127.0.0.1:27017")
db = client.bizDB
users = db.users

user_list = [
               {
                 "name" : "Homer Simpson",
                 "username" : "homer",
                 "password" : b"homer_s",
                 "email" : "homer@springfield.net",
                 "admin" : False
               },
               ...
             ]

for new_user in user_list:
      new_user["password"] = bcrypt.hashpw(
                        new_user["password"], bcrypt.gensalt())
      users.insert_one(new__user)
```

First, the code connects to the MongoDB server and selects the database and collection with which to work. Note that if a collection that does not currently exist is selected, it will be automatically created once the first attempt is made to insert data to it.

The collection of user information in the list **`data`** provides values for each of the fields but provides the password in plain text. Our specification requires the password to be encoded, so before calling the **`insert_one()`** method, we first use the **`hashpw()`** method of the **`bcrypt`** library to generate a hashed version of the password value. **`hashpw()`** takes two parameters – a ***bytes object*** representing the plaintext password and a ***salt*** which is a random value used in the encryption operation. The salt is created by the **`bcrypt`** library's **`gensalt()`** method. Note that in the specification of each user, the password value is

preceeded by **b** (i.e. **`b"homer_s"`**). This identifies the password as a ***bytes*** object rather than a string.

Before we can run ***create_staff_users.py*** to build the collection of staff accounts, we need to install the **`bcrypt`** package that provides the **`hashpw()`** and **`gensalt()`** methods. Once again, this is most easily done by using the **pip** package manager as follows:

```
(venv) P:\biz> pip install bcrypt
```

Do it now!	Install the **`bcrypt`** package as shown above and then copy the file create_users.py into your ***biz*** application folder. Make sure that your database server is running and then run the application to generate the collection of users. You should be able to use the MongoDB shell to verify that the collection has been created as shown in Figure 8.12, below.

Note when viewing the database contents that the password values have been encrypted and that one of the users has the "***admin***" field set to ***true***. Also note that each document has a unique **`_id`** field even though we did not specify one. MongoDB creates this automatically for each new document.

```
bizDB> db.users.find()
[
  {
    _id: ObjectId('66be045bce1158c7cdec58b7'),
    name: 'Homer Simpson',
    username: 'homer',
    password: Binary.createFromBase64('JDJiJDEyJG01VFFuOS9HMVY4T3dBemtXbnV4VnU0Z0RT
V4SXMvNzhT', 0),
    email: 'homer@springfield.net',
    admin: false
  },
  {
    _id: ObjectId('66be045cce1158c7cdec58b8'),
    name: 'Marge Simpson',
    username: 'marge',
    password: Binary.createFromBase64('JDJiJDEyJFFIb1BXWjFHcmxXTE9HYWlVRWVVb2VBZUZK
pkZUZqN3ZX', 0),
    email: 'marge@springfield.net',
    admin: false
  },
  {
    _id: ObjectId('66be045cce1158c7cdec58b9'),
    name: 'Bart Simpson',
    username: 'bart',
    password: Binary.createFromBase64('JDJiJDEyJHpEWnFRWHJrTmhEWnF4ejY0ZEg1VnVPa2Y5
UubWVEbG1T', 0),
    email: 'bart@springfield.net',
    admin: false
  },
  {
    _id: ObjectId('66be045cce1158c7cdec58ba'),
    name: 'Lisa Simpson',
    username: 'lisa',
    password: Binary.createFromBase64('JDJiJDEyJGc1YWZaa1NmYVlJMnVSQnViVnp1M3U5Ym5I
FYMllkZktl', 0),
    email: 'lisa@springfield.net',
    admin: true
  },
  {
    _id: ObjectId('66be045cce1158c7cdec58bb'),
    name: 'Maggie Simpson',
```

Figure 8.12 User Accounts Created

8.4.2 User Login

The previous version of the application implemented a login function that created a JWT token if a specified value was provided as the password. We now need to update this to check the username and password against the values held in the **`users`** collection.

In this version of the **`login()`** function, we first check for the presence of an authorization header. If one is found, we retrieve the ***username*** field from it and perform a database search for a document in the ***users*** collection with a ***username*** value that matches that in the header. If a user is found, we then use the **`bcrypt`** method **`checkpw()`** to compare the password in the authorization header to that retrieved from the database. If there is a match, then a new JWT token is generated, encoding the username and expiry stamp within the payload. Finally, the token is returned as the response to the login request. If any of the conditions fail, then a response indicates the problem that was encountered.

Note the format of the **`checkpw()`** method, which takes two parameters. The first is the plaintext entry, converted to a ***bytes*** object, while the second parameter is the encrypted version.

File: app.py

```
...

import bcrypt

...

users = db.users

...

def login():
    auth = request.authorization

    if auth:
        user = users.find_one( {'username':auth.username } )
        if user is not None:
            if bcrypt.checkpw(bytes(auth.password, 'UTF-8'),
                              user["password"]):
               token = jwt.encode( {
                 'user' : auth.username,
                 'exp' : datetime.datetime.now(datetime.UTC) +
                         datetime.timedelta(minutes=30) },
                 app.config['SECRET_KEY'],
                 algorithm="HS256")
               return make_response(
                  jsonify({'token' : token}), 200)

            else:
                return make_response(jsonify( {
                              'message':'Bad password'}), 401)
        else:
            return make_response(jsonify( {
                              'message':'Bad username'}), 401)

    return make_response(jsonify( {
                  'message':'Authentication required'}), 401)

...
```

Do it now!	Add code to import the **`bcrypt`** package and select the ***users*** collection and then modify the **`login()`** function as shown above. Using Postman, generate a **GET** request to http://localhost:5000/api/v1.0/login and select ***Basic Auth*** from the list of options under the ***Authorization*** tab. Now provide a username and password from those specified in ***create_users.py*** and click "Send". Check that the request is accepted and that you receive a JWT token as output, as shown in Figure 8.13, below.

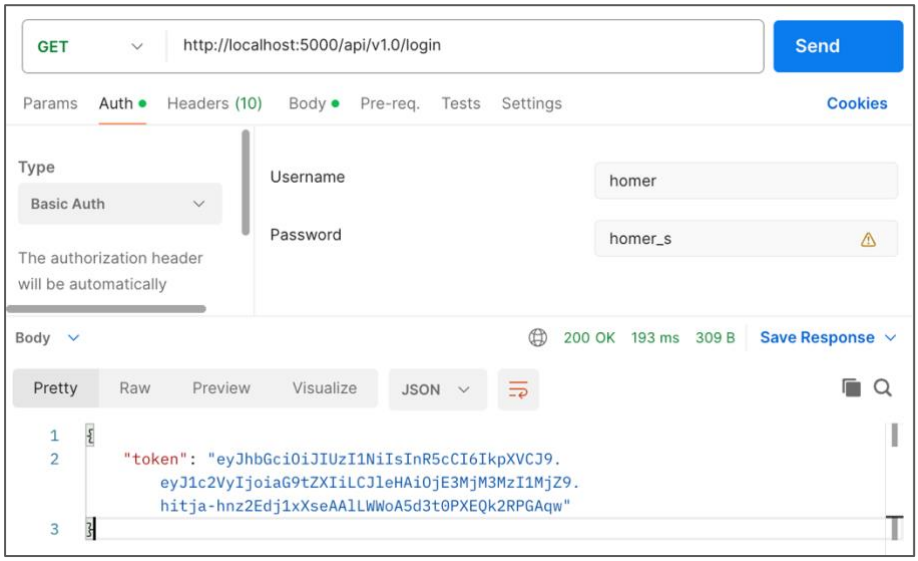


Figure 8.13 User Login

Do it now!	Check the validation in the **`login()`** function by the following tests i) Provide the correct username, but incorrect password ii) Provide a username that does not match any of the documents in the collection iii) Select ***No Auth*** from the list of Authorization types

8.4.3 User Logout

The token generated by the **`login()`** function has an expiry stamp after which time it will automatically become invalid. However, up until that time it will still permit actions to be performed, so it is a good idea to provide a logout mechanism so that a user can indicate that they have completed their session.

By design, JWT tokens cannot be destroyed other than through their own expiry, so we need a mechanism to specify that a token is no longer to be used, even if it is technically still

valid. The most common solution is to develop a **token blacklist**, which is updated with token values as users indicate that they want to log out of the application. When a user sends a GET request to the ***/api/v1.0/logout*** route, the application checks for the presence of a token in the request header and, if one is found, adds that token to the ***blacklist*** collection in the database.

File: app.py

```
...

users = db.users
blacklist = db.blacklist

...

@app.route('/api/v1.0/logout', methods=["GET"])
@jwt_required
def logout():
    token = request.headers['x-access-token']
    blacklist.insert_one({"token":token})
    return make_response(jsonify( {
                    'message' : 'Logout successful' } ), 200 )

if __name__ == "__main__":
      app.run(debug=True)
```

Note again that we don't need to explicitly create the ***blacklist*** collection in the database – it will be automatically created for us the first time we insert data to it.

Do it now!	Add code to select the ***blacklist*** collection and then add the **`logout()`** function as shown above. Using Postman, perform a valid login and copy the token generated as a response. Now create a 'x-access-token' key in the ***Headers*** tab and paste the new token as the value. Send a **GET** request to ***/api/v1.0/logout*** and ensure that you see a "Logout successful" message as shown in Figure 8.14 below.

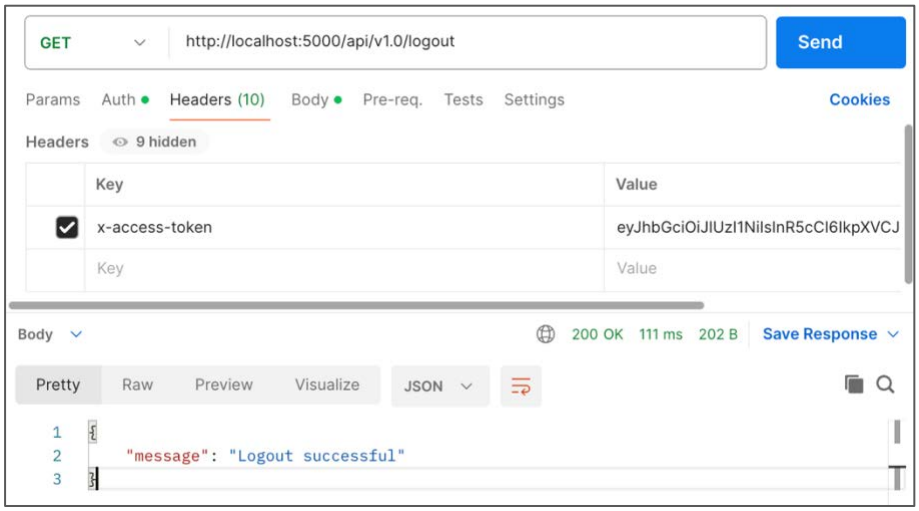


Figure 8.14 User Logout

To check that the token has been written to the blacklist collection, we can use the MongoDB Shell as illustrated in Figure 8.15 below. Here we see that the collection has been created and that it contains a single token value that matches that passed as the 'x-access-token' header.



Figure 8.15 Token Added to Blacklist after Logout

Once a token has been added to the blacklist, we need to make a change to the **`jwt_required()`** decorator to make sure that the token presented is not one that is contained in the blacklist.

This is easily achieved by adding a **`find_one()`** query to check for the token in the blacklist collection just before the **`func()`** passed to the decorator is executed. If the token is found in the blacklist, then an error message is returned, otherwise execution continues as normal.

File: app.py

```
...

def jwt_required(func):
    @wraps(func)
    def jwt_required_wrapper(*args, **kwargs):

        ...

        bl_token = blacklist.find_one({"token":token})
        if bl_token is not None:
            return make_response(jsonify( {
                'message' : 'Token has been cancelled' } ), 401)
        return func(*args, **kwargs)

    ...
```

Do it now!	Add the highlighted code above to the **`jwt_required()`** decorator and try to make a request to a route that requires a valid token (assuming the token you have just used to logout is still in the 'x-access-token' header). Ensure that you receive the "Token has been cancelled" message as illustrated in Figure 8.16, below.

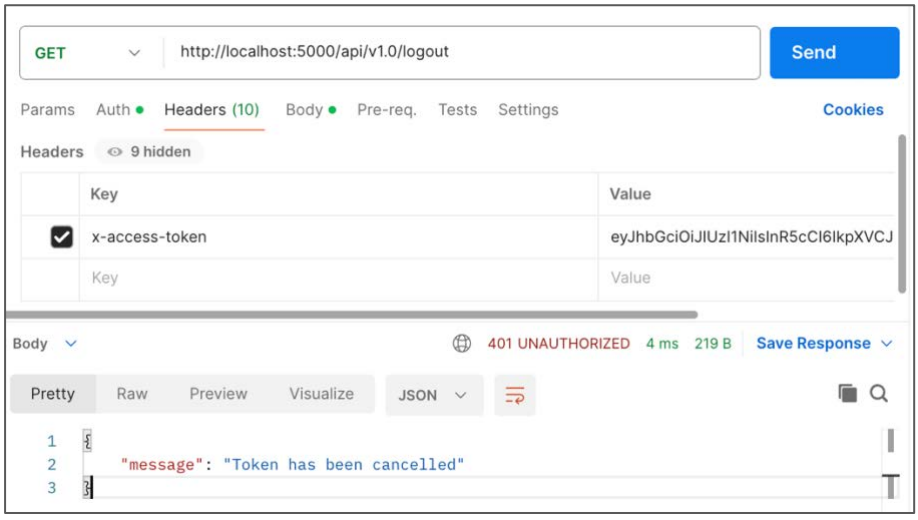


Figure 8.16 Attempting to Use a Blacklisted Token

8.4.4 Multi-level Access

Currently, our application has two levels of access – GET requests on businesses and reviews that are available to anyone, and POST/PUT/DELETE requests that are available to logged in

users with a valid JWT token. In this section, we will extend the access hierarchy to withdraw permission for DELETE requests from general logged-in users and only allow valid token holders with an admin credential to remove businesses and reviews.

In order to implement the new access level, we first modify the **`login()`** function so that the token payload includes the user's admin status flag.

File: app.py

```
...

def login():
    auth = request.authorization

    if auth:
        user = users.find_one( {'username':auth.username } )
        if user is not None:
            if bcrypt.checkpw(bytes(auth.password, 'UTF-8'),
                              user["password"]):
               token = jwt.encode( {
                 'user' : auth.username,
                 'admin' : user['admin'],
                 'exp' : datetime.datetime.now(datetime.UTC) +
                         datetime.timedelta(minutes=30) },
                 app.config['SECRET_KEY'],
                 algorithm="HS256")
               return make_response(
                  jsonify({'token' : token}), 200)
...
```

Next, we require another decorator **`admin_required()`** that checks for the JWT token presented having the ***admin*** flag set to true before allowing access to the operation. This decorator will be applied to delete routes **as well as** that which checks for a valid JWT token, so it need only check the admin flag in the token and does not have to repeat the checks for valid, non-blacklisted tokens.

File: app.py

```
...

def admin_required(func):
    @wraps(func)
    def admin_required_wrapper(*args, **kwargs):
        token = request.headers['x-access-token']
        data = jwt.decode(
           token, app.config['SECRET_KEY'], algorithms="HS256")
        if data["admin"]:
            return func(*args, **kwargs)
        else:
            return make_response(jsonify( {
               'message' : 'Admin access required' } ), 401 )
    return admin_required_wrapper

...
```

Finally, we apply the new decorator to the routes that service the delete requests on businesses and reviews.

File: app.py

```
...

@app.route("/api/v1.0/businesses/<string:id>",
                              methods=["DELETE"])
@jwt_required
@admin_required
def delete_business(id):

...

@app.route("/api/v1.0/businesses/<bid>/reviews/<rid>",
                                      methods=["DELETE"])
@jwt_required
@admin_required
def delete_review(bid, rid):

...
```

Do it now!	Add the additional code to the **`login()`** function and create the new decorator and apply it to the methods to delete businesses and reviews. Test it by logging in with admin and non-admin users and try to delete a business. Verify that the application behaves as shown in Figure 8.17 below.

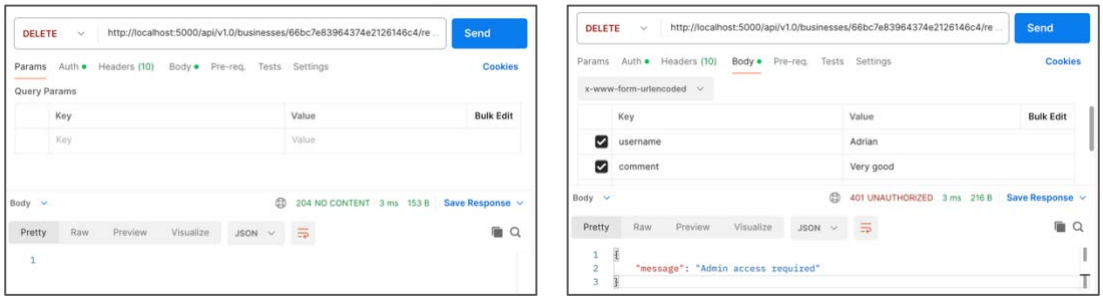


Figure 8.17 Delete Request for Admin and non-Admin Users

Try it now!	Test the new code by trying the following: i) Issue a POST request to add a review to a business ii) Issue a GET request to make sure that the review has been added iii) Repeat the POST request without the token and check the error that is generated iv) Repeat the POST request with an invalid token value and check the error that is generated

8.5 Further Information

- https://realpython.com/token-based-authentication-with-flask/
  Token-based authentication with Flask

- https://jwt.io/
  JSON Web Tokens home page

- https://pypi.org/project/jwt/
  Python jwt package – home page

- https://dev.to/apcelent/json-web-token-tutorial-with-example-in-python-23kb
  JSON Web Token Tutorial with example in Python

- https://www.youtube.com/watch?v=VW8qJxy4XcQ
  HTTP Basic Authentication - YouTube

- https://www.youtube.com/watch?v=J5bIPtEbS0Q
  Authenticating a Flask API with JSON Web Tokens – YouTube

- https://www.youtube.com/watch?v=WxGBoY5iNXY&t=601s
  Creating a RESTFul API in Flask with JSON Web Token Authentication and Flask-SQLAlchemy – YouTube

- https://www.youtube.com/watch?v=mZ5IwFfqvz8
  The basics of Python decorators – YouTube

- https://www.youtube.com/watch?v=nYDKH9fvlBY
  Python decorators tutorial – YouTube

- http://book.pythontips.com/en/latest/args_and_kwargs.html
  Using *args and **kwargs in Python

- https://www.youtube.com/watch?v=SKswJH7_plQ
  How to send JWT tokens as headers with Postman – YouTube

- https://pypi.org/project/bcrypt/
  Bcrypt package home page

- https://docs.python.org/3/library/stdtypes.html#bytes-objects
  Python bytes object - manual

- https://www.w3resource.com/python/python-bytes.php
  Python bytes and bytearrays

- https://stackoverflow.com/questions/21978658/invalidating-json-web-tokens
  Invalidating JSON Web Tokens

- https://dev.to/_arpy/how-to-log-out-when-using-jwt-4ajm
  How to log out using JWT

- https://blog.teclado.com/learn-python-defining-user-access-roles-in-flask/
  Defining user access levels in Flask

COM661 Full Stack Strategies and Development

BE09. Modular Design with Flask Blueprints

Aims

- To introduce Blueprints as a technique for developing modules Flask applications
- To develop a strategy for assigning endpoints to blueprints
- To introduce a simple multi-function application using blueprints
- To demonstrate the refactoring of a single-file application using blueprints
- To present techniques for handling shared resources in multi-file applications

Table of Contents

9.1 FLASK BLUEPRINTS 2

9.1.1 A Basic Application 2
9.1.2 Refactoring the Application 4
9.1.3 Adding a Second Blueprint 7
9.1.4 Resolving Naming Conflicts 9
9.1.5 Inter-Blueprint Communication 11

9.2 CASE STUDY – REFACTORING THE BIZ DIRECTORY 12

9.2.1 Identification and Implementation of Blueprints 12
9.2.2 Handling Shared Resources 16

9.3 FURTHER INFORMATION 24

9.1 Flask Blueprints

To this point, our back-end application has been constructed as a single code file, containing all of the application scaffold such as database initialization and Flask infrastructure, as well as the route management, user authentication, and API endpoint definition. For smaller applications, this will work fine, but as our application size increases, so the large single code file becomes more difficult to keep track of and maintain. Additionally, were we have components that we want to reuse in other applications, such as user authentication, or a shopping cart, then a single file structure makes it awkward to extract these and re-deploy them elsewhere.

To promote application modularity and code re-use, Flask provides a concept known as **blueprints** to support common patterns across application. Blueprints can greatly simplify how large applications are structured and make it much easier to share components across applications. In this section we examine the structure of blueprints by considering how a simple API application could be re-structured into separate modules.

9.1.1 A Basic Application

Let's take a simple API application with three endpoints that demonstrate different ways in which Flask routes can be defined as follows

`GET /`	a root endpoint that will display a simple message
`GET /hello`	an endpoint specified by a static path that prints the message "Hello World"
`GET /hello/<name>`	an endpoint with a variable parameter that prints a personalized message to the person identified by name.

We would normally implement this as a simple application as shown in the following code box.

Do it now!	Create a new project folder called ***blueprint*** and prepare it in the usual way by creating a new virtual environment and installing Flask. Inside the blueprint project, create the file ***app.py*** with code as shown in the code box overleaf. Run the application, and present URLs mapping to the three supported routes in a web browser to ensure that it is working as expected. Verify that you receive output such as that shown in Figures 9.1 to 9.3.

File: app.py

```
from flask import Flask

app = Flask(__name__)

@app.route("/", methods=['GET'])
def index():
    return "Index of Hello World"

@app.route("/hello", methods=['GET'])
def hello():
    return "Hello World!"

@app.route("/hello/<string:name>", methods=['GET'])
def hello_name(name):
    return "Hello " + name

if __name__ == '__main__':
    app.run(debug=True)
```

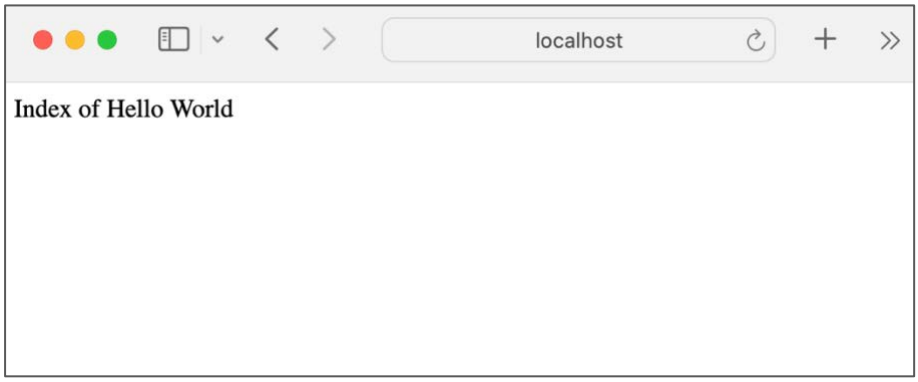


Figure 9.1 Response from http://localhost:5000/

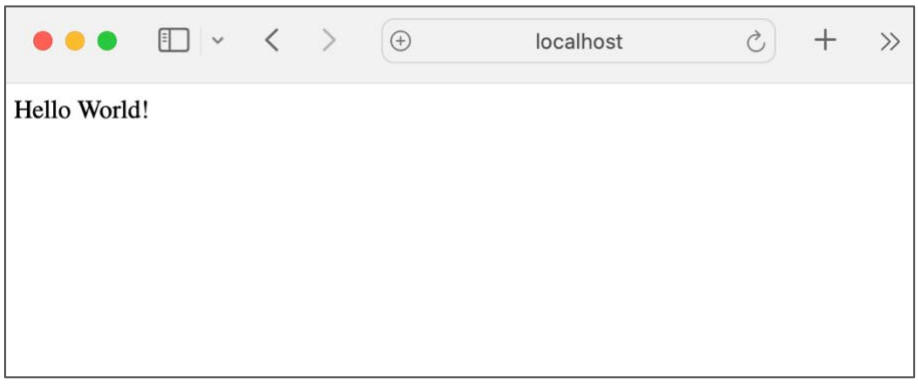


Figure 9.2 Response from http://localhost:5000/hello

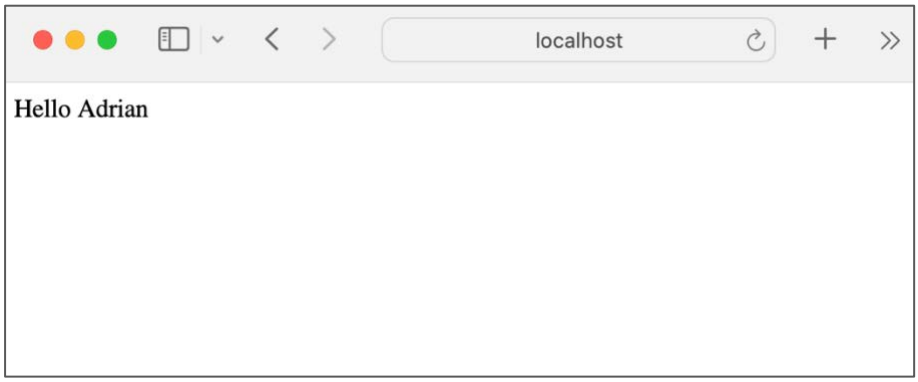


Figure 9.3 Response from http://localhost:5000/hello/Adrian

9.1.2 Refactoring the Application

As a first small example in using blueprints, we will now refactor (re-organise) our code to separate the API endpoint logic from the main application body. A Flask application will often contain many blueprints, so it is common to organize these in a ***blueprints*** sub-folder, giving a structure such as shown in Figure 9.4, below.

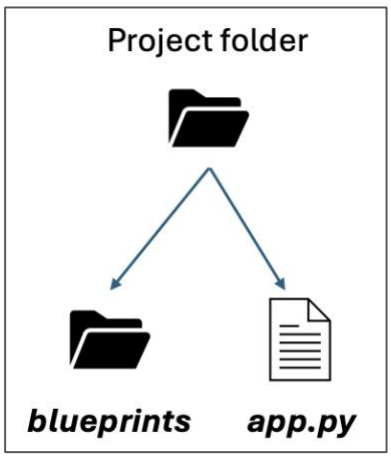


Figure 9.4 Blueprints folder structure

Next, we require a sub-folder inside ***blueprints*** to hold our blueprints. Initially, we only require a single blueprint that we will call ***helloworld***, so we create a ***helloworld*** folder inside ***blueprints*** and create a new (empty) Python file ***helloworld.py*** inside that folder, resulting in the structure shown in Figure 9.5, below.

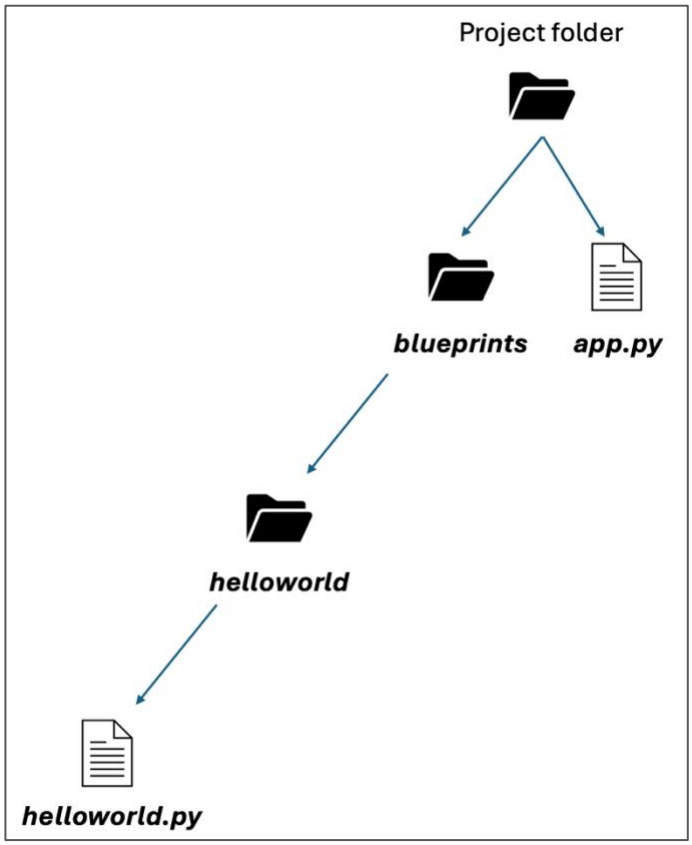


Figure 9.5 Structure with helloworld Blueprint

Do it now!	Prepare for code refactoring by creating the folder and file structure shown in Figure 9.5 within your blueprint application.

Now that the file and folder structure is in place, we can move the code that services the **helloworld** endpoints from ***app.py*** into the new file ***blueprints/helloworld/helloworld.py***. This leaves the code distributed across the two files as follows.

File: app.py

```
from flask import Flask

app = Flask(__name__)

if __name__ == '__main__':
    app.run(debug=True)
```

File: blueprints/helloworld/helloworld.py

```
@app.route("/", methods=['GET'])
def index():
    return "Index of Hello World"

@app.route("/hello", methods=['GET'])
def hello():
    return "Hello World!"

@app.route("/hello/<string:name>", methods=['GET'])
def hello_name(name):
    return "Hello " + name
```

The next step is to turn the functionality within **helloworld.py** into a Blueprint which could then be potentially attached to any other Flask application. We do this by importing the **`Blueprint`** class from the **`flask`** library and creating a new Blueprint object, providing it with the name by which it will be known. Finally, we update each route definition to specify that the endpoints are routes of the blueprint rather than of the top-level application.

File: blueprints/helloworld/helloworld.py

```
from flask import Blueprint

helloworld_bp = Blueprint("helloworld_bp", __name__)

@ helloworld_bp.route("/", methods=['GET'])
def index():
    return "Index of Hello World"

@helloworld_bp.route("/hello", methods=['GET'])
def hello():
    return "Hello World!"

@ helloworld_bp.route("/hello/<string:name>", methods=['GET'])
def hello_name(name):
    return "Hello " + name
```

With the blueprint defined and available for use, we can now import it and register it with our main application by using the **`register_blueprint()`** method of the Flask object that we have called **`app`**.

File: app.py

```
from flask import Flask
from blueprints.helloworld.helloworld import helloworld_bp

app = Flask(__name__)

app.register_blueprint(helloworld_bp)

if __name__ == '__main__':
    app.run(debug=True)
```

Do it now!	Refactor your code as shown in the code boxes for *app.py* and ***helloworld.py*** above and ensure that the application still runs as before, generating output as shown earlier in Figures 9.1 to 9.3.

9.1.3 Adding a Second Blueprint

We will demonstrate the addition of a second blueprint to our application by specifying a simple calculator function with endpoints to add and subtract a pair of integer values provided as parameters within the URL, such that the route **/add/1/2** will return the result of the sum 1+2 and the route **/subtract/3/2** will return the result of the sum 3-2.

This blueprint will be called **`calc_bp`** and will be specified as shown in the code box below.

File: blueprints/calculator/calculator.py

```
from flask import Blueprint

calc_bp = Blueprint("calc_bp", __name__)

@calc_bp.route("/add/<int:x>/<int:y>", methods=['GET'])
def add(x, y):
    return str(x + y)

@calc_bp.route("/subtract/<int:x>/<int:y>", methods=['GET'])
def subtract(x, y):
    return str(x - y)
```

Do it now!	Add a new folder ***calculator*** within the ***blueprints*** sub-folder and create the file ***calculator.py*** with code as shown above.

Now, we can register the calculator blueprint with the application by repeating the import and register process as previously.

File: app.py

```
from flask import Flask
from blueprints.helloworld.helloworld import helloworld_bp
from blueprints.calculator.calculator import calc_bp

app = Flask(__name__)

app.register_blueprint(helloworld_bp)
app.register_blueprint(calc_bp)

if __name__ == '__main__':
    app.run(debug=True)
```

Do it now!	Register the calculator blueprint with the application and test it. Make sure that all of the previous Hello World functionality is still working as expected, and that you also now have addition and subtraction operations available as shown in Figure 9.6 and 9.7, below.



Figure 9.6 Response from http://localhost:5000/add/1/2



Figure 9.7 Response from localhost:5000/subtract/3/2

9.1.4 Resolving Naming Conflicts

One potential issue when combining Blueprints from various sources to make new applications is the conflict that arises when different Blueprints have used the same URL in a route definition. Consider the case where we add a root path to the calculator blueprint as highlighted in the code box below.

File: blueprints/calculator/calculator.py

```
from flask import Blueprint

calc_bp = Blueprint("calc_bp", __name__)

@calc_bp.route("/", methods=['GET'])
def index():
    return "Index of Calculator"

@calc_bp.route("/add/<int:x>/<int:y>", methods=['GET'])
def add(x, y):
    return str(x + y)

@calc_bp.route("/subtract/<int:x>/<int:y>", methods=['GET'])
def subtract(x, y):
    return str(x - y)
```

When we add this route to the calculator and run the application, we now find that the URL ***http://localhost:5000/*** matches both the root path to HelloWorld and the root path to the Calculator. As the HelloWorld blueprint is the first to be registered with the app, it is its route definition that takes priority and that of the Calculator blueprint is unavailable.

Do it now!	Add the additional endpoint to the Calculator blueprint as shown above and verify that when you attempt to navigate to the URL http://localhost:5000/ it is the matching route of the first blueprint to be registered that is serviced.

Flask provides a means of resolving such conflicts by providing for an option **`url_prefix`** parameter in the **`register_blueprint()`** method. This allows us to guarantee the uniqueness of all routes in our application by specifying a path that will be prepended to the route specified for the endpoint.

For example, in ***app.py***, we might provide a **`url_prefix`** for the Calculator blueprint as follows.

File: app.py

```
from flask import Flask
from blueprints.helloworld.helloworld import helloworld_bp
from blueprints.calculator.calculator import calc_bp

app = Flask(__name__)

app.register_blueprint(helloworld_bp)
app.register_blueprint(calc_bp, url_prefix='/calculator')

if __name__ == '__main__':
    app.run(debug=True)
```

By this, the routes defined within the Calculator blueprint as

`/`

`/add/<x>/<y>`

`/subtract/<x>/<y>`

will now be matched instead by the URLs

`/calculator`

`/calculator``/add/<x>/<y>`

`/calculator``/subtract/<x>/<y>`

Do it now!	Add the **`url_prefix`** parameter to the statement that registers the Calculator blueprint as shown above. Run the application and verify that the calculator functional is now available at the ***/calculator/…*** URLs and that the calculator index is now available as shown in Figure 9.8, below.

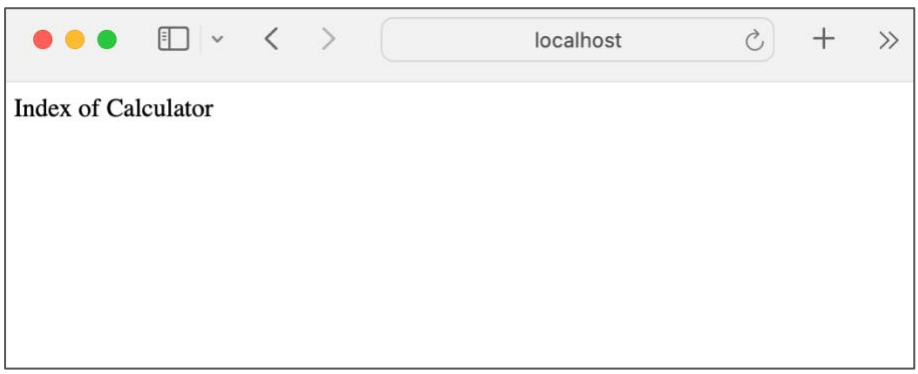


Figure 9.8 Response from http://localhost:5000/calculator

9.1.5 Inter-Blueprint Communication

One advantage of maintaining all endpoint definitions within a single application file is that any part of the application can be reached from any other. For example, if a user in an online cinema booking application makes a request to buy a seat from ***POST /cinema/booking***, the booking logic may determine that they are now eligible to a special offer and redirect them instead to the endpoint ***POST /cinema/bonusoffer***. However, when these endpoints are in different blueprints, they are no longer visible to each other.

Flask resolves this by providing **`redirect`** and **`url_for`** methods that invoke other endpoints in the application and automatically generates the URLs for those endpoints from the information gathered during the **`register_blueprint()`** process.

We can demonstrate this by adding an endpoint to the Calculator blueprint that redirects to an endpoint within the HelloWorld blueprint. In this example, the redirection is automatic, but we could easily use program logic to make the redirection subject to some condition or combination of conditions.

File: blueprints/calculator/calculator.py

```
from flask import Blueprint, redirect, url_for

calc_bp = Blueprint("calc_bp", __name__)

@calc_bp.route("/", methods=['GET'])
def index():
    return "Index of Calculator"

@calc_bp.route("/add/<int:x>/<int:y>", methods=['GET'])
def add(x, y):
    return str(x + y)

@calc_bp.route("/subtract/<int:x>/<int:y>", methods=['GET'])
def subtract(x, y):
    return str(x - y)

@calc_bp.route("/go_to_hello", methods=['GET'])
def go_to_hello():
    return redirect(url_for("helloworld_bp.hello"))
```

Here, we have added an new endpoint ***/calculator/go_to_hello*** which will invoke the **`go_to_hello()`** function. In this, the call to **`url_for()`** can be read as “get me the URL of

the endpoint associated with the function called **`hello()`** within the blueprint called **`helloworld_bp`**". The call to **`redirect()`** then makes an API request to that URL.

Do it now!	Make the changes to ***calculator.py*** as shown in the code box above. Then run the application and make sure that a request to ***/calculator/go_to_hello*** returns the output of a call to ***/hello***. You can see in the Python console how the request to ***/calculator/go_to_hello*** has generated HTTP Status Code 302 indicating redirection, before the call to ***/hello*** is made, as shown in Figure 9.9, below.

```
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 228-508-889
127.0.0.1 - - [20/Aug/2024 12:56:02] "GET /calculator/go_to_hello HTTP/1.1" 302 -
127.0.0.1 - - [20/Aug/2024 12:56:02] "GET /hello HTTP/1.1" 200 -
```

Figure 9.9 Redirection Generates Status Code 302

9.2 Case Study – Refactoring the Biz Directory

Having examined blueprints and their application in modular design, we can return to our Biz Directory example and apply blueprint modularization to the currently monolithic ***app.py***.

Do it now!	In preparation for refactoring the Biz Directory application, create a new application folder ***biz_bp***. Create a new virtual environment within this folder, activate it, and use **pip** to install **`flask`**, **`pymongo`**, **`pyjwt`** and **`bcrypt`**. Now copy your existing ***app.py*** from the Biz Directory into your new folder and check that the application is running as expected.

9.2.1 Identification and Implementation of Blueprints

Normally, we would aim to identify blueprints during the design process, but when refactoring a large application, the best approach is to identify the main actors in the system by considering the URLs used to identify routes. Remember that good RESTful design uses nouns rather than verbs in the URL, so it should be easy to quickly identify the main objects.

The list of endpoints in the Biz Directory application is presented in Table 9.1 below

Method	URL	Action
GET	/api/v1.0/businesses	Get all (or multiple) businesses
POST	/api/v1.0/businesses	Create a new business
GET	/api/v1.0/businesses/<id>	Get a specific business
PUT	/api/ v1.0/businesses/<id>	Update a specific business
DELETE	/api/ v1.0/businesses/<id>	Delete a specific business
GET	/api/ v1.0/businesses/<id>/reviews	Get all (or multiple) reviews for a specific business
POST	/api/ v1.0/businesses/<id>/reviews	Add a review for a specific business
GET	/api/ v1.0/businesses/<id>/reviews/<id>	Get a specific review for a specific business
PUT	/api/ v1.0/businesses/<id>/reviews/<id>	Update a specific review for a specific business
DELETE	/api/ v1.0/businesses/<id>/reviews/<id>	Delete a specific review
GET	/api/v1.0/businesses/login	User login
GET	/api/v1.0/businesses/logout	User logout

Table 9.1 Biz Directory Endpoints

One possible subdivision of the functionality is suggested by the colour coding in the table, which identifies groups as

Business-level endpoints those that deal with the collection of businesses or with individual businesses

Review-level endpoints those that deal with the collection of reviews for a specified business, or with individual reviews

User-level endpoints those that provide a user authentication service

This appears to be a reasonable sub-division of the functionality, so we can proceed to create our folder and file structure.

Do it now!	Inside the application folder, create a new sub-folder called ***blueprints***. Inside ***blueprints***, create additional subfolders called ***businesses***, ***reviews*** and ***auth***. Now create (empty) Python files ***businesses.py***, ***reviews.py***, and ***auth.py*** inside the new bottom-level folders. The folder and file structure should be as illustrated in Figure 9.10.

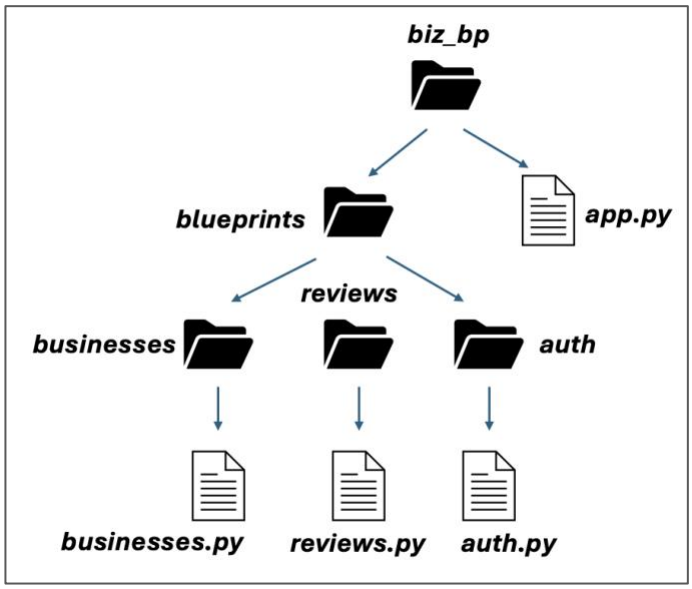


Figure 9.10 ***biz_bp*** *Folder and File Structure*

Now that the folder structure is created, we can begin to migrate functionality from ***app.py*** into the individual blueprint Python files in the way shown in Table 9.1.

Note:	As you move code from file to file, you may receive error indications from your code editor. This is to be expected as we have some modifications to perform to the code after it is moved.

Do it now!	Move the functionality from ***app.py*** to each of ***businesses.py***, ***reviews.py*** and ***auth.py*** in accordance with the allocation shown in Table 9.1. Remember that for each endpoint, you need to move **both** the `@app.route()` decorator and the associated function.

Now that the functionality has been moved, we need to package each of ***businesses.py***, ***reviews.py*** and ***auth.py*** as a Blueprint and register it with the main application. The Blueprint is created by adding a line to the top of each file, and changing the `app.route()` decorator for each endpoint to use the blueprint name as follows.

File: blueprints/businesses/businesses.py.

```
from flask import Blueprint
businesses_bp = Blueprint("businesses_bp", __name__)

@businesses_bp.route("/api/v1.0/businesses", methods=["GET"])
def show_all_businesses():

...

# repeat for each endpoint in businesses.py
```

File: blueprints/reviews/reviews.py.

```
from flask import Blueprint
reviews_bp = Blueprint("reviews_bp", __name__)

@reviews_bp.route(
        "/api/v1.0/businesses/<string business_id>/reviews",
        methods=["POST"])
@jwt_required
def add_new_review(business_id):

...

# repeat for each endpoint in reviews.py
```

File: blueprints/auth/auth.py.

```
from flask import Blueprint
auth_bp = Blueprint("auth_bp", __name__)

@auth_bp.route("/api/v1.0/login", methods=["GET"])
def login():

...

# repeat for each endpoint in auth.py
```

We can now register our new blueprints with the main application by adding the highlighted code below to ***app.py***.

File: app.py

```
...

from blueprints.businesses.businesses import businesses_bp
from blueprints.reviews.reviews import reviews_bp
from blueprints.auth.auth import auth_bp

app = Flask(__name__)
app.config['secret_key'] = 'mysecret'
app.register_blueprint(businesses_bp)
app.register_blueprint(reviews_bp)
app.register_blueprint(auth_bp)

...
```

Do it now!	Specify each of ***businesses.py***, ***reviews.py*** and ***auth.py*** as a blueprint as described in the code boxes above. Then register each of the blueprints with the main application by adding the highlighted code to **app.py**.

9.2.2 Handling Shared Resources

Although all of the functionality has been distributed to the appropriate blueprints, the application is not yet ready to run. Among the functionality not yet considered are the **`@jwt_required`** and **`@admin_required`** decorators and while these are no longer required in ***app.py***, they are needed in each of the blueprint files.

The easiest way to make these available across multiple blueprints is to extract them into a separate python module which can then be imported where they are needed.

Do it now!	Create a new file ***decorators.py*** in the same folder as ***app.py*** and move the definitions of **`jwt_required()`** and **`admin_required()`** into this new file. Note that you will also need to copy the imports (but **NOT** the blueprint imports) and the database connection code into ***decorators.py***.

Following this, the code in ***decorators.py*** should look as shown in the following code box.

File: decorators.py

```
from flask import Flask, request, jsonify, make_response
from bson import ObjectId
from pymongo import MongoClient
import jwt
import datetime
from functools import wraps
import bcrypt

client = MongoClient("mongodb://127.0.0.1:27017")
db = client.bizDB      # select the database
businesses = db.biz    # select the collection name
users = db.users
blacklist = db.blacklist

def jwt_required(func):
    @wraps(func)
    def jwt_required_wrapper(*args, **kwargs):

...

def admin_required(func):
    @wraps(func)
    def admin_required_wrapper(*args, **kwargs):

...
```

We have one final adjustment to make to ***decorators.py*** before we can import it into the blueprints. When encoding and decoding the JWT token, the code makes reference to a configuration variable **`app.config['secret_key']`** that is initialised in ***app.py***. However, the application variable **app** is no longer in scope since it is only accessible from ***app.py***. We could simply re-define **`secret_key`** as a string variable in each file where it is needed, but a better solution is to create another top-level Python file where any global variable can be maintained.

Do it now!	Create a new file ***globals.py*** in the same folder as ***app.py*** and add a definition for a variable **`secret_key`** to hold whatever string value you want to use as your secret. For example, you might have a definition such as that shown below.

File: globals.py

```
secret_key = 'mysecret'
```

Now we can import ***globals.py*** into any other file that requires one of our global values and make use of the value by referring to the module in which it is defined. For example, ***decorators.py*** can be modified as shown below.

File: decorators.py

```
from flask import Flask, request, jsonify, make_response
from bson import ObjectId
from pymongo import MongoClient
import jwt
import datetime
from functools import wraps
import bcrypt
import globals

...

def jwt_required(func):
    @wraps(func)
    def jwt_required_wrapper(*args, **kwargs):

        ...

        try:
            data = jwt.decode(token,
                              globals.secret_key,
                              algorithms="HS256")

...
```

Do it now!	Update ***decorators.py*** to import the new ***globals.py*** file as shown above. Make sure that you update **both** instances of **`app.config['SECRET_KEY']`** to **`globals.secret_key`**. Make similar changes in the auth blueprint file to import the globals and replace the instance of **`app.config['SECRET_KEY']`** found there. You can also now remove the definition of **`app.config['SECRET_KEY']`** from ***app.py*** as it is no longer required.

Now, we can import the **`@jwt_required`** and **`@admin_required`** decorators into the each of the blueprints by adding the following **`import`** statement to the top of each file.

File: decorators/businesses/businesses.py

File: decorators/reviews/reviews.py

File: decorators/auth/auth.py

```
from flask import Blueprint
from decorators import jwt_required, admin_required

...
```

Do it now!	Update each of the blueprint files to import the decorators as shown in the code box above.

The final stage before running the application is to make sure that all of the imports originally made at the top of ***app.py*** are moved to the correct place, even if they need to be duplicated. First, if you examine the code for ***businesses.py***, ***reviews.py*** and ***auth.py***, you will see that each instance of the Flask elements **`request`**, **`make_response()`** and **`jsonify()`** are indicating an error as they are not imported into the files where they are used.

File: blueprints/businesses/businesses.py

File: blueprints/reviews/reviews.py

File: blueprints/auth/auth.py

```
from flask import Blueprint, request, make_response, jsonify

...
```

Now, if we examine the code for ***decorators.py***, we see that the **`Flask`** object is being imported even though it is never used, so we can remove that from the **`import`** statement leaving just the following statement.

File: decorators.py

```
from flask import request, make_response, jsonify

...
```

Finally regarding imports from Flask, we find that the only object needed in ***app.py*** is the **`Flask`** object itself, so the corresponding **`import`** statement there can be modified to the following.

Do it now!	Update the Flask imports across the **`businesses.py`**, **`reviews.py`**, **`auth.py`**, **`decorators.py`** and **`app.py`** files so that each only imports the facilities it requires.

File: app.py

```
from flask import Flask

...
```

The final modification required is to make the database functionality available in each file where it is needed. In order to increase maintainability, it is better to add the database connection statement to the globals, so that a change of database server or database name only requires an amendment in one place. This also requires that the **pymongo** import is added to the file ***globals.py*** as shown below.

File: globals.py

```
from pymongo import MongoClient

secret_key = 'mysecret'

client = MongoClient("mongodb://127.0.0.1:27017")
db = client.bizDB           # select the database
```

Next, we need to create the appropriate collection references in each of the files that holds the functionality. The **businesses** and **reviews** blueprints require access to the ***businesses*** collection, while the **auth** blueprint requires access to ***users*** and ***blacklist***, and the **decorators** definition requires access to **blacklist**. This is accomplished by the following modifications. In each case we need to **`import globals`** to the corresponding file and create a connection to the required collections.

File: blueprints/businesses/businesses.py

```
...

import globals

businesses_bp = Blueprint("businesses_bp", __name__)

businesses = globals.db.biz    # select the collection name
```

File: blueprints/reviews/reviews.py

```
...
import globals

reviews_bp = Blueprint("reviews_bp", __name__)

businesses = globals.db.biz
```

File: blueprints/auth/auth.py

```
...
import globals

auth_bp = Blueprint("auth_bp", __name__)

users = globals.db.users
blacklist = globals.db.blacklilst
```

File: decorators.py

```
...

import globals

blacklist = globals.db.blacklist
```

Do it now!	Ensure that each file has the appropriate database access by performing the updates shown in the code boxes above.

The final step is to tidy up the remaining imports so that all components have exactly the information they require, but no more. In the businesses and reviews blueprints, both

modules need access to the **`ObjectId()`** constructor that was imported from **bson**, so that import needs to be included in each file.

File: blueprints/businesses/businesses.py

File: blueprints/reviews/reviews.py

```
from flask import Blueprint, request, jsonify, make_response
from bson import ObjectId
from decorators import jwt_required, admin_required
import globals

...
```

The auth blueprint requires access to **`bcrypt`**, **`jwt`** and **`datetime`**, so we can copy those statements from ***app.py*** to ***auth.py***.

File: blueprints/auth/auth.py

```
from flask import Blueprint, request, make_response, jsonify
import bcrypt
import jwt
import datetime
from decorators import jwt_required, admin_required
import globals

...
```

Finally, we can remove unneeded imports from ***decorators.py*** and ***app.py***. Initially, we populated ***decorators.py*** by copying all imports from ***app.py***, but not all of these are needed, so we can reduce the import section as shown below.

File: decorators.py

```
from flask import request, jsonify, make_response
import jwt
from functools import wraps
import globals

blacklist = globals.db.blacklist
...
```

That only leaves ***app.py***, where all functionality has been moved into either a blueprint, the decorators file, or the globals. All that remains of ***app.py*** is shown below in its entirety.

File: app.py

```
from flask import Flask
from blueprints.businesses.businesses import businesses_bp
from blueprints.reviews.reviews import reviews_bp
from blueprints.auth.auth import auth_bp

app = Flask(__name__)
app.register_blueprint(businesses_bp)
app.register_blueprint(reviews_bp)
app.register_blueprint(auth_bp)

if __name__ == "__main__":
      app.run(debug=True)
```

Do it now!	Make the final updates to the imports as described in the above code boxes. Run the application and ensure that it is still working exactly as before.

9.3 Further Information

- https://flask.palletsprojects.com/en/3.0.x/blueprints/
  Modular Applications with Blueprints

- https://realpython.com/flask-blueprint/
  Using a Flask Blueprint to Architect your Applications

- https://www.freecodecamp.org/news/how-to-use-blueprints-to-organize-flask-apps/
  How to use Blueprints to Organise your Flask Apps

- https://www.youtube.com/watch?v=_LMiUOYDxzE
  Flask Blueprints Make Your Apps Modular And Professional (YouTube)

- https://blog.miguelgrinberg.com/post/the-flask-mega-tutorial-part-xv-a-better-application-structure
  The Flask Mega-Tutorial, Part XV: A Better Application Structure

- https://www.youtube.com/watch?v=WteIH6J9v64
  Flask Tutorial #10 - Blueprints & Using Multiple Python Files (YouTube)

- https://nickjanetakis.com/blog/flask-nested-blueprints-example
  Flask Nested Blueprints Example

- https://dev.to/blankgodd/creating-a-blueprint-based-flask-app-199h
  Build a Flask Application with Blueprints

- https://hackersandslackers.com/flask-blueprints/
  Organise Flask Apps with Blueprints

- https://www.youtube.com/watch?v=pjVhrIJFUEs
  Intro to Flask Blueprints (YouTube)

- https://www.youtube.com/watch?v=GZ51so48YnM
  Blueprints - Flask Tutorial Series #9 (YouTube)

- https://www.youtube.com/watch?v=hc5tC3TCs8g
  Flask Blueprints - Hello World (YouTube)

COM661 Full Stack Strategies and Development

BE10. API Testing and Documentation

Aims

- To appreciate the importance of through API testing
- To develop a coherent test plan for the ***Biz*** API
- To introduce the Postman testing facility
- To develop an automated test script for the ***Biz*** API
- To introduce the Postman topol for automated API documentation
- To document the requests and parameters of the ***Biz*** API
- To illustrate the addition of examples to the test documentation
- To publish the API documentation on the Web using Postman's automated toolkit

Table of Contents

10.1 AUTOMATED API TESTING 2

10.1.1 CREATING A TEST PLAN 2
10.1.2 REGISTERED USER TESTING 3
10.1.3 GENERAL USER TESTING 12
10.1.4 ADMIN USER TESTING 13

10.2 AUTOMATED API DOCUMENTATION 14

10.2.1 DOCUMENTING REQUESTS 15
10.2.2 DOCUMENTING PARAMETERS 16
10.2.3 DOCUMENTING COLLECTIONS 17
10.2.4 ADDING EXAMPLES 18
10.2.5 VIEW DOCUMENTATION 19

10.3 FURTHER INFORMATION 22

10.1 Automated API Testing

API testing involves comparing the result of requests to API endpoints with the intended behavior. For example, in the API developed so far, a GET request to the endpoint http://localhost:5000/api/v1.0/businesses should result in the retrieval of a JSON formatted collection of business information, with the HTTP status code 200. If a code other than 200 is returned, then the request has failed due to incorrect input, program code error or non-availability of the database.

Comprehensive testing of an API can be a very laborious process as each request to each endpoint should be tested each time the code is modified or deployed to a new environment. This situation is greatly eased, however, by Postman, which provides a useful tool that allows us to specify a collection of tests that can be run with a single button click.

10.1.1 Creating a Test Plan

Our ***Biz Directory*** API supports the following endpoints and requests.

Endpoint	Method	Meaning	User type
/api/v1.0/businesses	GET	Fetch list of businesses	All
	POST	Add new business	Registered
/api/v1.0/businesses/<id>	GET	Fetch a single business	All
	PUT	Modify a business	Registered
	DELETE	Delete a business	Admin
/api/v1.0/businesses/<id>/reviews	GET	Fetch list of reviews	All
	POST	Add new review	Registered
/api/v1.0/businesses/<id>/reviews/<id>	GET	Fetch single review	All
	PUT	Edit single review	Registered
	DELETE	Delete a review	Admin
/api/v1.0/login	GET	Login	All
/api/v1.0/logout	GET	Logout	All (if logged in)

Our test plan should provide for all of these combinations, making sure that all provide the desired response for any combination of input. As our database is a "living" one, with businesses and reviews being added, modified and deleted as time progresses, the appropriate order is to begin with actions available to *Registered* users so that we can test the addition and modification of businesses and reviews, before proceeding to test the general retrieval operations open to all – in case the retrieval operations are adversely affected by addition and modification of the data. Finally, we will test the endpoints available to *Admin* users to delete the previously added reviews and businesses, so leaving the database in the state it was in before testing.

10.1.2 Registered User Testing

Postman allows us to arrange our tests in collections so that we can keep the tests for an API together and run them as a batch on request. Launch Postman and sign in by either creating a new account or using your Google account. Make sure that the sidebar is visible and click on "My Workspace" to reveal a screen as shown in Figure B10.1, below.

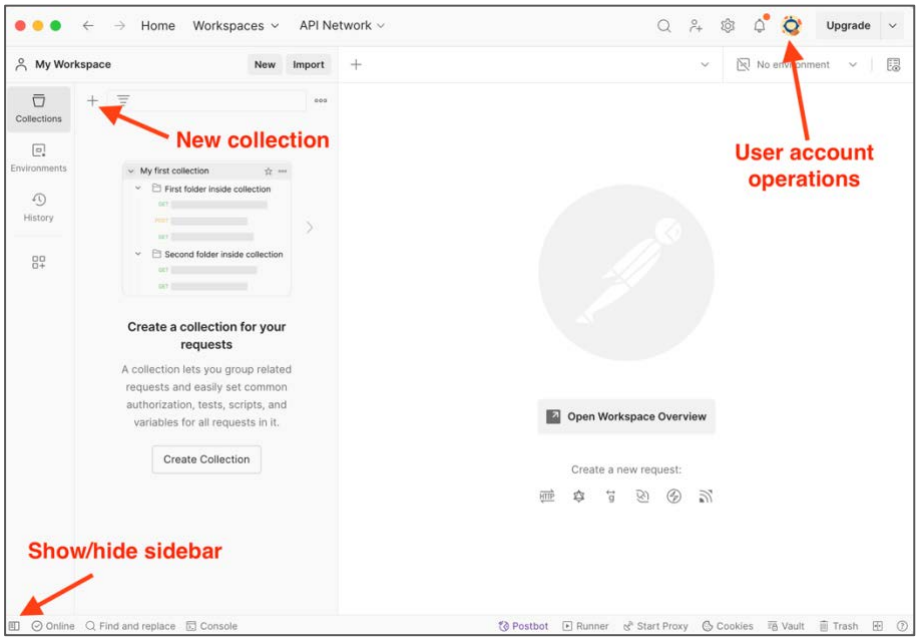


Figure 10.1 Postman Account Operations and Sidebar

Now use the ***New Collection*** button to create a new ***blank collection*** called ***Biz*** to hold your tests. Before we can add a new business to the database, we need to log in to the application and generate a JWT token, so create a new request in the Biz collection (by clicking on the three dots to the right of the collection name and selecting "Add request" from the menu) and specify a GET request to the endpoint http://localhost:5000/api/v1.0/login. Change the name of the request to "Login" by clicking

on the three dots to the right of the "GET New Request" label and selecting "Rename" from the menu. In the ***Authorization*** tab, select ***Basic Auth*** and provide the username and password for one of your registered (but not admin) users. If you click ***Send*** and issue the request, you will see that the request is processed and the JWT token is presented in the Postman output pane. Previously we manually copied this token value and pasted it as a **x-access-token** header, but we want the test process to run without user intervention, so we need to have the token value saved in a variable that we can then use in later requests.

Postman provides a global variable structure that allows us to save values returned from a request so that they can be used later. We do this by selecting the ***Scripts*** tab as shown in Figure 10.2, below and specifying the JavaScript code that retrieves the value from the result and saves it.

First, click on the ***Set a global variable*** option from the ***Snippets*** menu in the right-hand pane. This causes the line of code

```
pm.globals.set("variable_key", "variable_value");
```

to be added to the tests pane. Change the "**`variable_key`**" entry to read "**`jwt_token`**" (the name of our global variable). Now, we will retrieve the token value from the **`pm.response`** object by adding the line of code

```
response = pm.response.json();
```

above the previously inserted line. Finally, we replace the "**`variable_value`**" entry with **`response.token`** to extract the value of the token object and set it as the value of the global variable.

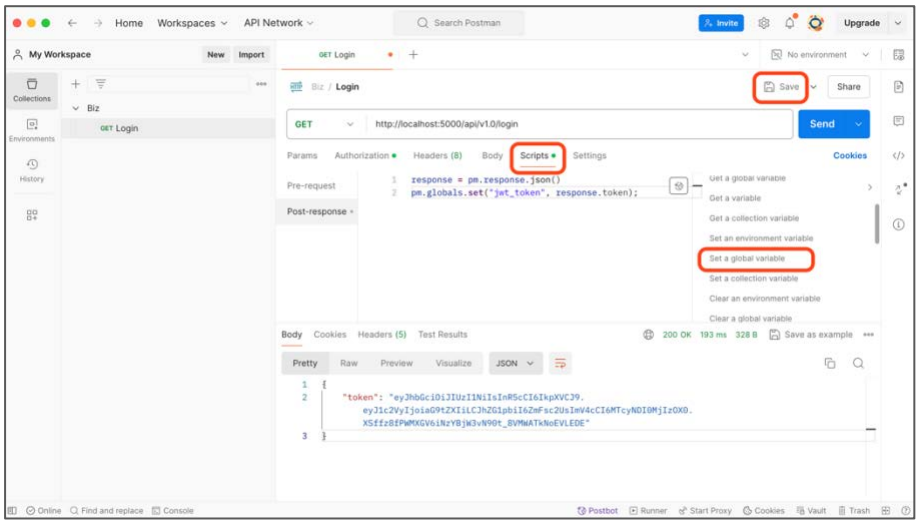


Figure 10.2 Fetch Token from Response Object and Store

Note:	Any valid JavaScript code can be used to specify the test, so we could check for the presence of a particular key field, that an array returned is of a certain length (or within particular boundaries) or any other format or value check.

Now, save the request by clicking the "Save" button (highlighted in Figure 10.2) and issue the request by clicking ***Send***. You can now view the value of the newly created global variable by clicking the ***Environment quick look*** icon as highlighted in Figure 10.3. The new token value should now be seen against the **`jwt_token`** entry in the ***Globals*** list.

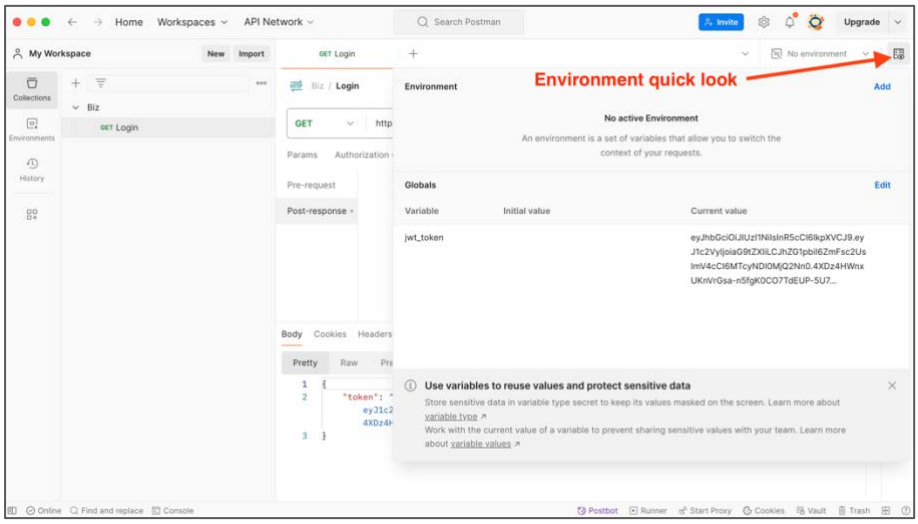


Figure 10.3 Inspect Global Variables

Now that we have a live JWT token available, we can use it in a request to add a new business. Add a new request called "*Add new business*" to the collection as a POST request to http://localhost:5000/api/v1.0/businesses. Specify values for the parameters **`name`**, **`town`** and **`rating`** in the ***Body*** tab as shown in Figure 10.4, below.

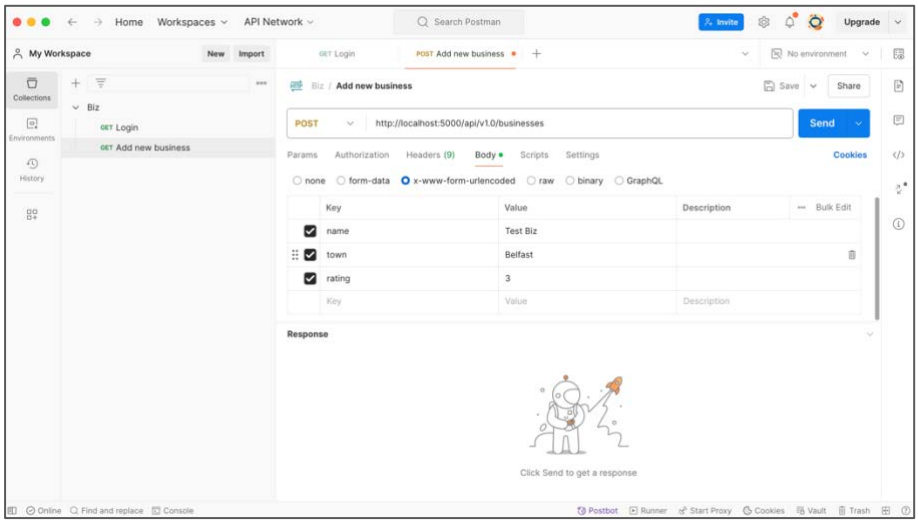


Figure 10.4 Parameters for a New Business

Now add the JWT token to the request by visiting the ***Headers*** tab and creating a new header called ***x-access-token***, as previously. For the value of the token, we can retrieve the global variable value by specifying the name of the variable in double curly brackets, such as **`{{jwt_token}}`**, as shown in Figure 10.5

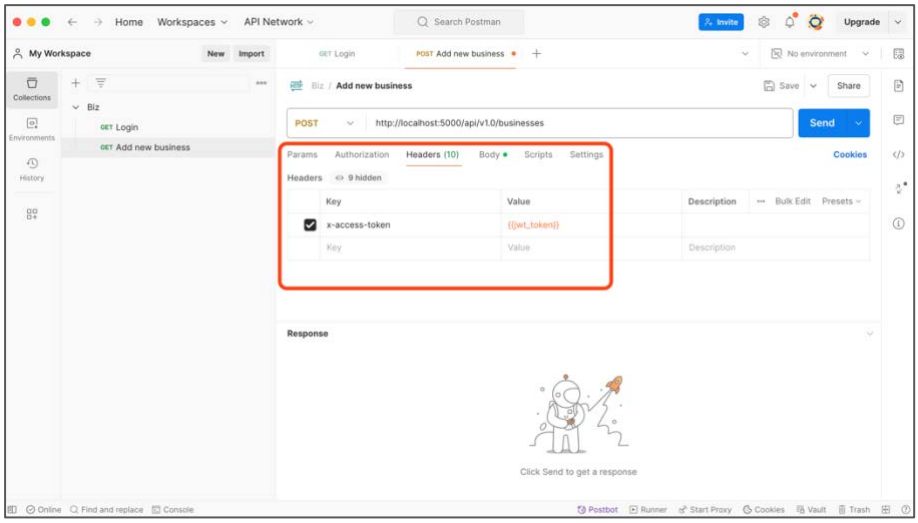


Figure 10.5 Add JWT Token to the Request Header

Now, we can specify the tests for the ***Add New Business*** request. The operation has been successful when a HTTP Status code of 201 is returned, so we begin by selecting the ***Scripts*** tab, clicking on the ***Status code: Code is 200*** option from the ***Snippets*** list and editing the code so that it checks for return code **201** instead of **200**. Next, we repeat the previous operation to set a global variable by retrieving the URL returned from the response and setting it to a new global variable called **`new_biz_url`** as shown in Figure 10.6, below.



Figure 10.6 Specify Tests for Add New Business Endpoint

Now, when we issue the request and check the global variables, we see that the new global variable has been created with a value of the URL of the new resource, shown in Figure 10.7, below.

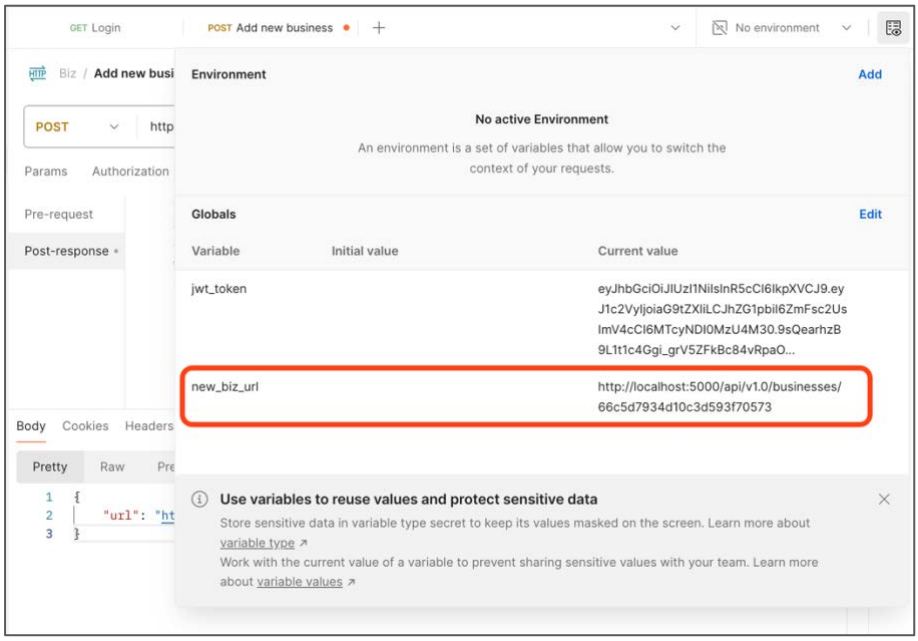


Figure 10.7 New URL Generated and Saved

Finally, we can run the login and add business actions specified so far as a single operation (make sure that both actions have been saved). Open the Collection pane by clicking on the ellipsis (three dots) to the right of the collection name and click ***Run collection***, highlighted in Figure 10.8, below.

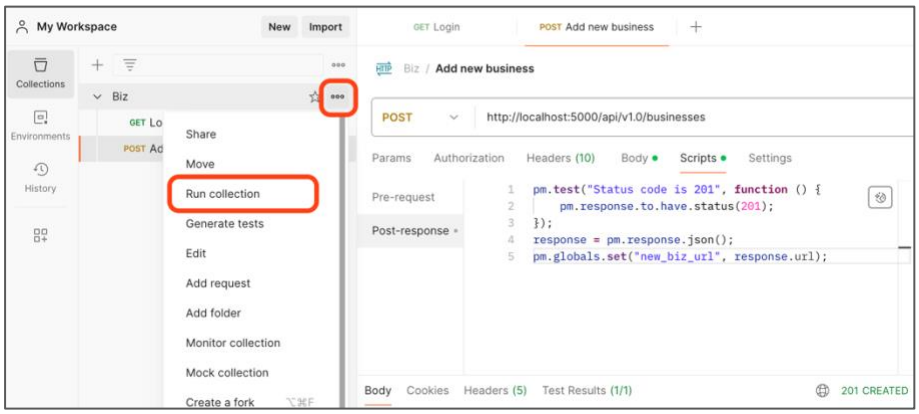


Figure 10.8 Open the Collection Runner

From the ***Collection Runner*** window shown in Figure 10.9 below, click the ***Run Biz*** button to run the tests.

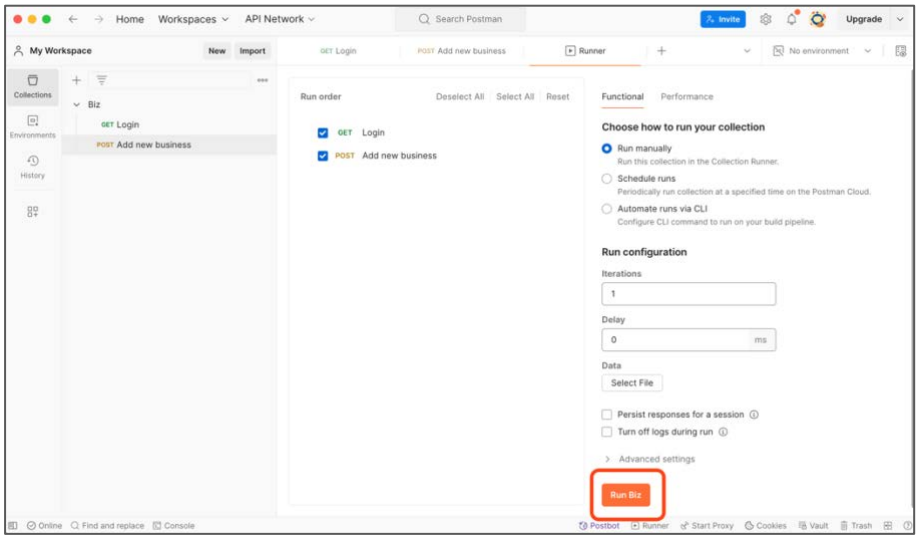


Figure 10.9 Collection Runner

Finally, you should see the result of the ***login*** and ***add business*** actions as described in Figure B6.10, below.

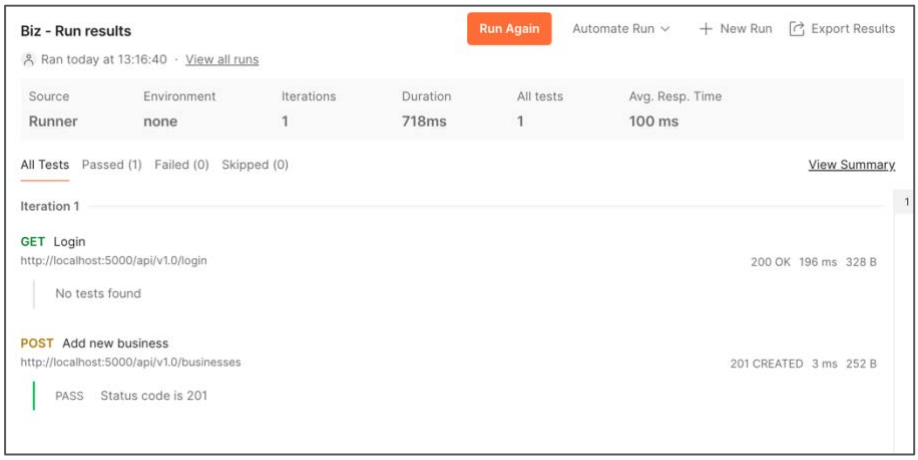


Figure 10.10 Test Collection Results

Do it now!	Follow the steps outlined above to create the ***login*** and ***add new business*** tests and verify their operation. Make sure that you have first run the MongoDB database server and the app.py application file.

Do it now!	Add a test to the ***login*** entry so that it checks for Status Code 200 being returned (i.e. a successful result). Run the collection of tests and verify that they work as expected

Do it now!	It is important to check that tests fail when invalid input is provided. Change the ***add new business*** test so that it checks for Status Code 200 instead of 201 and re-run the collection. Verify that you receive output such as that shown in Figure 10.11, below.

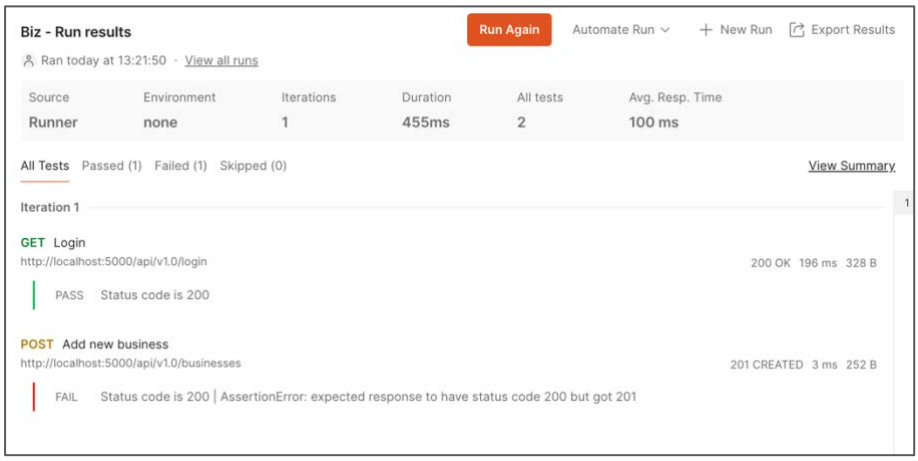


Figure 10.11 Test Failure Message

Do it now!	Restore the Status Code check to 201 and remove one of the input parameters ***name***, ***town*** or ***rating***. Make sure that the test fails with Status Code 404. Replace the missing parameter when you are satisfied with the operation of the tests.

We can now repeat the previous process to test the endpoint that adds a new review to a business. Remember that we created the **`new_biz_url`** global variable to hold the url of a newly added business, so we can make use of this to build the endpoint that allows us to add a review to that business. The **`new_biz_url`** value will be in the form http://localhost:5000/api/v1.0/<id>, so the URL for the request to add a review can be expressed as **`{{new_url}}/reviews`**.

Create a new test in the ***Biz*** collection to generate a POST request to **`{{new_biz_url}}/reviews`** and provide values for the input fields ***username***, ***comment*** and ***stars*** as shown in Figure 10.12 below. Remember to also add the **`x-access-token`** header as **`{{jwt_token}}`** as for adding a business earlier.

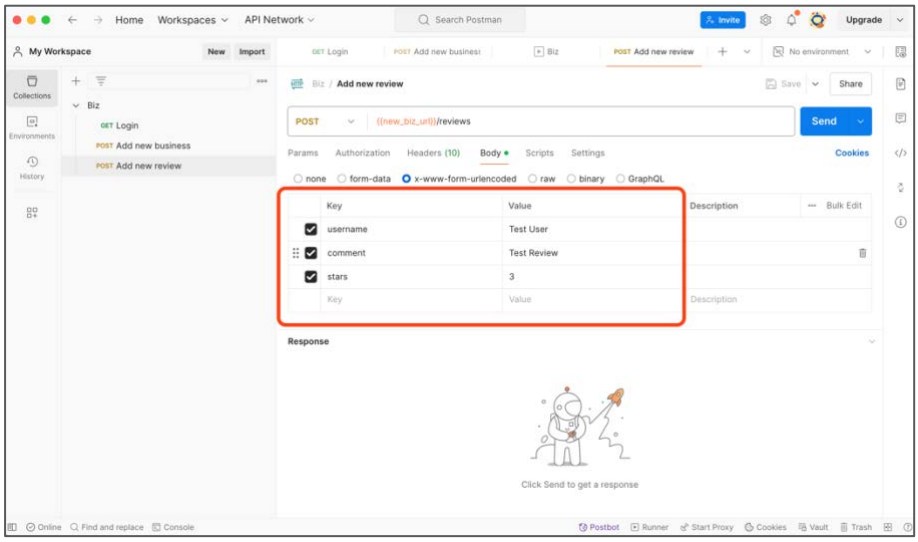


Figure 10.12 Add a New Review

Now we can add a test for the request as for adding a new business, by checking that the HTTP Status code returned is **201** and setting a new global variable called ***new_review_url*** to the value returned in the response body. The code is illustrated in Figure 10.13, below.

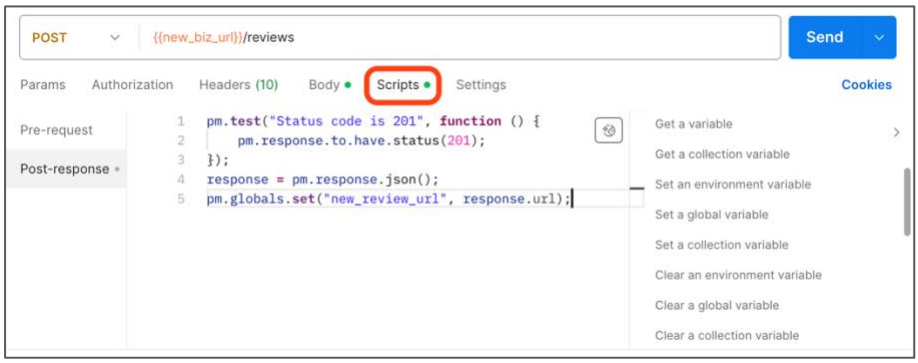


Figure 10.13 Test for New Review

Finally, we run the collection of tests and verify that they all succeed as shown in Figure 10.14, below.

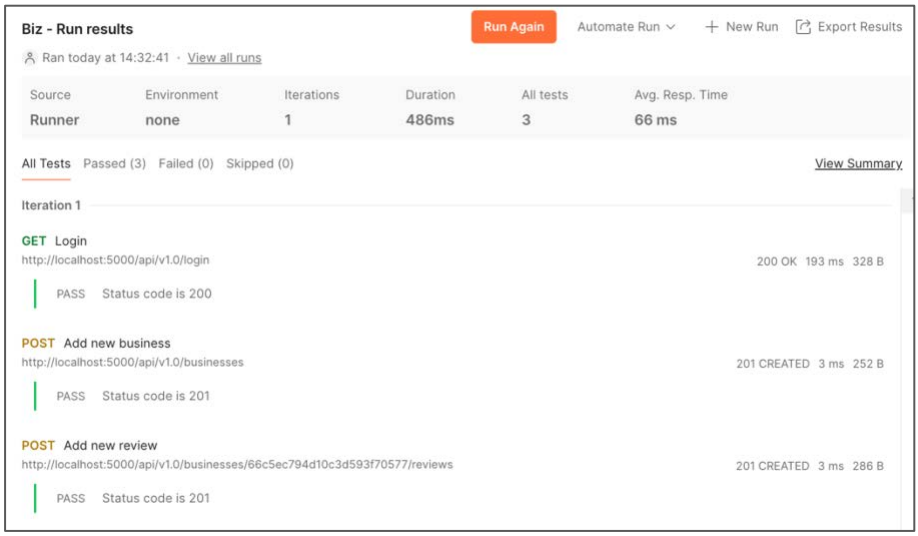


Figure 10.14 Check Tests

Do it now!	Follow the steps illustrated above to add a test for the ***add review*** endpoint and verify that they are successful. Check the global variables to make sure that the new **`new_review_url`** variable has been created.

Do it now!	Add additional tests to validate the **PUT** requests that allow a registered user to edit the details of a specified business and specified review by using the global variables **`new_url`** and **`new_review_url`** to construct the endpoints. In each case, the HTTP Status code returned should be 200.

Now that all of the registered user functionality has been added to the test collection, we can add an additional step to the collection to log the user out of the application. This is achieved by simply adding a new entry for a GET request to the URL http://localhost:5000/api/v1.0/logout as illustrated in Figure 10.15, below where the only information to be provided is the **`x-access-token`** header as **`{{jwt_token}}`** and the test for success is that Status Code 200 has been returned.

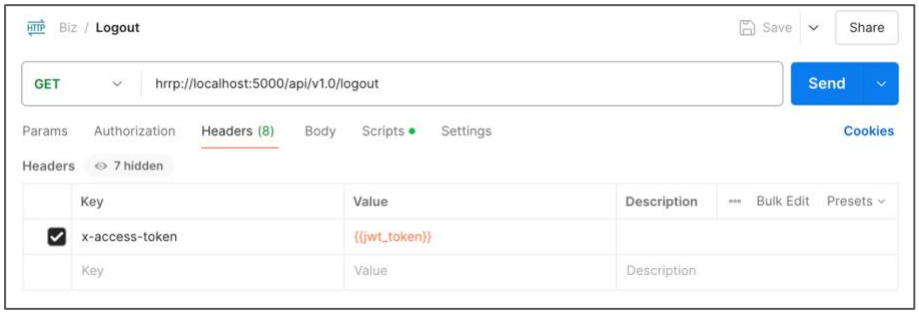


Figure 10.15 Logout User

Do it now!	Add the new entry to the collection to logout the current user. Add a test to the entry so that the HTTP response code is verified to be 200. Now, run the collection of tests and check that they still run successfully.

10.1.3 General User Testing

Now that the endpoints to add and edit businesses and reviews have been tested, we can implement tests for the routes available to general (i.e. non-logged in) users as described below.

Endpoint	Action	Test
GET /api/v1.0/businesses	Get all businesses	Status code is 200
GET /api/v1.0/businesses/<id>	Get a specific business	Status code is 200
GET /api/v1.0/businesses/<id>/reviews	Get all reviews for a business	Status code is 200
GET /api/v1.0/businesses/<id>/reviews/<id>	Get a specific review for a business	Status code is 200

Do it now!	Add the tests above to the ***Biz*** collection (immediately before the ***Logout*** test so that logout is the final action in the sequence) and verify that all operate as expected, generating the specified HTTP status code. Use the value of the global variables **`new_biz_url`** and **`new_review_url`** in the tests to retrieve a specific business and review, respectively. Check that the output from your complete set of tests is similar to that illustrated in Figure 10.16.

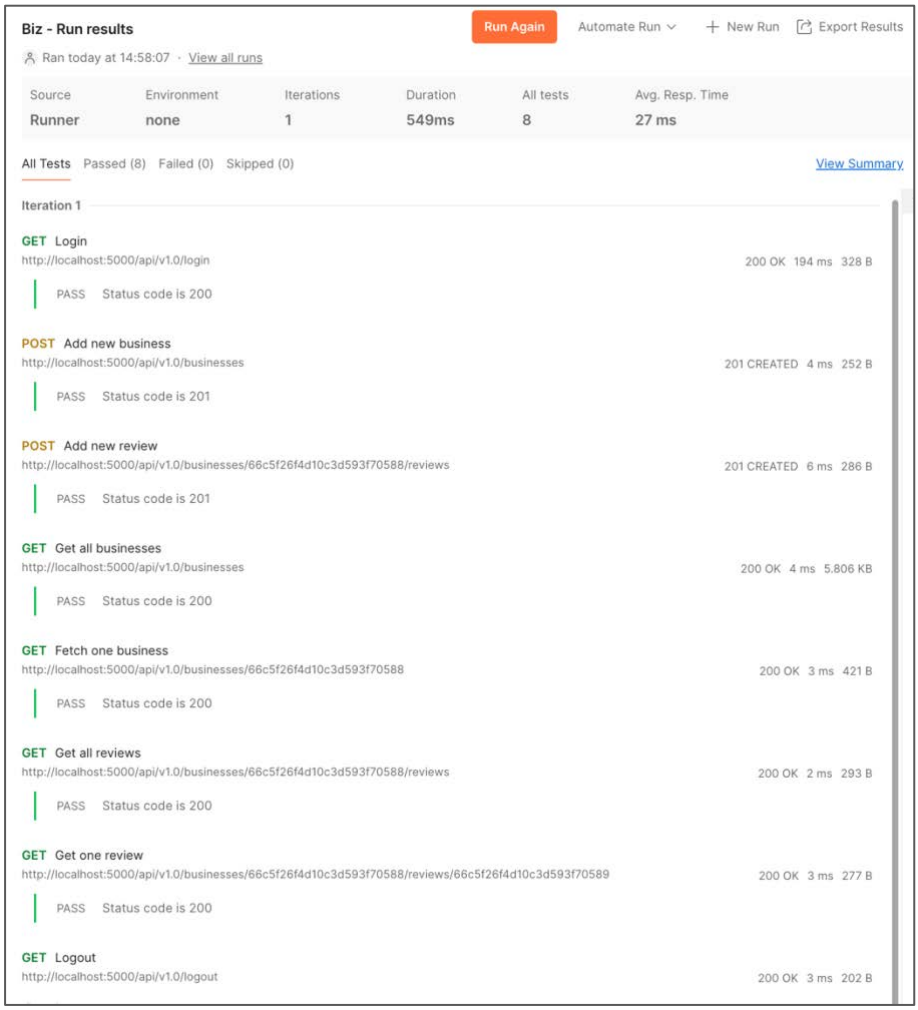


Figure 10.16 Checking all tests

10.1.4 Admin User Testing

The final endpoints to be added to the test collection are those which are reserved for admin users to delete a specific business or review. These are described in the table below.

Endpoint	Action	Test
DELETE /api/v1.0/businesses/<id>	Delete a specific business	Status code is 204
DELETE /api/v1.0/businesses/<id>/reviews/<id>	Delete a specific review for a business	Status code is 204

Do it now!	Add the tests for the DELETE operations above to the ***Biz*** collection and verify that all operate as expected. Use the value of the global variables **`new_url`** and **`new_review_url`** in the tests to retrieve a specific business and review, respectively. Remember to test the delete review route first, before testing the route to delete a business. **Hint:** Before the DELETE requests are sent, you should add an action to log in a user with "Admin" privileges. Add an additional action to log out the admin user after the tests are complete. Verify that the result of running all tests is similar to that shown in Figure 10.17. Note that the summary view of test results can be obtained by clicking the "View Summary" link seen in the upper right corner of Figure 10.16

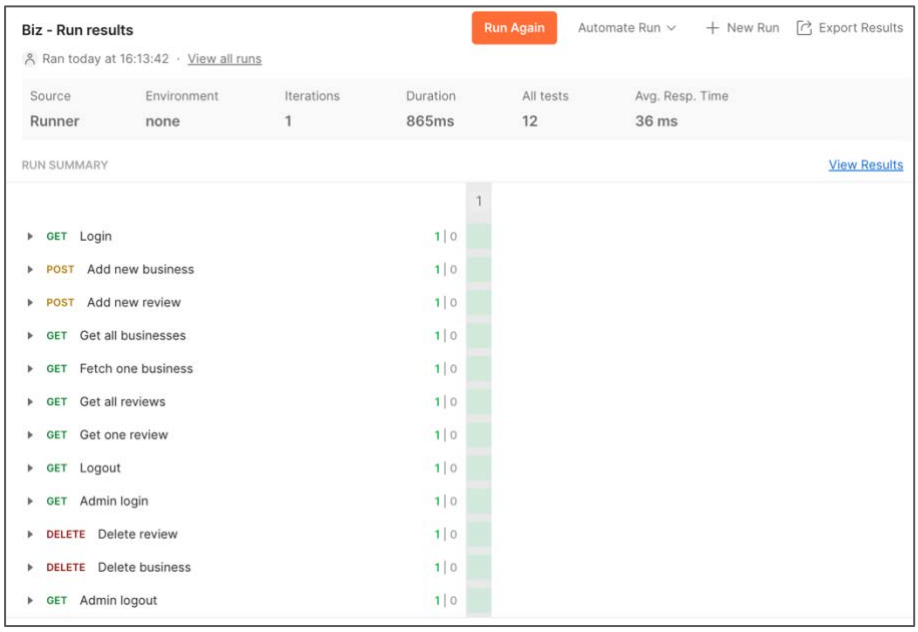


Figure 10.17 Checking All Tests

10.2 Automated API Documentation

Effective documentation is a very important aspect of API development. When an API to a data collection is released for use by the wider community, its potential for uptake is greatly enhanced by clear documentation that describes each endpoint in terms of its effect, the parameters required for it to be invoked and the format and organisation of the data that will be returned. Postman provides a very useful tool for the construction, ongoing management and online publication of API documentation in three areas – requests, parameters and collections. We will examine each of these in the sections that follow.

10.2.1 Documenting Requests

Each request can be documented by providing it with a meaningful name and a short piece of text that describes the purpose of the request. Figures 10.18 and 10.19, below, show a description applied to the user login request.

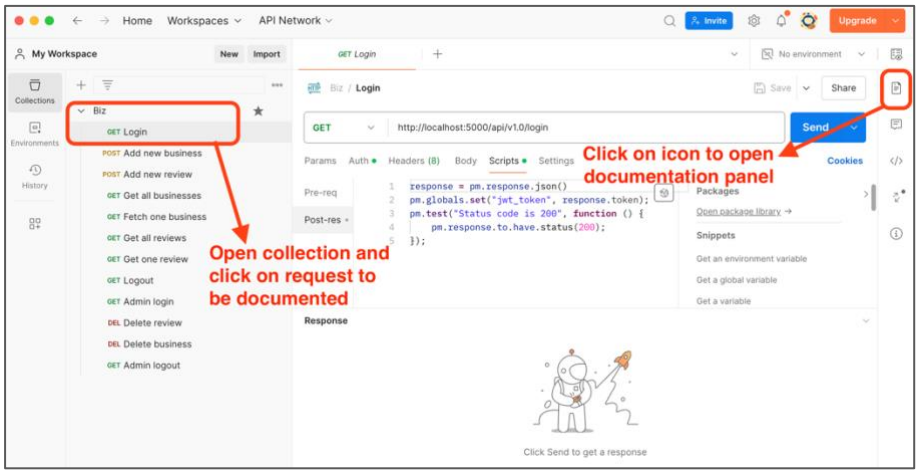


Fig 10.18 Open the Documentation Panel for a Request

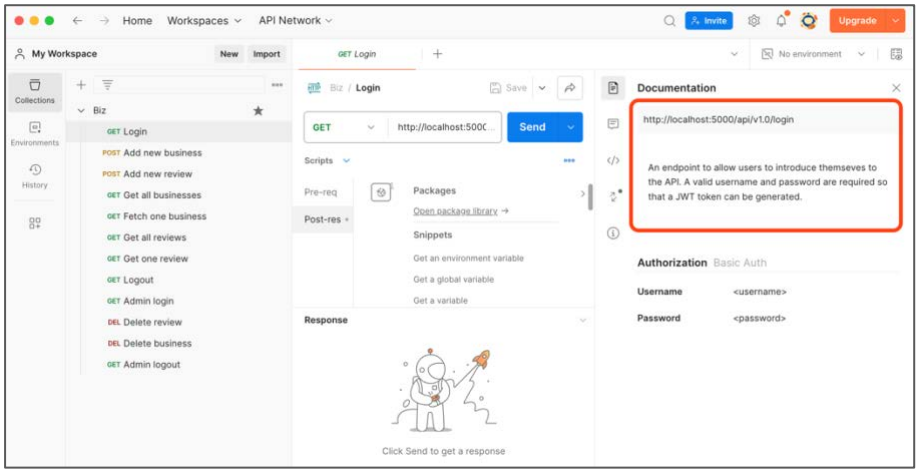


Fig 10.19 Request Documentation Provided

The ***Documentation*** panel is accessed by clicking on the "document" icon in the menu bar on the right-hand side (highlighted in Figure 10.18 above). Clicking in the text area highlighted in Figure 10.19 allows the user to enter or modify the text description of the request. This field can be completed in plain text or in ***Markdown*** – a text formatting notation that provides HTML-type notation for specification of lists, tables, images and other formatting options.

Note:	Markdown is beyond the scope of this module but links to information on it can be found in Section 10.3. You are encouraged to explore this and to use it to provide more professional documentation for your API.

Do it now!	Follow the steps above to create a documentation entry for the "User Login" request. Then, repeat the process to create documentation entries for all of the endpoints in the Biz Directory API.

10.2.2 Documenting Parameters

Our API frequently uses parameters specified as key/value pairs to provide information to be used in requests. These might be form fields (entered in the ***Body/x-www-form-urlencoded*** tab), headers (such as ***x-access-token*** values) or other data.

Anywhere that Postman provides a key/value data input facility also includes provision for a ***Description*** field to provide illustrative information for the value. This can be information regarding the allowable values (text, numeric, allowed range, etc.), the purpose of the parameter, whether it is compulsory or optional, or any other information that helps the user of the API provide the correct request format.

Figure 10.20, below, demonstrates the provision of description information for the parameters for the ***Add New Business*** request. Here, we specify that the business name and location are text strings, while the rating should be an integer value in the range 0-5.

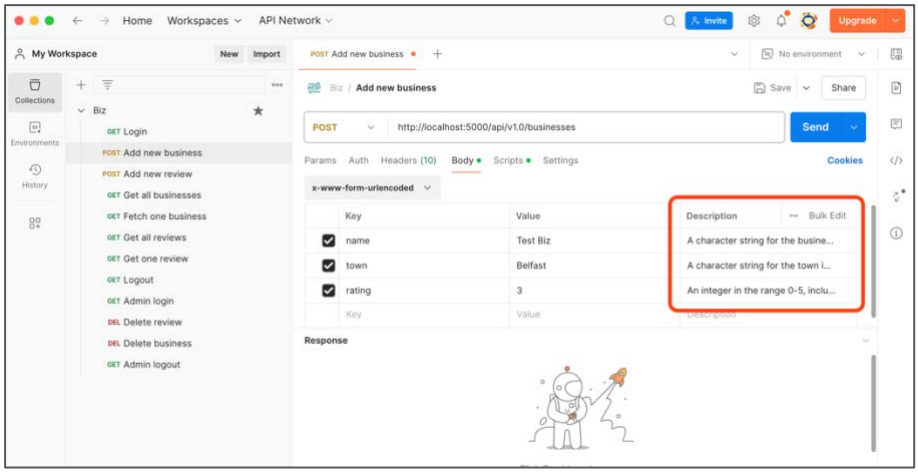


Figure 10.20 Documenting parameters

Do it now!	Provide description values for the parameters to the ***Add New Business*** request as shown in Figure 10.20. Then follow the same process to add documentation for all of the parameters in your endpoints – including those in the request headers.

10.2.3 Documenting Collections

The final option for providing textual help for the API consumer is summary documentation that describes the collection as a whole. This is useful in producing a general description of your API, indicating its purpose, intended use and limitations for certain types of users.

The collection documentation is accessed by clicking on the ellipsis (…) to the right of the collection name and selecting ***View Documentation*** from the drop-down menu that appears (illustrated by Figure 10.21, below).

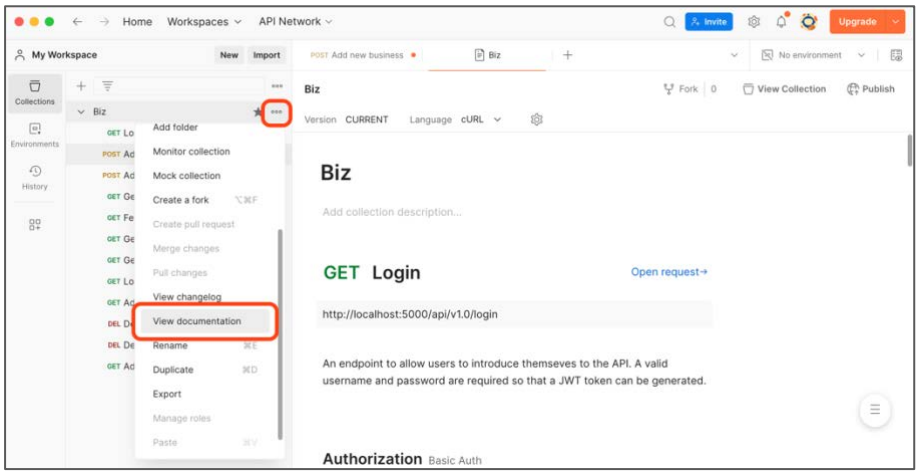


Figure 10.21 Edit a collection description

This reveals the ***Describe Collection*** window as shown in Figure 10.22, below. Here, we click on the prompt or the previously provided text to open the text editor and provide some summary text to give an overview of the API, detailing its purpose and usage restrictions.

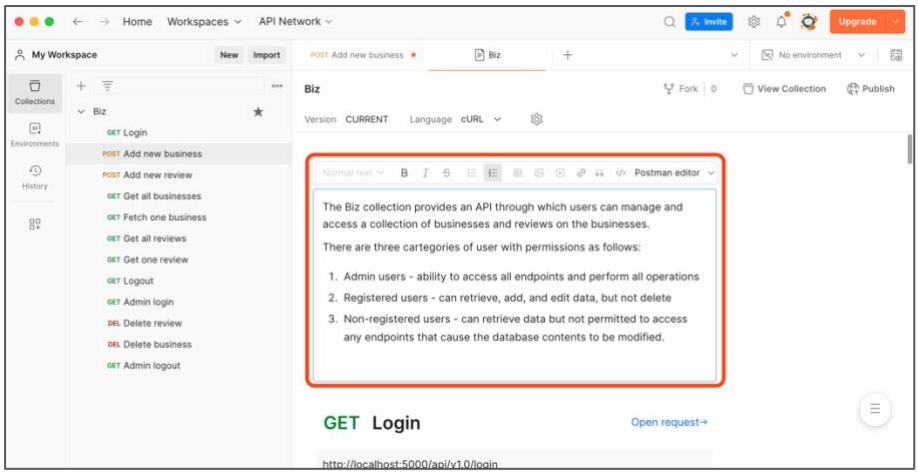


Figure 10.22 Provide a collection description

Do it now!	Add documentation to your collection as shown in Figure 10.22

10.2.4 Adding Examples

Another powerful feature of Postman's documentation tool is the ability to include examples of API requests to show the format of the request and sample data returned. This can be very useful for users in clearly illustrating how the API can be integrated with their application and how the data returned can be parsed and processed.

To add an example to the documentation, first select the request that you want to demonstrate from the list shown in the collection. Then, ensure that all of the request parameters are set up and click ***Send***. If you are happy with the response generated, click on the ***Save Response*** link on the top-right of the response panel and click on ***Save as example*** as shown in Figure 10.23, below. The example will then be added as a sub-item below the request in the Collections list in the sidebar from where its properties can be manipulated through the menu accessible from the ellipsis to the right of the example name. This is illustrated in Figure 10.24, where the name of the saved example has been changed to "Test Biz added".

Note that you can provide as many examples as you want for each request, if there are multiple elements of functionality that you want to demonstrate.

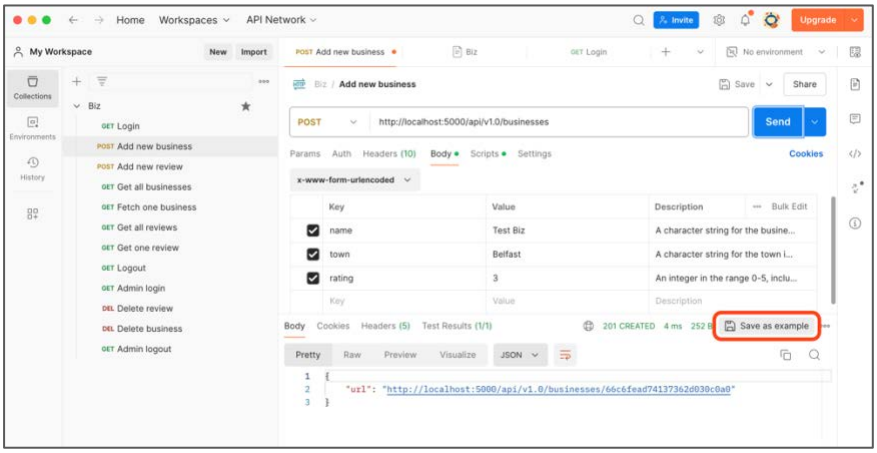


Figure 10.23 Adding an example

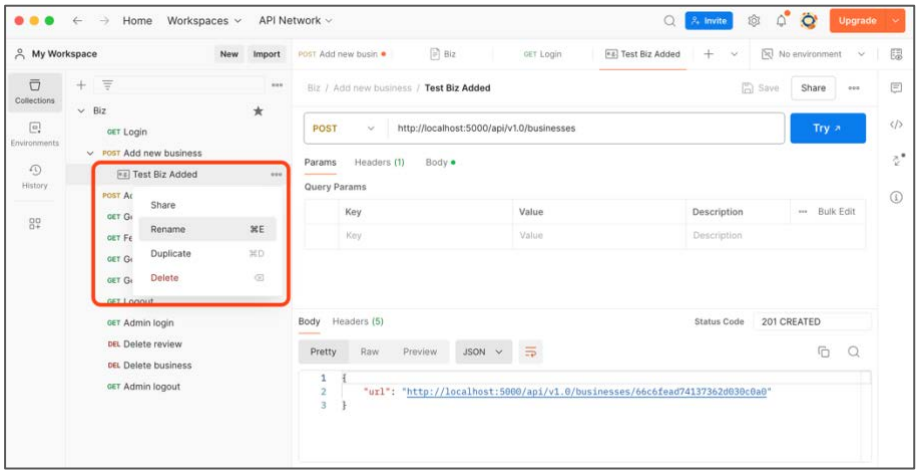


Figure 10.24 Example saved

Do it now!	Add an example to your ***Add New Business*** request as illustrated in Figure 10.24. Remember that you will need to have previously logged in to the application and provided a valid JWT token. Also add examples for other requests in your collection.

10.2.5 View Documentation

Once all of the documentation and examples have been provided, we can publish the documentation to the Web. Click on the ellipsis to the right of the collection name (in the sidebar) and then on the ***View documentation*** option on the menu that is revealed to see a preview of the documentation you have provided. Then, in the documentation panel, click the ***Publish*** icon on the top-right, as illustrated in Figure 10.25, below.

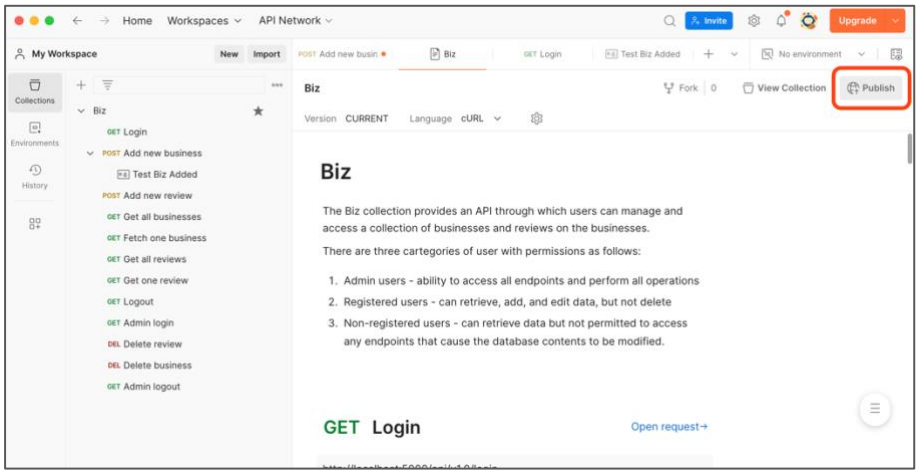


Figure 10.25 View documentation

This opens a web browser on the ***Publish Collection*** page, containing a form that enables you to customize your online documentation. Leave the various options at their default

values and click on the ***Preview documentation*** link on the left-hand side. This opens another web browser with a live preview of how your online documentation will be presented.

Note how the list of requests is presented on the left-hand side as a clickable menu giving direct access to each of your requests. The information about every request is presented in the centre pane with all of the descriptive information you have provided, while the right-hand pane shows code demonstrating how to call the endpoint and your example responses. The language used to demonstrate the calls to the API can be changed by the drop-down menu at the top of the pane. Figure 10.26, below, illustrates the web view of the documentation generated so far.

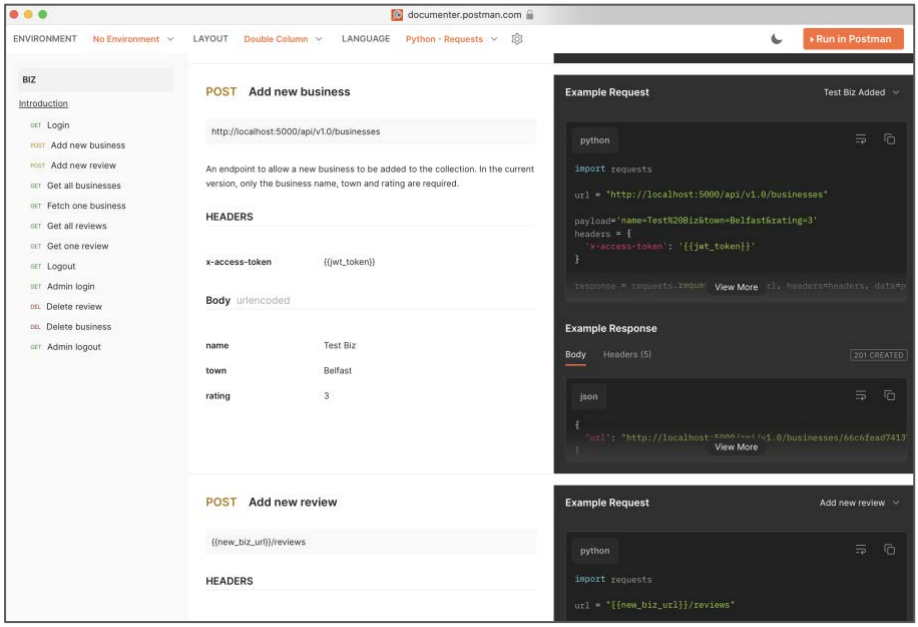


Figure 10.26 Preview documentation

Finally, when you are satisfied with the content and appearance of the documentation, you can publish it publicly by clicking on the ***Publish Collection*** button at the bottom of the ***Publish Collection*** page, shown in Figure 10.27, below.

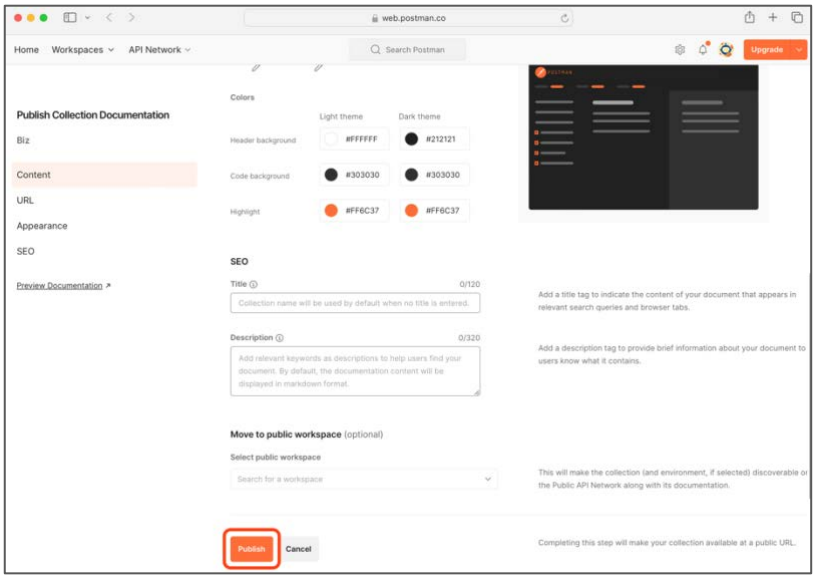


Figure 10.27 Publish documentation on the Web

Once published, clicking on the Publish link in the Documentation panel within Postman will open the ***Publication settings*** page, shown in Figure 10.28 below, which contains the URL that you can distribute for others to access your documentation and use your API.

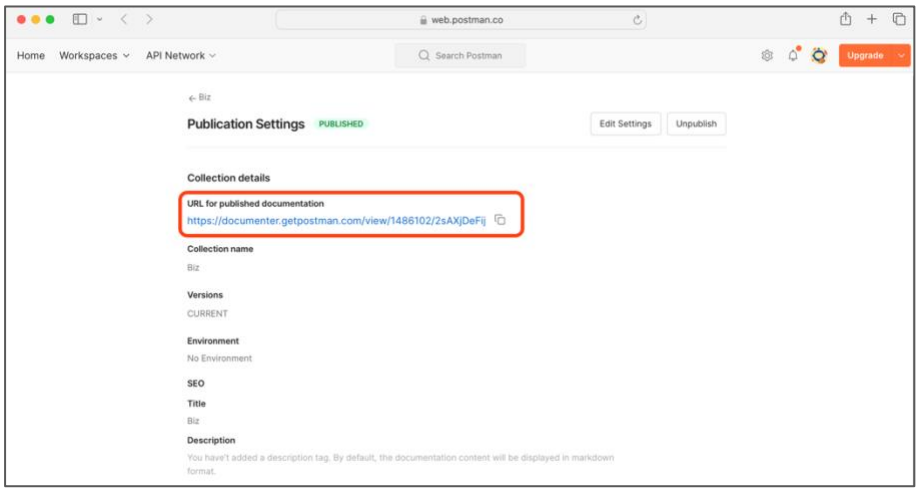


Figure 10.28 Documentation published

One of the best features of the Postman documentation tool is that any updates you make to the documentation are reflected in real-time in the public Web version – your documentation will always be up to date!

10.3 Further Information

- https://en.wikipedia.org/wiki/API_testing
  API testing overview
- https://www.katalon.com/resources-center/blog/api-testing-tips/
  10 API Testing Tips for Beginners
- https://www.softwaretestingmaterial.com/api-testing/
  API Testing Tutorial
- https://www.youtube.com/watch?v=t5n07Ybz7yI
  The Basics of Using Postman for API Testing (YouTube)
- https://www.guru99.com/postman-tutorial.html
  Postman Tutorial for Beginners with API Testing Example
- https://learning.getpostman.com/docs/postman/scripts/test_scripts
  Postman Learning Centre – Test Scripts
- https://medium.com/aubergine-solutions/api-testing-using-postman-323670c89f6d
  API Testing Using Postman
- https://www.markdownguide.org/
  Markdown Guide
- https://www.markdowntutorial.com/
  Markdown Tutorial
- https://github.com/adam-p/markdown-here/wiki/Markdown-Cheatsheet
  Markdown Cheatsheet

COM661 Full Stack Strategies and Development

FE11. Introducing Angular

Aims

- To appreciate the purpose of Angular in full-stack architecture
- To understand the code structure of an Angular application
- To introduce the structure and role of an Angular component
- To demonstrate injection of data values into the web presentation
- To introduce Angular bllocks
- To install Bootstrap through the Node Package Manager
- To style the initial application using Bootstrap cards
- To appreciate the use of ther Node Package Manager in managing an Angular application (npm)

Table of Contents

11.1 INSTALLATION AND FIRST USE 2

11.1.1 ANGULAR AND ANGULARJS 2
11.1.2 GETTING STARTED 3
11.1.3 BASIC STRUCTURE OF AN ANGULAR APP 5

11.2 SPECIFYING CUSTOM COMPONENTS 8

11.2.1 CREATING A COMPONENT 8
11.2.2 CONNECTING A COMPONENT TO THE APPLICATION 10

11.3 ANGULAR DATA PROCESSING 11

11.3.1 MANIPULATING JSON DATA 11
11.3.2 ITERATING ACROSS DATA WITH THE @FOR BLOCK 13

11.4 USING BOOTSTRAP 14

11.4.1 INSTALLING BOOTSTRAP 15
11.4.2 STYLING WITH BOOTSTRAP 16

11.5 NPM AND PACKAGE MANAGEMENT 18

11.5.1 UNDERSTANDING VERSION NUMBERS 19
11.5.2 BACKUP AND RESTORE OF THE ANGULAR APPLICATION 20

11.5 FURTHER INFORMATION 21

11.1 Installation and First Use

Angular is a front-end framework for the development of modern, responsive and scalable single-page Web applications. In this section of the module, we will use Angular to build a front-end to the API for the Biz application that we created in section B of the module.

11.1.1 Angular and AngularJS

AngularJS was first developed at Google in 2009 as an MVC (Model, View, Controller) JavaScript-based framework for the rapid development of front-end web applications. It was publicly launched as Version 1.0 in 2012 and quickly grew in popularity as an alternative to jQuery for the development of complex interactions.

The release of Version 2 in 2014 was the result of a complete re-write of the infrastructure and significant syntax changes, resulting in incompatibility between versions 1 and 2. As Version 1 had already become the most popular JavaScript framework by 2011, the change was met with some resistance, but simplifications in code structure and performance improvements saw the updated framework (now known simply as Angular) maintain its popularity. The current version of Angular is v20, but it is important to recognize that versions v2 and v4–v20 (there was no v3!) are largely compatible with each other (although there were significant differences between v16 and v17). Most changes were concerned with improvements under the bonnet – rather than in syntax and code structure. To emphasise this, v1 (which maintains a large user base) is commonly known as **AngularJS**, while versions 2+ are known simply as **Angular**.

Note:	When searching online for answers to Angular queries, always be aware of the version of Angular to which the answer applies. Any articles that are aimed at AngularJS should be ignored, and it is best to include a specific version number in your search.

One of the major changes between AngularJS and Angular is the introduction of TypeScript as a replacement for JavaScript. TypeScript was developed by Microsoft as a superset of JavaScript, adding optional data typing to the language. All JavaScript code is valid TypeScript and, although we will use TypeScript in this section of the module, we will not formally cover it – but will point out significant features as we meet them.

11.1.2 Getting Started

Note:	Angular has been provided for you on the lab machines but if you need to install it on your own computer, you can do so (as long as npm has previously been installed) by issuing the command **C:\> npm install –g @angular/cli** **npm** (Node Package Manager) is a component of Node.js. If you do not already have Node.js available on your machine, you will need to install it first from https://nodejs.org/en/.

As a first example, we will create the first version of our Biz Directory Angular application called **bizFE** by the command

```
C:\Users\B00123456\Desktop> ng new bizFE
```

Note:	Given the volume of data that needs to be created/written to build an Angular application framework (typically over 350 MBytes), it is a good idea to create the application on the Desktop (on the lab machines) so that delay due to network traffic is eliminated. You can minimise the application volume (see later) and copy it to your personal drive when the session is complete.

This will take a little while to execute – even up to a couple of minutes on a high-powered computer – but we only need do it once for each application. As the command executes, you will be asked a number of questions regarding the nature of the application. You should select the default value in each case by pressing the **<return>** key in response to each question.

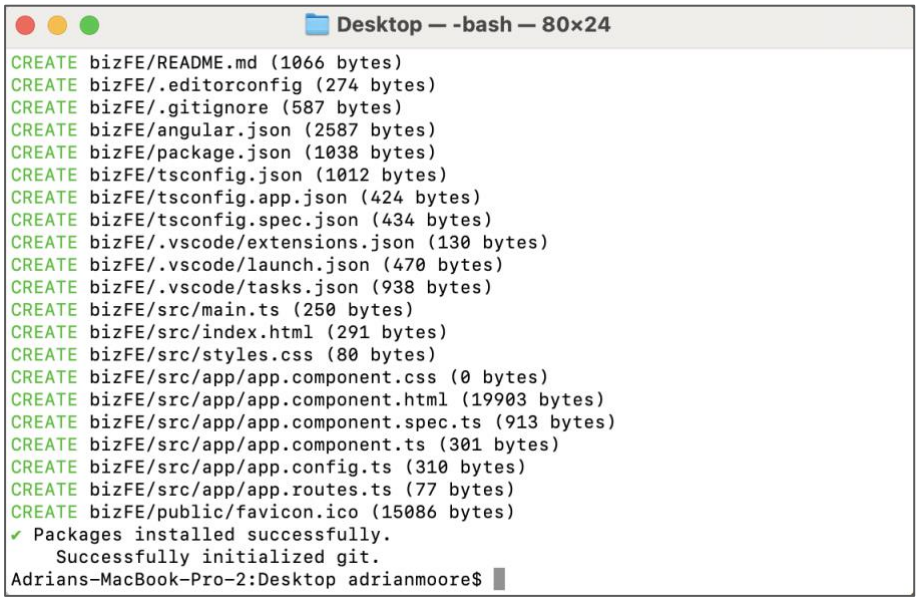


Figure 11.1 Creating a New Angular Application

Although we have not yet provided any code, we can test that the application has installed properly and that a full Angular application has been generated by navigating into the **biz** directory and launching it with the commands

```
C:\Users\B00123456\Desktop> cd bizFE
C:\Users\B00123456\Desktop\bizFE> ng serve
```

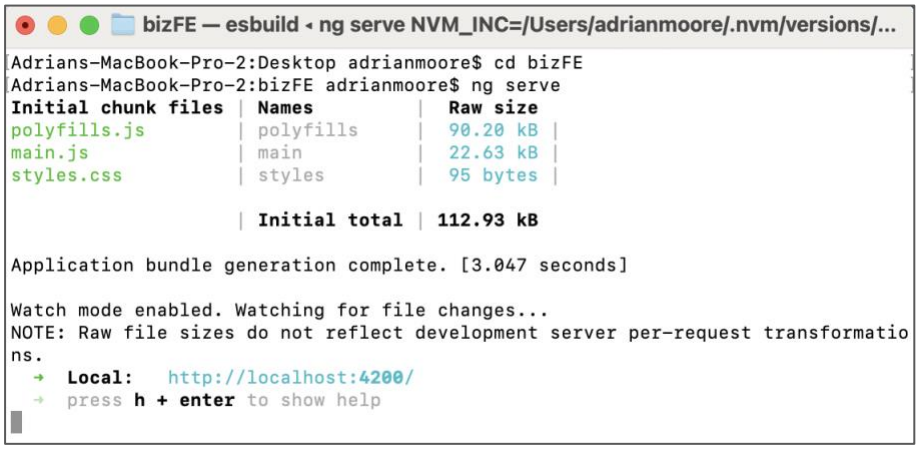


Figure 11.2 Running the Front-end Server

Now that the server is running, we can open a web browser and load the URL http://localhost:4200 to see the default Angular application homepage as shown below.

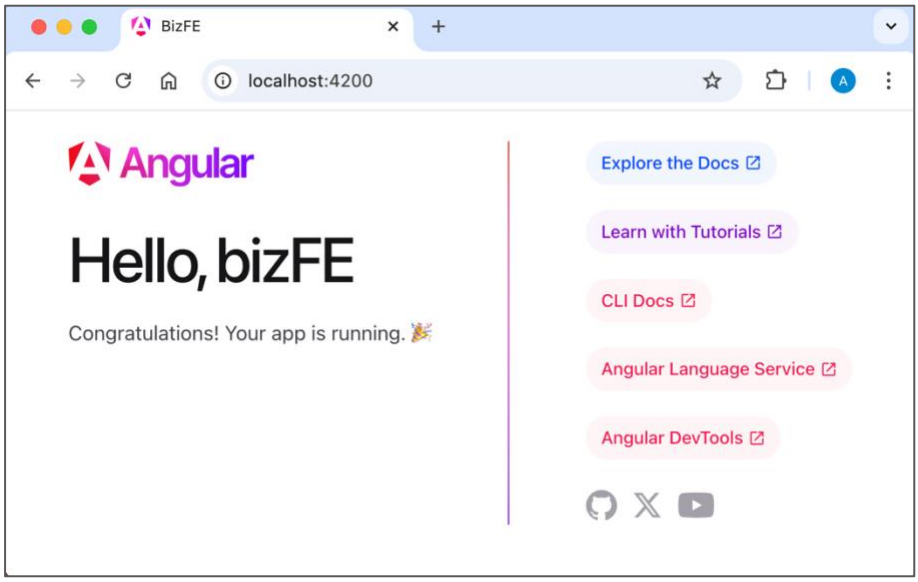


Figure 11.3 Default Angular Server Output

Do it now!	Create a default Angular application called **bizFE** by following the steps outlined above. Run the application and visit the URL http://localhost:4200 in a web browser to ensure that you are presented with the default Angular page shown in Figure 11.3 above.

Note:	Your default home page may look different depending on the version of Angular CLI that is installed. We will delete all of the code that specifies this page anyway, so any difference will not matter**.**

11.1.3 Basic Structure of an Angular App

The Angular CLI creates all the folders and files that will support our application development. All the files that we will edit (and add) will be in the **src** directory – the other directories and files are Angular system files to support the application's development and execution.

The base of the frontend application is the file ***index.html***, located in the **src** directory. If you examine the code in this file (shown in the code box below), you will see a standard HTML skeleton for a web application – enhanced by a curious **`<app-root></app-root>`** tag in the **<body>** section. This is an example of an Angular **Component** – the basic building block of an Angular application. Components enable us to inject content onto the page at a position of our choosing and it is worth exploring the code to illustrate how this default component generates the display shown in the browser

File: bizFE/src/index.html

```
<!doctype html>
<html lang="en">
<head>
    <meta charset="utf-8">
    <title>FE11</title>
    <base href="/">
    <meta name="viewport" content="width=device-width,
                                   initial-scale=1">
    <link rel="icon" type="image/x-icon" href="favicon.ico">
</head>
<body>
    <app-root></app-root>
</body>
</html>
```

A Controller is specified by a TypeScript file and an optional HTML template and CSS stylesheet. The files for components are defined in the **app** sub-directory of **src**, so first look at the default component TypeScript file, ***app.component.ts***.

File: bizFE/src/app/app.component.ts

```
import { Component } from '@angular/core';
import { RouterOutlet } from '@angular/router';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [RouterOutlet],
  templateUrl: './app.component.html',
  styleUrl: './app.component.css'
})
export class AppComponent {
  title = 'FE11';
}
```

The component TypeScript file consists of 3 sections

i) The **`import`** statement that enables the file to make use of the facilities of the Angular **`Component`** and **`RouterOutlet`** libraries

ii) The **Decorator** that specifies the selector (HTML tag) to be used to refer to the component (here, **`app-root`** – corresponding to the **`<app-root>`** tag used in

index.html) and the optional URLs for an HTML template and stylesheets to be used when rendering the component.

iii) Finally, the **class definition** that contains the logic for the component. Here, we set up a single variable called **`title`** with the value '**bizFE**'.

To best understand the structure of the default Angular application, we will replace all the code in ***app.component.html*** with that provided below.

File: biz/src/app/app.component.html

```
<h1>
     Welcome to {{ title }}
</h1>
```

Note here how the component variable **`title`** defined in the class definition can be used within the template. As we see in the browser, the value of the variable is inserted into the **<h1>** tag by enclosing it in **`{{ }}`**. This is an example of data binding – a central concept of Angular, of which we will see (much) more later.

Do it now!	Modify the code of ***app.component.html*** as shown above and verify that you receive output such as that illustrated by Figure 11.4 below.

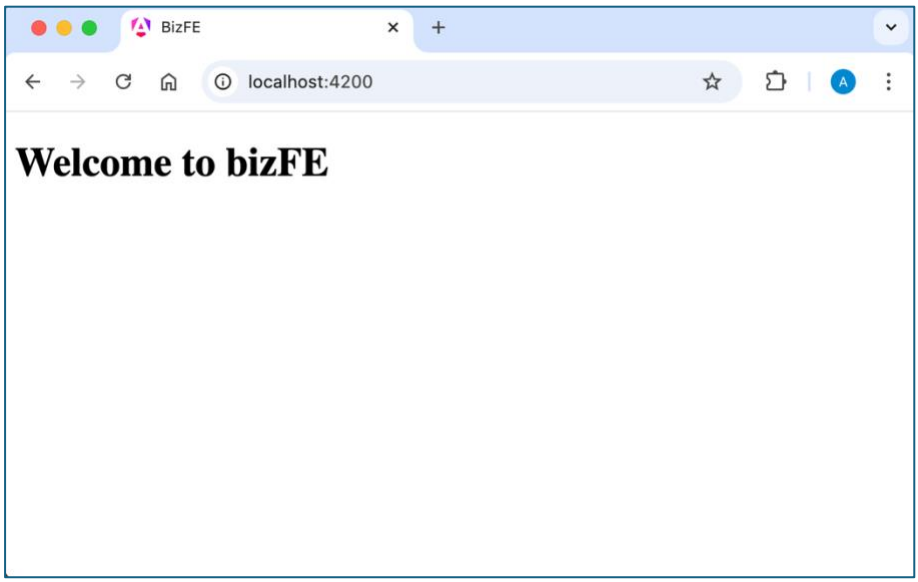


Figure 11.4 Modified Default Output

Try it now!	Develop your understanding of Angular code structure by attempting the following: i) Change the value of the title variable in the component TypeScript file and verify that the browser output changes accordingly. ii) Change the HTML template by adding some additional content of your own (but leave the **<h1>** element in place). iii) Add a style rule to the CSS template file to have the **<h1>** element displayed in red text on a yellow background. iv) Make the required changes so that the **<app-root>** component is called by the modified tag **<app-title>**. (Note: return it to **<app-root>** when you have shown that you can change it – we will refer to this element again later) v) Note how the changes take effect in the browser immediately when you save the source file – the application is automatically re-loaded after each change.

11.2 Specifying Custom Components

In this section, we will add a custom component to our front-end application. Eventually, this component will house our collection of businesses, but initially we will just display a simple heading.

11.2.1 Creating a Component

The first stage is to create the files that will be required to house our component. Our component will be called ***BusinessesComponent,*** so the first step is to create 3 empty files called

- ***businesses.component.ts***,
- ***businesses.component.html*** and
- ***businesses.component.css***

in the **src/app** folder of the **bizFE** application.

Now, we specify the content to be rendered by the component by adding a simple **<h1>** element to the HTML file

File: bizFE/src/app/businesses.component.html

```
<h1>
     Biz Directory
</h1>
```

and create the component TypeScript file by copying and pasting the content from ***app.component.ts*** into our new ***businesses.component.ts*** and changing the selector, template, style and class names to identify the new component as highlighted below.

File: bizFE/src/app/businesses.component.ts

```
import { Component } from '@angular/core';
import { RouterOutlet } from '@angular/router';

@Component({
  selector: 'businesses',
  standalone: true,
  imports: [RouterOutlet],
  templateUrl: './businesses.component.html',
  styleUrl: './businesses.component.css'
})
export class BusinessesComponent { }
```

Next, we use the new selector by adding a **<businesses></businesses>** tag to the ***app.component.html*** file to render our new **BusinessesComponent** component.

File: bizFE/src/app/app.component.html

```
<h1>
     Welcome to {{ title }}
</h1>

<businesses></businesses>
```

If we now examine the browser, we can see that the application fails to display our new content but instead produces an error message in the Terminal window reporting that the **businesses** element is unknown, as shown in Figure 11.5 below.

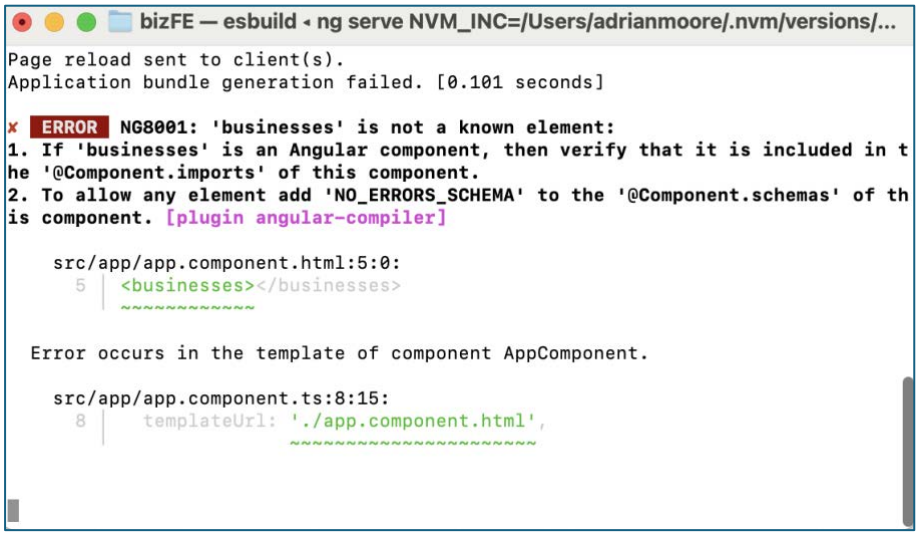


Figure 11.5 Unregistered Component

11.2.2 Connecting a Component to the Application

There are two further things we need to do to register the new component with the application. First, we need to **`import`** the new **`BusinessesComponent`** component into ***app.component.ts,*** as the component is rendered within its template. Then, we need to register the new component by adding it to the **`imports`** list.

File: bizFE/src/app/app.component.ts

```
import { Component } from '@angular/core';
import { RouterOutlet } from '@angular/router';
import { BusinessesComponent } from './businesses.component';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [RouterOutlet, BusinessesComponent],
  templateUrl: './app.component.html',
  styleUrl: './app.component.css'
})
export class AppComponent {
  title = 'FE11';
}
```

Now, when we view the content in the browser, we can see that our new component has been properly rendered on the page, as shown in Figure 11.6 below.



Figure 11.6 New Component Correctly Rendered

Do it now!	Register the **`BusinessesComponent`** with the application as shown above. Verify that you receive output as illustrated in Figure 11.6.

11.3 Angular Data Processing

So far, although our new component is connected to the application, its impact is limited to the display of a simple, static HTML element. We will now enhance the functionality of the component by having it process JSON data that we specify in the controller's TypeScript file.

11.3.1 Manipulating JSON Data

In the **`BusinessesComponent`** class of the ***businesses.component.ts*** file, we define a variable **`business_list`** as a JSON object holding information about 3 business objects. For each business, we specify value for fields "*name*", "*town*" and "*rating*" (reflecting 3 of the fields in our data set from the Biz Directory API application).

File: bizFE/src/app/businesses.component.ts

```
...

export class BusinessesComponent {

    business_list =  [
        {
            "name" : "Pizza Mountain",
            "town" : "Coleraine",
            "rating" : 5
        },
        {
            "name" : "Wine Lake",
            "town" : "Ballymoney",
            "rating" : 3
        },
        {
            "name" : "Sweet Desert",
            "town" : "Ballymena",
            "rating" : 4
        }
    ]

}
```

Now, we add code to the ***businesses.component.html*** file to display the size of the JSON structure (i.e. the number of elements it contains) and the **name** property of the first element.

File: bizFE/src/app/businesses.component.html

```
<h1>
    Biz Directory
</h1>

<h2>
    There are {{ business_list.length }}
    businesses in the directory.
</h2>

<h2>
    The first business is
    {{ business_list[0].name }}
</h2>
```

Examining the browser contents reveals that the information from the JSON structure is now embedded into the code that specifies the web page as shown in Figure 11.7 below.

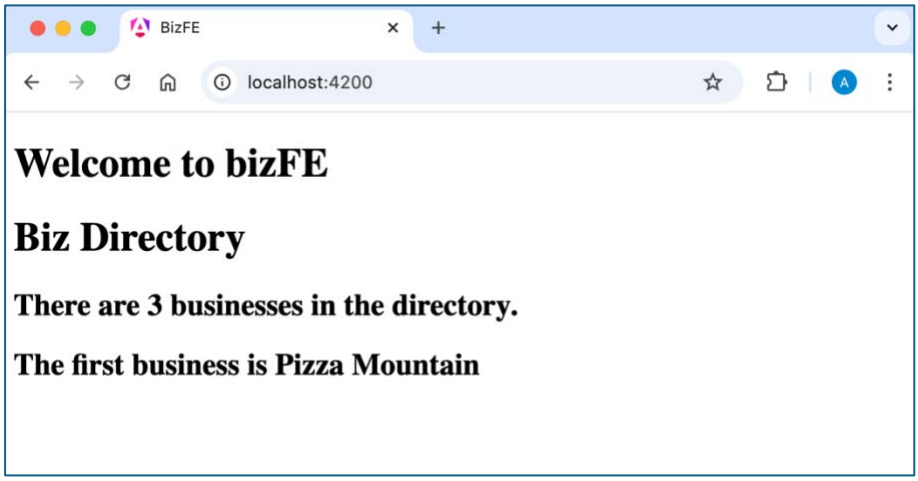


Figure 11.7 Injecting Data into the Web Page

Do it now!	Add the collection of data to ***businesses.component.ts*** and add the code shown above to ***businesses.component.html*** to report on the number of businesses in the collection and the name of the first business.

11.3.2 Iterating Across Data with the `@for` Block

Angular provides a number of very useful control flow blocks that enable us to introduce common programming constructs into our HTML specification. One of the most useful of these is the **`@for`** block, that provides a looping mechanism.

`@for` takes the form "***`variable`*** `of` ***`collection`***", by which the **`variable`** iterates across the **`collection`**, taking on each value in turn. To illustrate this, consider the code box below where the **`@for`** block is applied to a **`<div>`** element that contains a paragraph that presents the *name*, *town* and *rating* properties of each element of **`business_list`** in turn. Note the requirement for the **`track`** element where one of the fields is nominated as the "key" which is then tracked by Angular as it iterates across the collection.

File: bizFE/src/app/businesses.component.html

```
...

@for (business of business_list; track business.name) {
    <p>
        Name: {{ business.name }} <br>
        Town: {{ business.town }} <br>
        Rating: {{ business.rating }}
    </p>
}
```

The effect of this is to generate separate **<p>** elements for each business as shown in Figure 11.8 below.

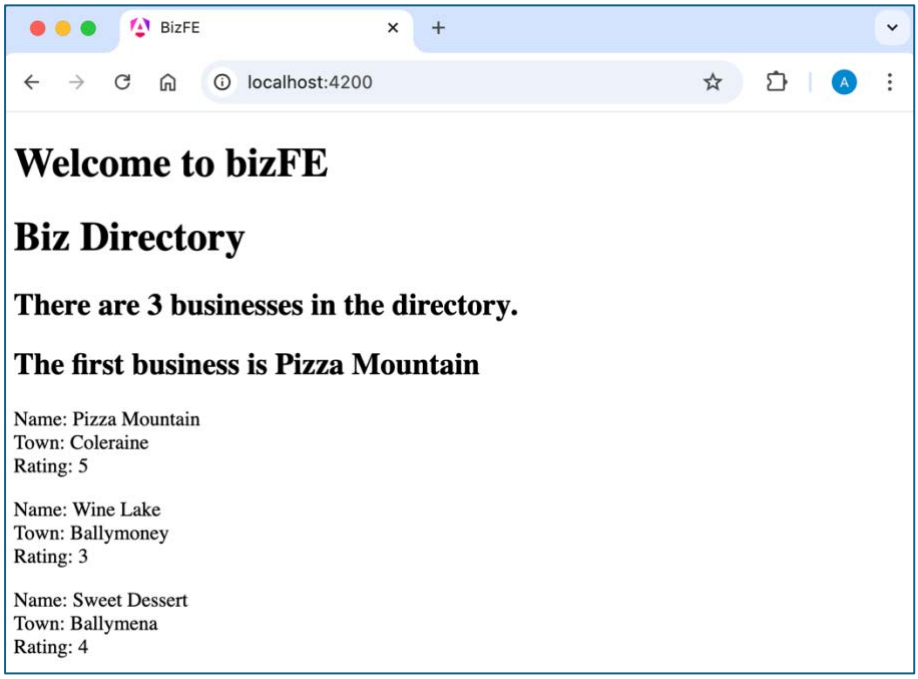


Figure 11.8 Using `@for()` to traverse a dataset

Do it now!	Verify the operation of the `@for` block by adding the code above to ***businesses.component.html*** and make sure that you obtain output such as that shown in Figure 11.8.

11.4 Using Bootstrap

As we build the front-end of the Biz application, we will style the interface using Bootstrap – a popular CSS framework for responsive front-end development. This will provide an easy

way of creating an attractive interface, as well as automatically providing dynamic adjustment for different devices and browser characteristics.

11.4.1 Installing Bootstrap

We could use Bootstrap by linking to a local copy or a CDN, but **npm** also provides it as a package that we can install into our application. To install Bootstrap and its dependent package jQuery, kill any currently running Angular server (using CTRL-C), navigate back to the **bizFE** directory and issue the following command to install the packages using the **Node Package Manager** (npm). (Note – the following command should be issued all on a single line.)

```
C:\Users\B00123456\Desktop\bizFE> npm install bootstrap
jquery --save
```

Now, we need to tell the Angular application that the Bootstrap CSS library is available by adding it to the list of global stylesheets. Open the file **/angular.json** in the code editor and locate the **`styles`** entry (Inside the `projects|FE11|architect|build|options` object). Now add the location of the file ***bootstrap.min.css*** to the styles entry as shown below.

File: bizFE/angular.json

```
      ...

         "styles": [
             "src/styles.css",
             "node_modules/bootstrap/dist/css/bootstrap.min.css"
         ],

      ...
```

We can check that Bootstrap has been installed by re-running the application (with **ng serve**), exploring the **node_modules** folder in our IDE and navigating to **node_modules/bootstrap/dist/css** to check that the Bootstrap files are now present.

Note:	You can also see visually that Bootstrap has been installed by simply observing the change in font and layout of your application contents in the browser.

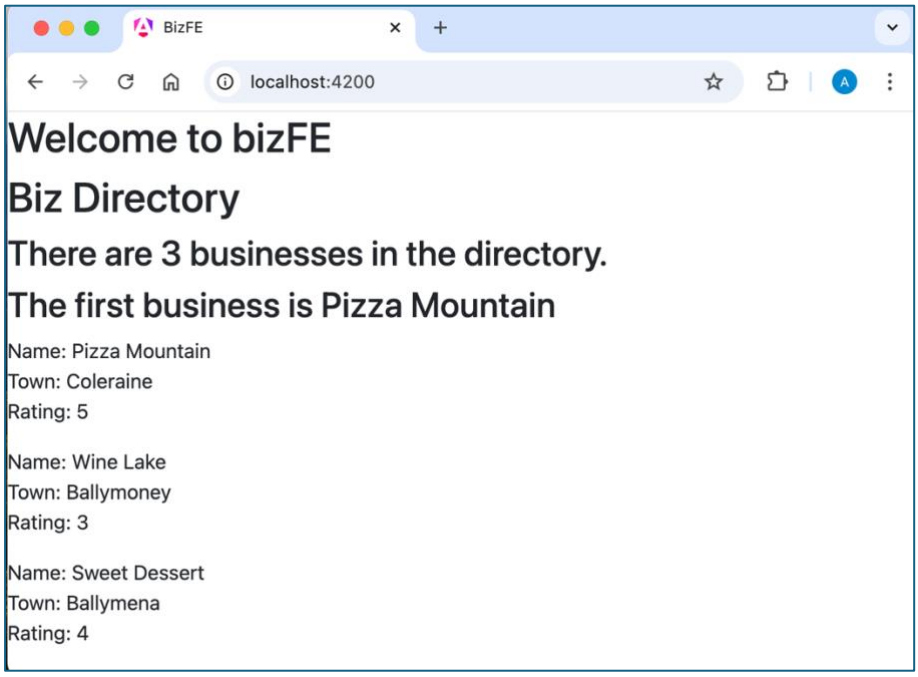


Figure 11.9 Verifying that Bootstrap is installed

Do it now!	Install Bootstrap and verify that it is available to the application by following the steps above.

11.4.2 Styling with Bootstrap

As a quick example of Bootstrap styling, we will present our JSON data as Bootstrap **`card`** elements. A Bootstrap card is a flexible and extensible content container. It includes options for headers and footers, a wide variety of content, contextual background colors, and powerful display options. The card is specified by applying style classes to **`<div>`** elements that define the card and its optional header, body and footer components as shown below.

```
<div class = ”card”>
     <div class = “card-header”>
          <!-- the card header text -->
     </div>

     <div class = “card-body”>
          <!-- the card body text -->
     </div>

     <div class = “card-footer”>
          <!-- the card footer text -->
     </div>
</div>
```

The remaining CSS classes applied (e.g. **`text-white`**, **`bg-primary`**, etc.) are Bootstrap classes to set text and background colours. A full specification of Bootstrap cards can be seen at https://getbootstrap.com/docs/5.3/components/card/. The code box below shows the full revised specification for *businesses.component.html*.

File: bizFE/src/app/businesses.component.html

```
<div class="container">
    <div class="row">
        <div class="col-sm-12">
           @for(business of business_list;
                track business.name) {
                <div class="card text-white bg-primary mb-3">
                     <div class="card-header">
                          {{ business.name }}
                     </div>
                     <div class="card-body">
                          This business is based in
                          {{ business.town }}
                     </div>
                     <div class="card-footer">
                          Rating: {{ business.rating }}
                     </div>
                </div>
           }
        </div> <!-- col -->
    </div>    <!-- row -->
</div>      <!-- container -->
```

Finally, we remove all other code from ***app.component.html*** so that the entire presentation consists of our own work in the **`businesses`** component.

File: bizFE/src/app/app.component.html

```
<businesses></businesses>
```

The effect of these changes can be seen in Figure 11.10, below.

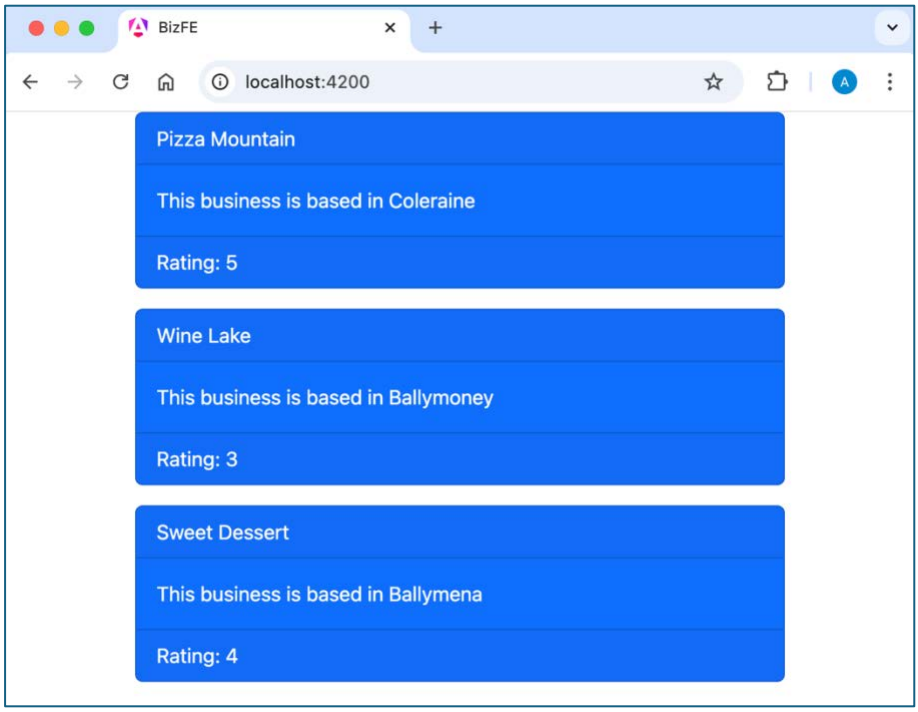


Figure 11.10 Using Bootstrap Cards

Do it now!	Apply Bootstrap styling to the application by following the instructions set out in this sub-section. Ensure that you obtain output as illustrated by Figure 11.10 above.

Try it now!	Add more businesses to the data set and experiment with Bootstrap and Cards to generate the most attractive layout with cards displayed 3 or 4 to a row.

11.5 npm and Package Management

In the previous section, we introduced the Node Package Manager (npm) as a tool for the installation of external components into our Angular application. npm is a **Node.js** tool that allows us to define and manage dependencies between code modules, libraries and packages. This helps promote the notion of re-usable code, where members of the Node.js community develop and share useful packages that others can include in their projects.

All npm-based applications contain a file, usually in the project root, called ***package.json***. This file holds various metadata relevant to the project and is used to give information to npm that allows it to identify the project as well as handle the project's dependencies. It can also contain other metadata such as a project description, the version of the project in a particular distribution, license information and configuration data.

Do it now!	Examine the file ***/package.json*** and see how it contains the specification of all of the components in our application. Note the entries for **bootstrap** and **jquery** that were added to ***package.json*** by our use of the **`npm install`** command in the previous section.

Note:	When using **`npm install`**, the **`--save`** flag is the command that tells npm to add entries for the components being added to *package.json*. Without specifying **`--save`**, the components would be added, but *package.json* would remain unchanged.

11.5.1 Understanding Version Numbers

npm packages are identified using the **Semantic Version Specification** (SemVer), which expresses software versions as three values identified as the **Major** version, the **Minor** version and the **Patch** version. If you examine the entry in ***package.json*** for Bootstrap, you will see that it is in the form

```
"bootstrap": "^5.3.3",
```

which denotes Major version **5**, Minor version **3** and Patch version **3**. In the SemVer scheme, given a version number in the format Major.Minor.Patch, developers should increment the

- MAJOR version when a change results in previous versions of the API being incompatible
- MINOR version when new functionality is added in a backwards compatible manner (i.e. existing software will still work)
- PATCH version when changes are backwards-compatible bug fixes

In other words, a user of a package should always be able to safely upgrade to the latest Patch or Minor version without fear of incompatibility but upgrading a Major version (i.e. to Bootstrap 6.x.x) may cause existing functionality to break and should only be done after thorough testing.

The use of the **^** symbol in the ***package.json*** entry

```
"bootstrap": "^5.3.3",
```

instructs npm that although it may automatically download the latest Patch and Minor versions as they become available, it should not automatically upgrade to any new Major version that may be released. This should future proof our application against updates to any of the components that we use.

11.5.2 Backup and Restore of the Angular Application

One of the most useful features of ***package.json*** is that it allows us to store or transfer a greatly reduced subset of the application code by first deleting the **node_modules** folder that contains all of the supporting packages, leaving only the application structure and our own code files.

If you examine the **biz** folder for our current application, you will find that it occupies in the region of 350 Mbytes across approximately 30,000 files. However, if you then examine the **node_modules** folder, you will find that it accounts for almost all of this bulk. Since ***package.json*** is essentially a script that tells npm how to recreate **node_modules**, we can therefore safely delete the entire **node_modules** folder.

When we want to restore the application, we simply use the command **`npm install`** from the application directory (the same directory where ***package.json*** is found) which opens ***package.json***, creates a **node_modules** folder and installs all of the components listed in ***package.json*** into it.

Do it now!	Stop the Angular server and delete the folder **node_modules** from the **biz** application. Then, from a Command prompt in the **bizFE** folder, issue the command **`npm install`** to re-create the application. Once installed, run it by the command **`ng serve`** and verify that everything works as previously.

11.5 Further Information

- https://angular.io/
  Angular home page

- https://www.youtube.com/watch?v=UeRaz971X4U
  How to Install Angular 18 on Windows 10/11

- https://kinsta.com/knowledgebase/install-angular/
  How To Install Angular on Windows, macOS, and Linux

- https://angular.dev/tutorials/learn-angular
  Getting Started with Angular

- https://angular.dev/guide/components
  Angular – Introduction to Components

- https://www.typescriptlang.org/
  TypeScript home page

- https://nodejs.org/en/
  Node.Js home page

- https://nodesource.com/blog/an-absolute-beginners-guide-to-using-npm/
  An Absolute Beginner's Guide to Using npm

- https://semver.org/
  Semantic Versioning

- https://getbootstrap.com/
  Bootstrap home page

- https://getbootstrap.com/docs/5.3/layout/grid/
  The Bootstrap Grid System – containers, rows and columns

- https://getbootstrap.com/docs/5.3/components/card/
  Bootstrap cards

- https://getbootstrap.com/docs/5.3/utilities/colors/
  Bootstrap colours

COM661 Full Stack Strategies and Development

FE12. Building a Data Service

Aims

- To demonstrate external JSON data files in an Angular application.
- To introduce Angular Services as a means of providing general purpose functionality to an application
- To create a first Data Service
- To demonstrate injecting a Service into a Component
- To use the data returned from a Data Service in the front end of an Angular application
- To build a basic pagination interface
- To implement browser storage using the Local Storage and Session Storage objects

Table of Contents

12.1 IMPORTING JSON DATA 2

12.1.1 Application Assets 2
12.1.2 Importing JSON 2

12.2 USING A DATA SERVICE 4

12.2.1 Creating the Data Service 5
12.2.2 Connecting to the Data Service 5

12.3 FRONT-END PAGINATION 8

12.3.1 Specifying the Slice to be Taken 8
12.3.2 Providing Front-end Navigation 10
12.3.3 Adding Component Functionality 12

12.4 USING BROWSER STORAGE 18

12.4.1 HTML5 Data Storage 18
12.4.2 Using Session Storage to Improve Pagination 18

12.5 FURTHER INFORMATION 21

12.1 Importing JSON Data

JSON (JavaScript Object Notation) is a text-based format for representing structured data. It is very often used for communicating data between web servers and clients and is particularly useful in our work because of the close relationship to the format of the MongoDB database architecture that we will integrate to the front-end soon. In this practical, we will show how JSON data files can be used as the secondary storage for our applications and build a Data Service that provides information storage and retrieval functionality.

12.1.1 Application Assets

The assets of a software application are those elements other than the program code that decorate or otherwise support the functionality. They might include videos, images, stylesheets or other contextual information, but in this case, we will explore how our **bizFE** application can be populated with an external collection of business information specified by a JSON data file. As a first step, lets create a new Angular application to investigate the import of JSON data.

Do it now!	Create a new Angular app called **FE12** by running the command **`ng new FE12`** from a terminal prompt. Remember that due to the volume of data generated, it is better to navigate to a non-networked drive (such as the desktop on the lab PCs) and create the new application there.

12.1.2 Importing JSON

In order to maintain the theme of working with business data, we will create a JSON data structure that reflects the database previously generated. Using the **mongoexport** tool, we can generate a JSON data file that reflects the current contents of the **biz** collection in our **bizDB** database and use it as the basis for our experiments. We will then create an **assets** sub-folder within the **src** folder of our new application and copy the file of businesses data to there.

Do it now!	Obtain a JSON representation of your MongoDB **biz** collection by using **mongoexport** to write the content of the collection to the file ***businesses.json***. Create a folder **assets** as a sub-folder of **bizFE/src** and place the file ***businesses.json*** into the assets folder.

Note:	Where you have different types of assets in your application, it may be useful to organize these in sub-folders of **assets** such as **assets/data**, **assets/images**, **assets/logos**, **assets/videos** etc. In such cases, the path in the **`import`** statement would change accordingly, for example **`import jsonData from '../assets/data/businesses.data'`**

Now that our database collection is represented in an external JSON file, we can demonstrate how to read the contents of that file into a Python variable. In many languages we would be required to read the file line by line and re-build the structure, but as the file contains a single JSON structure (a JSON array), we are able to import it in its entirety in a single statement. This is illustrated in the code box below, where the contents of the file ***src/assets/businesses.json*** is read into the Python variable **`jsonData`** which can then by output to the browser console by a **`console.log()`** statement. Note the definition of the **`ngOnInit()`** method within the **`AppComponent`** class. This is a method that is called automatically by Angular once a Component has been initialized and is often used to initialize variables or to set up connections (for example to back-end APIs).

File: FE12/src/app/app.component.ts

```
import { Component } from '@angular/core';
import { RouterOutlet } from '@angular/router';
import jsonData from '../assets/businesses.json';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [RouterOutlet],
  templateUrl: './app.component.html',
  styleUrl: './app.component.css'
})
export class AppComponent {

    ngOnInit() {
       console.log(jsonData);
    }

}
```

Do it now!	Modify ***app.component.ts*** as shown above to import the JSON data file from the **assets** folder and log the contents to the browser console. Verify that the data has been imported by running the application with **`ng serve`**, visiting http://localhost:4200 in the web browser, and opening the Browser Console. Check that you see data such as that in Figure 12.1, below.

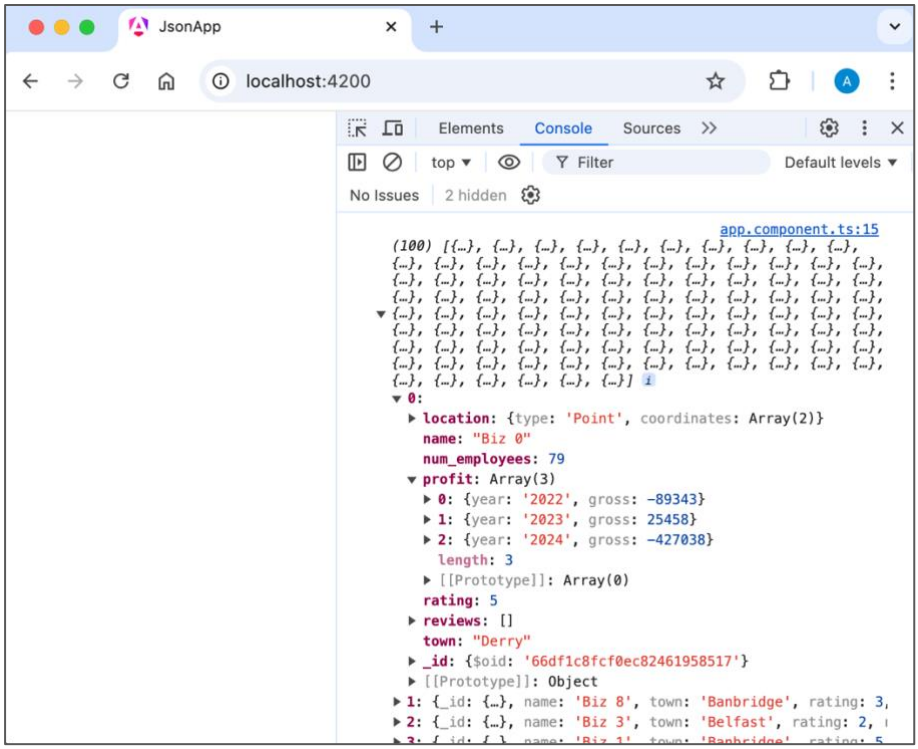


Figure 12.1 JSON Data in the Browser Console

Note:	It is a good idea to keep the Browser Console open when developing Angular applications as it is often the source of helpful error messages. In addition, use of the **`console.log()`** statement in your code can help you trace the flow of execution and trace the source of bugs.

12.2 Using a Data Service

Components in Angular relate to elements that will have a physical presence in the application and that are incorporated into the application by using the tag defined as the component selector (for example **`<businesses></businesses>`** in our **bizFE**

application). They comprise of a chuck of HTML, CSS and TypeScript that encapsulates some part of the application that we want to display.

Angular **Services** on the other hand, are used for common functionality that can be used across multiple parts of the application. They can be injected into any Component that requires that functionality so that the operation only has to be defined once. In this section, we introduce Angular Services by creating a Data Service that provides access to the collection of businesses.

12.2.1 Creating the Data Service

The Service is specified as a simple TypeScript class containing methods that provide the functionality that we want to make available. In this example, we return to our **bizFE** application, import the ***businesses.json*** collection as previously, and provide a Data Service method **`getBusinesses()`** that returns the imported data as a JSON array.

File: bizFE/src/app/data.service.ts

```
import jsonData from '../assets/businesses.json'

export class DataService {

    getBusinesses() {
        return jsonData;
    }
}
```

Do it now!	Close the **FE12** application and stop the Angular application with CTRL-C. In your **bizFE** application, create the new folder **src/assets** and copy your file ***businesses.json*** to the new folder. Then, create the new file ***src/app/data.service.ts*** as shown in the code box above.

12.2.2 Connecting to the Data Service

In order to use the new Service, we need to import into the relevant component and inject it into the component class. This is carried out in three stages as follows.

First, the Service is imported into the Component.

File: bizFE/src/app/businesses.component.ts

```
import { Component } from '@angular/core';
import { RouterOutlet } from '@angular/router';
import { DataService } from './data.service';
...
```

Next, we include the Service in the providers list within the Component decorator.

File: bizFE/src/app/businesses.component.ts

```
...

@Component({
  selector: 'businesses',
  standalone: true,
  imports: [RouterOutlet],
  providers: [DataService],
  templateUrl: './businesses.component.html',
  styleUrl: './businesses.component.css'
})}

...
```

Finally, we inject the Service into the Component (i.e. make it available for use) by providing it as a parameter to the constructor method of the Component. Here, we are making available an instance of the **`DataService`** under the name **`dataService`**. The private specifier demotes that the **`dataService`** object will only be available within the current class definition.

File: bizFE/src/app/businesses.component.ts

```
...

export class BusinessesComponent {
    ...

    constructor(private dataService: DataService) { }

    ...
}
```

Now that the Service is available, we can use it by re-defining the local Component variable **`business_list`** as an empty object (rather than the previous hard-coded list of businesses) and populating it from the values returned from the **`DataService`** by making a call to the **`DataService`** method **`getBusinesses()`** from within the Component's **`ngOnInit()`** method.

The full modified listing of ***businesses.component.ts*** is provided for convenience below.

File: bizFE/src/app/businesses.component.ts

```
import { Component } from '@angular/core';
import { RouterOutlet } from '@angular/router';
import { DataService } from './data.service';

@Component({
  selector: 'businesses',
  standalone: true,
  imports: [RouterOutlet],
  providers: [DataService],
  templateUrl: './businesses.component.html',
  styleUrl: './businesses.component.css'
})}

export class BusinessesComponent {
    business_list: any;

    constructor(private dataService: DataService) { }

    ngOnInit() {
       this.business_list = this.dataService.getBusinessea();
    }
}
```

Now, when we run the **bizFE** application, the browser should display all of the business data returned from the Data Service.

Do it now!	Connect the **`BusinessesComponent`** to the **`DataService`** as described in the steps above. Run the application and make sure that the browser displays the complete collection of business data as a stack of Bootstrap cards, as illustrated in Figure 12.2, below.

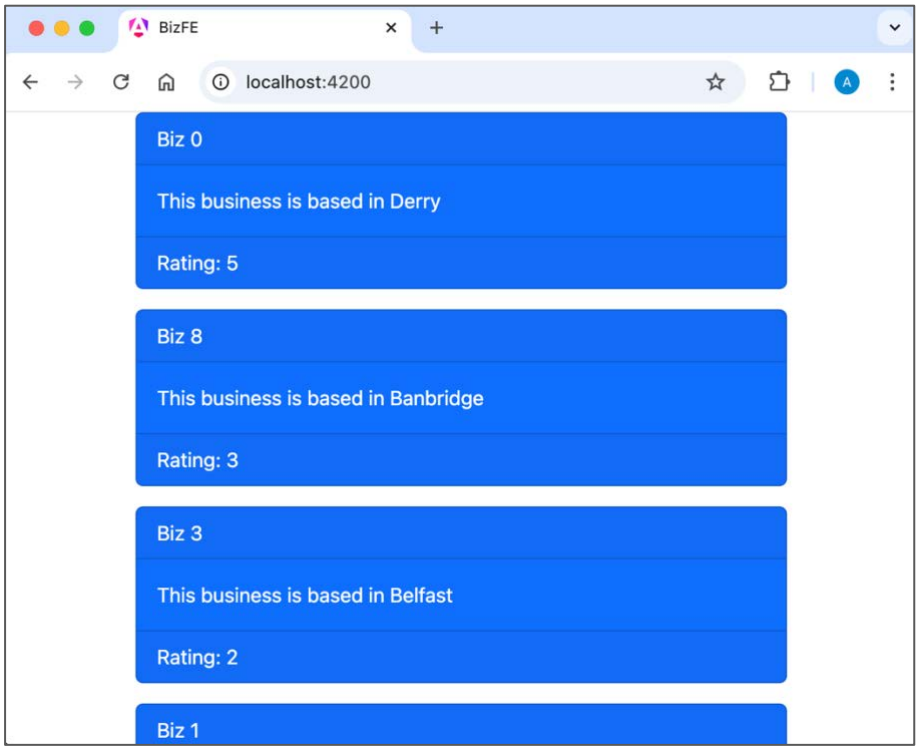


Figure 12.2 Data Service Provides Business Data

12.3 Front-end Pagination

The initial version of our **`DataService`** returns the entire collection as a single list. Depending on our application and data, this might be too much information to display within our interface in a single view, so it may be beneficial to paginate the data so that only a slice at a time is retrieved.

Pagination can be provided either at the front-end, the back-end, or both. We have already implemented back-end pagination in our API, it can be beneficial to have all data available to the front-end and paginate there to make it easier to implement searching and filtering with minimal network traffic.

12.3.1 Specifying the Slice to be Taken

We will modify our **`DataService`** so that only a specified portion of the dataset is returned, controlled by two values. The **`DataService`** local variable **`pageSize`** will specify how many businesses are to be returned, while the **`getBusinesses()`** method will accept a parameter called **`page`** which determines the page of businesses required.

Then, in **`getBusinesses()`** we calculate the values **`pageStart`** and **`pageEnd`**, and return the slice of the JSON array that is bounded by these values.

File: bizFE/src/app/data.service.ts

```
import jsonData from '../assets/businesses.json'

export class DataService {
    pageSize: number = 3;

    getBusinesses(page: number) {
        let pageStart = (page - 1) * this.pageSize;
        let pageEnd = pageStart + this.pageSize;
        return jsonData.slice(pageStart, pageEnd);
    }
}
```

Now, we can update the **`BusinessesComponent`** to include a local variable page that will store the page number currently being displayed and modify the call to the **`DataService getBusineses()`** method so that the desired page is selected. As the **`BusinessesComponent`** HTML template will automatically display all data that is returned, no updates to the template are required at this stage.

File: bizFE/src/app/businesses.component.ts

```
...

export class BusinessesComponent {
    business_list: any;
    page: number = 1;

    constructor(private dataService: DataService) { }

    ngOnInit() {
       this.business_list =
            this.dataService.getBusinessea(this.page);
    }
}
```

Running the application now should confirm that only the first page of three businesses is retrieved and displayed.

Do it now!	Modify the **`BusinessesComponent`** and **`DataService`** as shown to have the application display the first page of 3 businesses. Verify that you receive output such as that shown in Figure 12.3, below.

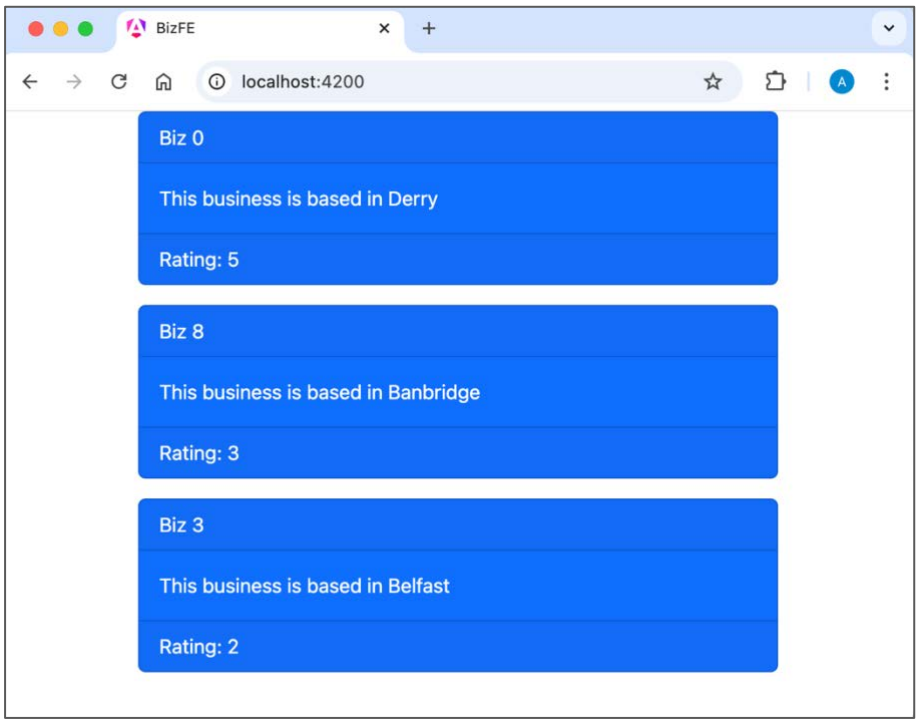


Figure 12.3 A Page of Three Businesses Returned

Try it now!	Verify the effect of the **page** variable in the Businesses Component and the **`pageSize`** property in the Data Service by assigning them different values and observing the effect in the browser.

12.3.2 Providing Front-end Navigation

The next step is to add a pair of buttons to the **`BusinessesComponent`** HTML template to trigger the previous and next pages of data to be displayed. Note the **`(click)`** notation that assigns the event handler to the button. This is the Angular equivalent to the familiar JavaScript **`onClick`** keyword.

File: bizFE/src/app/businesses.component.html

```
...

  <div class="row">
     <div class="col-sm-6">
        <button class="btn btn-primary"
                (click)="previousPage()">Previous</button>
     </div>
     <div class="col-sm-6 text-end">
        <button class="btn btn-primary"
                (click)="nextPage()">Next</button>
     </div>
  </div>

</div>      <!-- container -->
```

Since we have specified calls to functions **`previousPage()`** and **`nextPage()`**, we need to provide(initially) empty definitions for these in the component TypeScript file.

File: bizFE/src/app/businesses.component.ts

```
...

export class BusinessesComponent {
  ...

  previousPage() {
  }

  nextPage() {
  }

}
```

The application should now run, and we can verify that the buttons are added and are aligned to the left and right of the stack of business cards as shown in Figure 12.4 below.

Do it now!	Follow the steps above to add “Previous” and “Next” buttons to the end of the **`BusinessesComponent`** HTML template

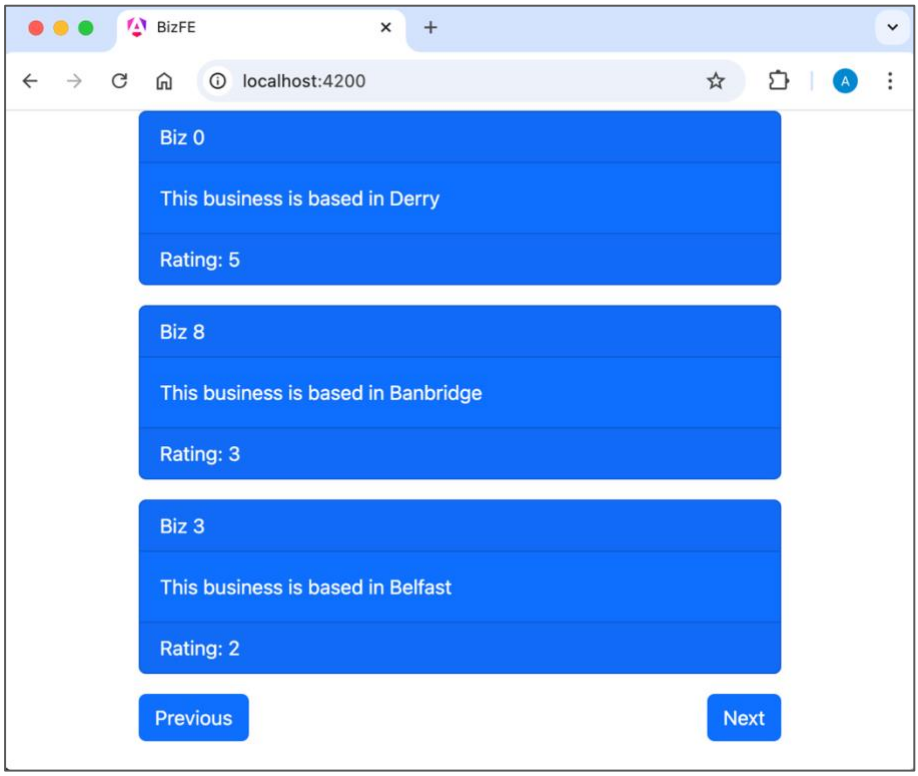


Figure 12.4 Adding Pagination Buttons

12.3.3 Adding Component Functionality

Now, we add the **`nextPage()`** and **`previousPage()`** methods to move forward or backward through the collection of business data. In each case, all that is required is to change the value of page accordingly (by either adding or subtracting 1) and then to call the **`DataService`** function that retrieves the data. As each of **`nextPage()`** and **`previousPage()`** request a new slice of data from **`getBusinesses()`** at the end of each call, the display will automatically change each time a new page of data is fetched.

Do it now!	Add the pagination functionality by providing code for **`previousPage()`** and **`nextPage()`** as shown below. Verify that the **Previous** and **Next** buttons now allow the user to navigate forwards and backwards through the dataset.

Try it now!	Attempt to navigate past the boundaries of the dataset by requesting the previous page when the first is being displayed or the next page when the last is being displayed.

File: bizFE/src/app/businesses.component.ts

```
...

export class BusinessesComponent {
  ...

  previousPage() {
    this.page = this.page – 1
    this.business_list =
                this.dataService.getBusinesses(this.page);
  }

  nextPage() {
    this.page = this.page + 1
    this.business_list =
                this.dataService.getBusinesses(this.page);
  }

}
```

When scrolling forward and backwards through the pages of data, we notice a problem when we attempt to go back past the first page or forward past the last page. In these cases, there is nothing to prevent the Data Service from attempting to take a slice of data from outside the boundaries of the array. We need to fix these edge cases by testing the page requested against the range of pages available. In the case of the "Previous" button, it is easy – we only allow the previous page to be selected if a page other than the first is currently being displayed.

File: bizFE/src/app/businesses.component.ts

```
...

export class BusinessesComponent {
  ...

  previousPage() {
    if (this.page > 1) {
       this.page = this.page – 1
       this.business_list =
                this.dataService.getBusinesses(this.page);
    }
  }
  ...
}
```

For the “Next” button, the concept is equally simple – do not allow navigation past the last page – but how can we tell which page is the last? If we know the number of elements in the dataset then it is straightforward, but in an application where the number of elements may change over time, we need additional support.

One technique is to add functionality to the **`DataService`** to return the number of the last page of data currently stored in the collection. The calculation is to divide the length of the dataset by the size of each page and to return that result – adding 1 if there is a remainder in the division. Hence for a page size of 10, a dataset size of 20 will require two pages, which a dataset of between 21-29 will require an extra page to hold those after the first two.

TypeScript provides a useful function **`Math.ceil()`** that performs this calculation by rounding up the result of a division to the next whole number and we can use it in a new **`DataService`** method **`getLastPageNumber()`** that returns the number of pages available in the dataset.

File: bizFE/src/app/data.service.ts

```
import jsonData from '../assets/businesses.json'

export class DataService {
    pageSize: number = 3;

    getBbusinesses(page: number) {
        let pageStart = (page - 1) * this.pageSize;
        let pageEnd = pageStart + this.pageSize;
        return jsonData.slice(pageStart, pageEnd);
    }

    getLastPageNumber() {
        return Math.ceil( jsonData.length / this.pageSize );
    }
}
```

With the new **`DataService`** capability in place, we can return to the **`BusinessesComponent`** and add the check for the end of the dataset to the **`nextPage()`** method.

File: bizFE/src/app/businesses.component.ts

```
...

export class BusinessesComponent {
  ...

  nextPage() {
    if (this.page < this.dataService.getLastPageNumber()) {
       this.page = this.page + 1
       this.business_list =
                  this.dataService.getBusinesses(this.page);
    }
  }

}
```

Do it now!	Add the pagination control logic by making the amendments to the **`DataService`** and the **`BusinessesComponent`** as shown above. Verify that the **Previous** button no longer has any effect when the first page is being displayed and that the **Next** button has no effect when the last page is being displayed.

An additional useful modification would be to add code to the **`BusinessesComponent`** HTML template to display the current page number as well as the number of pages available. We can make use of the space between the “Previous” and “Next” buttons by converting the Bootstrap row to one of three columns and displaying a message incorporating the current value of the Component **`page`** variable as well as the result of a call to the **`DataService getLastPageNumber()`** method.

File: bizFE/src/app/businesses.component.html

```
      ...

        <div class="row">
           <div class="col-sm-4">
              <button class="btn btn-primary"
                       (click)="previousPage()">Previous</button>
           </div>
           <div class="col-sm-4 text-center align-self-center">
              Showing page {{ this.page }}
              of {{ this.dataService.getLastPageNumber() }}
           </div>
           <div class="col-sm-4 text-end">
              <button class="btn btn-primary"
                       (click)="nextPage()">Next</button>
           </div>
        </div>

      </div>      <!-- container -->
```

Running the application now reveals an error as the **`dataService`** object was declared to be private when it was injected into the **`BusinessesComponent`**, as seen in Figure 12.5, below

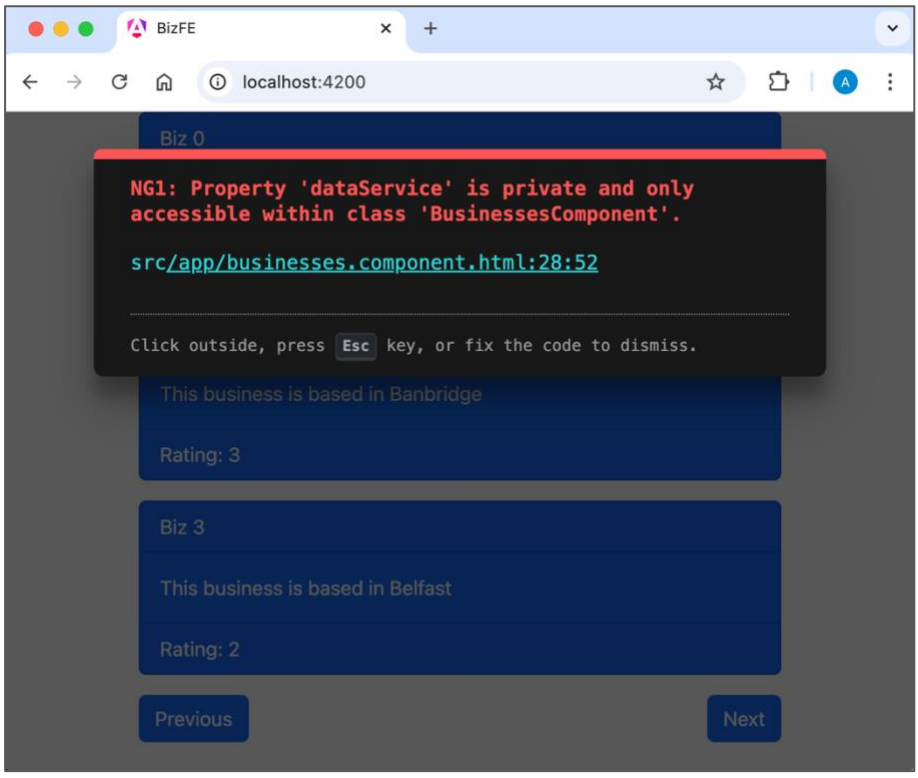


Figure 1256 **`DataService`** *is Private*

However, this is easily addressed by returning to the definition of the **`BusinessesComponent constructor()`** and changing the **`dataService`** definition to be public instead, thus allowing it to be used within the Component HTML template.

File: bizFE/src/app/businesses.component.ts

```
...

export class BusinessesComponent {
    business_list: any;
    page: number = 1;

    constructor(public dataService: DataService) { }

    ...
}
```

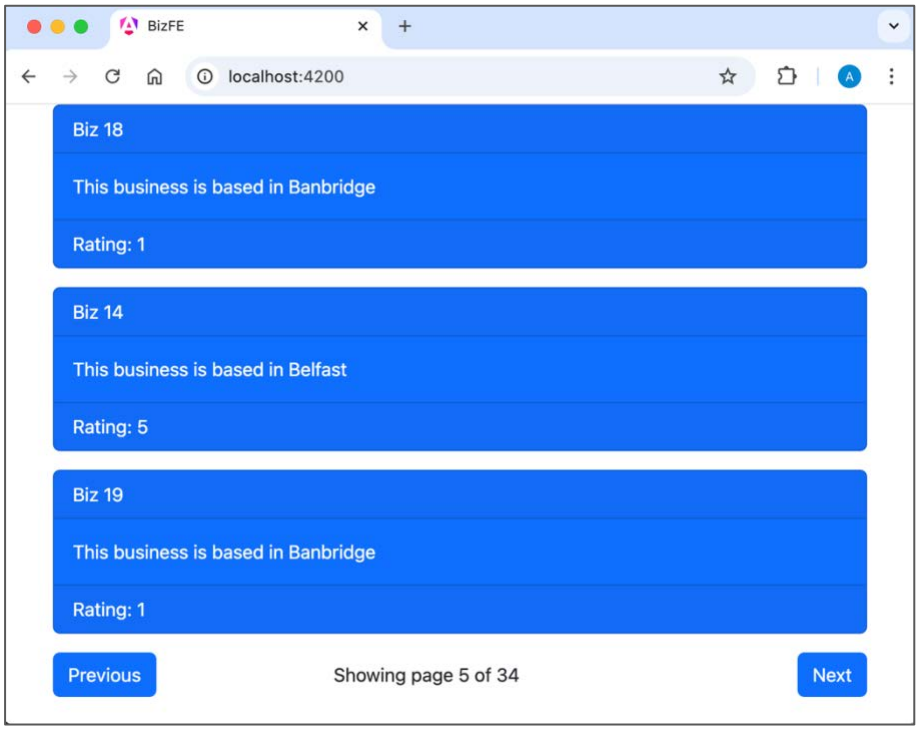


Figure 12.6 Display Pagination Status Information

Do it now!	Add the pagination status information by making the changes shown above and verify that it displays the page currently being viewed and the number of pages available, as seen in Figure 12.6, above. Change the **`pageSize`** property in the **`DataService`** and observe the effect on navigation.

12.4 Using Browser Storage

When we try the pagination in the browser it appears to work as expected. We are able to move forward and backward through the data one page at a time. However, if we refresh the browser, we are returned to the first page of information – not the page that we most recently visited. This is because the **`BusinessesComponent`** is re-initialised every time it is loaded, resetting the value of page to 1.

12.4.1 HTML5 Data Storage

HTML5 provides **`localStorage`** and **`sessionStorage`** as repositories for data on the client. The information is never passed to the server and can be used when we want to programmatically maintain the state of an application. Data stored in **`localStorage`** has no expiry date – it is not deleted when the browser is closed and will be available at any point in the future, while data in **`sessionStorage`** is removed when the browser tab is closed.

Do it now!	Illustrate the bug in our current pagination scheme by navigating to a page other than the first and refreshing the browser. Observe how the re-initialisation of the **`BusinessesComponent`** reverts to displaying the first page of information.

12.4.2 Using Session Storage to Improve Pagination

We can improve the usability of our application by using **`sessionStorage`** to store the page currently being displayed. Every time that the **`nextPage()`** or **`previousPage()`** methods are invoked, the new version of **`page`** is copied to **`sessionStorage`**. Then, when the component is loaded and **`ngOnInit()`** runs, we can check for the presence of a **`page`** value in **`sessionStorage`** and restore the value of **`page`** (first making sure that it is interpreted as a numeric value and not a string) if a previous value exists.

The modifications to the **`BusinessesComponent`** TypeScript file are presented in the following code box.

File: bizFE/src/app/businesses.component.ts

```
...

export class BusinessesComponent {
  ...

  ngOnInit() {
    if (sessionStorage['page']) {
       this.page = Number(sessionStorage['page']);
    }
    this.business_list =
            this.dataService.getBusinessea(this.page);
  }

  previousPage() {
    if (this.page > 0) {
       this.page = this.page – 1
       sessionStorage['page'] = this.page;
       this.business_list =
                  this.dataService.getBusinesses(this.page);
    }
  }

  nextPage() {
    if (this.page < this.dataService.getLastPageNumber()) {
       this.page = this.page + 1;
       sessionStorage['page' = this.page;
       this.business_list =
                   this.dataService.getBusinesses(this.page);
    }
  }
}
```

Now, running the application with the browser console open and displaying the application storage allows us to track the value of page that is maintained in **`sessionStorage`** as the application runs.

Do it now!	Add the code shown above to ***businesses.component.ts*** to maintain a copy of the current page number in the browser Session Storage. Now run the application with the browser console open and observe how the value for **page** in Session Storage updates as you navigate through the pages as illustrated in Figure 12.6, below. Observe also how refreshing the browser causes the last stored version of page to be retrieved so that the user is presented with the most recently viewed page.

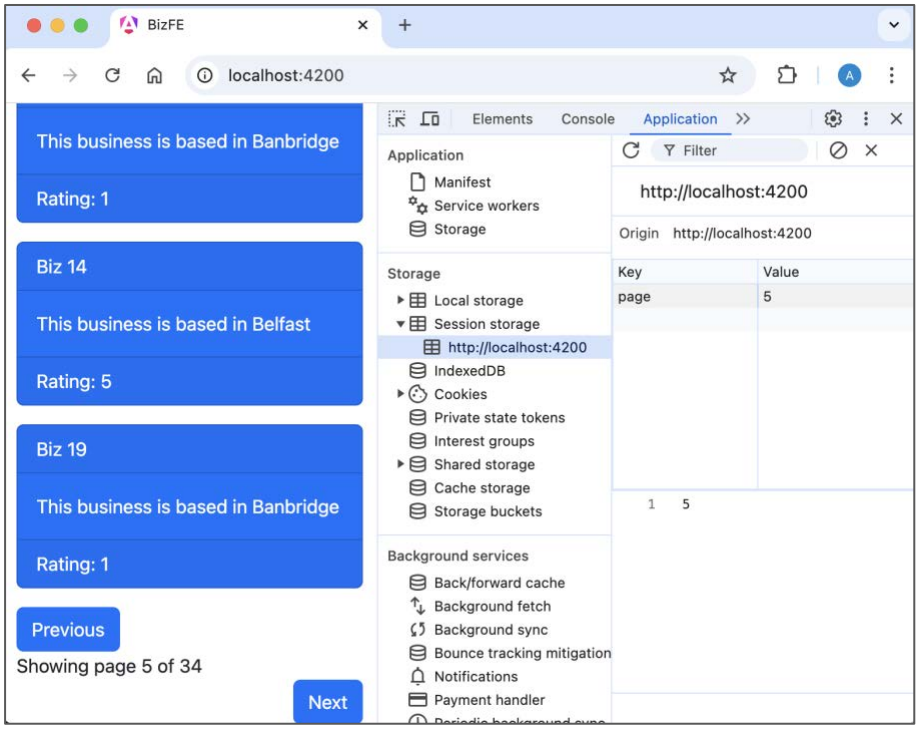


Figure 12.7 Showing the Page Number in Session Storage

Note:	There may be occasions when reverting to the first page of data is the desired response. For example, if the user navigates away from the directory of businesses and then returns to it, you may prefer them to be returned to the first page. This is easily achieved by either deleting the value in Session Storage or re-initialising it to 1 whenever you want the pagination to be reset.

12.5 Further Information

- https://www.thirdrocktechkno.com/blog/how-to-read-local-json-files-in-angular/
  How to read local JSON files in Angular

- https://developer.chrome.com/docs/devtools/console
  Using the Browser Console

- https://angular.dev/tutorials/first-app/09-services
  Angular Services

- https://angular.dev/guide/di
  Dependency Injection in Angular

- https://angular.io/guide/architecture-components
  Angular – Introduction to Components

- https://angulardive.com/blog/angular-services-vs-components-understanding-when-to-use-each/
  Angular Services vs Components: Understanding When to Use Each

- https://angular.dev/tutorials/learn-angular/21-constructor-based-di
  Constructor-based Dependency Injection

- https://angular.dev/api/core/OnInit
  Using **`ngOnInit()`**

- https://www.w3schools.com/bootstrap/bootstrap_buttons.asp
  Bootstrap Buttons

- https://www.tutorialrepublic.com/html-tutorial/html5-web-storage.php
  HTML5 Web Storage

- https://www.geeksforgeeks.org/what-are-the-different-types-of-storage-in-html5/
  Different Types of Storage in HTML5

- https://medium.com/@hassaanali.dev/efficient-data-pagination-in-angular-implementing-smooth-and-performant-pagination-for-large-data-637fe994bbbf
  Efficient Data Pagination in Angular

COM661 Full Stack Strategies and Development

FE13. Additional URLs and Routing

Aims

- To introduce another component and route
- To extend the existing Data Service
- To introduce the Angular RouterModule
- To support multiple URLs in a single app via the RouterModule
- To define a Navigation Component
- To explore the Bootstrap navbar classes
- To integrate the navbar into the existing application

Table of Contents

13.1 ADDING A NEW COMPONENT 2

13.1.1 A HOME COMPONENT 2
13.1.2 ROUTING IN AN ANGULAR APPLICATION 3

13.2 VIEWING A SINGLE DATA ITEM 5

13.2.1 CREATING THE NEW COMPONENT 5
13.2.2 EXTENDING THE DATA SERVICE 6
13.2.3 ADDING THE NEW ROUTE 7
13.2.4 PROVIDING THE FUNCTIONALITY 8
13.2.5 SPECIFYING THE FRONT-END OF THE NEW COMPONENT 10

13.3 CONNECTING THE COMPONENTS 12

13.4 ADDING A NAVIGATION BAR 14

13.4.1 CREATE THE NAVIGATION COMPONENT 14
13.4.2 CONNECT THE NAVIGATION COMPONENT TO THE APPLICATION 17

13.5 FURTHER INFORMATION 19

13.1 Adding a New Component

Although one of the main principles of Angular is that apps are single page in nature, this essentially means that the browser is never completely re-loaded. Even though the application supports multiple URLs to provide different views of the information being delivered, this means that different chunks of HTML, CSS and TypeScript are loaded into the existing page to reflect the currently active Component. In this section, we will create a plain page to use as the homepage for our application and then see how to specify different URLs to navigate between it and the previously implemented page that displays details of businesses.

13.1.1 A Home Component

A static HTML page is easily generated by creating TypeScript, HTML and (optional) CSS files for a new Component, but by providing only the minimal TypeScript definition and having all content served by the HTML template. Create new files in the **src/app** folder for ***home.component.ts***, ***home.component.html*** and ***home.component.css*** and provide minimal content as shown below. Note that the ***home.component.ts*** file is most easily created by copying the existing ***app.component.ts*** file and deleting/modifying the appropriate elements. We will leave ***home.component.css*** empty (for now).

File: bizFE/src/app/home.component.ts

```
import { Component } from '@angular/core';
import { RouterOutlet } from '@angular/router';

@Component({
  selector: 'home',
  standalone: true,
  imports: [RouterOutlet],
  templateUrl: './home.component.html',
  styleUrl: './home.component.css'
})
export class HomeComponent { }
```

File: bizFE/src/app/home.component.html

```
<h1>
    Biz Directory
</h1>
```

File: bizFE/src/app/home.component.css

```
<!-- file is empty -->
```

Do it now!	Create the files for the new Home Component as shown above.

13.1.2 Routing in an Angular Application

Angular provides a very useful Router module that manages multiple URLs within an application. In Angular, routing refers to the showing or hiding of portions of the display that correspond to Components, rather than fetching an entirely new page from the server. As users perform application tasks, so they move between the different views defined by the components of the application.

We can add the routes for the two Components in our current application by setting the **`HomeComponent`** as the default route (**/**) while the **`BusinessesComponent`** will be accessed by the URL **/businesses**. To achieve this, we modify the content of ***app.routes.ts***, importing the Components to which we want to define routes ,and specifying each route as a path (without a leading '**/**') and the component to be loaded if that path is requested. The import and the definition of the routes is illustrated in the following code box.

File: bizFE/src/app/app.routes.ts

```
import { Routes } from '@angular/router';
import { HomeComponent } from './home.component';
import { BusinessesComponent } from './businesses.component';

export const routes: Routes = [
    {
        path: '',
        component: HomeComponent
    },
    {
        path: 'businesses',
        component: BusinessesComponent
    }
];
```

Once the routes are defined, we need to specify when the output from the matching component should be injected into the application. Previously, we specified that the

`BusinessesComponent` output should be injected into ***app.component.html*** by using the **`<businesses></businesses>`** tag. We want to replace this with an instruction to instead display the output from the currently selected Component instead. First, we clear the current content of ***app.component.html*** as follows

File: bizFE/src/app/app.component.html

```
<!-- empty file -->
```

and replace it with the tag **`<router-outlet></router-outlet>`** to determine that the output of that Component that matches the currently matching path in the router should be shown.

File: bizFE/src/app/app.component.html

```
<router-outlet></router-outlet>
```

Do it now!	Add the routes specifications to ***app.routes.ts*** and make the change to ***app.component.html*** as shown above. Now run the application and confirm that the URL http://localhost:4200/ displays the new **`HomeComponent`** while http://localhost:4200/businesses displays the **`BusinessesComponent`** as shown in Figure 13.1 and Figure 13.2, below.

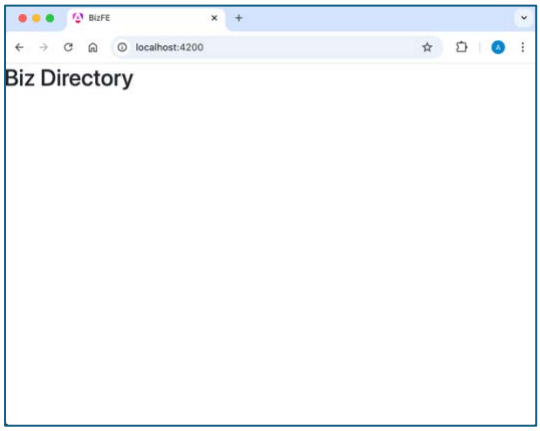


Figure 13.1 Home Page
http://localhost:4200/

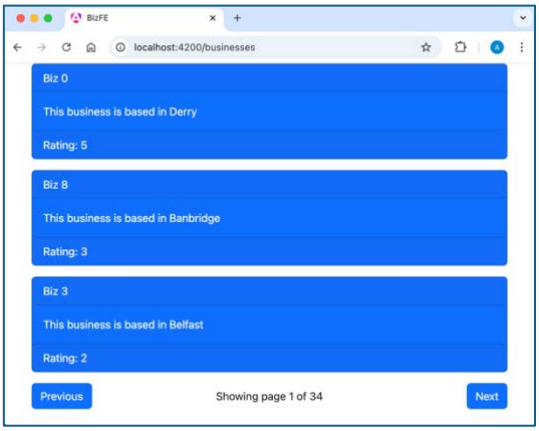


Figure 13.2 Businesses Directory
http://localhost:4200/businesses

13.2 Viewing a Single Data Item

Currently, the **`BusinessesComponent`** provides navigation through the entire collection of businesses data, but we want to add functionality such that when we click on a business entry from the main list, information about that business alone is displayed on the page. Initially, we will display the same information as is shown in the main list, but this could be used to display more complete information or allow the user to interact with the business in some way.

13.2.1 Creating the New Component

The first task is to add a new Component to our application to display a single business, where the business to be displayed is identified by its unique ID that is passed as a query string parameter.

The definition of a Component to display a single business is shown in the following code box. Note how we include the **`DataService`** in the **`import`**s and **`providers`** sections as we will want to retrieve details of the single business from the same Service that provides details on the collection of businesses, and also how we specify that the **`<business></business>`** tag is to be used as the selector for a single business.

File: bizFE/src/app/business.component.ts

```
import { Component } from '@angular/core';
import { RouterOutlet } from '@angular/router';
import { DataService } from './data.service';

@Component({
  selector: 'business',
  standalone: true,
  imports: [RouterOutlet],
  providers: [DataService],
  templateUrl: './business.component.html',
  styleUrl: './business.component.css'
})
export class BusinessComponent {}
```

Do it now!	Add the new file ***src/app/business.component.ts*** as described in the code box above. The easiest approach is to copy the code from the existing ***app.component.ts*** and make the necessary modifications.

13.2.2 Extending the Data Service

Once the new Component TypeScript definition is in place, the next step is to add functionality to the **`DataService`** to return a list containing the single business where the **`id`** field matches that passed as a parameter.

To achieve this, we add a method to the **`DataService`** that accepts an ID value as a parameter and iterates across the **`jsonData`** collection, returning that element with a matching ID. If we examine the format of the JSON dataset (as shown in Figure 13.3, below, we can see that the unique identifier is in the **`['_id']['$oid']`** property, so we use this as the comparator in the test for a matching business



Figure 13.3 Format of ID Field in the Dataset

The source of the new **`getBusiness()`** method in the **`DataService`** is shown in the code box below. Note how we continue to return the business data as a list, even though it is now a list of a single element. This will make the transition easier when we retrieve data from the live API later.

File: bizFE/src/app/data.service.ts

```
...

export class DataService {
    ...

    getBusiness(id: any) {
        let dataToReturn: any[] = [];
        jsonData.forEach( function(business) {
            if (business['_id']['$oid'] == id) {
                dataToReturn.push( business );
            }
        })
        return dataToReturn;
    }
}
```

Do it now!	Add the definition of the **`getBusiness()`** method to the **`DataService`** TypeScript file as shown above.

13.2.3 Adding the New Route

Now that the **`DataService`** functionality is in place, we can set up a route to the new **`BusinessComponent`** by importing the Component into ***app.routes.ts*** and specifying the route to an individual business. Note how the variable part of the URL (i.e. the business ID) is prefixed by a colon, giving the path **businesses/:id**.

File: bizFE/src/app/app.routes.ts

```
import { Routes } from '@angular/router';
import { HomeComponent } from './home.component';
import { BusinessesComponent } from './businesses.component';
import { BusinessComponent } from './business.component';

export const routes: Routes = [
    {
        path: '',
        component: HomeComponent
    },
    {
        path: 'businesses',
        component: BusinessesComponent
    },
    {
        path: 'businesses/:id',
        component: BusinessComponent
    }

];
```

Do it now!	Modify ***src/app/app.routes.ts*** to add the new route to the **`BusinessComponent`** as shown above. Consider how any new component can be added to the routes collection in this way.

13.2.4 Providing the Functionality

Now that the **`BusinessComponent`** has been created, the **`DataService`** has been extended, and the route is in place, we can implement the functionality of the **`BusinessComponent`** that retrieves the ID value from the URL and makes the request to the **`DataService`**.

As for the **`BusinessesComponent`**, we provide an implementation of the **`ngOnInit()`** method which makes a call to the **`DataService`** to retrieve the business information. Here, we call the **`DataService`** **`getBusiness()`** method, passing the ID defined in the route (as **businesses/:id**) as a parameter. The variable parameter is retrieved from the **`route.snapshot.param`** object by passing the query string parameter name as a parameter to the **`get()`** method.

File: bizFE/src/app/business.component.ts

```
...

export class BusinessComponent {
    business_list: any;

    ...

    ngOnInit() {
        this.business_list =
             this.dataService.getBusiness(
                      this.route.snapshot.paramMap.get('id'));
    }
}
```

However, access to **`paramMap`** requires that the **`ActivatedRoute`** Component is imported into the **`BusinessComponent`** and is injected into the Component by including it in the parameter list for the Component constructor.

File: bizFE/src/app/business.component.ts

```
...

import { RouterOutlet, ActivatedRoute } from '@angular/router';

...

export class BusinessComponent {
    ...

    constructor( public dataService: DataService,
                 private route: ActivatedRoute) {}

    ...
}
```

The complete modifications for the **`BusinessComponent`** Typescript file are presented in the code box below.

File: bizFE/src/app/business.component.ts

```
...

import { RouterOutlet, ActivatedRoute } from '@angular/router';

...

export class BusinessComponent {
    business_list: any;

    constructor( public dataService: DataService,
                 private route: ActivatedRoute) {}

    ngOnInit() {
        this.business_list =
             this.dataService.getBusiness(
                      this.route.snapshot.paramMap.get('id'));
    }
}
```

Do it now!	Complete the specification of the **`BusinessComponent`** TypeScript file as illustrated in the code box above.

13.2.5 Specifying the Front-End of the New Component

Finally, we can specify the **`BusinessComponent`** HTML template. This code is exactly as found in the **`BusinessesComponent`**, except that we omit the Bootstrap row that defines the pagination buttons. Remember that the information returned from the **`DataService`** is still formatted as list, even though it is a list containing a single element.

File: bizFE/src/app/business.component.html

```
<div class="container">
    <div class="row">
        <div class="col-sm-12">
            @for(business of business_list;
                            track business.name) {
                <div class="card text-white bg-primary mb-3">
                    <div class="card-header">
                        {{ business.name }}
                    </div>
                    <div class="card-body">
                        This business is based in
                        {{ business.town }}
                    </div>
                    <div class="card-footer">
                        Rating: {{ business.rating }}
                    </div>
                </div>
            }
        </div> <!-- col -->
    </div>    <!-- row -->
</div>      <!-- container -->}
```

We can also create a Component stylesheet file, though we choose at this stage to leave it empty since all style will be inherited from the parent application class.

File: bizFE/src/app/business.component.css

```
<!-- file is empty -->
```

Do it now!	Launch the application with **`ng serve`** and show that the **`BusinessComponent`** is working by presenting the URL http://localhost:4200/businesses/<id> to the browser, where <id> is the **`$oid`** property of one of the businesses in the file ***assets/businesses.json***. Verify that you receive output such as that presented in Figure 13.4, below.

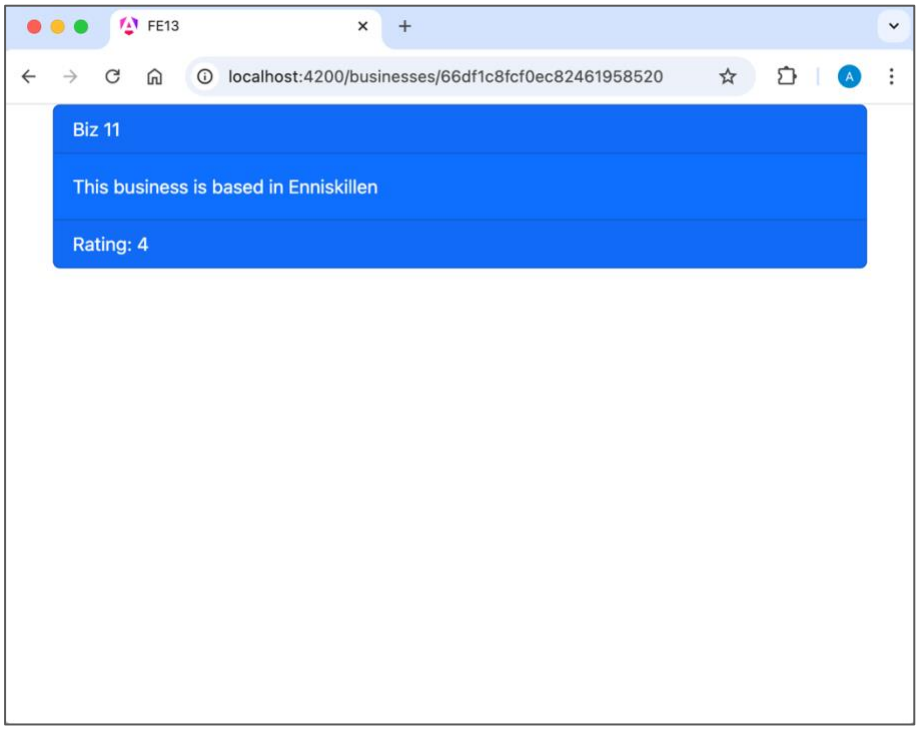


Figure 13.4 Display a Single Business

13.3 Connecting the Components

With the **`BusinessComponent`** that displays details of a single business running, we can now connect it to the **`BusinessesComponent`** that displays the collection of businesses. The way in which this is achieved is to make each element in the **`BusinessesComponent`** clickable, delivering the user to the page that displays information about the selected business.

To achieve this, we add a style rule to display the cursor as a pointer when it is positioned over the Bootstrap card that describes a business and make it clickable by binding a URL to its **`routerLink`** property. Note how the URL is specified as a list, where the component parts of the URL are individual list elements. Here the URL will be "**/businesses**" followed by the **`$oid`** field of the business **`_id`** property – hence the tatget URL will be (for example) **/businesses/1234a1234b1234c**.

File: bizFE/src/app/businesses.component.html

```
<div class="container">
    <div class="row">
        <div class="col-sm-12">
            @for(business of business_list;
                            track business.name) {
                <div class="card text-white bg-primary mb-3"
                     style = "cursor: pointer"
                     [routerLink] = "['/businesses',
                                     business._id.$oid]">
                    <div class="card-header">
                        {{ business.name }}
                    </div>

...
```

Then, to make the **`routerLink`** property available in the **`BusinessesComponent`**, we need to import **`RouterModule`** into the Component TypeScript file and add it to the Component's **`import`**s list.

File: bizFE/src/app/businesses.component.ts

```
import { Component } from '@angular/core';
import { RouterOutlet, RouterModule } from '@angular/router';
import { DataService } from './data.service';

@Component({
  selector: 'businesses',
  standalone: true,
  imports: [RouterOutlet, RouterModule],
  providers: [DataService],
  templateUrl: './businesses.component.html',
  styleUrl: './businesses.component.css'
})

...
```

Do it now!	Carefully follow the steps above to link the **`BusinessesComponent`** to the **`BusinessComponent`**. Verify that the businesses in the **`BusinessesComponent`** are now clickable and that clicking on one launches the **`BusinessComponent`** to display that business. You should obtain output such as that shown in Figures 13.5 and 13.6 below.

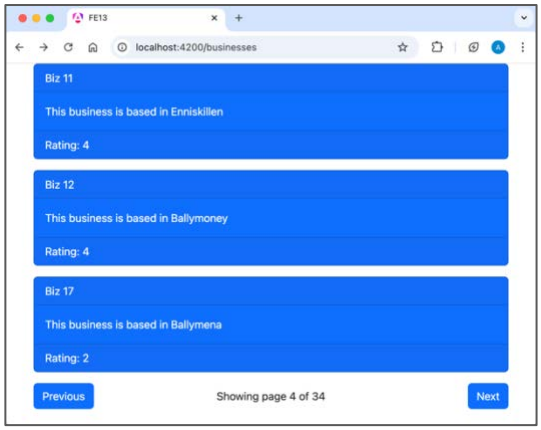


Figure 13.5 Clickable Page of Businesses

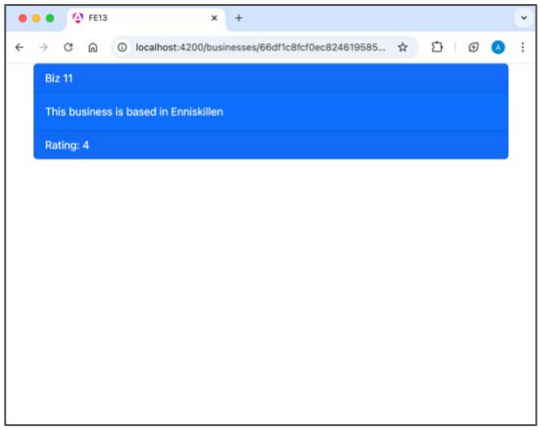


Figure 13.6 Individual Business Selected

13.4 Adding a Navigation Bar

Our Biz Directory application provides a retrieval interface to information on a collection of businesses stored in a MongoDB database. It also provides the facility for registered, logged-in users to add a review to the collection stored for each business.

In this development stage, we will implement a Bootstrap navigation bar as an Angular component, and have it automatically included on each page.

13.4.1 Create the Navigation Component

Creating the navigation component follows the now-familiar process or creating a TypeScript definition file and an HTML template. The TypeScript file can be most easily generated by copying the code from the ***app.component.ts*** file and making the necessary changes to result in the code box shown below.

File: bizFE/src/app/nav.component.ts

```
import { Component } from '@angular/core';
import { RouterOutlet } from '@angular/router';

@Component({
  selector: 'navigation',
  standalone: true,
  imports: [RouterOutlet],
  templateUrl: './nav.component.html'
})

export class NavComponent { }
```

Now, we can create the **`NavComponent`** template as a Bootstrap **`navbar`** element. Most of the Bootstrap classes used are self-explanatory, but some will require additional clarification.

The navigation bar is created by specifying a **`<div>`** element with style classes that describe its physical characteristics, such as

`navbar-expand-sm` the navbar will collapse in a responsive manner when the browser window drops below the level of a "small device" (typically tablet resolution). Alternatives to **-sm** are **-xs** (small, smartphones), **-md** (medium, desktop resolution) and **-lg** (large, large desktop).

`bg-primary` the background of the navbar will be in the Bootstrap **primary** colour (blue). Other defined colour values are **`secondary`** (grey), **`success`** (green), **`danger`** (red) and **`warning`** (yellow) – although you can also define colours using the standard CSS **`background-color`** property.

`navbar-dark` characterises the navbar as **`dark`**, so prompting the use of a contrasting light text colour.

`fixed-top` fixes the **`navbar`** to the top of the browser window, allowing content to scroll beneath it.

Inside the navbar we have a text element defined as a **`<span>`** with class **`navbar-brand`**. This causes the "Biz Directory" text to be formatted as a heading within the navbar contents

Next, we have the menu items displayed as **<li>** items within a **<ul>** element. By applying the **navbar-nav** class to the **<ul>** element, we define it as a container for the navigation links, with each **<li>** specified as a **nav-item**. The actual link is an **<a>** anchor tag, specified as class **nav-link**. Note that we do not provide a **href** attribute for the **<a>**, instead using the Angular **routerLink** property to identify the route to be invoked when the link is clicked.

File: bizFE/src/app/nav.component.html

```
<div class="navbar navbar-expand-sm bg-primary
            navbar-dark fixed-top">
    <span class="navbar-brand">Biz Directory</span>
    <ul class="navbar-nav">
        <li class="nav-item">
            <a class="nav-link"
               [routerLink]="['/']">Home</a>
        </li>
        <li class="nav-item">
            <a class="nav-link"
               [routerLink]="['/businesses']">Businesses</a>
        </li>
    </ul>
</div>
```

Once again, we need to make **routerLink** available to the Component by importing **RouterModule** and adding it to the **imports** list.

File: bizFE/src/app/nav.component.ts

```
import { Component } from '@angular/core';
import { RouterOutlet, RouterModule } from '@angular/router';

@Component({
  selector: 'navigation',
  standalone: true,
  imports: [RouterOutlet, RouterModule],
  templateUrl: './nav.component.html'
})

export class NavComponent { }
```

Do it now!	Create the **NavComponent** by specifying the code shown above to the files ***nav.component.ts*** and ***nav.component.html***.

13.4.2 Connect the Navigation Component to the Application

With the new **NavComponent** created, we can now have it displayed on every page by adding it to the **AppComponent** template. First, we make it available to the **AppComponent** by **import**ing it and adding it to the list of Component **imports**.

File: bizFE/src/app/app.component.ts

```
import { Component } from '@angular/core';
import { RouterOutlet } from '@angular/router';
import { BusinessesComponent } from './businesses.component'
import { NavComponent } from './nav.component'

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [RouterOutlet, BusinessesComponent, NavComponent],
  templateUrl: './app.component.html',
  styleUrl: './app.component.css'
})
export class AppComponent {
  title = 'bizFE';
}
```

Then, we place the **<navigation>** selector before the **<router-outlet>** within **app.component.html** to instruct the browser to render it before the content from the selected route, so effectively adding the navbar code at the top of each page.

File: bizFE/src/app/app.component.html

```
<navigation></navigation>
<router-outlet></router-outlet>
```

At this stage, we also add a **margin-top** property to the **container ** in the **BusinessesComponent**, **BusinessComponent** and **HomeComponent** template files to push the page content below the fixed navbar.

File: bizFE/src/app/businesses.component.html

File: bizFE/src/app/business.component.html

```
<div class="container" style="margin-top: 70px">

...
```

We also add similar spacing to the top of the content for the **`HomeComponent`**.

File: bizFE/src/app/home.component.html

```
<h1 style="margin-top:70px">
    Biz Directory
</h1>
```

The effect of these changes can be seen in Figure 13.7, which demonstrates the new navbar on the ***/businesses*** route.

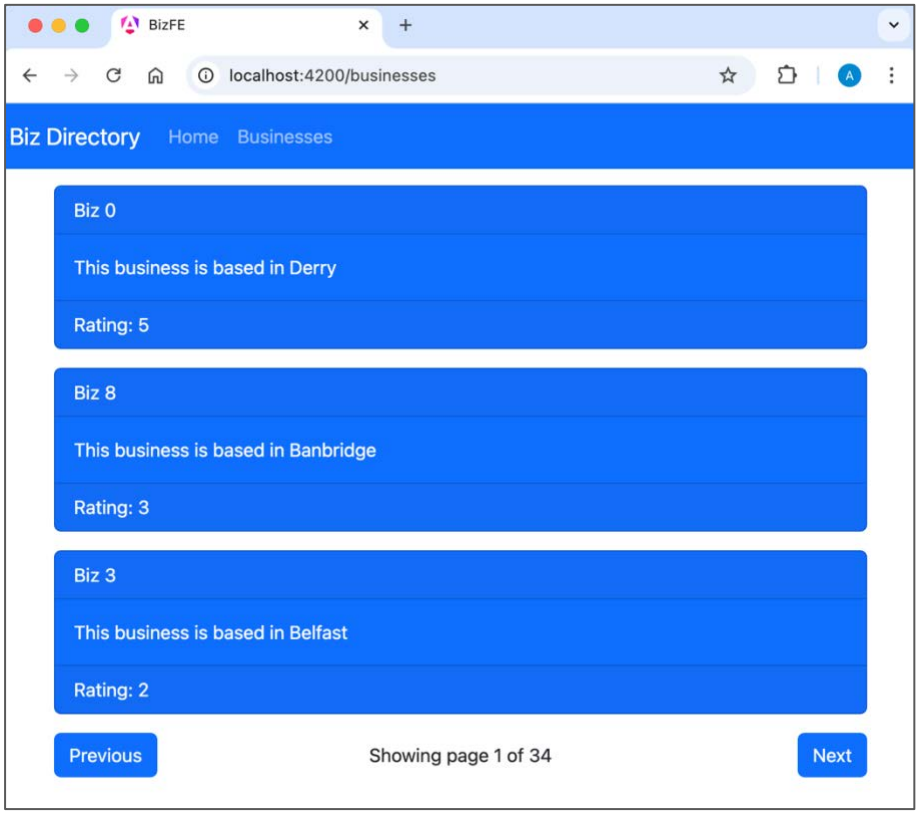


Figure 13.7 Navigation Bar Added

Do it now!	Connect the navigation bar to the application as shown above. Run the application and navigate through it, making sure that the navigation bar is present on all pages and that the links in the navigation bar work as expected.

13.5 Further Information

- https://angular.dev/guide/routing
  Angular Routing

- https://angular.dev/guide/routing/router-tutorial
  Using Angular Routes in a Single Page Application

- https://dev.to/codev206/how-to-set-up-routing-in-angular-to-create-single-page-applications-4ch2
  How to set up Routing in Angular

- https://www.typescriptlang.org/docs/handbook/variable-declarations.html
  Variable Declarations in TypeScript

- https://www.typescriptlang.org/docs/handbook/2/objects.html
  TypeScript Objects

- https://angular.dev/api/router/RouterLink
  Angular RouterLink class

- https://www.telerik.com/blogs/angular-basics-router-link-overview
  Angular RouterLink Overview

- https://css-tricks.com/almanac/properties/c/cursor/
  CSS Cursor Properties

- https://www.w3schools.com/html/html5_webstorage.asp
  HTML5 Web Storage API

- https://www.digitalocean.com/community/tutorials/how-to-use-the-javascript-developer-console
  How to use the JavaScript Developer Console

- https://getbootstrap.com/docs/5.3/components/navbar/
  The Bootstrap NavBar

- https://www.w3schools.com/bootstrap/bootstrap_navbar.asp
  Bootstrap Navigation Bar

COM661 Full Stack Strategies and Development

FE14. Data-driven Functionality

Aims

- To introduce Angular Directives
- To represent numeric data in a visual fashion
- To integrate Google Maps into the application
- To make use of representative placeholder content to populate the interface of an application
- To harvest data from an external API
- To introduce SVG for graphical representation of data
- To demonstrate layout design for effective representation of information

Table of Contents

14.1 DATA VISUALISATION 2

14.2 USING COORDINATE DATA 4

14.2.1 USING GOOGLE MAPS 4
14.2.2 GENERATING A FIRST MAP 5

14.3 GENERATING PLACEHOLDER CONTENT 10

14.3.1 A LOREM IPSUM GENERATOR 10

14.4 USING APPLICATION DATA WITH EXTERNAL APIS 15

14.4.1 CONNECTING TO OPENWEATHERMAP API 16
14.4.2 GRAPHICAL REPRESENTATION OF THE WEATHER DATA 18

14.5 BUILDING A LAYOUT 23

14.6 FURTHER INFORMATION 26

14.1 Data Visualisation

In this practical we will demonstrate how we can use various techniques and plug-ins to generate additional dynamic functionality from our data. Currently, each of the business in our collection is described by a name, a town, a rating, a coordinate location and a set of profit/loss figures for the previous three years, and although displaying these as text within a Bootstrap Card does impart the information, we can to this in a much more interesting way by considering how the information might otherwise be represented on the web page.

First, let's consider how we display the business rating as a value from 1-5. Presently, it is displayed as a simple text message within the footer of the Bootstrap card, but it might be more clearly displayed by a star representation, where the number of stars is the figure for the business rating.

We have already seen the Angular **`@for()`** block that allows blocks of HTML content to be repeated for a given number of times. The same effect can be achieved in a more compact form by the **`*ngFor`** directive that iterates over a collection by the syntax

```
*ngFor = "let item of collection"
```

where the loop variable ***`item`*** takes the value of each element of ***`collection`*** in turn. For our "stars" use case, we can make use of this by defining an array containing as many items as stars that we want to display. This can be achieved by using the constructor of the array element and passing the number of elements required as a parameter. Hence the expression

```
[].constructor(business.rating)
```

will create an array with a number of elements dictated by the value of the business **`rating`** field.

By wrapping the display of a star image with a **`<span>`** element that deploys **`*ngFor`** to display that image a certain number of times, we can generate a number of stars equal to the rating of the business. This is shown by the code box below which we can deploy to the card footer instead of the current business rating message.

Note how the image file ***star.png*** is represente3d in the ***/images*** folder. This is a sub-folder of the ***public*** folder that will have been created at the top level of your Angular project structure. ***public*** can be used for those assets that need to be accessible directly by the browser, such as images, logos, sounds and other media elements.

File: bizFE/src/app/business.component.html

```
...

 <div class="card-footer">
    <span *ngFor="let star of [].constructor(business.rating)">
          <img src="images/star.png"
               style="width:30px; height:30px">
    </span>
 </div>

...
```

To use *ngFor in the component it needs to be imported in the BusinessComponent TypeScript file and added to the list of component imports as below.

File: bizFE/src/app/business.component.ts

```
import { Component } from '@angular/core';
import { RouterOutlet, ActivatedRoute } from '@angular/router';
import { DataService } from './data.service';
import { CommonModule } from '@angular/common';

@Component({
  selector: 'business',
  standalone: true,
  imports: [RouterOutlet, CommonModule],
  providers: [DataService],
  templateUrl: './business.component.html',
  styleUrl: './business.component.css'
})
...
```

Making these changes and visiting the page for an individual business shows how the rating is displayed as illustrated in Figure 14.1 below.

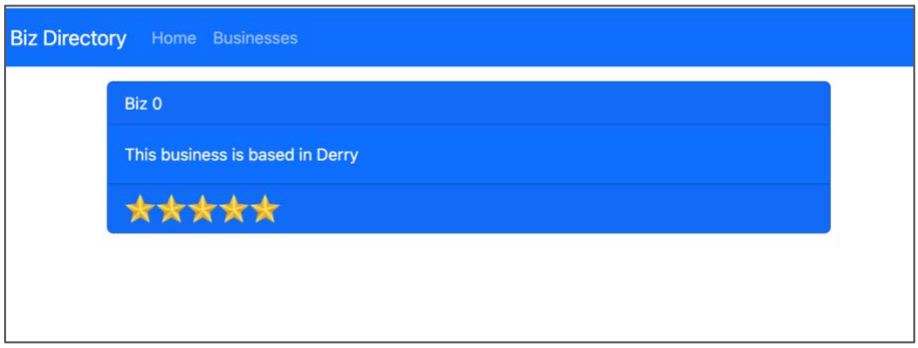


Figure 14.1 Graphical Representation of the Business Rating

Do it now!	Make the change shown above to ***business.component.html*** to change the way in which the rating of a business is displayed. Remember to create ***images*** as a sub-folder of ***public*** and to copy the ***stars.png*** file found in the **Practical BE13 Files** to this new folder.

14.2 Using Coordinate Data

One of the most interesting features of our dataset is the provision of a realistic coordinate position for each business through latitude and longitude values. An obvious way in which these could be used would be in the display of a map that provides a visual representation of the location of the business.

14.2.1 Using Google Maps

Note:	Using Google Maps within your applications requires that you have a **Google Cloud Platform developer account** and that you create a **Google Maps API key**. This will require that you register a payment card with Google, but the level of free credit provided is such that you should never be charged. However, if you are uncomfortable with registering a card, please feel free to skip this section and replace the Map that we will add to our **Business Component** template with a representative image from the internet.

To create a new Google Cloud Platform developer account and a Google Maps project, visit https://developers.google.com/maps/get-started and following the 3-step process described. Once you have created your project in Step 1, visit the **Maps API Library Page** in Step 2 and enable the **Maps JavaScript API**. Then, in Step 3, click the link for the **Google Maps Platform Credentials** page and follow the link to **Maps Platform API Key**. From here, you will be able to copy the API key that you will need to embed in your application.

Do it now!	Create a Google Cloud Platform developer account and create a new project. In that project, enable the Google Maps JavaScript API and obtain an API key.

To use Maps in our Angular applications, there is a very useful Google Maps module that we can install using **npm** by the command

```
npm install @angular/google-maps --save
```

Then, we need to import the **`GoogleMapsModule`** into the **`BusinessComponent`** in which we want to display a map. This is achieved by the modifications to ***business.component.ts*** shown in the following code box.

File: bizFE/src/app/business.component.ts

```
...
import { GoogleMapsModule } from '@angular/google-maps'

@Component({
  selector: 'business',
  standalone: true,
  imports: [RouterOutlet, CommonModule, GoogleMapsModule],
  providers: [DataService],
  templateUrl: './business.component.html',
  styleUrl: './business.component.css'
}

...
```

14.2.2 Generating a First Map

With the **`GoogleMapsModule`** installed, we can now add a Map object to our **`BusinessComponent`** HTML template. The key information for the Map is the coordinate position at which it is centered, so we retrieve the latitude and longitude values from the currently selected business and use them to initialize the **`lat`** and **`lng`** properties of the **`MapOptions`** object.

To accomplish this, we add new properties to the **`BusinessComponent`** to store the latitude value as **`business_lat`**, the longitude value as **`business_lng`**, and the

`MapOptions` object as **`map_options`**. We also initialise the **`map_locations`** property that will hold map markers as an empty list.

Then, we modify the component **`ngOnInit()`** method to initialise the latitude and longitude values to those associated with the current business object, as well as initializing the **`map_options`** with the default **`mapId`** and centering the map on the latitude and longitude values.

The modifications in ***business.component.ts*** are described in the code box that follows.

File: bizFE/src/app/business.component.ts

```
...

export class BusinessComponent {

    business_list: any;
    business_lat: any;
    business_lng: any;
    map_options: google.maps.MapOptions = {};
    map_locations: any[] = [ ]

    ...

    ngOnInit() {
        this.business_list =
             this.dataService.getBusiness(
                  this.route.snapshot.paramMap.get('id'))
        this.business_lat =
             this.business_list[0].location.coordinates[0];
        this.business_lng =
             this.business_list[0].location.coordinates[1];

        this.map_options = {
          mapId: "DEMO_MAP_ID",
          center: { lat: this.business_lat,
                    lng: this.business_lng },
          zoom: 13,
        };
      }

...
```

Now that the **ngOnit()** method has been set to initialize the map when the **BusinessComponent** loads, we can add the HTML tag that inserts the map onto the web page. In the business.component.html template, we add a **<google-map>** tag which specifies the dimensions of the map on the page and binds the **options** property to the **map_options** element specified above.

File: bizFE/src/app/business.component.html

```
...

<google-map height="400px" width="400px"
        [options]="map_options">
</google-map>

...
```

Finally, we need to add the following script to the application's ***src/index.html*** file and paste the JavaScript content provided in the Google Maps Script.html file immediately before the closing **</body>** tag. Remember to replace the placeholder **YOUR_API_KEY** with your corresponding Google Maps developer API key.

File: bizFE/src/index.html

```
...

<body>
  ...
<!—- Paste the <script> element provided in the FE14 Practical
     Files here -->
</body>

...
```

With the Typescript setup code and the HTML template in place, we can then run the application and select an individual business to see its location centered on a Google Map that is displayed on the page.

Do it now!	Make the modifications to the TypeScript and HTML files of the **`BusinessComponent`** as described above. Run the application and select an individual business and confirm that the map displayed is centered on the business location as shown in Figure 14.2, below. Try different businesses to see how the map moves from town to town with the business location.

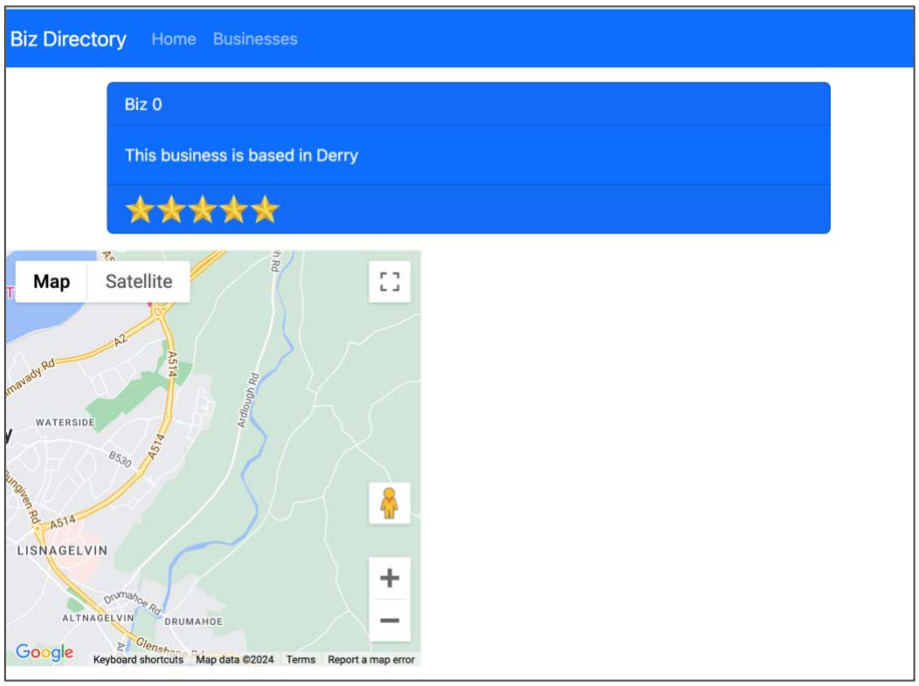


Figure 14.2 Creating a Google Map

With the Map displayed and centered on the business, we can now add a marker to the Map that indicates the precise location. Earlier, we defined **`map_locations`** in the **`BusinessComponent`** as an array so that the code the follows will support multiple markers, but here we will add a single marker indicating the business coordinate position.ng

For each marker to be displayed, we need to push an object containing **`lat`** and **`lng`** values to the **`map_locations`** property of the component. Here, we push a single location by using the **`business_lat`** and **`business_lng`** values that were assigned for the current business from the values retrieved from the **`DataService`**.

File: bizFE/src/app/business.component.ts

```
...

    ngOnInit() {
        this.business_list =
             this.dataService.getBusiness(
                  this.route.snapshot.paramMap.get('id'))
        this.business_lat =
             this.business_list[0].location.coordinates[0];
        this.business_lng =
             this.business_list[0].location.coordinates[1];

        this.map_locations.push( {
          lat: this.business_lat, lng: this.business_lng
        });

        ...
```

Now, we can add a **`@for()`** block to the **`<google-map>`** element in the **`BusinessCompoent`** template to iterate over the contents of **`map_locations`** and add a **`<map-advanced-marker>`** element for each location specified.

File: bizFE/src/app/business.component.html

```
...

<google-map height="400px" width="400px"
        [options]="map_options">
     @for (location of map_locations; track location) {
        <map-advanced-marker
          #markerElem="mapAdvancedMarker"
          [position]="{ lat: location.lat,
                        lng: location.lng }">
        </map-advanced-marker>
     }
 </google-map>
...
```

Do it now!	Make the modifications to ***business.compoennt.ts*** and ***business.component.html*** described above and run the application. Verify that a marker has been added to the map at the location of the business being displayed as shown in Figure 14.3, below.

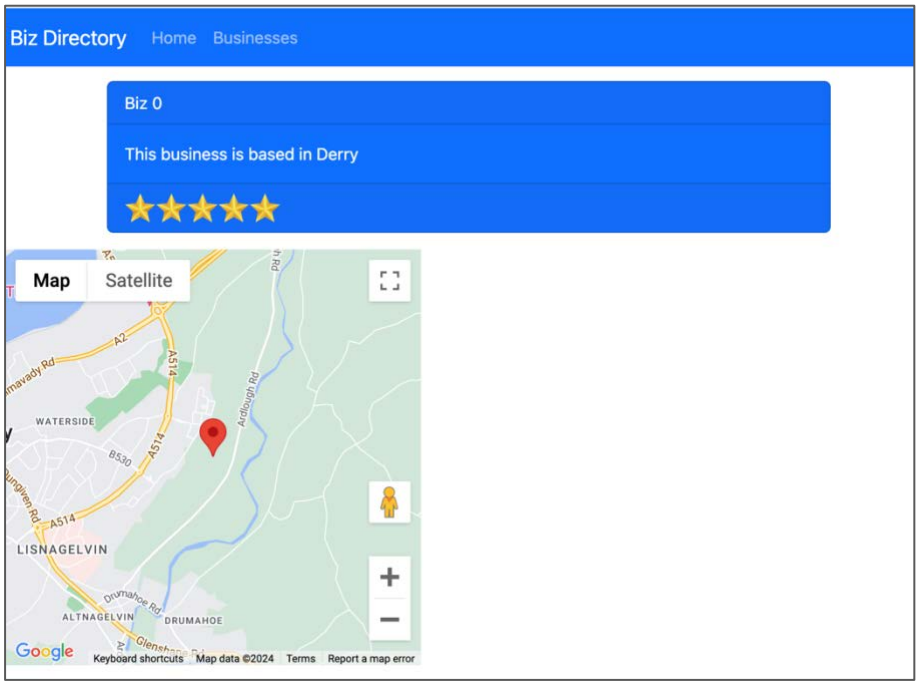


Figure 14.3 Google Map with a Marker

14.3 Generating Placeholder Content

Sometimes, our application is a demonstration of some concept or idea that either has no live data or where the data being used is incomplete. In such cases, we can use external sources of dummy data to populate the front-end and there are many potential sources for this. In this section, we make use of an external API from **api-ninjas.com** to generate dummy text that will represent a description of a business in the absence of a corresponding field in our randomly generated dataset.

14.3.1 A Lorem Ipsum Generator

In publishing and graphic design, **Lorem Ipsum** is meaningless placeholder text that occupies the space where the real information would be displayed. It is often used in industry when layout is to be presented but actual content is not yet available.

api-ninjas.com makes available a very useful range of APIs which can be used to generate a wide range of types of sample data. Among their catalogue is an API that supplies Lorem Ipsum text which we will use to populate our business description.

Do it now!	Sign up for a free account at https://www.api-ninjas.com and obtain an API key for the Lorem Ipsum API.

With the API key available, we can now add a new method to the DataService to return a specified number of random Lorem Ipsum paragraphs from the API.

The call to the external API requires that we make an **HTTP GET** request to the API endpoint https://api.api-ninjas.com/v1/loremipsum?paragraphs=2 where the query string parameter '2' is the number of paragraphs being requested. We also pass our API key as an **X-Api-Key** request header. Note in the code box below how the HTTP request returns an Observable object.We will cover Observables in more detail later when we connect our front end to the API built earlier in the module, but for now we can describe an Observable as a stream of data that happens over time. Rather than read a single value, we subscribe to the stream and accept and process data as it arrives. The new **`getLoremIpsum()`** method is shown in the code box below.

File: bizFE/src/app/data.service.ts

```
    ...

        getLoremIpsum(paragraphs: number): Observable<any> {

            let API_key = 'YOUR-API-KEY';
            return this.http.get<any>(
                        'https://api.api-ninjas.com/v1/' +
                        'loremipsum?paragraphs=' + paragraphs,
                        { headers: { 'X-Api-Key': API_key } }
            );

    ...
```

In order to support the new method, we also need to import new modules **`HttpClient`**, **`Injectable`** and **`Observable`** into the **`DataService`** as described below. We also need to make the component injectable by adding the **`@Injectable`** decorator to the component class and make the **`HttpModule`** available for use by injecting it to the component by providing it as a parameter to the component constructor

File: bizFE/src/app/data.service.ts

```
import jsonData from '../assets/businesses.json';
import { HttpClient } from '@angular/common/http';
import { Injectable } from '@angular/core';
import { Observable } from 'rxjs';

@Injectable( {
    providedIn: "root",
})
export class DataService {
    pageSize: number = 3;

    constructor(private http: HttpClient) {}

...
```

The final stage in enabling the **`HttpClient`** is to **`import`** the **`provideHttpClient`** module into ***app.config.ts*** and add it to the list of **`providers`** specified.

File: bizFE/src/app/app.config.ts

```
import { ApplicationConfig,
         provideZoneChangeDetection } from '@angular/core';
import { provideRouter } from '@angular/router';
import { routes } from './app.routes';
import { provideHttpClient } from '@angular/common/http';

export const appConfig: ApplicationConfig = {
  providers: [
     provideZoneChangeDetection({ eventCoalescing: true }),
     provideRouter(routes),
     provideHttpClient()
  ]
};
```

With the **`DataService`** updated and the **`HttpClient`** registered with the application, we can now update the **`BusinessComponent`** to call the Lorem Ipsum generator from the **`ngOnInit()`** method and log the text received to the console.

Note in the code box below how, as the call to **`getLoremIpsum()`** returns an Observable, we need to subscribe to it by chaining the **`subscribe()`** method to the request. The **`subscribe()`** method takes as a parameter a function that is written in the “fat arrow”

style. Fat arrow functions (sometimes just called “arrow functions” are a concise notation for short (often single line) functions that do not have a function name and do not return a value. The parameter to the function is on the left-hand side of the **=>** symbol, while the code to be executed is on the right-hand side. Here, the parameter is the data returned from the Observable, while the response is simply to log that value to the browser console.

File: bizFE/src/app/business.component.ts

```
...

    ngOnInit() {

        ...

        this.dataService.getLoremIpsum(1)
          .subscribe((response: any) => {
              console.log(response.text);
        })};
    }
...
```

Do it now!	Update the files ***data.service.ts***, ***app.service.ts*** and ***business.component.ts*** with the code presented above and run the application with the browser console open. Verify that displaying the page for an individual business results in a stream of Lorem Ipsum text being retrieved from the API and displayed in the browser console as shown in Figure 14.4, below.

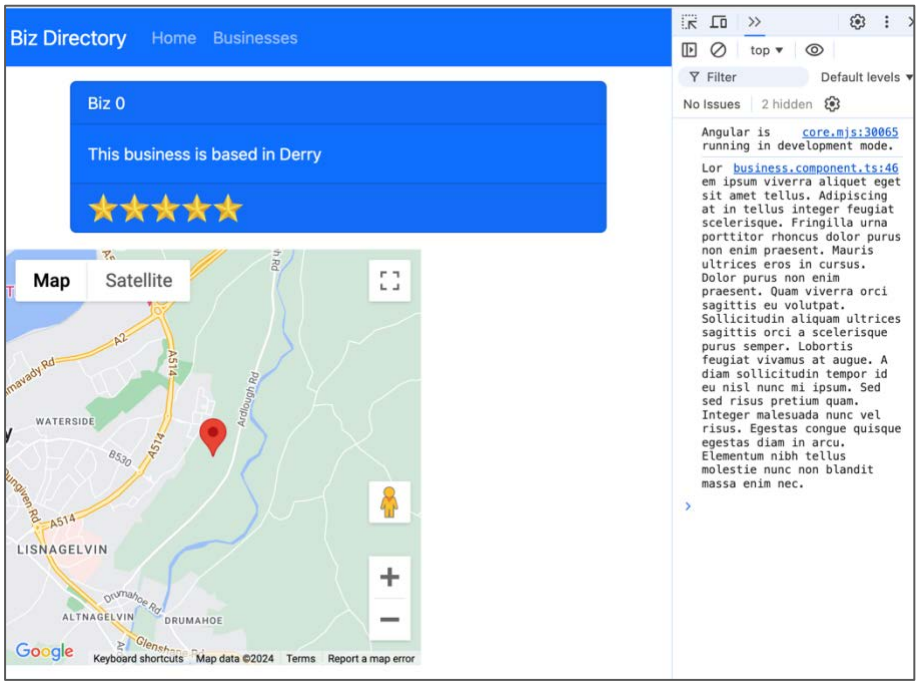


Figure 14.4 Lorem Ipsum Retrieved as a Placeholder

Now, we can add the Lorem Ipsum retrieved from the API as placeholder text for the description of the business. First, we define a new component property **`loremIpsum`** in which to store the text, and then we update the **`ngOnInit()`** method, replacing the previous **`console.log()`** with code to return the first 400 characters of the retrieved text of the new **`loremIpsum`** property.

File: bizFE/src/app/business.component.ts

```
...

export class BusinessComponent {
    ...

    loremIpsum: any;
    ...

    ngOnInit() {

        ...

        this.dataService.getLoremIpsum(1)
          .subscribe((response: any) => {
              this.loremIpsum = response.text.slice(0,400);
        })};
    }
...
```

Finally, we add new content to the **`BusinessCompoent`** HTML template to display the Lorem Ipsum under a “Description” heading.

File: bizFE/src/app/business.component.html

```
...

<h1>Description</h1>
{{ loremIpsum }}
```

Do it now!	Update the ***business.component.ts*** and ***business.component.html*** files with the code presented and navigate to the page for an individual business. Verify that the Lorem Ipsum description of the business is displayed as shown in Figure 14.5 below, and that the description text is randomly generated for each business selected.

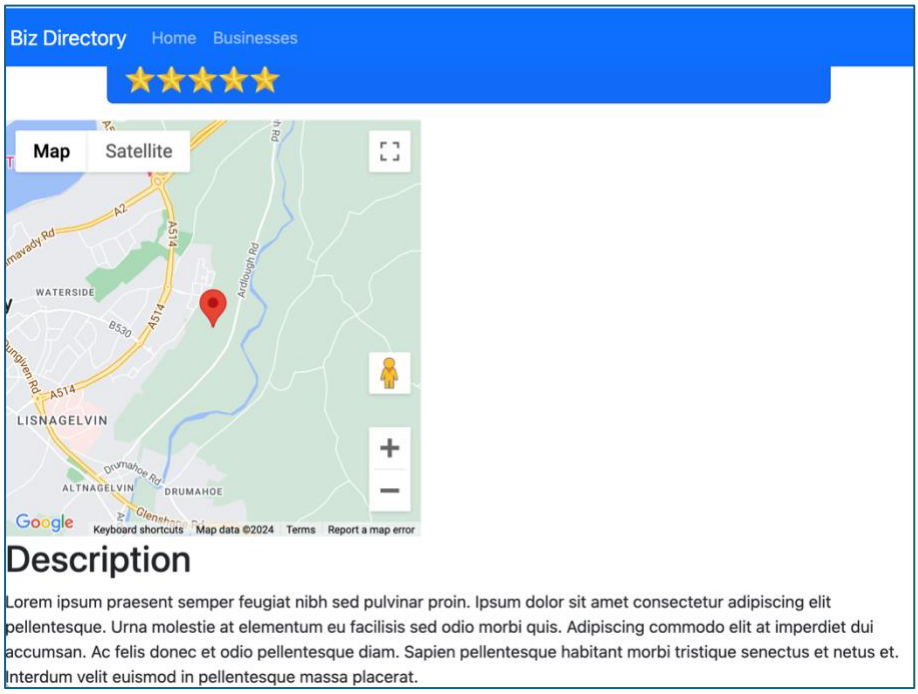


Figure 14.5 Lorem Ipsum in the Browser

14.4 Using Application Data from External APIs

The previous example made use of an external API to obtain random data that can enhance the presentation of our application, but there are also opportunities to use information from our dataset as parameters to an external API call and retrieve real-world live data that can be integrated into our front end.

14.4.1 Connecting to OpenWeatherMap API

In a previous practical, we have already seen the **OpenWeatherMap** API as an example of a commercial API that provides live data. Since each business in our dataset is associated with a location and has a realistic latitude and longitude coordinate position, we can use that data to obtain an accurate weather report for the location and display it on the page of business information.

First, we add a new method **`getCurrentWeather()`** to the Data Service, that takes the latitude and longitude values of the business as a parameter, and returns the packet of data representing the current weather conditions at that location. The call to the API endpoint is built as before, with the latitude and longitude values specified as query string parameters as well as specifying "metric" for the "units" parameter and adding our individual API key. The code for **`getCurrentWeather()`** is shown in the code box below.

File: bizFE/src/app/data.service.ts

```
...

    getCurrentWeather(lat: number, lon: number) {

        let API_key = "YOUR-API-KEY";
        return this.http.get<any>(
             'https://api.openweathermap.org/data/2.5/' +
             'weather?lat=' + lat +
             '&lon=' + lon + '&units=metric&appid=' +
             API_key);
    }
...
```

Next, we update the business component to call the new **`DataService`** method and extract the current temperature and weather description from the data retrieved from the API.

First, we add two new properties to the **`BusinessComponent`** to store the current **`temperature`** and **`weather`** description. Then, in the **`subscribe()`** method following the call to **`getCurrentWeather()`**, we extract the values for each and assign them to the respective properties. Finally, we format the weather description for display by capitalizing the first letter while leaving the rest of the description in lower case. The new code added to ***business.component.ts*** is highlighted in the code box below.

Do it now!	Update ***data.service.ts*** with the new **`getCurrentWeather()`** method. Remember to add your personal API key at the place indicated in the first line of the method body.

File: bizFE/src/app/business.component.ts

```
...

export class BusinessComponent {

    ...

    temperature: any;
    weather: any;

    ...

    ngOnInit() {

        ...

        this.dataService.getCurrentWeather(
            this.business_lat, this.business_lng)
          .subscribe((response: any) => {
              let weatherResponse =
                  response['weather'][0]['description'];
              this.temperature =
                  Math.round((response['main']['temp']));
              this.weather =
                  weatherResponse[0].toUpperCase() +
                  weatherResponse.slice(1);
          });
...
```

With the changes to ngOnInit() described above, loading the BusinessComponent for any individual business will now populate the properties temperature and weather with the values retrieved from OpenWeatherAPI for that specific location. We can now add these to the HTML template of the component so that they are made available to the user.

File: bizFE/src/app/business.component.html

```
...

<h1>Weather</h1>
Description: {{ weather }} <br>
Temperature: {{ temperature }} <br>
```

Do it now!	Update the ***business.component.ts*** and ***business.component.html*** files with the code presented and navigate to the page for an individual business. Verify that the description of the current weather and the temperature value are displayed as shown in Figure 14.6 below.

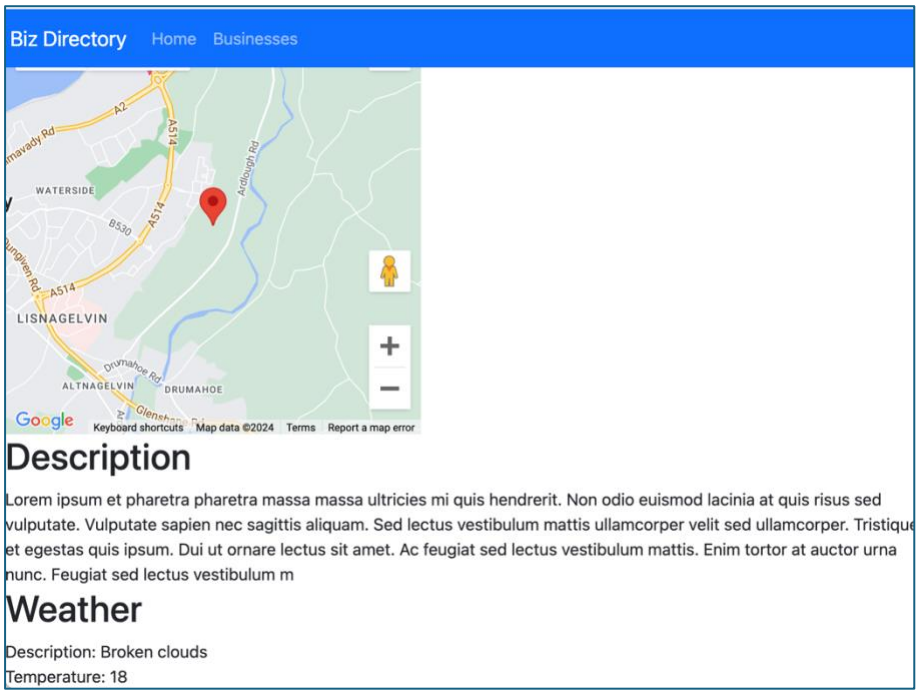


Figure 14.6 Current Weather Information Displayed

14.4.2 Graphical Representation of the Weather Data

The previous section has successfully retrieved and displayed the live weather information for the location of the selected business, but it would be more informatively and attractively displayed in graphical form. One of the data items retrieved from the OpenWeatherMap API is the URL of a symbol describing the current weather conditons and we can use this as additional graphical information.

The weather symbols are in the form https://openweathermap.org/img/wn/10d@2x.png where **10d** is the symbol identifier retrieved from the **`weather[0]['icon']`** element of

the data returned from the API and **2x** is the magnification factor at which the symbol is to be rendered. We add this to our BusinessComponent by creating a new Component property weatherIcon to store the URL of the image and weatherIconURL to store the URL from which the icon can be retrieved. We populate these by an additional retrieval from the API data as shown in the code box below.

File: bizFE/src/app/business.component.ts

```
...

export class BusinessComponent {

    ...

    temperature: any;
    weather: any;
    weatherIcon: any;
    weatherIconURL: any;

    ...

    ngOnInit() {

        ...

        this.dataService.getCurrentWeather(
            this.business_lat, this.business_lng)
          .subscribe((response: any) => {

              ...

              this.weatherIcon =
                  response['weather'][0]['icon'];
              this.weatherIconURL =
                  "https://openweathermap.org/img/wn/" +
                  this.weatherIcon + "@4x.png";
          });
...
```

Do it now!	Update the ***business.component.ts*** file with the code shown above to retrieve the weather icon code and construct its URL

Additionally, we can display the current temperature in a graphical symbol of the kind frequently shown in weather maps in broadcast and online media where the temperature is displayed inside a solid disc where the colour of the disc represents the band in which the temperature falls.

First, we add an additional method **`getTemperatureColour()`** to the **`DataService`** which returns a colour according to the temperature value. Following convention, temperatures lower than 5°C are represented in blue, from 5-12 °C in green, from 13-17 °C in yellow, from 18-25 °C in orange and temperatures of 26 °C or higher are in red. The new method is presented in the code box below.

File: bizFE/src/app/data.service.ts

```
    ...

        getTemperatureColour(temp: number) {
            if (temp <= 5) return "#0000ff";
            else if (temp <= 12) return "#00ff00";
            else if (temp <= 17) return "#ffff00";
            else if (temp <= 25) return "#ff7f00";
            else return "#ff0000"
        }

    ...
```

Now, we return to the **`BusinessComponent`** **`ngOnInit()`** method and add a new **`temperatureColour`** property which we populate by a call to **`getTemperatureColour()`** passing the current temperature value as a parameter.

File: bizFE/src/app/business.component.ts

```
...

export class BusinessComponent {

    ...

    temperatureColour: any
    ...

    ngOnInit() {

        ...

        this.dataService.getCurrentWeather(
            this.business_lat, this.business_lng)
          .subscribe((response: any) => {

            ...

            this.temperatureColour =
                this.dataService.getTemperatureColour(
                    this.temperature)
          });
...
```

Finally, we update the **`BusinessComponent`** HTML template and add an **SVG** object in which we specify a circle of radius 40 pixels, filled with the calculated **`temperatureColour`**, and with the current temperature added in text, centred within the circle.

Note:	SVG (Scalable Vector Graphics) is an XML format for the description of 2-dimensional vector-based graphics that is supported by all major browsers. It is outside of the scope of this module, but if you would like to explore it further, links to tutorials and examples are provided in Section 14.6 of this practical.

File: bizFE/src/app/business.component.html

```
...

<svg height="100" width="100">
   <g>
      <circle [style.fill]="temperatureColour"
              cx="50" cy="50" r="40">
      </circle>
      <text x="50%" y="50%" font-size="1.3em"
            text-anchor="middle" stroke="#000"
            stroke-width="1px" dy=".3em">
          {{ temperature}}
          <tspan dy="-10">o</tspan>
          <tspan dy="10"> C</tspan>
       </text>
   </g>
</svg>

<img src="{{ weatherIconURL }}"><br>
Current weather: {{ weather }}
```

Do it now!	Add the new method **`getTemperatureColour`** to the **`DataService`** and update the **`BusinessComponent`** TypeScript file to call the method and obtain the **`temperatureColour`** property for the business. Then add the code shown above to the **`BusinessComponent`** HTML template and verify that the current temperature for the selected business is displayed on a coloured disc along with the weather icon and descriptive text as shown in Figure 14.7, below.

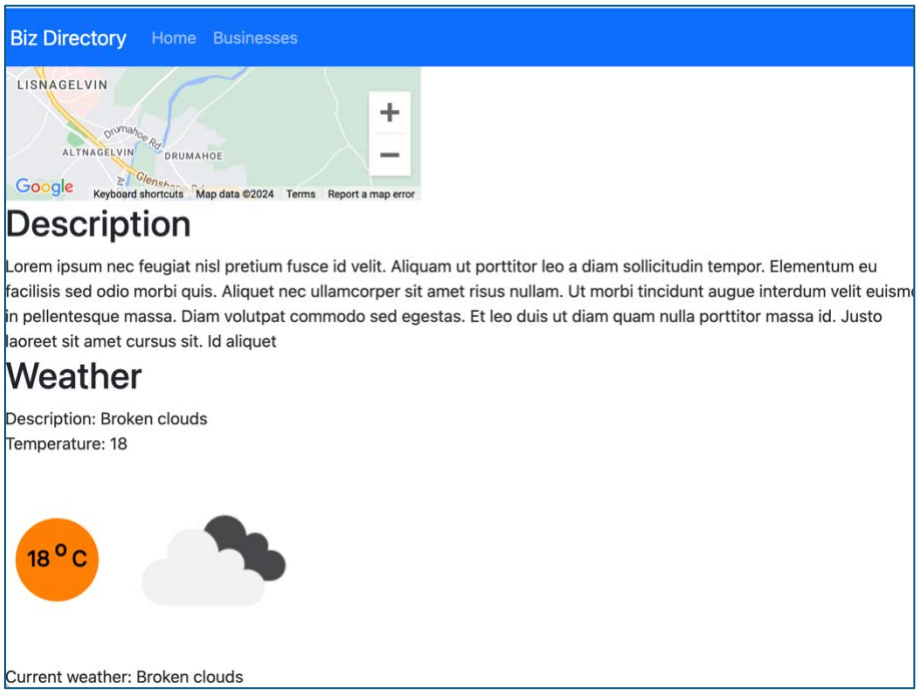


Figure 14.7 Weather Information Displayed Graphically

14.5 Building a Layout

Finally, we can organise the display of the Business Component to provide an attractive layout. To differentiate from the Bootstrap Card display of the list of businesses, we remove the card from the Business Component and instead generate a layout as shown below, where we have a Bootstrap row containing a single column with the business name and star rating above a row split into two equal columns, once containing the Lorem Ipsum description and the weather report and another containing the Google Map (or a representative image if you prefer not to sign up for a Google Cloud Platform developer account).

<table>
<tr><td colspan="2">Business name and rating</td></tr>
<tr><td>Business information as Lorem Ipsum text

Current weather report</td><td>Google Map centered on the business location</td></tr>
</table>

The code structure for the layout is described in the code box below, with the place to add the specific items of content indicated by comments.

File: bizFE/src/app/business.component.html

```
<div class="container"  style="margin-top: 70px">
    @for ( business of business_list; track business.name ) {

        <div class="row">
            <div class="col-sm-12">
                <div class="display-1">
                    {{ business.name }} of {{ business.town }}
                </div>
                <!-- Star rating here -->
            </div>
        </div>

        <div class="row">
            <div class="col-sm-6">
                <!-- Lorem Ipsum description here -->

                <div>
                    <p align="center">
                        <!—- Weather information here -->
                    </p>
                </div>
            </div>
            <div class="col sm-6 text-center">
                <!-- Google Map here -->
            </div>
        </div>
    }
</div>
```

Do it now!	Replace the current contents of ***business.component.html*** with the code structure shown above, inserting the code for the star rating, Lorem Ipsum descripton, weather information, and Google Map (or a representative image) at the locations indicated. Verify that your display of a single business now resembles that shown in Figure 14.8, below and that the information is updated correctly as each new business is selected from the directory.

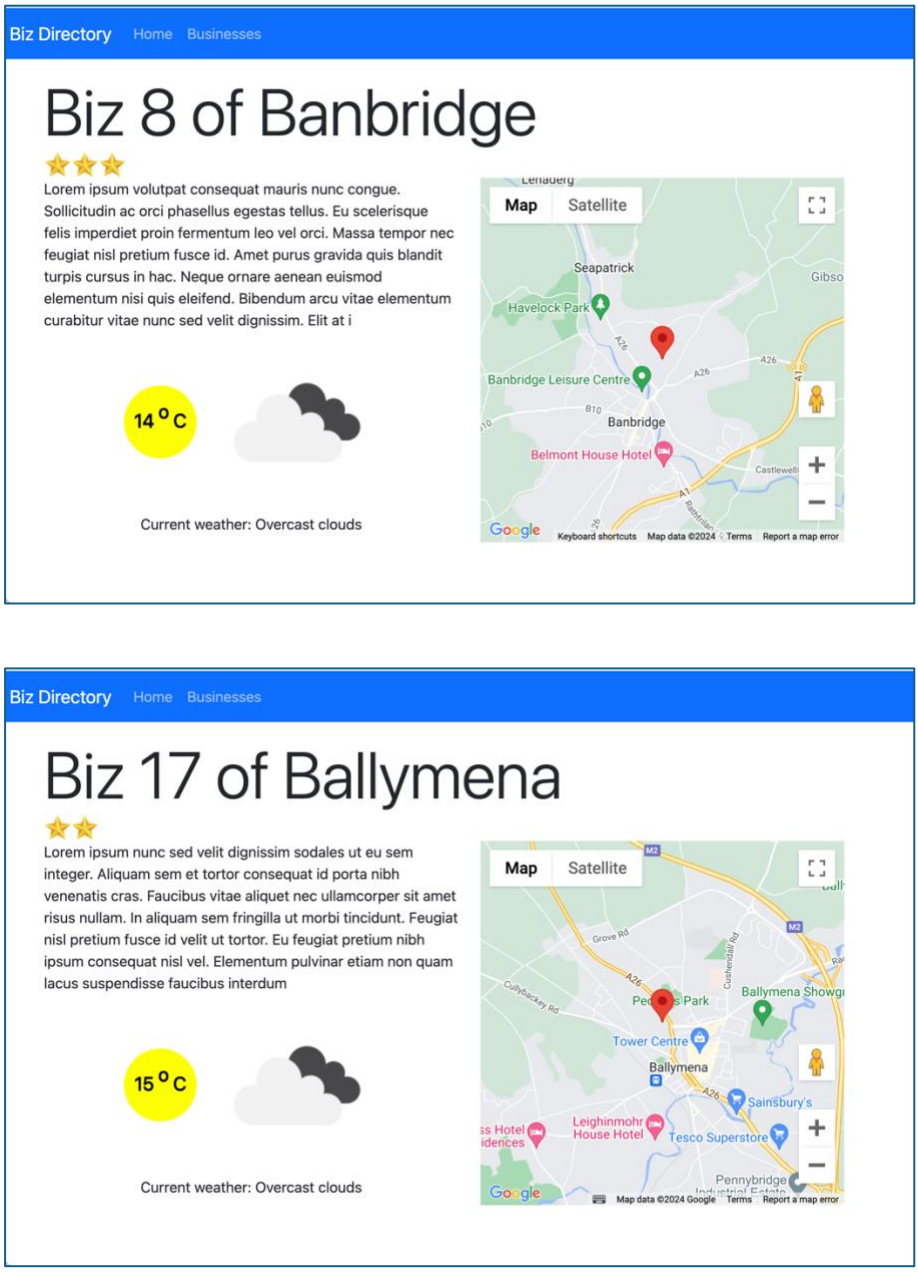


Figure 14.8 The Improved Business Component Template

14.6 Further Information

- https://www.techiediaries.com/angular-18-assets-folder/
  The Angular 18 Assets folder

- https://www.programiz.com/javascript/library/array/constructor
  JavaScript Array Constructor

- https://cloud.google.com/
  Google Cloud Platform

- https://developers.google.com/maps/get-started
  Getting Started with Google Maps Platform

- https://medium.com/@selsa-pingardi/integrating-google-maps-in-angular-17-66487ed2238c
  Integrating Google Maps in Angular

- https://timdeschryver.dev/blog/google-maps-as-an-angular-component
  Google Maps Component for Angular

- https://developers.google.com/maps/documentation/javascript/advanced-markers/migration
  Google Maps Advanced Markers

- https://api-ninjas.com/
  API Ninjas – Build Real Projects with Real Data

- https://api-ninjas.com/api/loremipsum
  Lorem Ipsum API

- https://openweathermap.org/current
  OpenWeatherMap Current weather API

- https://www.w3.org/TR/SVG/
  SVG Specification

- https://www.tutorialscampus.com/html5/svg-draw-circle-with-text.htm -google_vignette
  SVG Text in a Circle

- https://getbootstrap.com/docs/4.1/layout/grid/
  Bootstrap Grid System

COM661 Full Stack Strategies and Development

FE15. Sub-documents and Reactive Forms

Aims

- To generate sample reviews and add to the dataset
- To add functionality to retrieve and display sub-documents
- To introduce Bootstrap form classes
- To use the Angular FormBuilder to connect the form template to the component logic
- To introduce validation with Angular ReactiveForms
- To provide visual feedback associated with form validation
- To introduce two-way data binding

Table of Contents

15.1 RETRIEVING SUB-DOCUMENTS 2

15.1.1 GENERATING SAMPLE REVIEWS 2
15.1.2 DISPLAYING THE REVIEWS 8

15.2 FORMS 9

15.2.1 CREATE THE FORM 10
15.2.2 USING ANGULAR FORMBUILDER 12
15.2.3 SUBMITTING FORM VALUES 14

15.3 VALIDATION WITH REACTIVE FORMS 15

15.4 RETRIEVING AND POSTING FORM DATA 20

15.4.1 RETRIEVING THE DATA 21
15.4.2 SENDING THE POST REQUEST 21

15.5 FURTHER INFORMATION 24

15.1 Retrieving Sub-documents

So far, we have provided our Angular application with the capability to browse a paginated collection of businesses and select a single business to display in more detail. Each business object also has the facility to store a collection of user reviews, and the next stage of development is to retrieve and display the reviews for a selected business and to provide a means by which a user can add their own review.

15.1.1 Generating Sample Reviews

Just as we provided a collection of dummy data in the file ***businesses.json***, so we can modify the data file to include a random number of randomly-generated reviews for each business. The reviews will be generated by a new function that we will add to the Data Service and invoke when the application runs by adding a call to the new function in the **`ngOnInit()`** method of the App Component. The new Data Service function will be called **`populateReviews()`**, so the first step is to modify the App Component to import the **`DataService`**, make it available by injecting it into the class constructor, and provide an **`ngOnInit()`** method to call the new function.

File: bizFE/src/app.component.ts

```
...
import { DataService } from  './data.service';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [RouterOutlet, BusinessesComponent, NavComponent],
  providers: [DataService],
  templateUrl: './app.component.html',
  styleUrl: './app.component.css'
})
export class AppComponent {
  title = 'bizFE';

  constructor(private dataService: DataService) { }

  ngOnInit() {
    this.dataService.populateReviews();
  }
}
```

Do it now!	Make the modifications shown above to the App Component. The call to the **`populateReviews()`** function will generate an error at this point as it has not yet been defined, but this will be rectified in the following section.

Next, we add the new **`populateReviews()`** function to the Data Service. In this, we will make use of techniques for generating random data so that the appearance of the review on the web page is as realistic as possible. There is a lot going on in this function, so we will consider it a section at a time.

First, we declare a variable to hold the *Lorem Ipsum* string from which we will randomly select the review text, as well as the dummy review that we will add to the dataset

```
let loremIpsum = <String>"";
let dummyReview = <any>{};
```

Next, we make a call to our existing Data Service function **`getLoremIpsum()`**to request a single paragraph of dummy text from the **api-ninjas** endpoint that we have already used. As the call to **`getLoremIpsum()`** returns an **`Observable`**, we need to **`subscribe`** to it so that we can use the result in this function. Once the API produces the text, we copy it to our local **`loremIpsum`** variable.

```
...

this.getLoremIpsum(1).subscribe((response: any) => {
    loremIpsum = response.text;
}
```

Now that the Lorem Ipsum has been retrieved, we can visit each business in the collection and generate a random number of reviews (in the range 0-9) that we will add to that business.

```
...

this.getLoremIpsum(1).subscribe((response: any) => {
    loremIpsum = response.text;
    jsonData.forEach( function(business) {
       let numReviews = Math.floor(Math.random() * 10);
       for (var i = 0; i < numReviews; i++) {

       }
    })
}
```

Now, to generate the text for each review, we generate the number of characters in the text as a random value in the range 10-300 and then choose a random starting point in the *Lorem Ipsum* making sure that the starting point leaves sufficient text in the string to satisfy the review length that has been generated.

```
...

       for (var i = 0; i < numReviews; i++) {
           let textSize = Math.floor(Math.random() * 290 + 10);
           let textStart = Math.floor(Math.random() *
                          (loremIpsum.length - textSize));

       }
...
```

With all the required values in place, we can now generate the dummy review. The **`username`** field will be "User " plus a random integer in the range 1-9999; the **`comment`** field is a slice of the *Lorem Ipsum* beginning at the calculated **`textStart`** position and containing **`textSize`** characters, and the **`stars`** field is a random integer value in the range 1-5.

Once generated, we add the review by pushing it onto the **`reviews`** list for the business.

```
    ...

            for (var i = 0; i < numReviews; i++) {
                let textSize = Math.floor(Math.random() * 290 + 10);
                let textStart = Math.floor(Math.random() *
                                (loremIpsum.length - textSize));
                dummyReview = {
                     'username' : 'User ' +
                           Math.floor(Math.random() * 9999 + 1),
                     'comment' : loremIpsum.slice(textStart,
                                          textStart + textSize),
                     'stars' : Math.floor(Math.random() * 5) + 1
                 };
                 business['reviews'].push(dummyReview);

            }
    ...
```

The full **`populateReviews()`** function is shown in the following code box.

Do it now!	Implement the **`populateReviews()`** function in the Data Service as shown in the code box below.

File: bizFE/src/data.service.ts

```
...

populateReviews() {

  let loremIpsum = <String>"";
  let dummyReview = <any>{};

  this.getLoremIpsum(1).subscribe((response: any) => {
    loremIpsum = response.text;

     jsonData.forEach( function(business) {
        let numReviews = Math.floor(Math.random() * 10);
        for (var i = 0; i < numReviews; i++) {
           let textSize = Math.floor(Math.random() * 290 + 10);
           let textStart = Math.floor(Math.random() *
                          (loremIpsum.length - textSize));
           dummyReview = {
                'username' : 'User ' +
                      Math.floor(Math.random() * 9999 + 1),
                'comment' : loremIpsum.slice(textStart,
                                      textStart + textSize),
                'stars' : Math.floor(Math.random() * 5) + 1
            };
            business['reviews'].push(dummyReview);
        }
     })
  }

...
```

When you code **`populateReviews()`** you may see that the final operation to push the new review into the reviews list of the business generates a code error as shown in Figure 15.1, below.

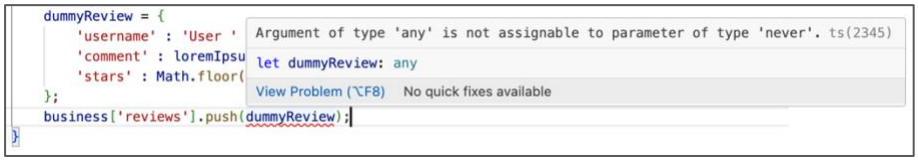


Figure 15.1 Reviews Object Assigned Type **never**.

This error occurs because of the strong typing enforced by TypeScript. When the file ***businesses.json*** contains an empty reviews list for every business, TypeScript assumes that this will always be the case and automatically assigns reviews a type of **never** – denoting

that the field will never contain a value. Obviously, this is not our intention and, since this is dummy data, the easiest fix is to provide one of the entries in the ***businesses.json*** data file with a dummy review, as shown in Figure 15.2, below.



```
[
    {
        "_id": {
            "$oid": "66df1c8fcf0ec82461958517"
        },
        "name": "Biz 0",
        "town": "Derry",
        "rating": 5,
        "reviews": [
            {
                "username" : "User 999",
                "comment" : "Lorem ipsum",
                "stars" : 3
            }
        ],
        "num_employees": 79,
```

*Figure 15.2 Adding a Dummy Review to the **businesses.json** Dataset*

Now that the error has been resolved, we can show that the reviews have been created by logging them to the Browser Console once the business data has been returned from the Data Service by a call from the Business Component **`ngOnInit()`** function.

File: bizFE/src/business.component.ts

```
...

    ngOnInit() {
        this.business_list =  this.dataService.getBusiness(
                              this.route.snapshot.paramMap.get('id');)
        console.log(this.business_list[0]['reviews'])

...
```

We can now run the application with the Browser Console open, select one of the businesses, and see how the randomly generated reviews have been generated and retrieved, as shown in Figure 15.3, below.

Do it now!	Modify the dataset by adding a review to one of the businesses and run the application with the Browser Console open. Verify that you see the reviews that have been generated by expanding the element in the Console.

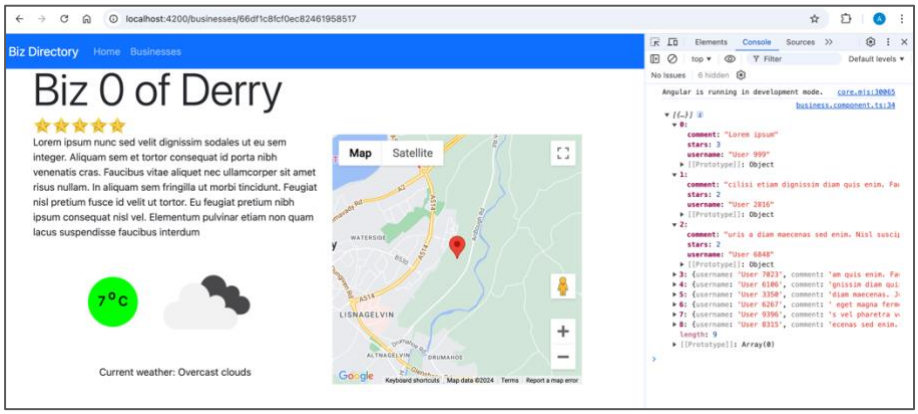


Figure 15.3 Randomly Generated Reviews in the Browser Console

15.1.2 Displaying the Reviews

With the reviews available in the Business Component for display in the browser, we can now add code to the Component HTML template to render them to the display.

We can add this to our existing layout by providing a new Bootstrap row containing a single column that spans the entire container width. Inside this, we use a **`@for`** block to iterate across each element of the **`business.reviews`** list. As for the Businesses Component, we will display each review as a Bootstrap **`card`**, except this time we will use the **`bg-light`** style class to distinguish a review **`card`** from a business **`card`**.

Inside the card, we display the **`username`** property of the review in the card header, the randomly generated **`comment`** text in the card body, and an indication of the random **`rating`** in the card footer. The effect is to display the collection of reviews below the business details as illustrated in Figure 15.4 below.

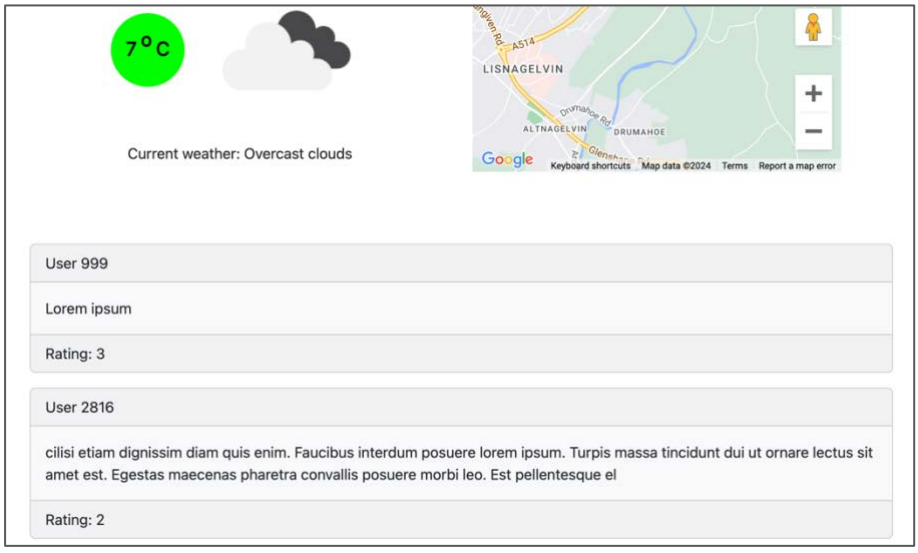


Figure 15.4 Displaying the Reviews

File: bizFE/src/business.component.html

```
      <div class="container"  style="margin-top: 70px">
         @for ( business of business_list; track business.name ) {
         ...

           <div class="row" style="margin-top: 70px">
              <div class="col-sm-12">

                @for ( review of business.reviews; track review ) {

                    <div class="card bg-light mb-3">

                        <div class="card-header">
                            {{ review.username }}
                        </div>
                        <div class="card-body">
                            {{ review.comment }}
                        </div>
                        <div class="card-footer">
                            Rating: {{ review.stars }}
                        </div>

                    </div>
                }

              </div>
           </div>


         }
      </div>
```

Do it now!	Add code to the Business Component HTML template as shown in the code box above and verify that you can see the randomly generated collection of reviews as demonstrated by Figure 15.4.

15.2 Forms

In order to invite the user to contribute a new review for the business, we will add an HTML form below the list of reviews. Angular provides a powerful form specification and manipulation facility that implements the form structure as a model and enables two-way binding between the model and the form fields. We will see in this section how this two-

way binding provides a powerful validation tool as well as making it easy for us to retrieve data from the form after submission.

As an initial step, we need to import the **`ReactiveFormsModule`** class into the Business Component and add it to the Component **`imports`** list.

File: bizFE/src/business.component.ts

```
...

import { ReactiveFormsModule } from '@angular/forms';

@Component({
  selector: 'business',
  standalone: true,
  imports: [RouterOutlet, CommonModule, GoogleMapsModule,
            ReactiveFormsModule],
  providers: [DataService],
  templateUrl: './business.component.html',
  styleUrl: './business.component.css'
})

...
```

15.2.1 Create the Form

Next, we add a form to the Business Component template with 3 fields

- A user name
- A free text review
- A star rating in the range 1-5

The username is implemented as an HTML **`input`** box, the review is implemented as a **`textarea`** component, and the star rating is provided as a drop-down list (using the **`<select>`** and **`<option>`** tags). All style classes in the code at this stage are basic Bootstrap 5 properties – with the exception of **`formControlName`** which is an Angular property that is used to bind the form field to an element in the model that describes the data structure representing the form.

File: bizFE/src/business.component.html

```
...

<div class="container" style="margin-top: 50px;">
    <div class="row">
        <div class="col-sm-12">
            <h2>Please review this business</h2>
            <form>
                <div class="form-group">
                    <label for="username">Username</label>
                    <input type="text" name="username"
                           id="username" class="form-control"
                           formControlName="username">
                </div>
                <div class="form-group">
                    <label for="comment">
                        Please leave your review below</label>
                    <textarea name="comment" id="comment"
                              class="form-control"
                              formControlName="comment">
                    </textarea>
                </div>
                <div class="form-group">
                    <label for="stars">
                          Please provide a rating (5 = best)
                    </label>
                    <select name="stars" id="stars"
                            class="form-control"
                            formControlName="stars">
                        <option value="1">1 star</option>
                        <option value="2">2 stars</option>
                        <option value="3">3 stars</option>
                        <option value="4">4 stars</option>
                        <option value="5">5 stars</option>
                    </select>
                </div>
                <button class="btn btn-primary"
                        type="submit"
                        style="margin-top: 20px">
                    Submit
                </button>
            </form>
        </div>
    </div>
</div>
```

15.2.2 Using Angular FormBuilder

Now that the form has been implemented in the template, we update the Component's TypeScript file to import the **`FormBuilder`** class, inject it into the Component **`class`** by providing it as a parameter to the class constructor, and specify the model that describes the data structure represented by the form.

Note that we use '***username'***, '***comment*** and '***stars***' as the field names in the model – matching the values used for **`formControlName`** properties in the template.

File: bizFE/src/business.component.ts

```
...

import { FormBuilder } from '@angular/forms';

...

export class BusinessComponent {

    ...

    reviewForm: any;

    constructor( public dataService: DataService,
                 private route: ActivatedRoute,
                 private formBuilder: FormBuilder) {}

    ngOnInit() {

        this.reviewForm = this.formBuilder.group( {
          username: '',
          comment: '',
          stars: 5
        })

    ...
```

We then complete the connection between the **`formBuilder.group()`** and the form by specifying the **`reviewForm`** model as the value of the **`formGroup`** property in the **`<form>`** tag.

File: bizFE/src/business.component.html

```
...

<div class="col-sm-12">
            <h2>Please review this business</h2>
            <form [formGroup] = "reviewForm">
                <div class="form-group">

...
```

Now, when we run the application and click on an individual business entry, we should now see the form to provide a new review displayed at the bottom of the page, as seen in Figure 15.5 below.

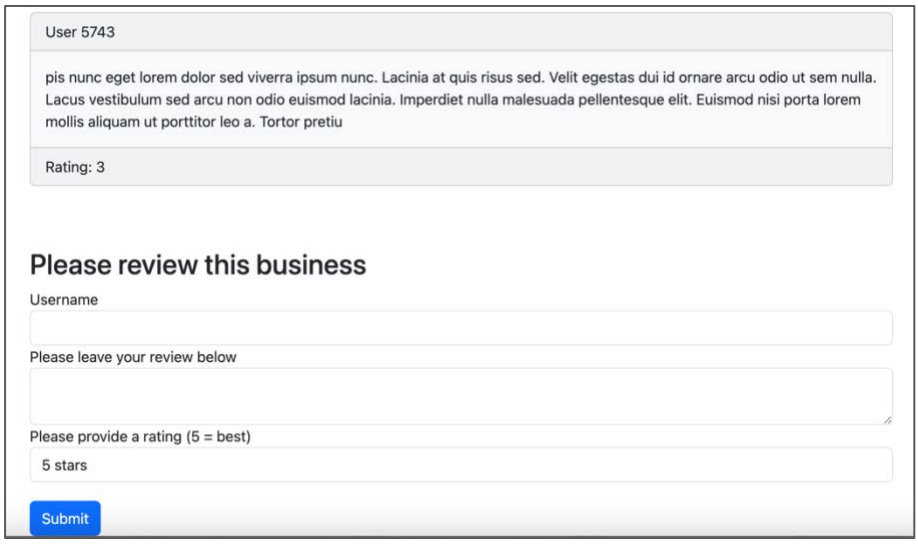


Figure 15.5 A Review Form

This is another demonstration of the binding provided by Angular. Note that we did not specify any of the 5 options for the **`<select>`** element buttons as selected in the form – yet Angular has chosen to use the '5 stars' option as the default. This is actually specified in the **`formBuilder.group()`** definition that we added to the **`BusinessComponent`** TypeScript file.

Do it now!	Add the code to the Business Component HTML template and TypeScript files as shown above to create the review form and display it below the list of reviews for the business display it. Confirm that you see output such as that demonstrated in Figure 15.5, above.

15.2.3 Submitting Form Values

Angular forms are submitted by binding a function to the form's **`ngSubmit`** property. As a first step, we will bind the **`onSubmit()`** function, which we then implement in the **`BusinessComponent`** as a simple **`console.log()`** of the model.

File: bizFE/src/business.component.html

```
...

<div class="col-sm-12">
            <h2>Please review this business</h2>
            <form [formGroup] = "reviewForm"
                  (ngSubmit) = "onSubmit()">
                <div class="form-group">

...
```

File: bizFE/src/business.component.ts

```
...

export class BusinessComponent{

    ...

    onSubmit() {
      console.log(this.reviewForm.value);
    }

...
```

Entering values into the form fields and clicking the submit button generates a browser console message as shown in Figure 15.6 below.

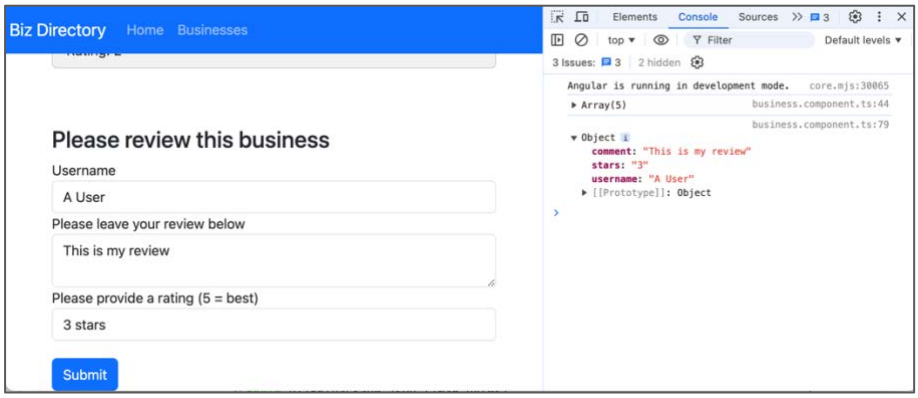


Figure 15.6 Form Values Submitted

Do it now!	Add the **`onSubmit()`** functionality as described above. Provide values in the review form and make sure that you can see the submitted values in the Browser Console.

15.3 Validation with Reactive Forms

Angular provides a powerful set of validation elements that can be used to change the appearance or operation of the form in response to user input (or lack of input). In order to apply validation, we first need to import the **`Validators`** class into the **`BusinessComponent`**.

Next, we specify that we want to ensure that data is provided for the **`name`** and **`review`** fields of the form by adding the **`Validators.required`** property to the **`name`** and **`review`** fields. We do this by converting the value of each **`formBuilder.group()`** property as a list, where the first element is the default value to be used and the second is the validation requirement. As the **`stars`** rating does not require validation (it is impossible to avoid choosing one of the values), we have no change to make for that element.

Finally, we observe the effect of the validation by modifying the **`console.log`** to output the form's **`valid`** property.

File: bizFE/src/business.component.ts

```
...

import { FormBuilder, Validators } from '@angular/forms';

...

    ngOnInit() {

        this.reviewForm = this.formBuilder.group( {
          username : ['', Validators.required],
          comment: ['', Validators.required],
          stars: 5
        })

...

    onSubmit() {
      console.log(this.reviewForm.valid);
    }

...
```

When we check this in the browser, we see that the value **`false`** is logged to the browser console if the form is submitted with either of the **`username`** or **`comment`** fields left blank, while true is only reported if values are provided for both text fields.

Do it now!	Add validation to the Business Component TypeScript file as shown in the code box above. Make sure that the message **`false`** is shown in the Browser Console when you submit a review with one of the text fields left empty.

We would like to use this validation to provide visual feedback to the user informing them that a value for the field is required. First, we will add a style rule to the ***business.component.css*** file that will be used to apply a light red background to unfilled text entry fields.

File: bizFE/src/business.component.css

```
.error { background-color: #fff0f0 }
```

Now, we bind the new error class to the **<input>** box for the user name when the **invalid** property of the **name** control is true. The **ngClass** binding takes a JSON object (in the form **{ name : value }**) where the **name** element is the style rule and the **value** element is the condition that must be satisfied for the rule to be applied. This code can be read as "apply the error class to the name element when the contents of the form field are invalid". Whether or not the value is invalid is determined by the validation rule described in the **formBuilder.group()** definition – i.e. that the value is **required**.

File: bizFE/src/business.component.html

```
...

<input type="text" name="username" id="username"
       class="form-control" formControlName="username"
       [ngClass]="{ 'error' :
                    reviewForm.controls.username.invalid }">

...
```

Checking in the browser shows that the text box is highlighted even before the user has had an opportunity to enter any data. This is not exactly what we want, so we add another rule to the **[ngClass]** definition to apply the background colour only if the field has been **touched** (i.e. modified).

File: bizFE/src/business.component.html

```
...

[ngClass]="{ 'error' : reviewForm.controls.username.invalid &&
                       reviewForm.controls.username.touched }">

...
```

This is a much more pleasing effect, so we apply the same rule to the **comment** textbox.

File: bizFE/src/business.component.html

```
...

<textarea name="comment" id="comment" class="form-control"
    formControlName="comment"
    [ngClass]="
        { 'error' : reviewForm.controls.comment.invalid &&
                    reviewForm.controls.comment.touched }">
</textarea>

...
```

This validation now works as required, but it can be easily seen that if we had a larger number of text inputs on the form, this approach would lead to a large volume of duplicated code. We can avoid this by re-factoring the code to implement the check for invalid input in a function which accepts the name of the form control as a parameter

File: bizFE/src/business.component.ts

```
...

isInvalid(control: any) {
   return this.reviewForm.controls[control].invalid &&
          this.reviewForm.controls[control].touched;
}

...
```

We can then modify the **ngClass** binding rule in the form to call the new function, passing the name of the control as a parameter.

File: bizFE/src/business.component.html

```
...

 <input type="text" name="username" id="username"
        class="form-control" formControlName="username"
        [ngClass]="{ 'error' : isInvalid('username') }">

...

 <textarea name="comment" id="comment" class="form-control"
           formControlName="comment"
           [ngClass]="{ 'error' : isInvalid('comment') }">
 </textarea>

...
```

Our final validation stage will be to provide a feedback message if the user attempts to leave a required field blank, while at the same time removing the submit button to prevent invalid submission. First we create new functions **`isUntouched()`** and **`isIncomplete()`** that return **`true`** if either field has not been touched (is still **`pristine`**) or either text input is invalid. The **`isUntouched()`** function covers the initial state where no user input has been provided to either field, while **`isIncomplete()`** combines this with the situation where data has been provided and then removed.

File: bizFE/src/business.component.ts

```
...

    isUntouched() {
      return this.reviewForm.controls.username.pristine ||
             this.reviewForm.controls.comment.pristine;
    }

    isIncomplete() {
      return this.isInvalid('username') ||
             this.isInvalid('comment') ||
             this.isUntouched();
    }

...
```

Now, we apply this to a new **`<span>`** object containing a message – and use the Angular **`*ngIf`** directive to display either this message or the "submit" button. **`*ngIf`** is a very

useful feature that can be applied to any HTML element to dynamically modify the structure and content of the page in response to dynamic activity.

File: bizFE/src/business.component.html

```
...

   <span *ngIf="isIncomplete()">
       You must complete all fields
   </span>
   <button *ngIf="!isIncomplete()" class="btn btn-primary"
           type="submit"
           style="margin-top: 20px">Submit</button>

...
```

Viewing the application in the browser verifies that we now have the desired validation.



Figure 15.7 Invalid Data and User Feedback

Do it now!	Implement the validation code as described in the above sections. Test it in the browser and ensure that the correct visual feedback is provided when required fields are not completed.

15.4 Retrieving and POSTing Form Data

Now that the review form is specified and validation is in place, we will add the functionality that allows the values provided by the user to be POSTed to the Data Service.

15.4.1 Retrieving the Data

We have already how the username, comment and stars values entered by the user on the form are available in the **`reviewForm.value`** object of the Business Component. For clarity, we can repeat that check by modifying the **`onSubmit()`** method of ***business.component.ts*** as shown below.

File: bizFE/src/business.component.ts

```
...

    onSubmit() {
      console.log(this.reviewForm.value);
    }

...
```

If we run the application and check the message in the Browser Console, we can again see that the username, review text and star rating are available. We can now create the new Data Service method **`postReview()`** to add the review to the collection. **`postReview()`** takes as parameters the **`_id`** value of the business to which the review should be attached and the contents of the review form. First, it constructs the **`newReview`** object by extracting the fields from the form. Then, it iterates across the businesses in the collection, checking for that with a matching **`_id`** value. When a match is found, the new review is pushed onto the list of reviews for that business, as shown in the code box below.

File: bizFE/src/data.service.ts

```
...

    postReview(id: any, review: any) {
        let newReview = {
            'username' : review.username,
            'comment' : review.comment,
            'stars' : review.stars
        };
        jsonData.forEach( function(business) {
            if ( business['_id']['$oid'] == id ) {
                business['reviews'].push( newReview );
            }
        });
    }

...
```

Next, we amend the Business Component **`onSubmit()`** to call the Data Service **postReview()** method. We also include a call to the form's **`reset()`** method to restore the form to its original state once the new review has been accepted.

File: bizFE/src/business.component.ts

```
...

    onSubmit() {
        this.dataService.postReview(
                        this.route.snapshot.paramMap.get('id'),
                        this.reviewForm.value);
        this.reviewForm.reset();
    }

...
```

Running the application in the browser and submitting a review should confirm that the new functionality is now complete.

Do it now!	Make the amendments to the Business Component and Data Service shown above and run the application. Select a business and observe its reviews displayed on the page. Now submit a review for the business and confirm that it has been added to the collection as shown in Figure 15.8 and 15.9, below.

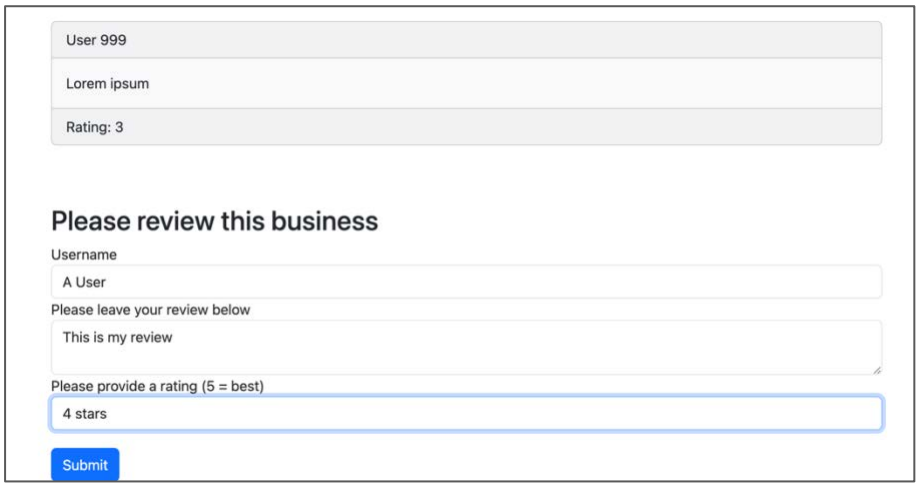


Figure 15.8 Submitting a Review



Figure 15.9 Review Accepted and Added to the Collection

15.5 Further Information

- https://coreui.io/docs/components/forms/
  Bootstrap Forms

- https://www.w3schools.com/Bootstrap/bootstrap_forms.asp
  Bootstrap Forms – W3Schools

- https://angular.dev/guide/forms/reactive-forms
  Angular ReactiveForms – the online manual

- https://malcoded.com/posts/angular-fundamentals-reactive-forms/
  Reactive Forms with Angular – using easy examples!

- https://angular.dev/api/forms/FormBuilder
  Angular FormBuilder class

- https://coryrylan.com/blog/angular-form-builder-and-validation-management
  Angular Form Builder and Validation Management

- https://malcoded.com/posts/angular-reactive-form-validation/
  Validating Reactive forms in Angular

- https://angular.dev/guide/forms/form-validation
  Angular Form Validation

- https://angular.dev/guide/templates/two-way-binding
  Two-way binding in Angular

COM661 Full Stack Strategies and Development

FE16. Front-end Authentication

Aims

- To identify authentication as a key component in online applications
- To introduce Authentication as a Service and Auth0 as an authentication provider
- To install Auth0 and integrate it with an Angular application
- To create an Authentication Service
- To implement authenticated login and logout
- To implement restricted access to selected content
- To add authentication functionality into the navigation bar

Table of Contents

16.1 AUTH0 AUTHENTICATION 2

16.1.1 Authentication as a Service 2
16.1.2 Signing up for Auth0 2
16.1.3 Configuring Auth0 3
16.1.4 Install the Auth0 package 5

16.2 INTEGRATING AUTH0 WITH THE APPLICATION 5

16.2.1 Auth0 Setup 6
16.2.2 Create the Auth Button Component 7
16.2.3 Adding Logout Functionality 10
16.2.4 User Profile 12

16.3 ADD AUTHENTICATION FUNCTIONALITY TO THE NAVBAR 15

16.3.1 Move Authentication Functionality 15
16.3.2 Adding an Inline Button 17
16.3.2 Redirecting After Login 18
16.3.3 Restricting Access to Selected Content 20

16.4 FURTHER INFORMATION 23

16.1 Auth0 Authentication

Authentication is emerging as one of the most critical aspects of web-based software development. Digital security has become front-page news and a lack of user confidence in the security of their data is one of the main factors that limits the uptake of new systems.

When developing our back-end API, we implemented authentication ourselves using JSON Web Tokens that were passed in the header of requests to the API. Another option is to use a 3rd party provider, and we will investigate this in providing an authentication layer to our front-and application.

16.1.1 Authentication as a Service

Traditionally, software developers have handled authentication in-house, using self-written routines to handle login, authenticate transactions and protect from unauthorized users; but recently a number of 3rd party vendors have made available cloud-based systems which accept requests from user software and handle user authentication before passing control back to the host application.

One of the most popular providers of Authentication as a Service (AaaS) is **Auth0** (https://auth0.com), who provide a powerful free service which handles user authentication along with an extensive dashboard for user analysis.

16.1.2 Signing up for Auth0

To use Auth0 in our application, we first need to sign up for a free account. Visit https://auth0.com and click on the "Sign Up" button in the top-right corner. You should then be presented with the screen shown in Figure 15.1 below. You can sign up by providing a username and password or through a social media account.

Do it now!	Sign up for Auth0 and open a free account. You can provide a dedicated Auth0 account or signup using one of your existing social media accounts.

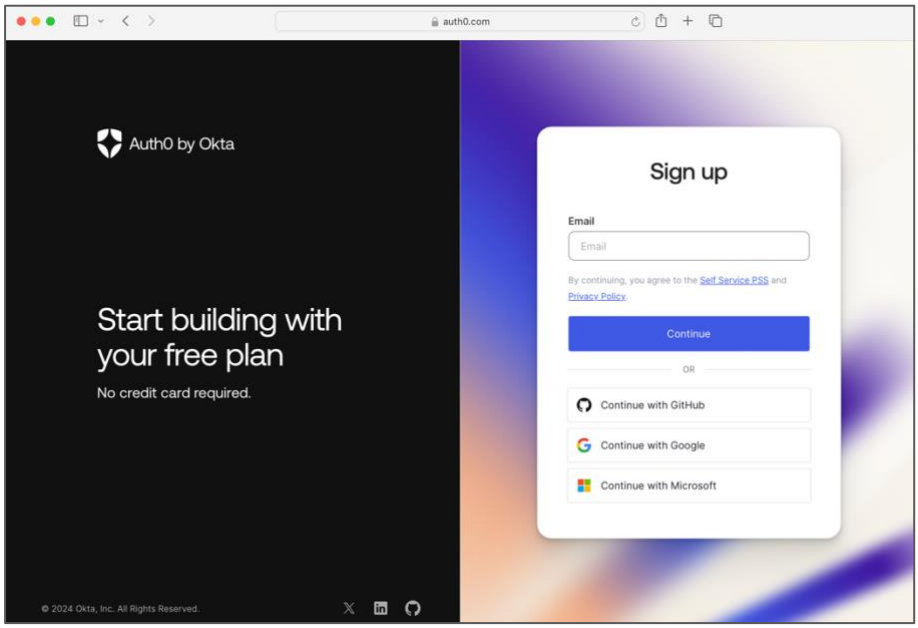


Figure 16.1 Auth0 Sign up

16.1.3 Configuring Auth0

When your Auth0 account is created and your profile has been established, you will be presented with the Auth0 Dashboard as shown in Figure 16.2, below.

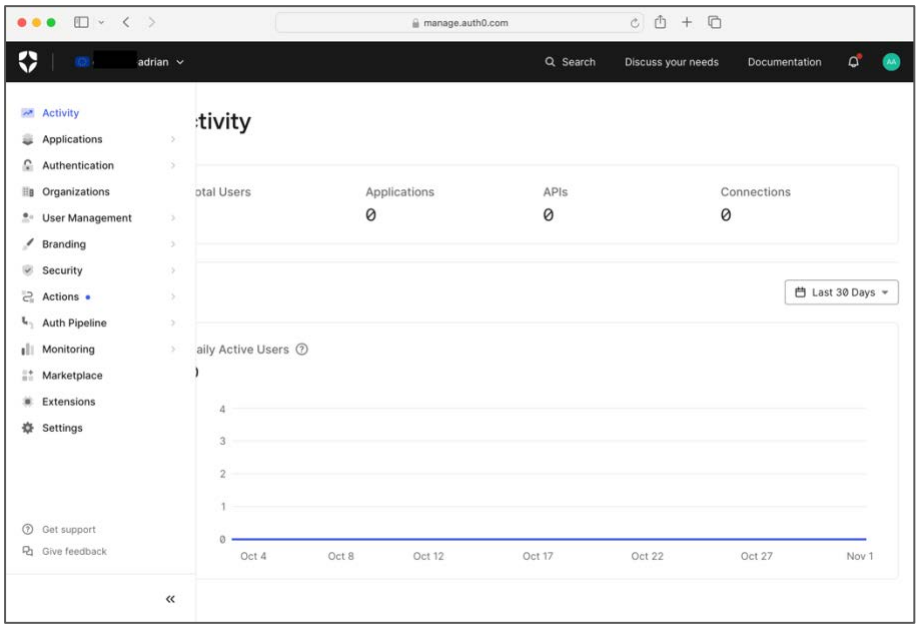


Figure 16.2 The Auth0 Dashboard

The dashboard is the control centre from where you can manage your applications and users. For our purposes the first activity is to create a new application by clicking on the "Applications" link on the menu on the left, then "Applications" from the sub-menu, and finally the "Create Application" button in the top-right of the browser. In response, you will

be presented with a screen where you provide the name for your application and a choice of application types, from which you should choose “Single Page Web Applications”, as illustrated in Figure 16.3, below.

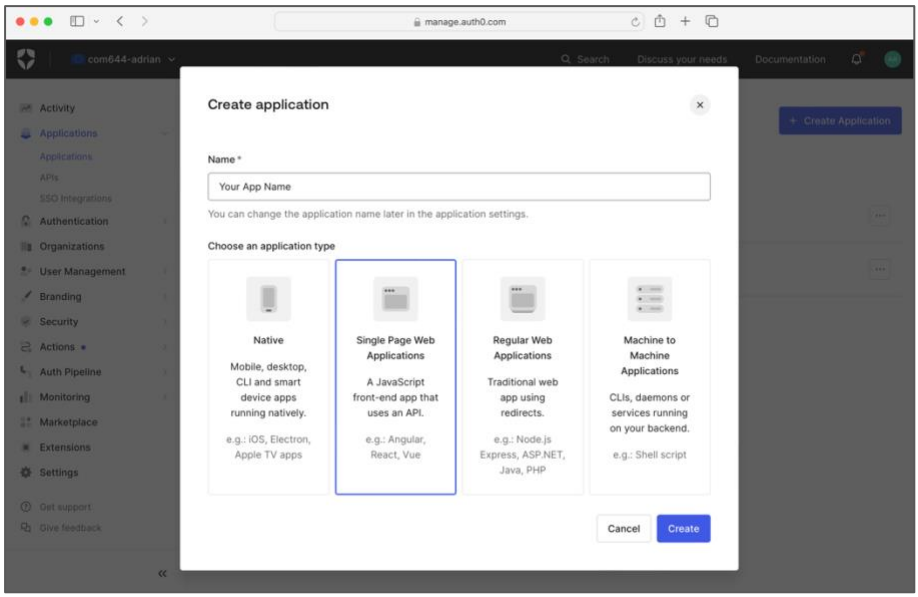


Figure 16.3 Create a New Application

Next, in response to the question ‘What technology are you using for your web app?’, click on the icon for ‘Angular’, as shown in Figure 16.4, below.

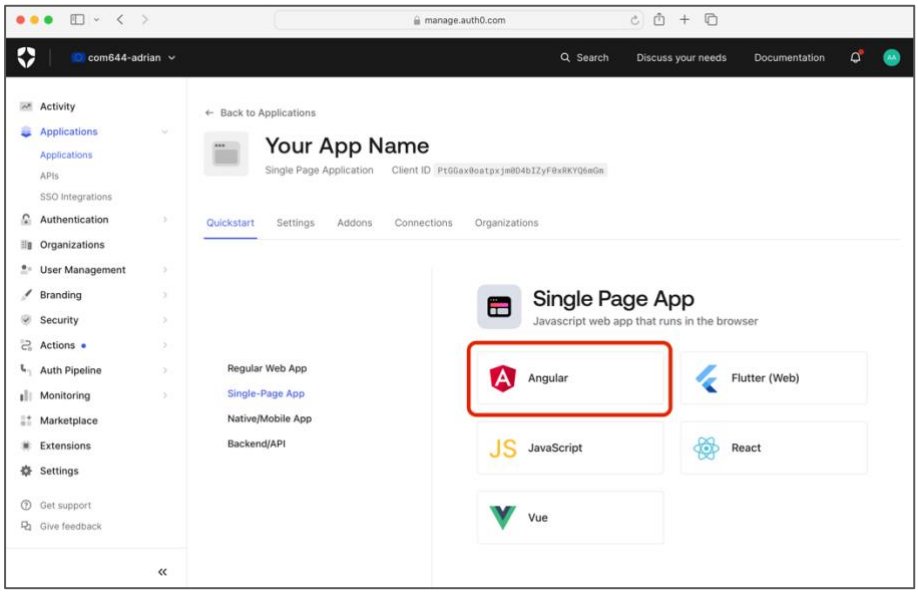


Figure 16.4 Choose Platform

Then, from the application configuration page, we need to click the “Settings” tab from the menu and set the “Allowed Callback URLs”, “Allowed Logout URLs” and “Allowed Web Origins” each to http://localhost:4200, as shown in Figure 16.5, below.

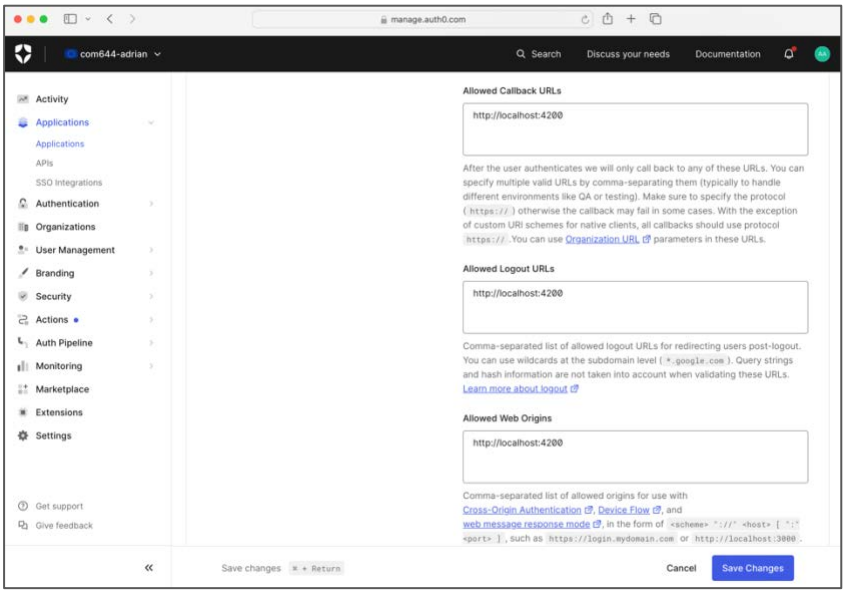


Figure 16.5 Application Settings

The Callback URL and Logout URLs are the locations to which the user will be redirected after a login and logout event, respectively; while the Allowed Web Origin permits Auth0 to silently refresh the authentication tokens when the user visits or refreshes a page, so preventing repeated login.

Do it now!	Create a new Auth0 application as directed above and provide the settings for the application configuration. Remember to click “Save Changes” at the bottom of the configuration page

16.1.4 Install the Auth0 package

The final preparation step is to add the Auth0 package to our Angular application. We do this by using use **npm** to install the **auth0-angular** package as shown in the following command.

```
C:\bizFE> npm install @auth0/auth0-angular --save
```

Do it now!	Install the auth0-angular package to the bizFE application as shown above.

16.2 Integrating Auth0 with the Application

When we install the **auth0-angular** package, we make available a Software Development Kit (SDK) that exposes several elements that enable us to integrate Auth0 with our application.

In this section, we perform the necessary setup and initialization, and then add login and logout buttons to our application home page.

16.2.1 Auth0 Setup

The Auth0 website provides an excellent Quickstart guide that you can access from the home page of your Auth0 application. When you select the QuickStart link and choose Angular from the list of available platforms that are supported, you are greeted with the step-by-step guide shown in Figure16.6, below.

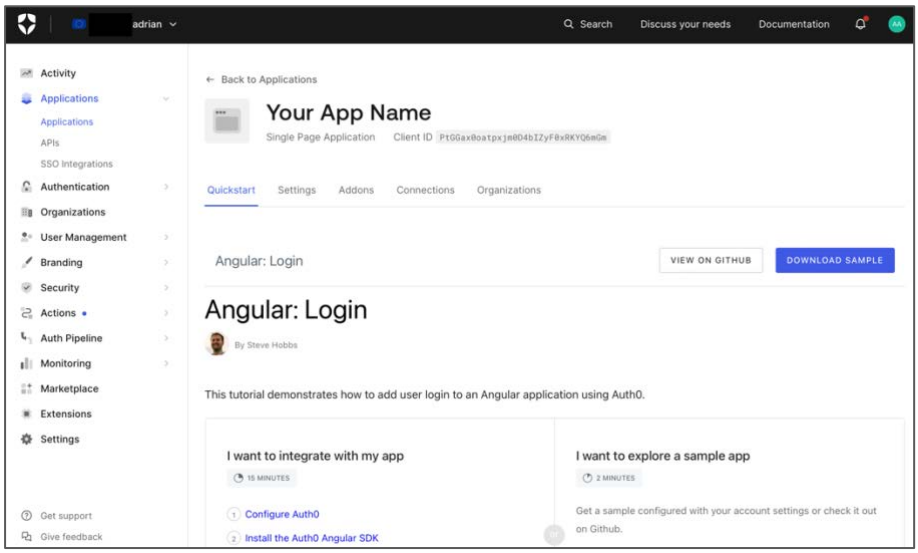


Figure 16.6 The Auth0 Quickstart Guide

Note:	In the sections that follow, you can copy and paste the code from the Quickstart guide into your application, but please note that the code presented in the code boxes in this section differs from the Quickstart code in some areas to reflect functionality already implemented in our application.

The first step is to replace the code in the file ***main.ts*** with that presented in the code box below. Note that your personal **`domain`** and **`clientId`** values will already be populated for you in the code seen in the Quickstart guide. Note also that this code differs from that in the Quickstart guide since we have had to integrate the Quickstart code with the existing functionality of our ***main.ts***.

File: bizFE/src/main.ts

```
import { bootstrapApplication } from
                                  '@angular/platform-browser';
import { AppComponent } from './app/app.component';
import { appConfig } from './app/pri';
import { provideAuth0 } from '@auth0/auth0-angular';
import { provideHttpClient } from '@angular/common/http';

bootstrapApplication(AppComponent, {
  providers: [
    provideAuth0({
      domain: 'YOUR-DOMAIN-NAME,
      clientId: 'YOUR-CLIENT-ID',
      authorizationParams: {
        redirect_uri: window.location.origin
      }
    }),
    provideHttpClient(),
    appConfig.providers
  ]
}).catch((err) => console.error(err));
```

Do it now!	Update ***main.ts*** with the code shown in the code box above. You can also run the application at this stage to verify that no errors have been introduced by Auth0 integration.

16.2.2 Create the Auth Button Component

Next, we add the **`AuthButton`** Component that will be added to the application to enable login and login, though initially, we will just consider login. As usual, the component is represented by a TypeScript class definition and a HTML template (we choose not to provide an associated CSS file).

The TypeScript file imports the standard Component package and the Auth0 Authentication Service and makes the service available by injecting it to the Component by the usual method of providing it as a parameter to the component constructor.

File: bizFE/src/app/authbutton.component.ts

```
import { Component } from '@angular/core';
import { AuthService } from '@auth0/auth0-angular';

@Component({
  selector: 'auth-button',
  templateUrl: 'authbutton.component.html',
  standalone: true
})

export class AuthButtonComponent {

  constructor(public auth: AuthService) {}
}
```

In the HTML template, we define the button as a standard Bootstrap button and bind the button click event to the **`AuthService loginWithRedirect()`** method.

File: bizFE/src/app/authbutton.component.html

```
<button class="btn btn-primary"
        (click)="auth.loginWithRedirect()">
    Log in
</button>
```

Now we can add the **`AuthButton`** Component to the home page of our application by importing it into the TypeScript file and adding it to the list of imports.

File: bizFE/src/app/home.component.ts

```
import { Component } from '@angular/core';
import { RouterOutlet } from '@angular/router';
import { AuthButtonComponent } from "./authbutton.component";

@Component({
  selector: 'home',
  standalone: true,
  imports: [RouterOutlet, AuthButtonComponent],
  templateUrl: './home.component.html',
  styleUrl: './home.component.css'
})
export class HomeComponent {}
```

Finally, we can display the button on the home page by adding its selector tag below the existing heading.

File: bizFE/src/app/home.component.html

```
<h1 style="margin-top: 70px">
    Biz Directory
</h1>

<auth-button></auth-button>
```

Do it now!	Add the **`AuthButton`** to the home page by making the modification shown in the four code boxes above. Verify that the button has been added by visiting the home page of the application and observing the display in the browser as shown by Figure 16.7, below.

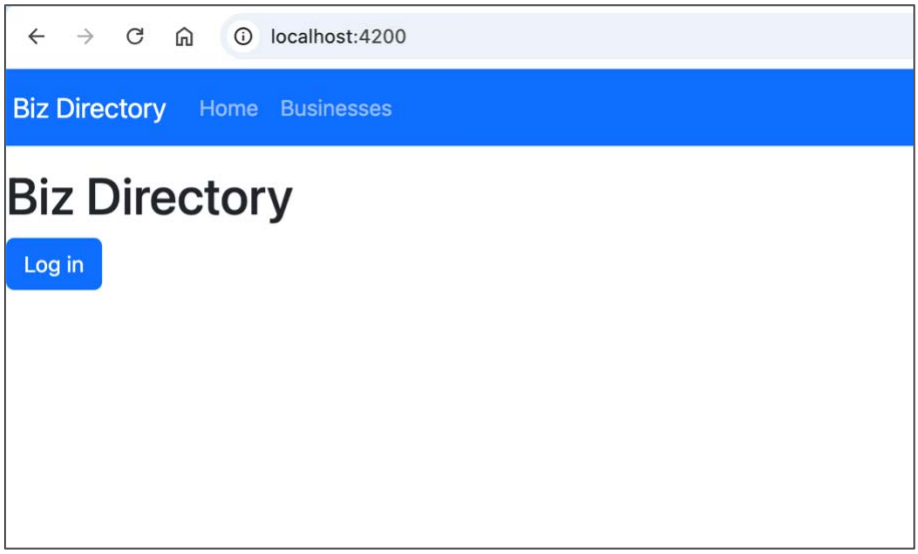


Figure 16.7 Login Button added to Home Page

Do it now!	At this stage, the button is already functional so click it and verify that you are redirected to the Auth0 Log in and Sign up screen. If you sign up for the application, using either a username and password or a social media account, you should find that you are redirected to the home page of the application.

Note:	It is important to distinguish between signing up for Auth0 as you did earlier and signing up for the application. The first is opening a developer account with Auth0 to enable you to deploy their authentication services in your applications. The second is signing up as a user of the application being protected by the authentication service.

16.2.3 Adding Logout Functionality

Now that users can sign in to the application, we to provide a way for them to sign out. This is implemented by using the **`logout()`** method provided by the **`AuthService`** service. Rather than specify separate components for login and logout, we will extend the functionality of our existing **`AuthButton`** so that is can cope with both uses.

The **`AuthService logout()`** method requires access to the Document Object Model to redirect the browser back to the specified origin. This is provided by the Angular **`DOCUMENT`** service that we need to import into the **`AuthButton`** TypeScript file and inject into the Component. This is achieved by the additional code highlighted in the code box below. The remaining new imports are to enable new functionality in the HTML template that we will see shortly.

File: bizFE/src/app/authbutton.component.ts

```
import { Component, Inject } from '@angular/core';
import { AuthService } from '@auth0/auth0-angular';
import { DOCUMENT } from '@angular/common';
import { AsyncPipe } from '@angular/common';
import { CommonModule } from '@angular/common';

@Component({
  selector: 'auth-button',
  templateUrl: 'authbutton.component.html',
  standalone: true,
  imports: [AsyncPipe, CommonModule]
})

export class AuthButtonComponent {

  constructor(@Inject(DOCUMENT) public document: Document,
              public auth: AuthService) {}
}
```

The enhanced version of the **AuthButton** HTML template demonstrates how we can have a single element with two separate functions. The **ng-container** directive provides us with a way of delimiting a bunch of page content as a single entity – much as the **div** and **span** tags in HTML. However, when used in conjunction with ***ngIf** and **ng-template**, it provides an easy way of specifying multiple content blocks, only one of which will be rendered according to some condition. In effect, the **ng-template** element acts as a reserve, ready to replace the **ng-container** content when required. The code here specifies the logout button as the default content to be displayed during "normal" operation, i.e. when a user is logged in to the application. However, the **ng-container** includes an ***ngIf** directive

```
*ngIf = "authService.isAuthenticated$ | async;
else loggedOut"
```

which is read as "if the **isAuthenticated$** property of the **authService** is true, then display this content, otherwise replace this content with that in the **loggedOut** template". The final thing to note is the trailing **$** on the **isAuthenticated$** property name. This is an Angular convention (though not a requirement) to denote that the property is an Observable. As such, we cannot its value directly but need to subscribe to it. The **async** pipe provides a convenient way to subscribe to an Observable in the HTML template. The alternative would be to use the **subscribe()** method in the TypeScript file (as we have already seen) and update a component property with the value.

The full updated code for ***authbutton.component.html*** is provided in the code box below.

File: bizFE/src/app/authbutton.component.html

```
    <ng-container *ngIf="auth.isAuthenticated$ | async;
                  else loggedOut">
        <button class="btn btn-primary"
                (click)="auth.logout({
                            logoutParams: {
                               returnTo: document.location.origin }
                        })">
             Log out
        </button>
    </ng-container>

    <ng-template #loggedOut>
        <button class="btn btn-primary"
                (click)="auth.loginWithRedirect()">Log in</button>
```

```
    </ng-template>
```

Do it now!	Update the AuthButton component TypeScript and HTML files according to the code presented above. Then run the application and navigate to the home page. Click on the login button and see how you are transferred to the Auth0 sign in page. Provide the sign in details that you created earlier and note now you are returned to the application home page, but the button is now a logout button. Finally, click on logout and verify that you are returned to the home page with a login button. The sequence of browser content should be as that shown in Figure 16.8, below.

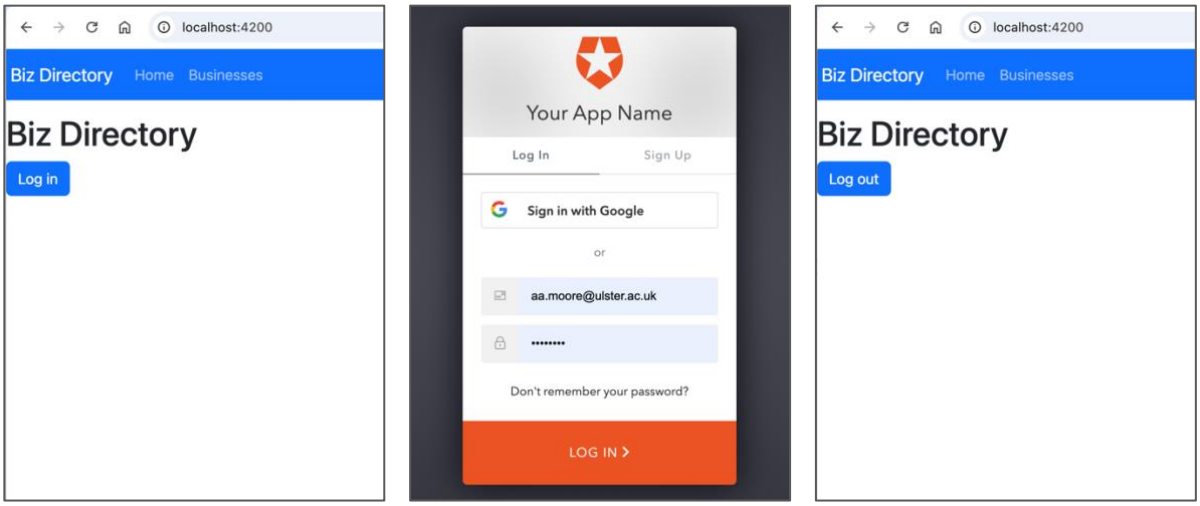


Figure 16.8 Ready to login, Auth0 Sign in, and Ready to Log out

16.2.4 User Profile

Now that users can register and login to our application, we can retrieve selected properties from the Auth0 User Profile object to personalize the application for the logged in user. The Auth0 **`AuthService`** provides a **`user$`** Observable that contains a range of fields. The full set of fields can be seen at the link in Section 16.4, but here, we will display the registered name and email address of the logged in user.

First, we define a new **`AuthUser`** component which will be represented by the selector auth-user and makes the **`AuthService`** available by injecting it into the component as usual. The following code box presents the source for ***authuser.component.ts***.

File: bizFE/src/app/authuser.component.ts

```
import { Component } from '@angular/core';
import { AuthService } from '@auth0/auth0-angular';
import { AsyncPipe } from '@angular/common';
import { CommonModule } from '@angular/common';

@Component({
  selector: 'user-profile',
  templateUrl: 'authuser.component.html',
  standalone: true,
  imports: [AsyncPipe, CommonModule]
})

export class AuthUserComponent {
  constructor(public auth: AuthService) {}
}
```

Now we can define the representation of the **`AuthUser`** component by specifying the HTML template. Here, we check that the **`AuthService user$`** Observable has a value (i.e. that someone is currently logged in), and if so, we display their name and email attributes.

File: bizFE/src/app/authuser.component.html

```
<h3 style="margin-top:30px">
    Current User Details
</h3>

<ul *ngIf="auth.user$ | async as user">
   <li>{{ user.name }}</li>
   <li>{{ user.email }}</li>
</ul>
```

Now that the **`AuthUser`** component is available, we can import it into the **`HomeComponent`** to enable the user details to be added to the home page.

File: bizFE/src/app/home.component.ts

```
import { Component } from '@angular/core';
import { RouterOutlet } from '@angular/router';
import { AuthButtonComponent } from "./authbutton.component";
import { AuthUserComponent } from "./authuser.component";

@Component({
  selector: 'home',
  standalone: true,
  imports: [RouterOutlet, AuthButtonComponent,
                                              AuthUserComponent]
  templateUrl: './home.component.html',
  styleUrl: './home.component.css'
})
export class HomeComponent {}
```

Finally, we are able to add the **AuthUser** selector to the HTML template using the **<user-profile>** tag so that the details of a logged in user will be displayed when available.

File: bizFE/src/app/home.component.html

```
<h1 style="margin-top: 70px">
    Biz Directory
</h1>

<auth-button></auth-button>
<user-profile></user-profile>
```

Do it now!	Create the **AuthUser** component and add it to the home page by making the modifications described above. Run the application and log in to confirm that the user details are displayed as illustrated in Figure 16.9, below.

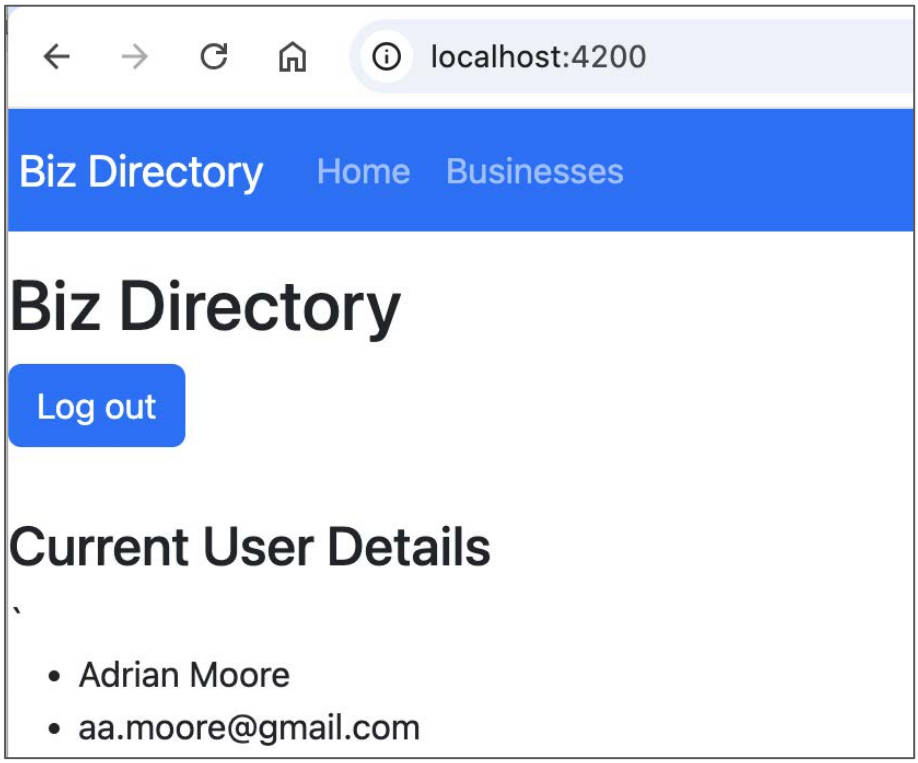


Figure 16.9 Displaying the User Profile

16.3 Add Authentication Functionality to the NavBar

Our authentication functionality is now in place, but the home paged is not the best place to implement it. We will want our users to be able to login from any part of our application and to provide login and logout on each page will require significant repetition of code.

Fortunately, we have a **`NavComponent`** that is constantly present regardless of our users' location within the application. If we move the authentication functionality to there, we can implement once and have it available from any page.

16.3.1 Move Authentication Functionality

Moving the functionality is quite straightforward and only requires that the **`AuthButton`** and **`AuthUser`** components are imported into the **`NavComponent`** and removed from the **`HomeComponent`**.

First, we import the required components into the **`NavComponent`**.

File: bizFE/src/app/nav.component.ts

```
import { Component } from '@angular/core';
import { RouterOutlet, RouterModule } from '@angular/router';
import { AuthButtonComponent } from "./authbutton.component";
import { AuthUserComponent } from "./authuser.component";

@Component({
  selector: 'navigation',
  standalone: true,
  imports: [RouterOutlet, RouterModule, AuthButtonComponent,
            AuthUserComponent],
  templateUrl: './nav.component.html'
})
export class NavComponent {}
```

Then, we remove the same content from the **`HomeComponent`**.

File: bizFE/src/app/home.component.ts

```
import { Component } from '@angular/core';
import { RouterOutlet } from '@angular/router';
~~import { AuthButtonComponent } from "./authbutton.component";~~
~~import { AuthUserComponent } from "./authuser.component";~~

@Component({
  selector: 'home',
  standalone: true,
  imports: [RouterOutlet~~, AuthButtonComponent,~~
            ~~AuthUserComponent~~],
  templateUrl: './home.component.html',
  styleUrl: './home.component.css'
})
export class HomeComponent {}
```

Do it now!	Move the imports related to authentication from the **`HomeComponent`** to the **`NavComponent`** as described above.

Note:	Note that removing the imports from the **`HomeComponent`** before removing the corresponding HTML template content will cause an error in the application if it is currently running, but this will be resolved in the next section.

16.3.2 Adding an Inline Button

Now that the components have been updated, we can make the corresponding changes to the HTML templates. First, we will add the **`AuthButton`** component to the navigation bar by specifying it in an inline **`<div>`** element. This is positioned below the **`<ul>`** element that contains the navbar links, but inside the **`<div>`** element that contains the navbar.

Note the class **`ms-auto`**, which specifies that this form should be right-aligned (i.e. allow the margin-start [**`ms`**] to automatically take as much space as is available).

File: bizFE/src/app/nav.component.html

```
      ...

          <div class="form-inline ms-auto"
               style="margin-right: 10px">
               <auth-button></auth-button>
          </div>
      </div>
```

We also take the opportunity to change the appearance of the button by describing it using the Bootstrap **`btn-outline-light`** class. This requires the following change in the **`AuthButton`** HTML template.

File: bizFE/src/app/authbutton.component.html

```
      ...

      <button class="btn btn-outline-light"

      ...

      <button class="btn btn-outline-light"

      ...
```

Finally, we can remove the authentication content from the **`HomeComponent`** HTML template.

File: bizFE/src/app/home.component.html

```
<h1 style="margin-top: 70px">
    Biz Directory
</h1>

<auth-button></auth-button>

<user-profile></user-profile>
```

Do it now!	Make the changes described above to move the template authentication functionality from the **`HomeComponent`** to the **`NavComponent`**. Run the application and confirm that you see the login/logout button in the navigation bar as illustrated in Figure 16.10, below.

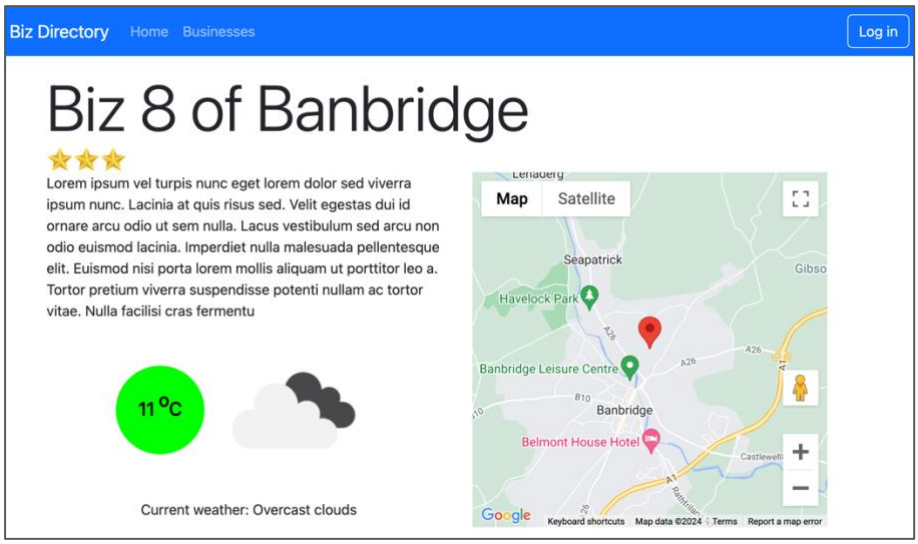


Figure 16.10 Authentication in the NavBar

16.3.2 Redirecting After Login

The login and logout mechanism implemented so far works well, but the user is redirected to the (currently empty) home page after every login/logout event. This is perhaps reasonable for logout, as the use may be on a page that is only available for logged in users, but there is no reason for a user to be redirected following a login event. Indeed, in our case it is desirable that the user remains on the current page, since they may have logged in order to leave a review for the specific business being viewed at that time (we will restrict the provision of reviews to logged in users in the next section).

Auth0 supports redirection after login by allowing us to specify the route to which the user is to be redirected as a parameter to the **`AuthService loginWithRedirect()`** method. The modification that we need to make to our **`AuthButton`** Component is illustrated below.

File: bizFE/src/app/authbutton.component.html

```
...

<ng-template #loggedOut>
    <button class="btn btn-outline-light"
            (click)="auth.loginWithRedirect( {
                appState: { target: this.router.url }
            })">Log in</button>
</ng-template>
```

To access the **`url`** attribute of the **`router`** service, we also need to import it into the **`AuthButton`** component TypeScript file, add it to the (currently not present) list of providers, and inject it into the Component constructor.

File: bizFE/src/app/authbutton.component.ts

```
...

import { Router } from '@angular/router';

@Component({
  selector: 'auth-button',
  templateUrl: 'authbutton.component.html',
  standalone: true,
  imports: [AsyncPipe, CommonModule],
  providers: [Router]
})

export class AuthButtonComponent {

    constructor( ...
        public auth: AuthService,
        public router: Router) {}

}
```

Do it now!	Make the changes to the **AuthButton** component described above to redirect a logged in user back to the page they were previously viewing. Run the application (logging out if you are currently logged in) and navigate to a single business. Now log in and confirm that you are returned to the page on which you initiated the login event.

16.3.3 Restricting Access to Selected Content

Ideally, we want to restrict the ability to leave a review only to users who have been successfully authenticated. We achieve this in the same way as we have already seen with the **AuthButton** component, using the **AuthService isAuthenticated$** Observable and the ***ngIf** directive to only display the review form if an authenticated user is present. Where there is no logged in user, we provide a message that instructs users that they should log in to leave a review.

This time, the **ng-container** contains the form which the user completes to submit a review, while the **ng-template** that replaces it in the event of no user being logged in is a message requesting that the user logs in to review the business.

File: bizFE/src/app/business.component.html

```
...

   <ng-container
       *ngIf = "authService.isAuthenticated$ | async;
        else loggedOut">

        <h2>Please review this business</h2>
        <form [formGroup]="reviewForm"
              (ngSubmit)=onSubmit()">

        ...

   </ng-container>

   <ng-template #loggedOut>
        <h2>Please login to review this business</h2>
   </ng-template>

...
```

As usual, to make the **`AuthService`** functionality available in the **`BusinessComponent`**, we need to import it into the Component and inject it into the constructor.

File: bizFE/src/app/business.component.ts

```
...

import { AuthService } from '@auth0/auth0-angular';

...

    constructor(public webService: WebService,
                private route: ActivatedRoute,
                private formBuilder: FormBuilder,
                public authService: AuthService) {}

...
```

The effect of this is that only users that are logged in will see the review form, while others will see a message as shown in Figures 16.11 and 16.12, below.

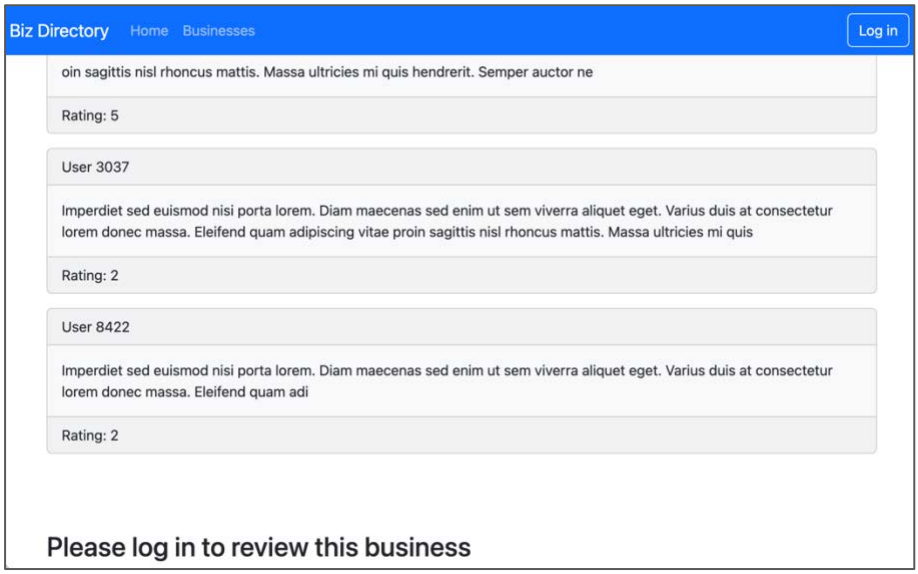


Figure 16.11 Logged out User Cannot Review

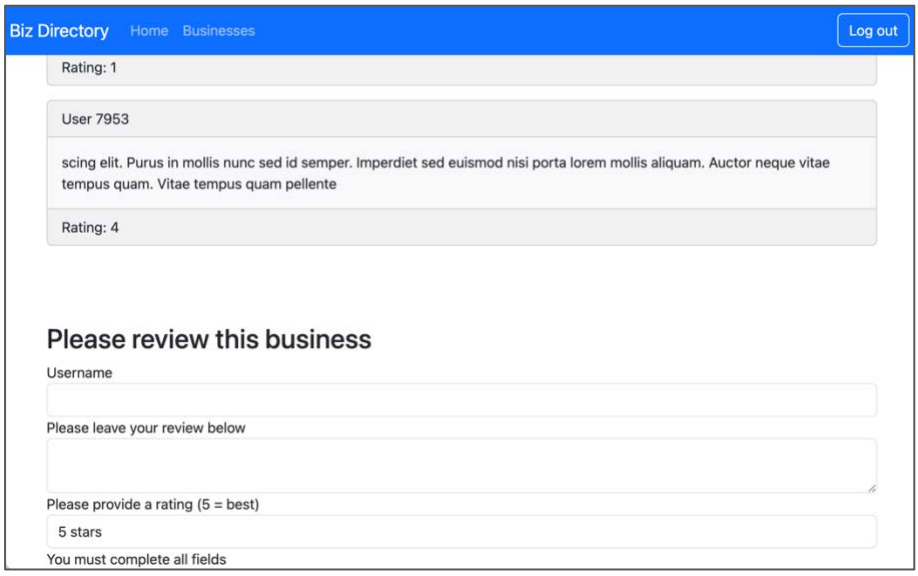


Figure 16.12 Logged in User Invited to Review

Do it now!	Modify the **`BusinessComponent`** so that the review form is enclosed in an **`ng-component`** directive as shown above, with an **`ng_template`** directive as an alternative "Please login" message for non-logged in users. Use the **`AuthService isAuthenticated$`** Observable to select between the content blocks by use of an **`*ngIf`** directive. Ensure that you are able to generate output such as that shown in Figures 16.11 and 16.12

16.4 Further Information

- https://coreui.io/docs/components/forms/
  Bootstrap Forms

- https://www.w3schools.com/Bootstrap/bootstrap_forms.asp
  Bootstrap Forms – W3Schools

- https://angular.dev/guide/forms/reactive-forms
  Angular ReactiveForms – the online manual

- https://malcoded.com/posts/angular-fundamentals-reactive-forms/
  Reactive Forms with Angular – using easy examples!

- https://angular.dev/api/forms/FormBuilder
  Angular FormBuilder class

- https://coryrylan.com/blog/angular-form-builder-and-validation-management
  Angular Form Builder and Validation Management

- https://malcoded.com/posts/angular-reactive-form-validation/
  Validating Reactive forms in Angular

- https://angular.dev/guide/forms/form-validation
  Angular Form Validation

- https://angular.dev/guide/templates/two-way-binding
  Two-way binding in Angular

COM661 Full Stack Strategies and Development

FE17. Connecting to the Back-end

Aims

- To create a Web Service to handle communication with an external API
- To describe the Single-origin Policy for Web Security and implement Cross-Origin-Resource-Sharing
- To subscribe to Observables and demonstrate retrieval of the data
- To demonstrate using multiple Web Service calls to facilitate flexible update of the content
- To identify opportunities for additonal functionality
- To describe an end-to-end authentication system

Table of Contents

17.1 BUILDING A WEB SERVICE 2

17.1.1 CREATE THE WEB SERVICE 2
17.1.2 CROSS-ORIGIN RESOURCE SHARING 4
17.1.3 DISPLAYING THE DATA RETRIEVED 6

17.2 FETCHING A SINGLE BUSINESS 10

17.2.1 WEB SERVICE 10
17.2.2 BUSINESS COMPONENT 10
17.2.3 FRONT-END 12

17.3 ADDING A REVIEW 13

17.3.1 WEB SERVICE 13
17.3.2 BUSINESS COMPONENT 14
17.3.2 SEPARATING BUSINESSES AND REVIEWS 17

17.4 OTHER FUNCTIONALITY 20

17.5 FURTHER INFORMATION 22

17.1 Building A Web Service

So far, all of the functionality of the front-end Angular application has been driven by hard-coded JSON data provided by the Data Service. In this practical, we explore how to connect the front-end app with the back-end Flask API previously built.

17.1.1 Create the Web Service

Just as our Data Service provides facilities that are accessed by a number of components, we will construct the bridge between the front- and back-ends of the application within a Web Service. The Web Service will not entirely replace the Data Service as the resources accessed from external sites such as the Lorem Ipsum and Google Maps APIs will still be provided there, but the Web Service will be the sole provider of services on businesses and reviews, so that all interaction is with our back-end database rather than the hard-coded data.

As a first step, we build an initial version of the Web Service with a class property to hold the default page size and a method **`getBusinesses()`** to make a HTTP GET request to the API endpoint **GET /api/v1.0/businesses**, passing the page requested and the default page size as query string parameters.

File: bizFE/src/app/web.service.ts

```
import { HttpClient } from "@angular/common/http";
import { Injectable } from "@angular/core";

@Injectable()
export  class WebService{

    pageSize: number = 3;

    constructor(private http: HttpClient) { }

    getBusinesses(page: number) {
        return this.http.get<any>(
            'http://localhost:5000/api/v1.0/businesses?pn=' +
             page + '&ps=' + this.pageSize);
    }
}
```

Do it now!	Add the new Web Service to your front-end application as provided in the code box above. You should be able to verify that no errors have been introduced and that the application still runs as before.

In order to use the **`getBusinesses()`** method of the new Web Service, we need to import it into the Businesses Component and add it to the component's **`providers`** list.

Note:	Remember that when importing elements into Angular components, other **components are added to the imports list**, while **services are added to the providers list**.

File: bizFE/src/app/businesses.component.ts

```
import { Component } from '@angular/core';
import { RouterModule } from '@angular/router';
import { DataService } from './data.service';
import { WebService } from './web.service';

@Component({
  selector: 'businesses',
  standalone: true,
  imports: [RouterModule],
  providers: [DataService, WebService],
  templateUrl: './businesses.component.html',
  styleUrl: './businesses.component.css'
})

...
```

With the new Web Service introduced to the Businesses Component, we can now attempt to make use of its **`getBusinesses()`** method. First, we make the Service available to the Component class by injecting it as a parameter to the Component constructor.

Then, we make a call to the Web Service **`getBusinesses()`** method, recognising that as the **`http.get()`** request invoked by the Web Service returns an Observable that we need to subscribe to. In the subscription function, we accept the response object returned by the Observable and initially log it to the browser console.

File: bizFE/src/app/businesses.component.ts

```
...

constructor(public dataService: DataService,
            private webService: WebService) { }

ngOnInit() {
   if (sessionStorage['page']) {
       this.page = Number(sessionStorage['page']);
   }

   // this.business_list =
   //     this.dataService.getBusinesses(this.page);


   this.webService.getBusinesses(this.page)
       .subscribe((response) => {
            console.log(response);
    })

}

...
```

Do it now!	Update the Businesses Component as shown above and run the application. Make sure that your MongoDB Database service and back-end API are already running.

17.1.2 Cross-Origin Resource Sharing

When we run the modified Angular application (remember to start the MongoDB database server and the back-end application first), we encounter an error message in the browser console when we attempt to fetch a page of businesses. The error message indicates that there has been an Access-Control-Allow-Origin error, as illustrated in Figure 17.1, below.

This error is a result of the **same-origin policy** which is is a web browser security mechanism that aims to prevent websites from attacking each other by restricting scripts on one origin from accessing data from another origin. For API development, however, this behaviour is exactly what we want to provide, so we need to enable **Cross-Origin-Resource-Sharing** (CORS) on our API.

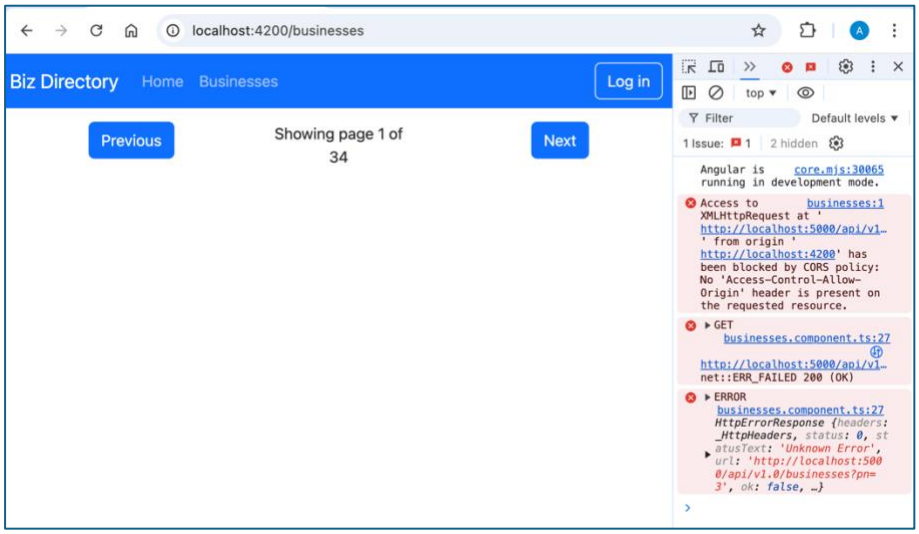


Figure 17.1 Cross Origin Resource Sharing Error

This requires us to stop our API application and install the package **flask_cors** by the command

```
(venv) C:\Desktop\Biz> pip install flask_cors
```

With the package installed, we can enable CORS by adding the code highlighted below to our API

File: biz\app.py

```
from flask import Flask
from flask_cors import CORS

...

app = Flask(__name__)
CORS(app)

...
```

Now, with CORS enabled, we can re-run the application and demonstrate that a page of businesses is being returned from the back-end and displayed in the browser console, as illustrated in Figure 17.2, below.

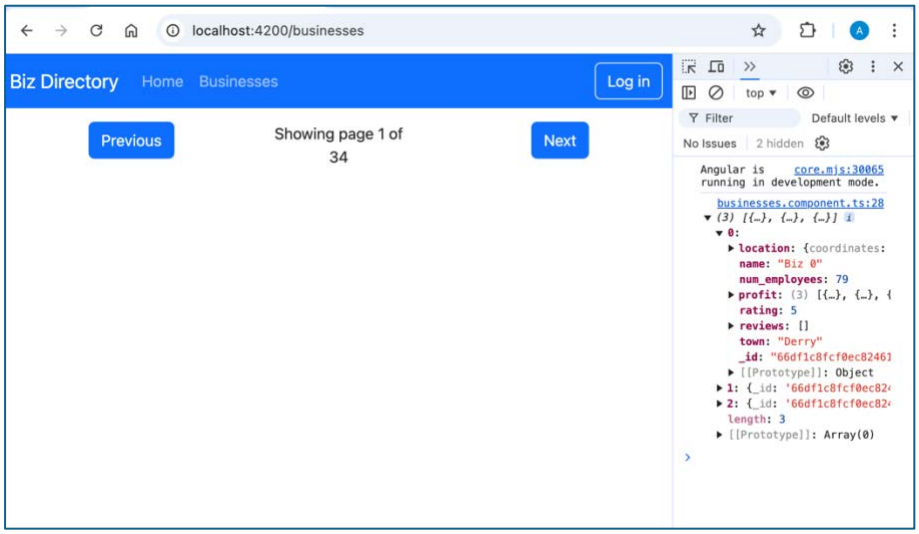


Figure 17.2 CORS Error Resolved

Do it now!	Update the Flask API application by enabling CORS as shown above. Re-run both the back-end and front-end and ensure that the data is retrieved and displayed as shown in Figure 17.2, above.

Note:	When building an API, you should enable CORS by design from the outset. In this example, we wanted to demonstrate the effect of not enabling CORS, so have introduced it deliberately at this stage.

17.1.3 Displaying the Data Retrieved

Logging the data to the console has proved that the communication with the back-end is operating as expected, so we can now display that data on the front-end instead of the data previously returned by the Data Service.

To achieve this, all we need to do is replace the **`console.log()`** of the response from the **`http.get()`** Observable with an assignment to the **`business_list`** property of the Businesses Component.

Making this change and running the application results in the first page of businesses being fetched from the MongoDB database, via the API endpoint and Web Service, and being displayed on the web page, as shown in Figure 17.3 below.

File: bizFE/src/app/businesses.component.ts

```
...

ngOnInit() {
   if (sessionStorage['page']) {
       this.page = Number(sessionStorage['page']);
   }

   /*
   this.business_list =
        this.dataService.getBusinesses(this.page);
   */


   this.webService.getBusinesses(this.page)
       .subscribe((response) => {
            this.business_list = response;
    })

}

...
```

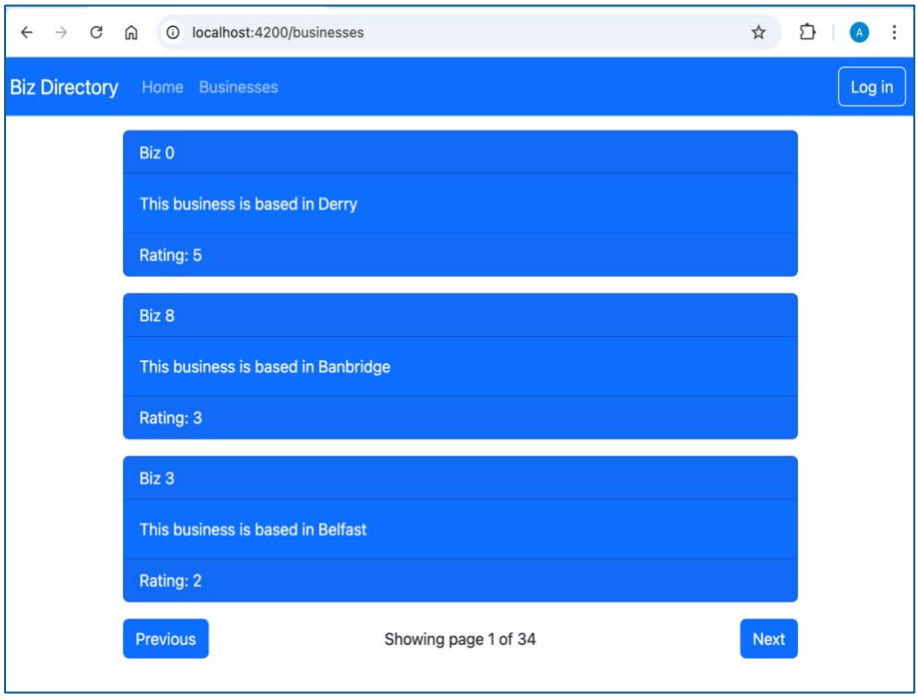


Figure 17.3 Businesses Displayed in Browser

Do it now!	Make the update to the Businesses Component **`ngOnInit()`** method as shown above and confirm that the first page of businesses are displayed in the browser.

Although we can be confident that we have fetched the data from the new Web Service rather than the previous Data Service, the display in the browser is exactly as we have seen previously. However, we can confirm that the data really is being retrieved from the back-end by making a small change to the Web Service that modifies the number of businesses requested per page.

File: bizFE/src/app/web.service.ts

```
...

export class WebService {

    pageSize: number = 4;

...
```

Saving this change and re-running the application results in a page of four businesses being displayed – leaving no doubt that the Web Service and the call to the back-end is the source of the data.

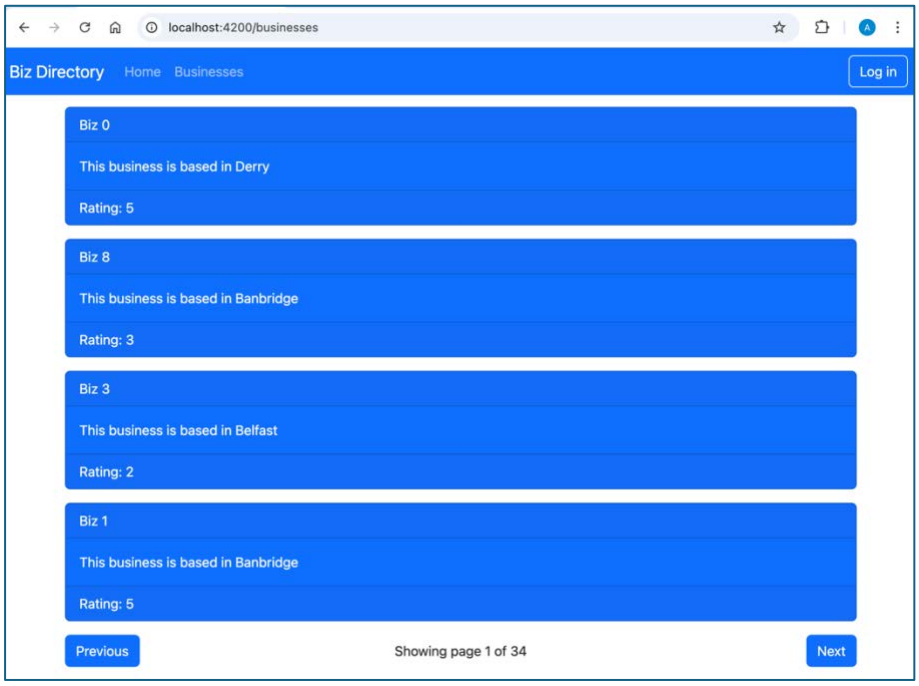


Figure 17.3 Page Size Changed

Do it now!	Change the default page size specified in the Web Service and confirm that the change has the appropriate effect on the number of business returned and displayed.

Note:	You can leave the default page size at four or change it back to three (or any other value) according to your preference.

Now that the collection of businesses is being fetched from the back-end, we need to ensure that all instances of data being fetched are implemented in the same way. Apart from the initial fetch operation in the Businesses Component **`ngOnInit()`** method, we also retrieve a fresh page of businesses following a "next page" or "previous page" operation. The final modification for the Businesses Component is to update these as shown below.

File: bizFE/src/app/businesses.component.ts

```
...

    previousPage() {
        if (this.page > 1) {
            this.page = this.page - 1;
            sessionStorage['page'] = this.page;
            this.webService.getBusinesses(this.page)
                .subscribe((response: any) => {
                   this.business_list = response;
                })
        }
    }

    nextPage() {
        if (this.page < this.dataService.getLastPageNumber()) {
            this.page = this.page + 1;
            sessionStorage['page'] = this.page;
            this.webService.getBusinesses(this.page)
            .subscribe((response: any) => {
               this.business_list = response;
            })
        }
    }

...
```

Do it now!	Change the **`previousPage()`** and **`nextPage()`** functions as shown above and re-run the application. Confirm that you can navigate forward and backwards through the collection of businesses.

17.2 Fetching a Single Business

With the Web Service established and demonstrated to correctly retrieve a page of businesses from the API, we can now add the functionality to retrieve a selected business by ID. This is tackled in three phases – adding new functionality to the Web Service, modifying the Business Component to use the Web Service rather than the Data Service, and updating the front-end HTML template.

17.2.1 Web Service

Providing the Web Service with the means of retrieving a single business by ID is straightforward, and simply involves the specification of a new method that accepts the business id as a parameter and uses it in an **`http.get()`** request to the API endpoint that returns a specified business.

File: bizFE/src/app/web.service.ts

```
    ...

        getBusiness(id: any) {
            return this.http.get<any>(
                'http://localhost:5000/api/v1.0/businesses/' + id);
        }
    }
```

Do it now!	Add the **`getBusiness()`** method to the Web Service as shown above.

17.2.2 Business Component

The first update to the Business Component is to import the Web Service and make it available to the Component class. This is carried out in the now familiar steps as highlighted in the code box below.

File: bizFE/src/app/business.component.ts

```
...

import { WebService } from './web.service';

@Component({
  selector: 'business',
  standalone: true,
  imports: [CommonModule, GoogleMapsModule,
            ReactiveFormsModule],
  providers: [DataService, WebService],
  templateUrl: './business.component.html',
  styleUrl: './business.component.css'
})

...

constructor( public dataService: DataService,
             private route: ActivatedRoute,
             private formBuilder: FormBuilder,
             public authService: AuthService,
             private webService: WebService) {}

...
```

Now, we can replace the existing call to the Data Service **`getBusiness()`** method with one to the Web Service equivalent. However, we need to be careful to recognize the fundamental difference between the methods. As the Data Service **`getBusiness()`** method returned the JSON object representing the business, we could assign the value returned from **`getBusiness()`** directly to the **`business_list`** component property. However, since the **`http.get()`** request returns an Observable, we need to subscribe to it in order to retrieve the value.

This has a significant consequence for the remainder of the code in **`ngOnInit()`**. As the requests to the **OpenWeatherMap** and **Google Maps** APIs depend on coordinate values that are retrieved from the business object returned, these API requests cannot be allowed to proceed until after the call to the Biz Directory API has returned. To ensure that this is the case, the code that uses external API requests to generate additional presentation information is moved inside the subscription function of the **`getBusiness()`** API call.

This is demonstrated in the following code box.

File: bizFE/src/app/business.component.ts

```
...

this.webService.getBusiness(
                        this.route.snapshot.paramMap.get('id'))
     .subscribe( (response: any)  => {
          this.business_list = [response];

          this.business_lat = ...
          this.business_lng = ...

          ...
          // remaining ngOnInit() code inside the subscription
          // function

     }
}
```

Do it now!	Update the ***business.component.ts*** file by importing the Web Service and modifying **`ngOnIOnit()`** as shown in the code boxes above.

17.2.3 Front-end

The only change required to the front end is in the Businesses Component rather than the Business Component. As the data was previously fetched from the Data Service, the **id** field that identified each business and which was specified in the **`routerlink`** property of the Bootstrap card was the **`business._id.$oid`** field (as this is how it is presented when the dataset is exported using **mongoexport**).

Now that the data is being retrieved directly from the database, the field is simply **`_id`**, so we can make the corresponding change in the Business Component HTML template.

File: bizFE/src/app/businesses.component.html

```
...

<div class="card text-white bg-primary mb-3"
     style = "cursor: pointer"
     [routerLink] = "['/businesses', business._id]">

...
```

Now, when we run the application and select a business, its data is fetched from MongoDB via the API and Web Service and is displayed in the browser as illustrated by Figure 17.5, below.

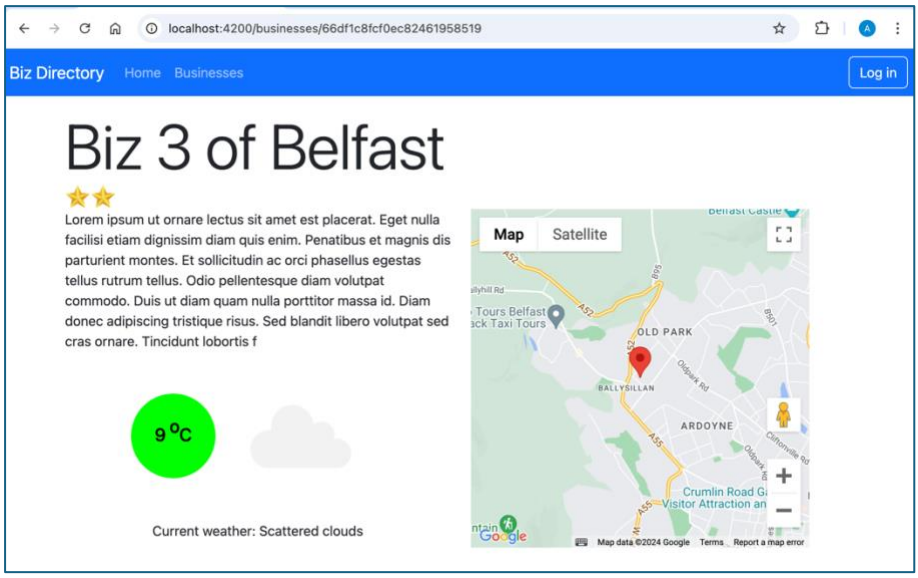


Figure 17.5 Business Retrieved

Do it now!	Update the ***businesses.component.html*** file as shown in the code box above and verify that you can navigate the collection of businesses and choose a business for display in detailed form as shown in Figure 17.5, above

17.3 Adding a Review

The next element of Data Service functionality that needs to be replaced with a Web Service equivalent is the ability to add a user review to the collection for the business being displayed. At the minute, the review is added to the collection maintained by the Data Service, but as this data is refreshed from the file in the assets folder every time the application is run, any review added are lost. By switching to the Web Service, the reviews will be added to the MongoDB database and will persist over the life of the application.

17.3.1 Web Service

The first step is to add an new method to the Web Service that accepts a business ID and a review data structure as parameters, and makes an **`http.post()`** request to the API endpoint that adds a review to the collection.

The TypeScript **`FormData`** object is a representation of a form that is structurally equivalent to an HTML form sent using the **`multipart/form-data`** encoding. This means that we can send it as the second parameter of a **`http.post()`** request and it will be received by the API just as if it had originated from an HTML **`<form>`** (or from a Postman request).

The Web Service **`postReview()`** method adds the user provided data to the **`FormData`** object by taking each field in turn and appending it with a field name. Hence, in the code box below, the value of the field "*username*" from the review form is added to the **`FormData`** object, again using "*username*" as the field name.

The full specification of **`postReview()`** is provided in the code box below.

File: bizFE/src/app/web.service.ts

```
...

postReview(id: any, review: any) {
   let postData = new FormData();
   postData.append("username", review.username);
   postData.append("comment", review.comment);
   postData.append("stars", review.stars);
   return this.http.post<any>(
          'http://localhost:5000/api/v1.0/businesses/' +
           id + "/reviews", postData);
}
```

Do it now!	Add the **`postReview()`** method to the Web Service as shown in the code box above.

17.3.2 Business Component

With the Web Service functionality in place, we can now move to the Business Component and invoke the new method, passing it the review form object and the _id of the business for which the review is intended. Again, as the **`http.post()`** request returns an Observable, we need to subscribe to it, moving the statement to reset the form inside the subscription function.

File: bizFE/src/app/business.component.ts

```
...

onSubmit() {
     this.webService.postReview(
               this.route.snapshot.paramMap.get('id'),
               this.reviewForm.value)
          .subscribe( (response) => {
             this.reviewForm.reset();
          });
}

...
```

When we save the file and run the application, sending a review generates a **401 Unauthorised Request** error which can be seen in the Browser Console as shown in Figure 17.6, below.

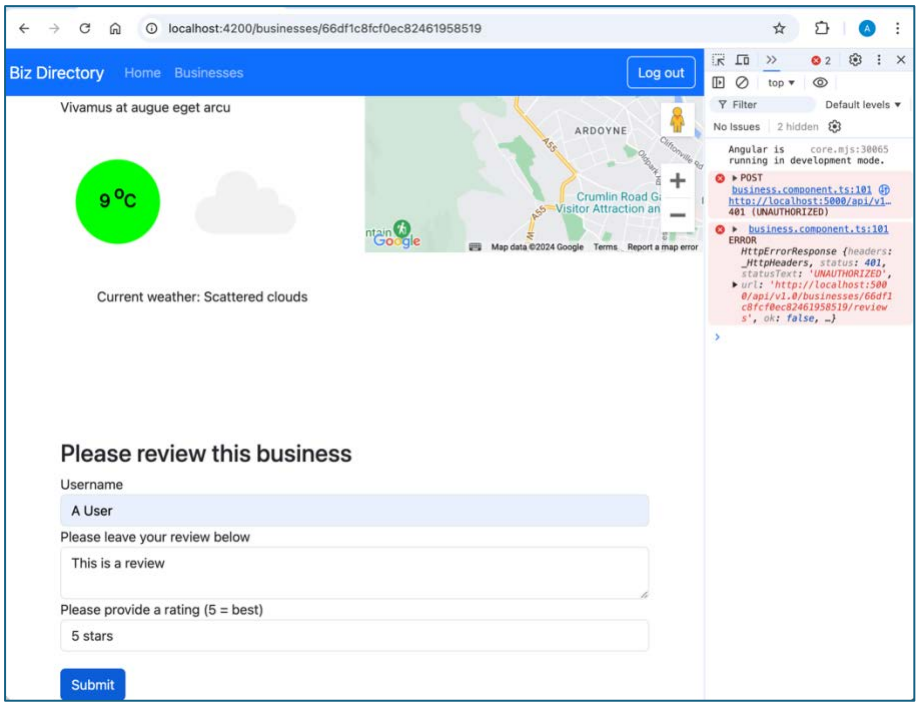


Figure 17.6 Unauthorised Access Error

Do it now!	Modify the **`onSubmit()`** method in the Business Component as shown in the code box above. Run the application with the Browser Console open and verify that you receive the 401 error code as shown in Figure 17.6, above.

The error is occurring because we specified in the API that the POST review endpoint should only be available to requests that enclosed a valid JWT token. We will discuss how to achieve this later, but for now we will progress by removing the requirement for a token from the API endpoint.

File: biz/bueprints/reviews/reviews.py

```
...

@reviews_bp.route("/api/v1.0/businesses/<string:id>/reviews",
                  methods=['POST'])
# @jwt_required - COMMENTED OUT (for now)
def add_new_review(id):
    new_review = {

...
```

Now when we re-run the application and submit a review, we can navigate back to the business and confirm that the review has been added, as seen in Figure 17.7, below.

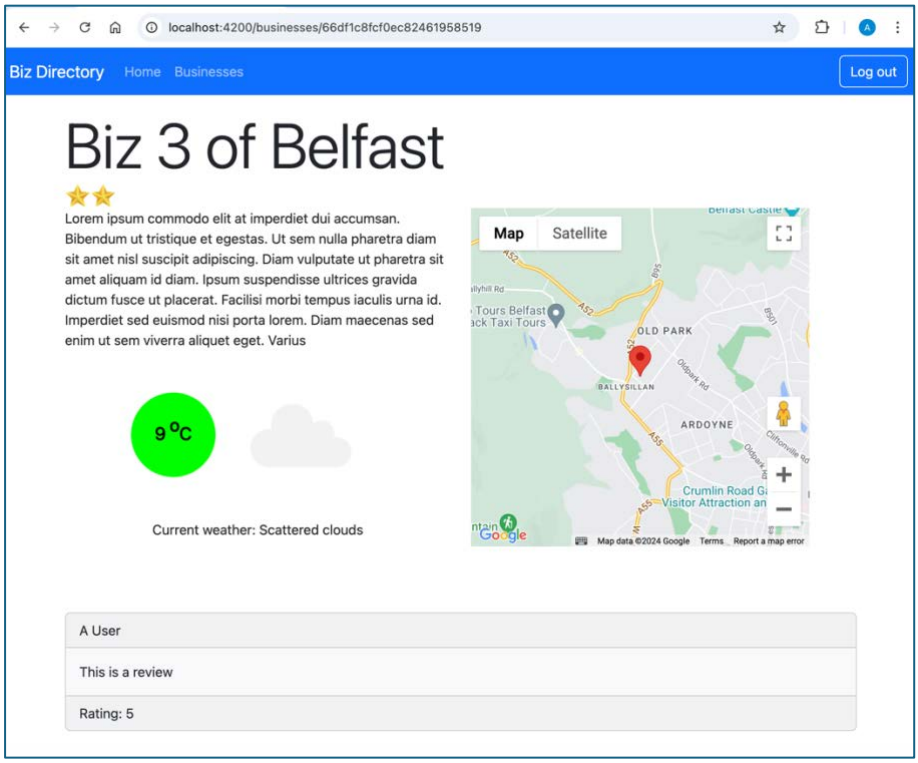


Figure 17.7 Review Added

Do it now!	Modify the API as shown above and verify that the review is added by refreshing the page after it has been submitted.

17.3.2 Separating Businesses and Reviews

A limitation of the **`postReview()`** implementation is that we need to navigate away from the business and return, or refresh the page in order to see the updated list of reviews. We could simply fetch the business again and update the browser that way, but fetching a business also involves fetching the description, weather information and map from 3rd party APIs so would be a very inefficient solution.

A better approach is to separate fetching a business from fetching its reviews, so that we can request that the reviews only are refreshed when a new review is added.

The first step is to add a Web Service method to retrieve the reviews of a specified business from the API.

File: bizFE/src/app/web.service.ts

```
...

    getReviews(id: any) {
        return this.http.get<any>('
               http://localhost:5000/api/v1.0/businesses/' +
               id + '/reviews');
    }}

...
```

Do it now!	Add the new **`getReviews()`** method to the Web Service as shown above.

Then, we update the Business Component to add a new property to hold the reviews list and to make a call to the new Web Service method that fetches the reviews immediately after the call to the method that fetches the business. Once again, since the **`http.get()`** request returns an Observable, we subscribe to it and assign the response returned from the Observable to the **`reviews_list`** property of the business.

Then, in the **`onSubmit()`** function, in the subscription function for the call to **`postReview()`**, we add a call to **`getReviews()`** to fetch the updated reviews list for the business. In the subscription function for **`getReviews()`** we retrieve the response from the Observable and update the **`review_list`** with this new value. This has the effect of updating the **`review_list`** after each new review is added, without requiring that the rest of the page is refreshed. Since the reviews displayed in the Business Component HTML

template are bound to the **`review_list`** variable, the list of reviews displayed is automatically updated every time a new review is posted.

File: bizFE/src/app/business.component.ts

```
...

export class BusinessComponent {

    business_list: any;
    ...
    review_list: any;

...

   this.webService.getBusiness(
                  this.route.snapshot.paramMap.get('id'))
       .subscribe( (response: any)  => {
            this.business_list = [response];

            ...
   }

   this.webService.getReviews(
                  this.route.snapshot.paramMap.get('id'))
       .subscribe( (response) => {
            this.review_list = response;
   });
}

onSubmit() {
     this.webService.postReview(
          this.route.snapshot.paramMap.get('id'),
          this.reviewForm.value)
        .subscribe( (response) => {
            this.reviewForm.reset();

            this.webService.getReviews(
            this.route.snapshot.paramMap.get('id'))
                 .subscribe( (response) => {
                      this.review_list = response;
            });
     });
}
```

Finally, we bind the reviews displayed in the browser to the content of the new **`review_list`** property (rather than the **`reviews`** element of the current business) as shown in the code box below.

File: bizFE/src/app/business.component.html

```
...

 <div class="row" style="margin-top: 70px">
      <div class="col-sm-12">

           @for ( review of review_list; track review ) {

...
```

Now when we run the application and visit a business, we can leave a review and see that the page is automatically updated without any other page elements requiring a refresh

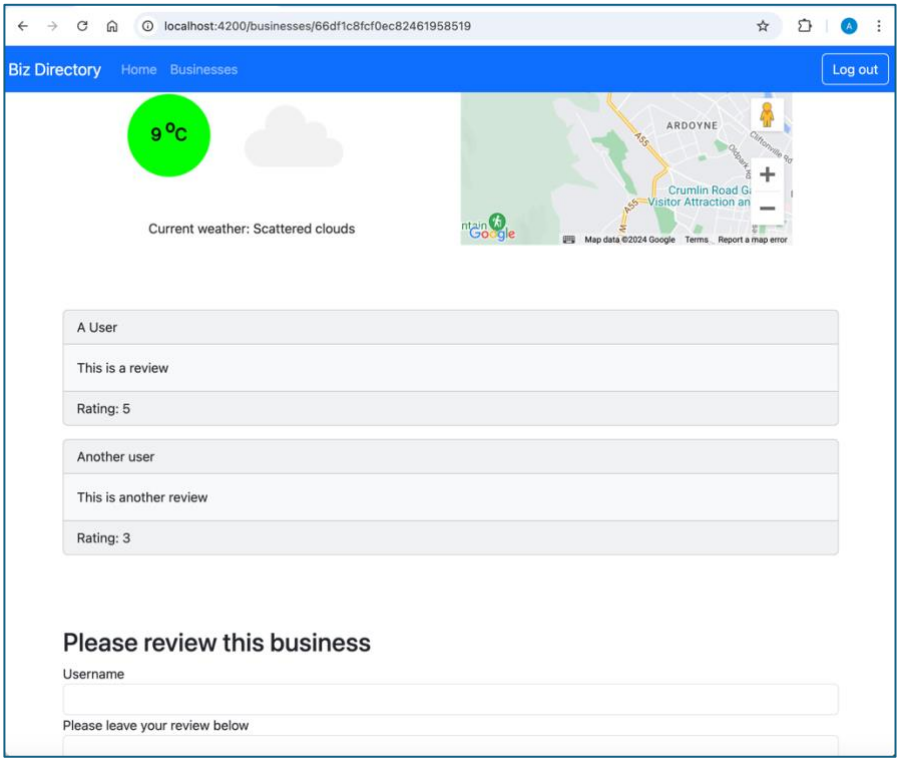


Figure 17.7 Review Added and Displayed Instantly

Do it now!	Make the modifications to the Business Component TypeScript and HTML files as shown above and verify that submitted reviews are automatically added to the displayed collection.

17.4 Other Functionality

As we build the Web Service and migrate the businesses and reviews functionality from the Data Service, we identify additional opportunities for development that require that either the back-end adapts to the requirements of the front-end, or the front-end adapts to the provision of the back-end.

We will identify one example of each here, but the actual implementation is left for you as an exercise and an opportunity to contribute original functionality to your own applications.

17.4.1 Providing Navigation Information

In our page that provides navigation of the collection of businesses, we have a message "Showing page x of y" which gives the user awareness of the scale of the collection and their current position within it.



Figure 17.7 Navigation Information Message

This message is facilitated by the Data Service method **`getLastPageNumber()`** which calculates and returns the number of the last page of information according to the size of the collection and the number of businesses requested per page. The same method also provides the value used in the Businesses Component method **`nextPage()`** which prevents the user from navigating past the last page of content.

This functionality is not possible with the current version of the Biz Directory API as there are no endpoints that return information about the collection. This could be provided in a variety of ways as follow:

i) Provide a new endpoint such as **GET /api/v1.0/businessesInfo** that returns a packet of information about the businesses collection. This packet could contain the size of the collection as well as other data that might be useful such as the different types of businesses represented, the different locations, or other meta-data that could be useful in building the front-end of the application.

ii) Restructure the data returned from the **GET /api/v1.0/businesses** endpoint so that summary information is returned alongside the businesses data. For example, the data returned might be in the form

```
{
  'thisPage' : 5,
  'totalPages' : 70,
  'businesses' : [ { business1 }, { business2 }, etc. ]
}
```

Note:	In general, we want to avoid making changes to the back-end to support front-end functionality as we might compromise other applications that use the API in its existing form. We should ensure that any changes we make are non-breaking for other users, otherwise they should be incorporated into a new version of the API that exists alongside the old one. For example, the endpoint **GET /api/v1.0/businesses** might return data in the current form, while the endpoint **GET /api/v2.0/businesses** could return the expanded data packet suggested above.

17.4.2 Passing a JWT Token

When we implemented our **`postReview()`** Web Service method, we had to remove the @jwt_required decorator from the API application so that the review could be accepted. If we want to make use of back-end JWT tokens at the front end, we need to make Web Service calls to our back-end authentication methods. The following technique could be implemented alongside the Auth0 authentication or (more likely) it could replace Auth0 authentication completely so that we provide our own "end-to-end" authentication service.

i) Provide a front-end interface (for example a button on the NavBar) that makes a call (via the Web Service) to your back-end **login** endpoint.

ii) Subscribe to the Observable returned by the login endpoint and store the JWT token received in browser storage.

iii) When making a request to an endpoint that requires a JWT token, retrieve the token from browser storage in the corresponding Web Service method and add it to the headers of the http request. (You can see the technique for adding request headers by reviewing the request for Lorem Ipsum content in Practical FE14).

17.5 Further Information

- https://angular.dev/guide/http
  Angular HttpClient

- https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Access-Control-Allow-Origin
  Access-Control-Allow-Origin HTTP Response Header

- https://developer.mozilla.org/en-US/docs/Web/Security/Same-origin_policy
  Same-origin Policy

- https://web.dev/articles/same-origin-policy
  Same-origin Policy – WebDev Article

- https://flask-cors.readthedocs.io/en/stable/api.html
  The Flask-CORS Package

- https://www.techiediaries.com/angular-formdata/
  Working with FormData in Angular 18

- https://www.telerik.com/blogs/angular-basics-introduction-observables-rxjs-part-2
  Angular Basics – Introduction to Observables

- https://aidankmcbride.medium.com/observables-and-subscriptions-in-angular-c0c703bb910d
  Observables and Subscriptions in Angular

- https://tekloon.medium.com/how-to-add-http-header-to-angular-httpclient-d38d4ccd0eb8
  How to Add Http Header to Angular HttpClient

COM661 Full Stack Strategies and Development

FE18. The Angular Ecosystem

Aims

- To summarise the structure of the Angular framework
- To describe the provision of 3rd party libraries
- To identify the AG Grid lilbrary to add data grid funcitonality
- To update the back-end API to provide a data feed for the grid component
- To build the basic AG Grid
- To configure the functionality and presentation of the grid
- To identify opportunities for further development

Table of Contents

18.1 ANGULAR FRAMEWORK 2

18.1.1 ANGULAR STRUCTURE 2
18.1.2 ANGULAR LIBRARIES 2
18.1.3 CASE STUDY – AG GRID 3
18.1.4 INSTALLATION AND PREPARATION 4

18.2 USING AG GRID 5

18.2.1 BUILDING THE WEB SERVICE 5
18.2.2 A BASIC GRID 6

18.3 CONFIGURING ADDITIONAL FUNCTIONALITY 11

18.3.1 FILTERING 11
18.3.2 COLUMN OPTIONS 14
18.3.3 PAGINATION 16
18.3.4 STYLING 18
18.3.5 OTHER EXTENSIONS 20

18.4 FURTHER INFORMATION 21

18.1 Angular Framework

The Angular framework has consistently been in widespread use since its first release in 2010 and has been in continuous development since then, with a release pattern that has now settled down to a new major release each year. Angular is maintained and developed by a single-focus team at Google, who devote significant resource to the project. Although the React framework maintains its overall lead in the most popular front-end web frameworks, Angular is still increasing in popularity among developers due to its suitability for rapid development, performance (especially in Single Page Apps), and code security.

18.1.1 Angular Structure

Angular applications are comprised of services and components, and it is important to understand the difference between these key elements. Services are typically used when you need to share data or functionality between different components in your application. For example, if you have multiple components that need to access the same data, you can create a service to provide that data and inject it into each component. Another common use case for services is when you need to interact with an API. You can create a service that makes HTTP requests to the API and provides the data to your components. Services are characterized as ***singletons***, meaning that they are created once and used throughout the entire lifespan of the application.

Angular components define and manage the view-related logic for a piece of your application. They consist of a TypeScript class and an HTML template, and they can be styled with CSS. Components are created and destroyed as needed by the Angular framework, and they are typically used to compose the structure of your application. You can use components to create reusable UI elements, such as buttons and forms, or to define specific views for different parts of your application. Components represent visible content elements on the web page and are included in the HTML stream by use of a tag which mirrors the text of the component selector.

18.1.2 Angular Libraries

An Angular Library is a project that cannot be run on its own but needs to be incorporated into an application to extend Angular's base features. For example, to add reactive forms to an application, we add the library package using **`ng add @angular/forms`**, then import the **`ReactiveFormsModule`** from the **`@angular/forms`** library in our application code.

One of the strengths of Angular is the range of libraries available from 3rd party developers and many commercial software companies produce Angular Libraries that are made

available – often with a free tier of use for anyone, and a subscription tier that releases more advanced features. The production of libraries to enhance the features of Angular is encouraged by the Angular Core Team who make available detailed documentation on how libraries should be produced and released.

18.1.3 Case Study – AG Grid

In this practical, we will investigate the incorporation of an Angular Library into an application by examining **AG Grid**, which provides data grid functionality on a collection of data.

A Data Grid is a table-based approach to browsing a collection of data. It often includes filtering to select a subset of the elements to be displayed, pagination to control the display of large volumes of data, and sorting on one or more fields. **AG Grid** provides all of these and more, and its fully configurable and programmable structure makes it an ideal candidate for close investigation.

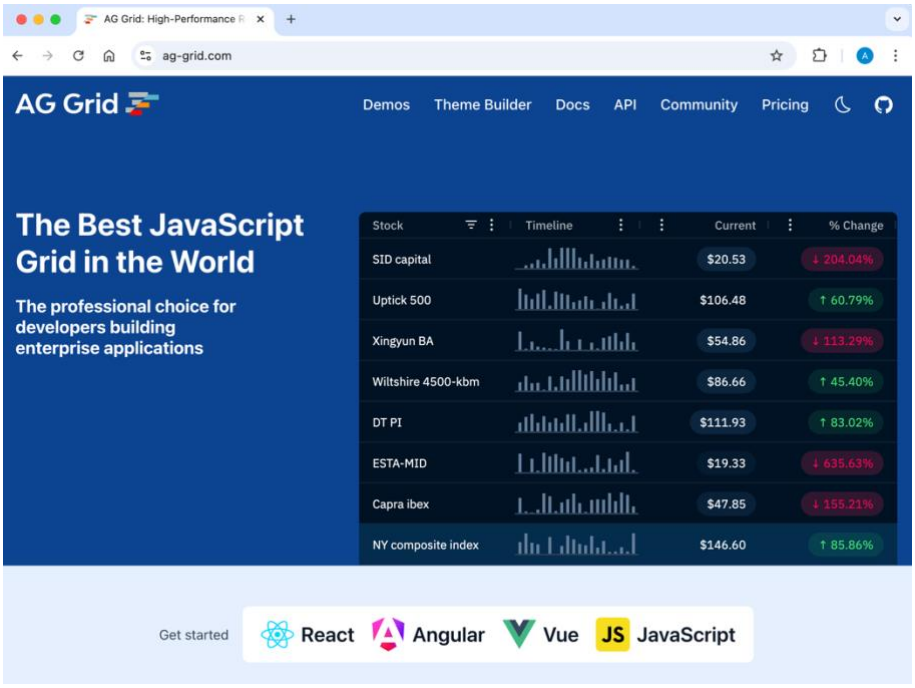


Figure 18.1 AG Grid Home Page

In the sections of this practical that follow, we will install **AG Grid** into an Angular application, configure it to display our collection of businesses, and demonstrate some of the additional functionality that can be incorporated.

18.1.4 Installation and Preparation

AG Grid is made available in the global **npm** library, so can be easily installed into our Angular applications. Our first task is to create a new Angular application in which to showcase AG Grid and then to install it.

First, we create a new Angular application by the following command. Note that we specify C:\ as an example of the location where the new application might be created – you can create it at any suitable location on your system.

```
C:\> ng new datagrid
```

Next, we navigate into the newly created **datagrid** folder and add the AG Grid library into the new application by the following command. As usual, we append the **`--save`** flag to request that the file ***package.json*** is updated with the newly installed library.

```
C:\datagrid> npm install ag-grid-angular --save
```

Note:	Choose here to build the demonstration application in a new Angular project rather than within our existing **BizFE** structure. If you prefer to integrate it within **BizFE**, you should create a new component and develop the following content there, rather than in **AppComponent.**

Do it now!	Create the new Angular application and install the **`ag-grid-angular`** package into it, as described above

One of the features of the Data Grid is that it works on the entire collection of data, rather than a single page as is produced by our current back-end API. In order to feed data to the grid, we need to add a new endpoint to the API that returns the entire dataset – without any reference to page size of page number parameters. The code that follows provides this endpoint, which is essentially a copy of the existing code to fetch a collection of businesses, but at a new endpoint and returning all businesses in the MongoDB collection.

File: biz\app.py

```
...

@businesses_bp.route("/api/v1.0/allbusinesses",
                     methods=['GET'])
def get_all_businesses():
   data_to_return = []
   for business in businesses.find():
     business['_id'] = str(business['_id'])
     for review in business['reviews']:
         review['_id'] = str( review['_id'] )
     data_to_return.append(business)

   return make_response( jsonify( data_to_return ), 200 )

...
```

Do it now!	Add a new endpoint to the back-end API to retrieve the complete collection of businesses, as presented in the code box above.

18.2 Using AG Grid

With the back-end facility to return the entire collection in place, we can now concentrate on the front-end app that will incorporate the AG Grid component.

18.2.1 Building the Web Service

The first task is to provide the Web Service that will fetch data from the back-end and return it to the front-end application. The Web Service requires only a single method **`getBusinesses()`** which makes an http.get() request to the new endpoint and returns the Observable from which the businesses collection data can be retrieved.

File: src/app/web.service.ts

```
import { HttpClient } from "@angular/common/http";
import { Injectable } from "@angular/core";

@Injectable()
export class WebService {

    constructor( private http: HttpClient) { }

    getBusinesses() {
        return this.http.get<any>(
            'http://localhost:5000/api/v1.0/allbusinesses');
    }
}
```

Do it now!	Add the Web Service to the Angular application to make a GET request to the new endpoint, as demonstrated in the code box above.

18.2.2 A Basic Grid

As a first step in using the **AG Grid** library, we modify the App Component TypeScript file to import the required packages that have been made available by installing the **ag-grid-angular** library, as shown in the code box below.

File: src/app/app.component.ts

```
import { Component } from '@angular/core';
import { RouterOutlet } from '@angular/router';
import { WebService } from './web.service';
import { AgGridAngular } from 'ag-grid-angular';
import { ColDef } from 'ag-grid-community';

@Component({
   selector: 'app-root',
   standalone: true,
   imports: [RouterOutlet, AgGridAngular],
   providers: [WebService],
   templateUrl: './app.component.html',
   styleUrl: './app.component.css'
})
export class AppComponent {
   title = 'datagrid';
}
```

Next, we identify the fields to be presented as columns in the grid and declare a property **`data`** that will store the data to be retrieved from the back-end via the Web Service. Note that even though the API returns all fields in each document, the **`headings`** definition selects only those fields that are to be included in the grid – other fields retrieved from the back-end will be ignored.

File: src/app/app.component.ts

```
...

export class AppComponent {

   headings: ColDef[] = [
      { field: "name" },
      { field: "town" },
      { field: "rating" }
   ];

   data: any = [];
}
```

To populate the data grid, we first make the Web Service available by injecting it as a parameter to the component constructor. Then, we specify the **`ngOnInit()`** method that runs automatically when the Component is loaded and make a call to the Web Service **`getBusinesses()`** method that returns the complete collection. This request populates the **`data`** property of the grid without any modification required to the API response.

File: src/app/app.component.ts

```
...

export class AppComponent {

   ...

   constructor(private webService: WebService) {}

   ngOnInit() {
      this.webService.getBusinesses()
          .subscribe( (response) => {
        this.data = response;
      });
   }
}
```

Do it now!	Complete the file ***app.component.ts*** using the code provided in the code boxes above.

With the TypeScript specification in place, we can move to the HTML and CSS elements of the App Component. First, in the HTML template, we replace the current contents of ***app.component.html*** with the **`<ag-grid-angular>`** tag presented in the following code box. Note now the **`rowData`** and **`columnDefs`** attributes of the **`ag-grid-angular`** element are bound to the previously established values of the component **`data`** and **`headings`** properties.

File: src/app/app.component.html

```
<ag-grid-angular
    [rowData]="data"
    [columnDefs]="headings" />
```

Do it now!	Replace the default content of app.component.html with the code provided in the code box above.

Installing the **ag-grid-angular** library also makes a number of stylesheet files available within the **node_modules** folder for the component. We can include these in the application by adding their paths to the styles entry within ***angular.json***.

File: angular.json

```
...

"styles": [
    "src/styles.css",
    "./node_modules/ag-grid-community/styles/ag-grid.css",
    "./node_modules/ag-grid-community/styles/ag-theme-quartz.css",
 ],

 ...
```

Do it now!	Add the style entries shown above to ***angular.json***. Note that additional stylesheets providing different themes are also available within the **node_modules** folder and you can experiment with these to change the appearance of your grid.

Once the style are added we can apply them to the grid by applying the ag-theme-quartz class to the grid component. We also apply a height of 500px to the element, leaving the width to occupy as much space as is required.

File: src/app/app.component.html

```
<ag-grid-angular
    class="ag-theme-quartz"
    style="height: 500px"
    [rowData]="data"
    [columnDefs]="headings" />
```

Do it now!	Apply the **`class`** and **`style`** attributes demonstrated in the code box above to ***app.component.html***.

Finally, we need to modify ***main.js***, which is the entry point of the Angular application, to add provision for the **`HttpClient`**, reconfiguring the file to take account of the existing code.

File: src/main.js

```
import { bootstrapApplication } from '@angular/platform-browser';
import { appConfig } from './app/app.config';
import { AppComponent } from './app/app.component';
import { provideHttpClient } from '@angular/common/http';

bootstrapApplication(AppComponent, {
  providers: [
    provideHttpClient(),
    appConfig.providers
  ]
}).catch((err) => console.error(err));
```

Do it now!	Update ***main.js*** with the highlighted code above.

Now we can view the data grid in action, so we run the application using ng serve and see the grid component as shown in Figure 18.23, below. If your application is already running, you should stop it first (by CTRL-C) and then re-start it to make sure that the stylesheets are loaded.

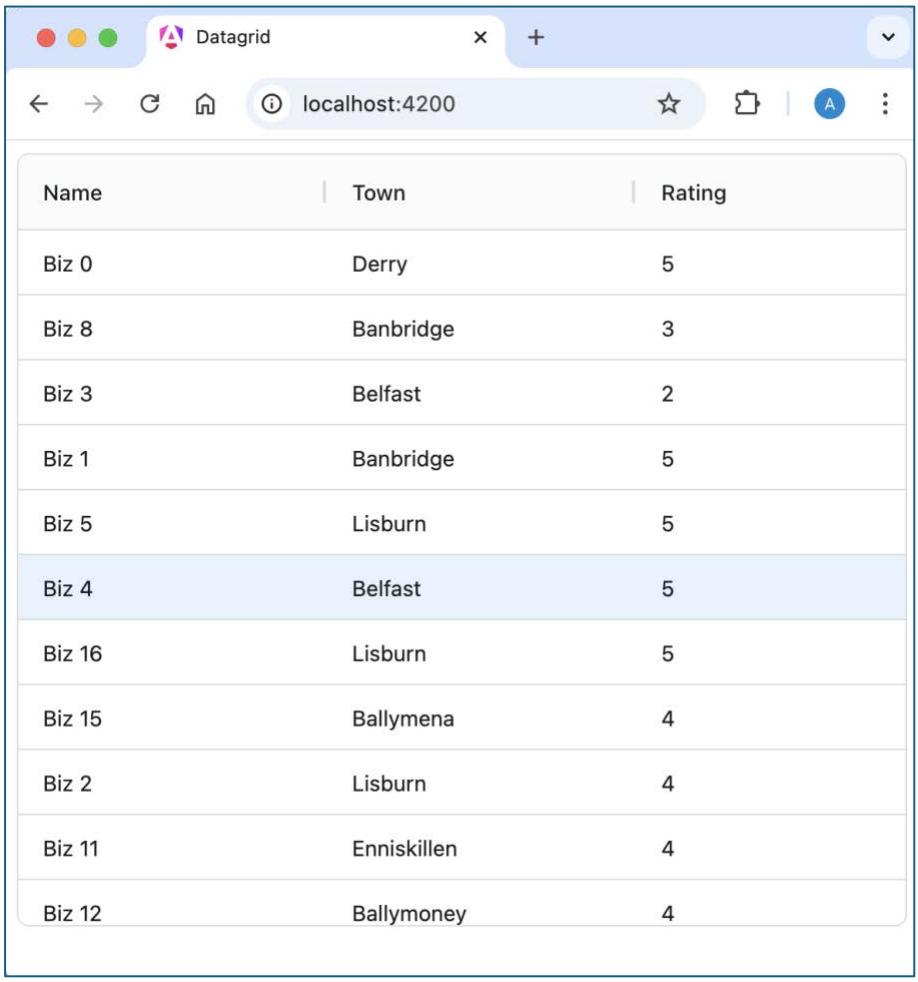


Figure 18.2 Basic AG Grid

Even in its most basic form, the gris contains some useful functionality. As we specified a fixed width, it provides vertical scrolling within the grid so that all rows can be seen. In addition, each of the row heading is clickable, with a click toggling between sorting that field in ascending order, in descending order, and with no sort applied.

Do it now!	Run the application with **`ng serve`** and visit http://localhost:4200 in a Web Browser. Verify that you see an interactive data grid as illustrated in Figure 18.2, above. Experiment with clicking on the column headings to demonstrate dynamic sorting on any selected field and see how the grid scrolls vertically to display the entire collection of businesses.

18.3 Configuring Additional Functionality

The **AG Grid** component already provides a useful means of browsing a large collection of data, but there are many additional elements of functionality and presentation that we can specify.

18.3.1 Filtering

Filtering a table of data on a selected column is when we allow the user to provide a value to be used as the filter and only rows where the column matches that value (either partially or fully) are displayed. In AG Grid, we can specify a partial match filter for any column by adding the filter: true attribute to the definition of the heading. This is demonstrated in the code box below where we add a filter to the town column.

File: src/app/app.component.ts

```
...

export class AppComponent {

   headings: ColDef[] = [
      { field: "name" },
      { field: "town", filter: true },
      { field: "rating" }
   ];

   data: any = [];

   ...
}
```

The effect of adding the filter can be seen from Figure 18.3 below. Here, we provide the text "Ball" in the ***town*** filter, resulting in only towns that include the character sequence "ball" being displayed. All towns where the name does not include the sequence "ball" are hidden from view.

Note that the filter updates the display in real time as the user enters the filter value and that the column value does not have to begin with the filter sequence. Hence, entering "***r***" as the filter string displays Businesses in De***r***ry, Banb***r***idge, Cole***r***aine, Lisbu***r***n and New***r***y, while extending the sequence to "***ri***" immediately limits the set of businesses displayed to those in Banb***ri***dge.

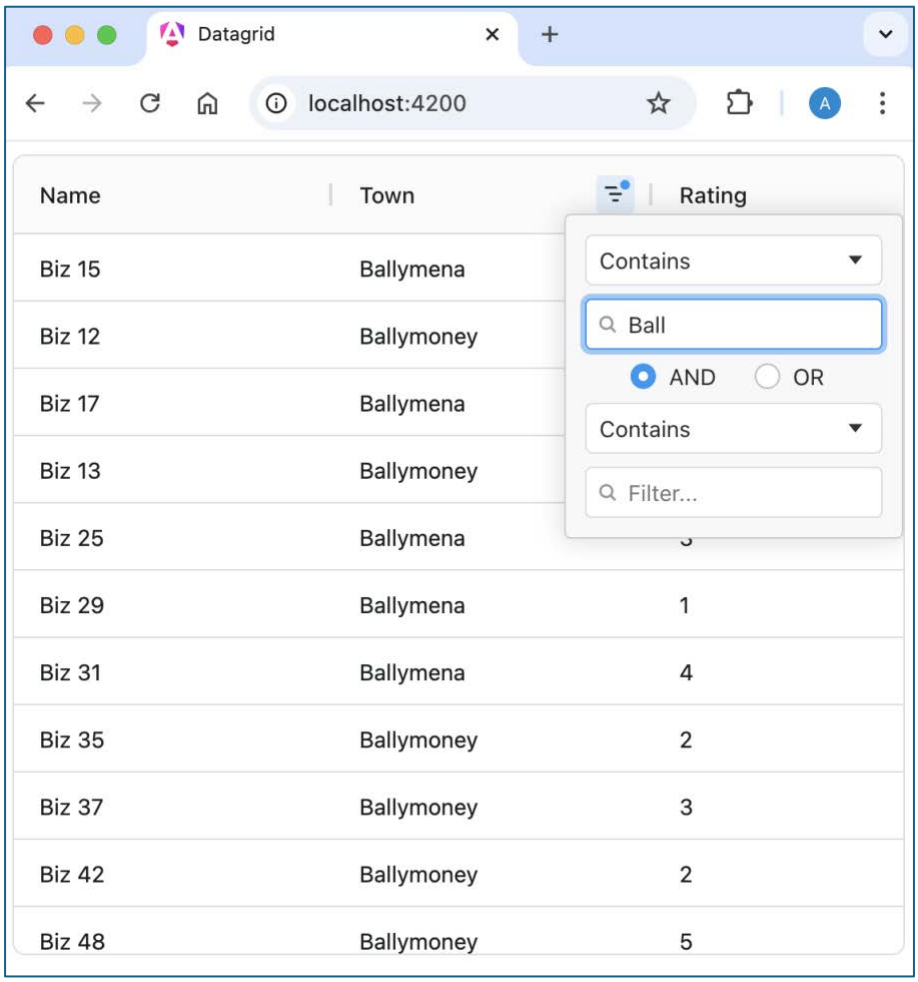


Figure 18.3 Adding a Filter

By default, the filter is hidden and appears when the icon to the right of the column heading is displayed. However, by adding **`floatingFillter: true`** to the heading definition, we can request that the filter is permanently displayed below the column heading.

File: src/app/app.component.ts

```
...

export class AppComponent {

   headings: ColDef[] = [
      { field: "name" },
      { field: "town", filter: true, floatingFilter: true },
      { field: "rating" }
   ];

   data: any = [];
   ...
}
```

The floating filter applied to the town column is shown in Figure 18.4, below.

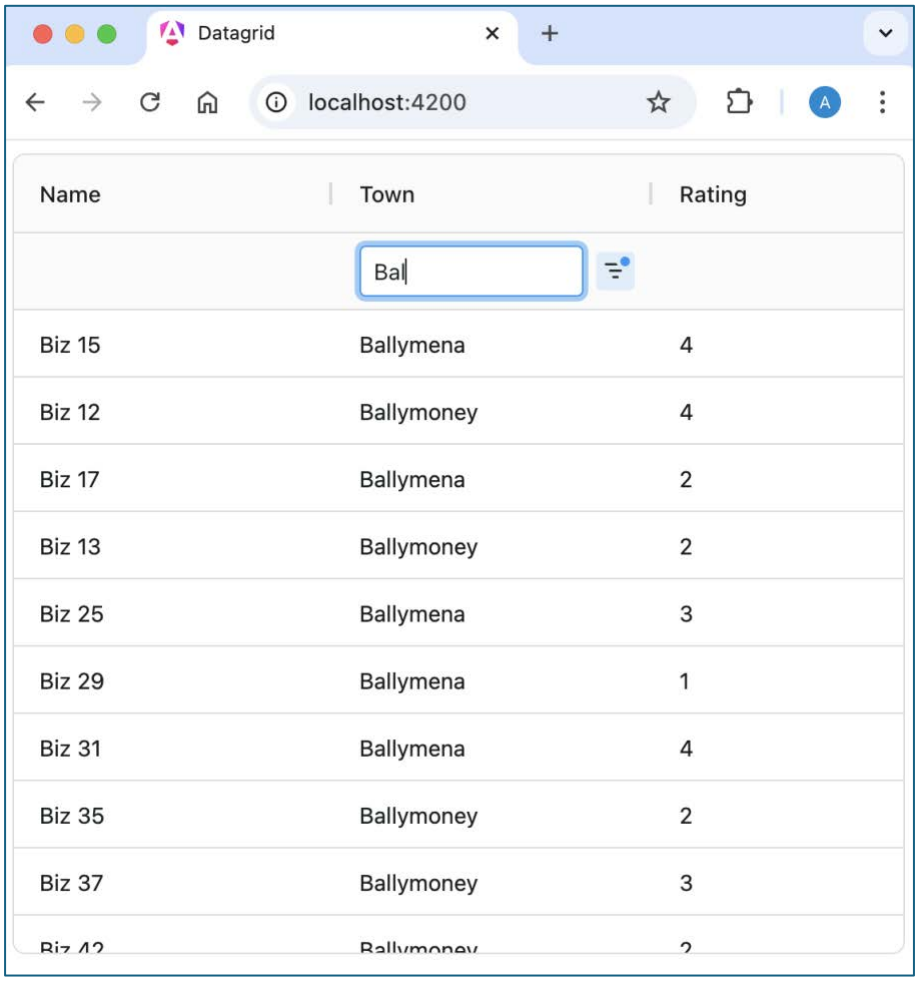


Figure 18.4 Floating Filter

Do it now!	Add a filter to the town field as described in this section. Try both the fixed and floating filter options and determine which you prefer to use going forward. Try adding additional filters for other fields and see how they can be combined.

18.3.2 Column Options

By default, the column names are derived automatically from the field name by capitalising it – hence our fields **`name`**, **`town`** and **`rating`** become the headings ***Name***, ***Town*** and ***Rating***. In addition, if the field name contains multiple words in camelCase, AG Grid will split and capitalise such that a field name of **`myTopScore`** will become the heading ***My Top Score***.

However, sometimes we want to use a custom heading that is unrelated to the field name and AG Grid provides for this by the **`headerName`** attribute that can be specified as part of the header definition. In the code box that follows, we can see the **`num_employees`** field added to the grid, but with a heading of ***Workforce***.

The code box below also demonstrates how to access data values that are embedded within the data structure. Our specification for a business includes the gross profit or loss for each of the past three years as shown below.

```
"profit" : [
   {
      "year" : "2022",
      "gross" : 50000
   },
   {
      "year" : "2023",
      "gross" : -10000,
   },
   {
      "year" : "2024",
      "gross" : 15000
   }
]
```

In order to access the individual values for gross profit, AG Grid provides the **`valueGetter`** attribute that allows us to access such fields using notation such as **`profit[0].gross`** for the 2022 value, **`profit[1].gross`** for the 2023 value, and so on.

File: src/app/app.component.ts

```
...

export class AppComponent {

 headings: ColDef[] = [
   { field: "name" },
   { field: "town", filter: true, floatingFilter: true },
   { field: "rating" },
   { field: "num_employees", headerName: "Workforce" },
   { valueGetter: "data.profit[0].gross", headerName: "2022" },
   { valueGetter: "data.profit[1].gross", headerName: "2023" },
   { valueGetter: "data.profit[2].gross", headerName: "2024" }
 ];

 ...
}
```

Adding these fields to the headings definition results in the expanded grid as shown in Figure 18.5, below.

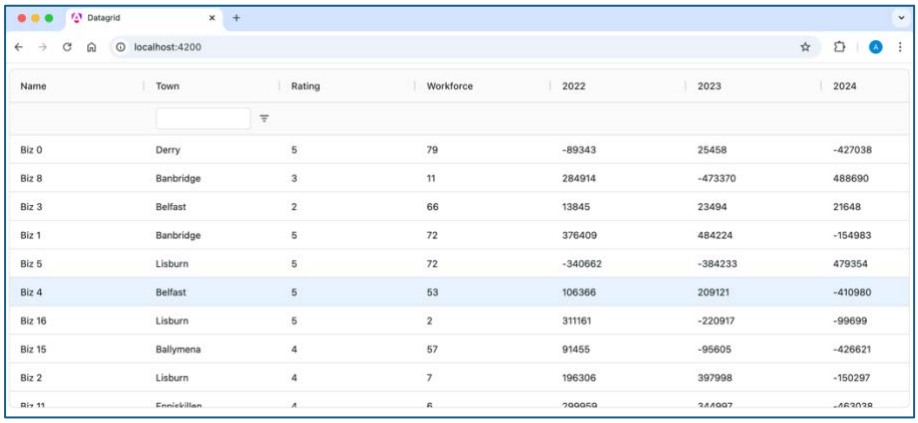


Figure 18.5 Custom Headers and Embedded Data Fields

Now that the grid has a larger number of fields, we can also demonstrate the horizontal scrolling provided by reducing the width of the browser window and observing that the grid only displays those columns for which there is space and makes those outside of that area available by scrolling. Figure 18.6 demonstrates a situation where there is enough horizontal space to show four columns with the others available by scrolling to the left or to the right.

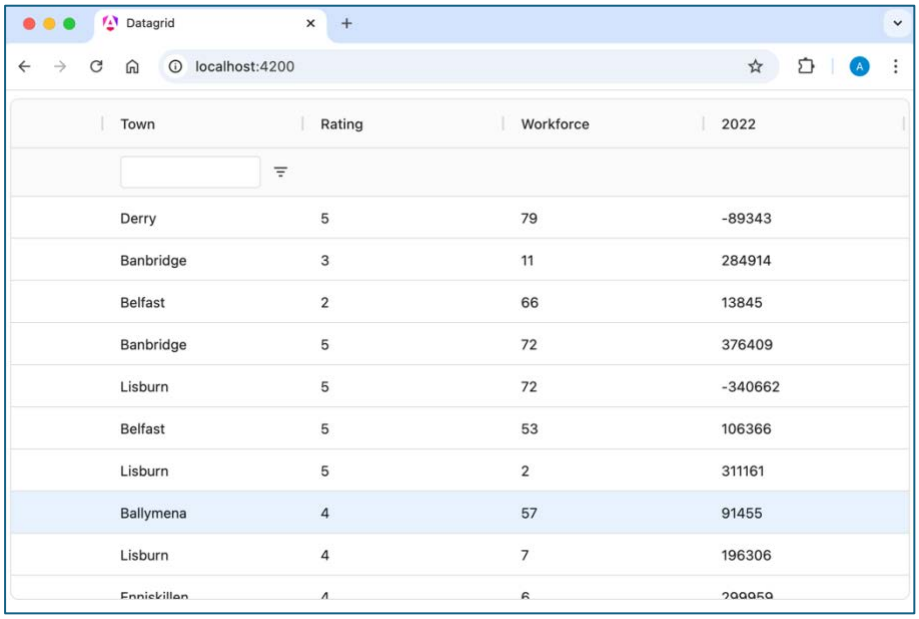


Figure 18.6 Horizontal Scrolling

Do it now!	Add the additional fields to the headings definition as shown in the code box above and see the horizontal scrolling in action.

18.3.3 Pagination

Pagination is a commonly found feature in data grid components and AG Grid provides a very flexible implementation based around three component attributes described as follows.

`pagination`	takes the value **`true`** if pagination is to be enabled, and **`false`** otherwise
`paginationPagesize`	the initial number of rows to display on a page. Should be one of the values specified in the **`paginationPageSizeSelector`** list
`paginationPageSizeSelector`	the list of possible page sizes offered to the user

The following code box shows values for these attributes initialized as properties of the App Component.

File: src/app/app.component.ts

```
...

data: any = [];

pagination = true;
paginationPageSize = 10;
paginationPageSizeSelector = [10, 25, 50];

...
```

Next, in the HTML template, we bind each of the ag-grid-angular attributes to the corresponding component property

File: src/app/app.component.html

```
<ag-grid-angular
    class="ag-theme-quartz"
    stye="height: 500px"
    [pagination]="pagination"
    [paginationPageSize]="paginationPageSize"
    [paginationPageSizeSelector]="paginationPageSizeSelector"
    [rowData]="data"
    [columnDefs]="headings" />
```

Now, when we re-run the application, we see the pagination controls added below the grid, as shown in Figure 18.7, below.

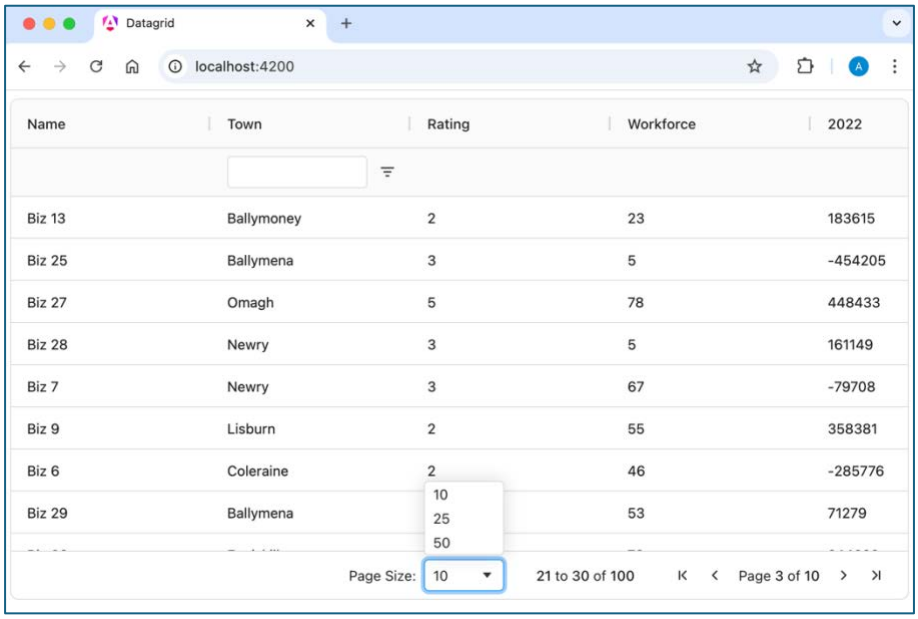


Figure 18.7 Adding Pagination

Do it now!	Add pagination to the data grid by making the modifications to ***app.component.ts*** and ***app.component.html*** shown in the code boxes above. Try different page size options and confirm their operation. Also combine pagination with filtering and verify that only the rows matching the filter are counted in the pagination calculation.

18.3.4 Styling

AG Grid comes with a number of themes that can be enabled by linking to their stylesheet files in ***angular.json***, but each of these is fully customizable by overwriting their SS rules in a local component CSS file.

The code box below illustrates this by providing new values for the CSS rules for the **ag-theme-quartz** theme that we specified to be used in the **`<ag-grid-angular>`** tag in ***app.component.html***.

File: src/app/app.component.css

```
.ag-theme-quartz {
    --ag-foreground-color: rgb(126, 46, 132);
    --ag-background-color: rgb(249, 245, 227);
    --ag-header-foreground-color: rgb(204, 245, 172);
    --ag-header-background-color: rgb(209, 64, 129);
    --ag-odd-row-background-color: rgb(0, 0, 0, 0.03);
    --ag-header-column-resize-handle-color: rgb(126, 46, 132);

    --ag-font-size: 17px;
    --ag-font-family: monospace;
}
```

Adding this code to app.component.css results in the grid style that we see in Figure 18.8, below.

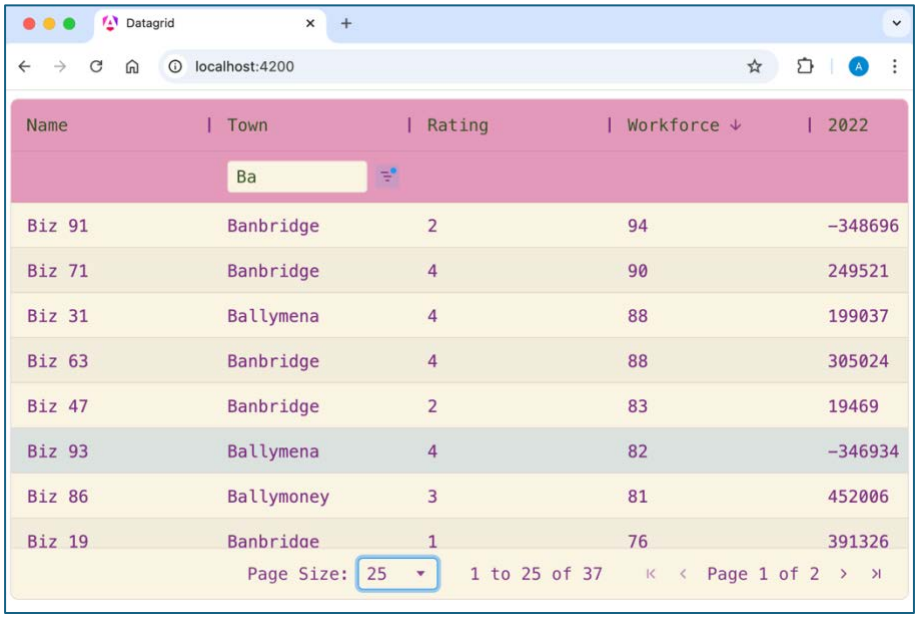


Figure 18.8 Custom Style

Do it now!	Add code to ***app.component.css*** as shown above an modify the presentation style of your grid. Manipulate the colour values to create your own theme. You can return to the original theme at any time by removing the content from ***app.component.css*** and reverting to the default values.

18.3.5 Other Extensions

AG Grid contains many other features (even in the free tier) that can provide additional functionality and display options, including those identified below.

Cell components	Allow HTML elements such as buttons, checkboxes and images to be displayed in grid cells
Row and value selection	Add a column of checkboxes to the grid so that the user can indicate those in which they are interested. The selected rows can then be extracted using the **AG Grid API** for further processing.
Editing	AG Grid supports editing of the cell values, and these can be saved back to the component data structure. If required, changes can be POSTed back to the database via the Web Service
Export	Data can be exported to CSV format by using an **AG Grid** API call

Information and examples of these and many other facilities are available in he comprehensive online documentation at https://www.ag-grid.com/angular-data-grid/getting-started/.

18.4 Further Information

- https://taglineinfotech.com/why-use-angular/
  Why use Angular in 2024?

- https://medium.com/@simon.sharp_25406/the-state-of-angular-by-the-google-angular-team-9ea8f6571c5f
  The State of Angular by the Google Angular Team

- https://angulardive.com/blog/angular-services-vs-components-understanding-when-to-use-each/
  Angular Components v Services

- https://angular.dev/tools/libraries
  Overview of Angular Libraries

- https://angular.dev/tools/libraries/creating-libraries
  Creating Libraries

- https://www.ag-grid.com/
  AG Grid Home Page

- https://www.ag-grid.com/angular-data-grid/compatibility/
  AG Grid and Angular Version Compatibility Table

- https://www.youtube.com/@agGrid
  Ag Grid YouTube Channel

- https://thinkster.io/tutorials/fundamentals-of-ag-grid-with-angular
  Fundamentals of AG Grid with Angular

- https://www.linkedin.com/pulse/elevate-your-angular-applications-ag-grid-guide-examples-jaiswal/
  Elevate Your Angular Applications with AG Grid: A Comprehensive Guide with Examples

COM661 Full Stack Strategies and Development

FE19. Testing the Front-end

Aims

- To introduce the Angular testing framework
- To present test specificactions in Jasmine
- To demonstrate test presentation in Karma
- To design a bespoke test framework for the Biz Directory application
- To test the retrieval of a specified page of businesses
- To test the retrieval of a specified business
- To test the retrieval of the reviews for a specified business
- To test the addition of a review for a business

Table of Contents

19.1 ANGULAR TESTING FRAMEWORK 2

19.1.1 ANGULAR COMPONENT TESTING 2
19.1.2 A CALCULATOR APP 5
19.1.3 A FIRST TEST 8
19.1.4 ADDING TESTS 10

19.2 A CUSTOM TESTING FRAMEWORK 12

19.2.1 THE TEST COMPONENT 12
19.2.2 SPECIFYING A TEST 15
19.2.3 TESTING OTHER FUNCTIONALITY 16

19.3 FURTHER INFORMATION 23

19.1 Angular Testing Framework

Software testing is the process of verifying that the expected output is produced for any combination of input. In Angular, it requires that the Components and Services that make up an application respond appropriately when they are activated. In this section, we examine the built-in Component testing provided by the platform, as well as developing a bespoke testing framework for the Biz Directory sample application.

19.1.1 Angular Component Testing

When the Angular framework creates a Component, four files are written to the application. We have so far seen three of these as described below for a component **x**:

x.component.hml	the HTML template
x.component.css	the stylesheet for the component
x.component.ts	the TypeScript definition of the component structure and behaviour.

The fourth "default" file is one that we have not so far created ourselves but can be seen by browsing the **`AppComponent`** files created by Angular when we create a new application. The file ***app.component.spec.ts*** is the default test script for the App Component and checks that the component has been created and is available for use. If we create components using the Angular framework command

```
C:\angularapp> ng generate component newname
```

then ***newname.component.spec.ts*** is generated alongside ***newname.component.html***, ***newname.component.ts*** and ***newname.component.css***. The structure of a default test script can be seen by creating a new Angular application called **calculator** (that we will use later in this practical) and examining the source of ***app.component.spec.ts*** as shown in the code box below.

Do it now!	Create a new Angular application called **calculator** by the command **`ng new calculator`** and verify that the test file ***app.component.spec.ts*** is created and that it has the structure shown in the code box below.

File: calculator/src/app/app.component.spec.ts

```
import { TestBed } from '@angular/core/testing';
import { AppComponent } from './app.component';

describe('AppComponent', () => {
  beforeEach(async () => {
    await TestBed.configureTestingModule({
      imports: [AppComponent],
    }).compileComponents();
  });

  it('should create the app', () => {
    const fixture = TestBed.createComponent(AppComponent);
    const app = fixture.componentInstance;
    expect(app).toBeTruthy();
  });

  it(`should have the 'calculator' title`, () => {
    const fixture = TestBed.createComponent(AppComponent);
    const app = fixture.componentInstance;
    expect(app.title).toEqual('calculator');
  });

  it('should render title', () => {
    const fixture = TestBed.createComponent(AppComponent);
    fixture.detectChanges();
    const compiled = fixture.nativeElement as HTMLElement;
    expect(compiled.querySelector('h1')?.textContent).toContain(
          'Hello, calculator');
  });
});
```

Angular tests are written in a JavaScript framework called **Jasmine** that helps us write test cases in a human-readable way. The Angular testing package includes a utility called **`TestBed`** that is structured around a container called **`describe`**. The **`describe`** container contains a number of blocks, the most important of which are **`beforeEach`**, which runs before each individual test, and **`it`**, which defines a test. The code above defines three tests, verifying (i) that the component has been created, (ii) that the title property of the app is "**calculator**", and (iii) that the HTML template contains a **<h1>** element with the value "**Hello, calculator**". The structure of the test specification is shown in Figure 19.1, below, where the **`describe()`** block is a container for the suite of tests, the **`beforeEach()`** block contains configuration or installation code that is run before each test, each **`it()`** block

contains the description of a single test, and the **`expect()`** block describes the condition that must be met for the test to pass.

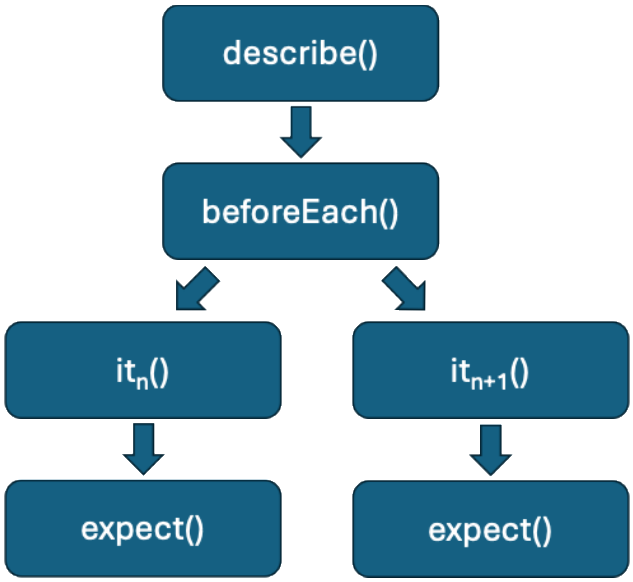


Figure 19.1 Angular Testing Structure

The Angular tests are run by the command **`ng test`**, which opens a Karma server that executes the test code in a browser. Karma is a test runner that executes the tests in a local server. It provides features such as live reloading, code coverage reporting and integration with a range of Continuous Integration tools. The instruction is entered from the command prompt as shown in Figure 19.2, below.

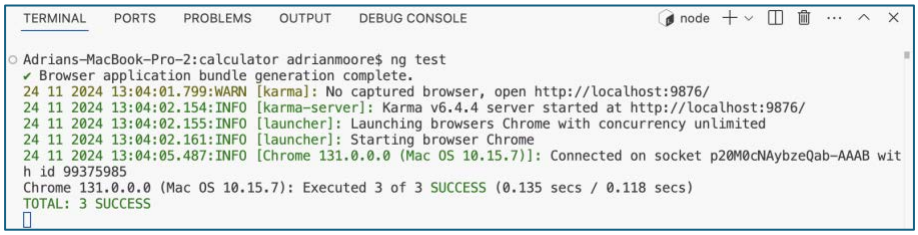


Figure 19.2 Running the Angular tests

When the test is invoked, it spawns a web browser and displays the results of the tests, as illustrated by Figure 19.3, below. Here, we see that the tests described by the three **`it()`** blocks in the code box above have successfully passed. Note the use of the word “should” in the title of each **`it()`** block (provided as the first parameter). This is a convention that should be observed and helps us to write appropriate tests.

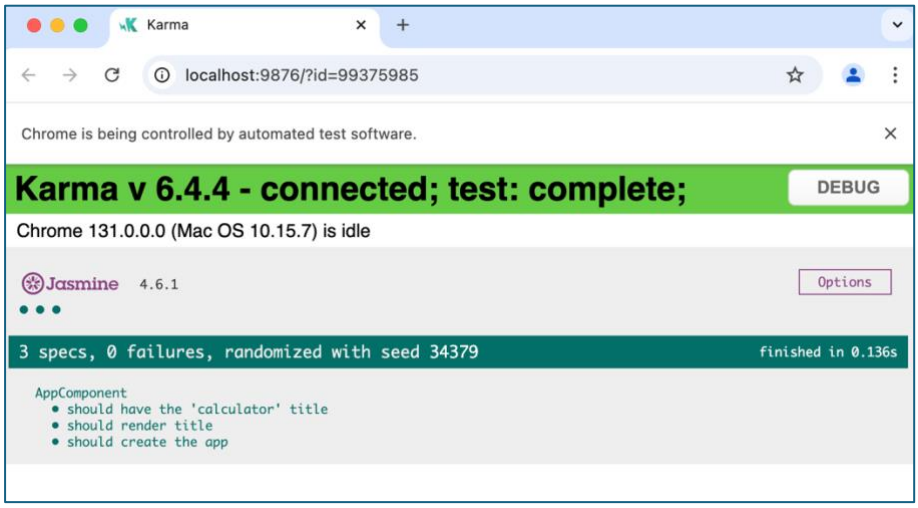


Figure 19.3 Test Results

19.1.2 A Calculator App

As a demonstration of Angular Component testing, we will build a small calculator application on which to specify a test suite. First, we add the files ***calculator.component.ts*** and ***calculator.component.html*** to our calculator application as shown in the code boxes below.

File: calculator/src/app/calculator.component.ts

```
import { Component } from '@angular/core';
import { RouterOutlet } from '@angular/router';

@Component({
  selector: 'calculator',
  standalone: true,
  imports: [RouterOutlet],
  templateUrl: './calculator.component.html'
})
export class CalculatorComponent { }
```

File: calculator/src/app/calculator.component.html

```
{ file is empty }
```

Next, we add to the component Typescript file, a property called **`calculations`** to hold the collection of operations that we will execute, as well as a method to implement the multiplication operation and add a report to the **`calculations`** collection. We also implement **`ngOnInit()`** to call the **`multiply()`** method for a range of calculations.

File: calculator/src/app/calculator.component.ts

```
...

export class CalculatorComponent {
  calculations: string[] = [];

  public multiply(x: number, y: number)  {
    this.calculations.push(x + " * " + y + " = " + (x * y) );
    return x * y;
  }

  ngOnInit() {
    this.multiply(2, 4);
    this.multiply(-3, -3);
    this.multiply(-3, 2);
    this.multiply(5, 0);
  }

}
```

Then, we add a **@for** block to the HTML template to display the collection of reports on the web page.

File: calculator/src/app/calculator.component.html

```
@for( calculation of calculations; track calculation ) {
    <h1>{{ calculation }}</h1>
}
```

Finally, in the setup stage, we replace the default code in ***app.component.html*** with an instance of the **calculator** selector so that the list of calculator reports is displayed in the browser and import the Calculator component into the App component specification.

File: calculator/src/app/app.component.html

```
<calculator></calculator>
```

File: calculator/src/app/app.component.ts

```
import { Component } from '@angular/core';
import { RouterOutlet } from '@angular/router';
import { CalculatorComponent } from './calculator.component';
@Component({
  selector: 'app-root',
  standalone: true,
  imports: [RouterOutlet, CalculatorComponent],
  templateUrl: './app.component.html',
  styleUrl: './app.component.css'
})
export class AppComponent {
  title = calculator;
}
```

Now, when we run the application with **`ng serve`**, we see the report of the four calls to the **`multiply()`** method reported in the browser, as shown in Figure 19.4, below.

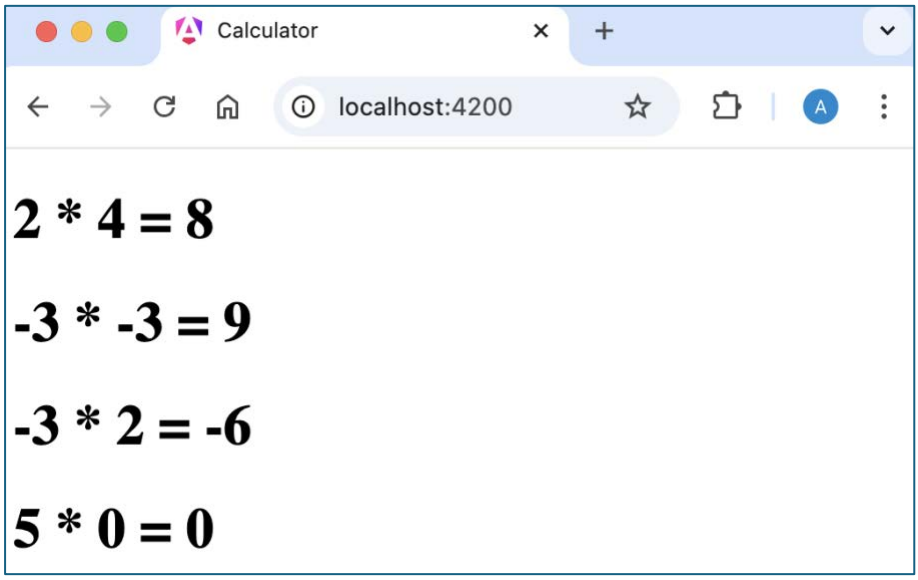


Figure 19.4 A Calculator App

Do it now!	Implement the Calculator component as described in the code boxes above and verify that the calls to the **`multiply()`** method are logged in the browser.

19.1.3 A First Test

Now that the calculator app has a working function, we can specify tests to ensure that the correct result is generated for all combinations of input. Initially, we specify the boilerplate code that imports the test framework components and creates an instance of the **Calculator** component on which to run the tests.

File: calculator/src/app/calculator.component.spec.ts

```
import { ComponentFixture, TestBed }
         from '@angular/core/testing';
import { CalculatorComponent } from './calculator.component';

describe('CalculatorComponent', () => {
  let component: CalculatorComponent;
  let fixture: ComponentFixture<CalculatorComponent>;

  beforeEach( () => {
    fixture = TestBed.createComponent(CalculatorComponent);
    component = fixture.componentInstance;
    fixture.detectChanges();
  });

// tests go here

});
```

Do it now!	Add the file ***calculator.component.spec.ts*** to the application and populate it with the standard test template code as shown in the code box above.

Next, we add a test to ensure that two positive numbers are correctly multiplied. Each test is specified in an **`it()`** block, where the first parameter is a string that describes the desired outcome of the test and the second parameter is a function that calls the method and compares the result obtained to that which is expected. Here, we expect the result of the multiplication of 2 and 4 to be 8.

File: calculator/src/app/calculator.component.spec.ts

```
...

// tests go here

it('2 times 4 should be 8', () => {
    let result = component.multiply(2, 4);
    expect(result).toEqual(8);
  });

...
```

Now, when we run the tests, the result shown in Figure 19.5 is shown in the browser. Note that we have also commented out the final test in ***app.component.spec.ts*** as the **`title`** property is no longer being rendered to the **<h1>** element in the browser.

File: calculator/src/app/app.component.spec.ts

```
...

  /*
  it('should render title', () => {
    const fixture = TestBed.createComponent(AppComponent);
    fixture.detectChanges();
    const compiled = fixture.nativeElement as HTMLElement;
    expect(compiled.querySelector('h1')?.textContent)
          .toContain('Hello, calculator');
  });
  */

});
```

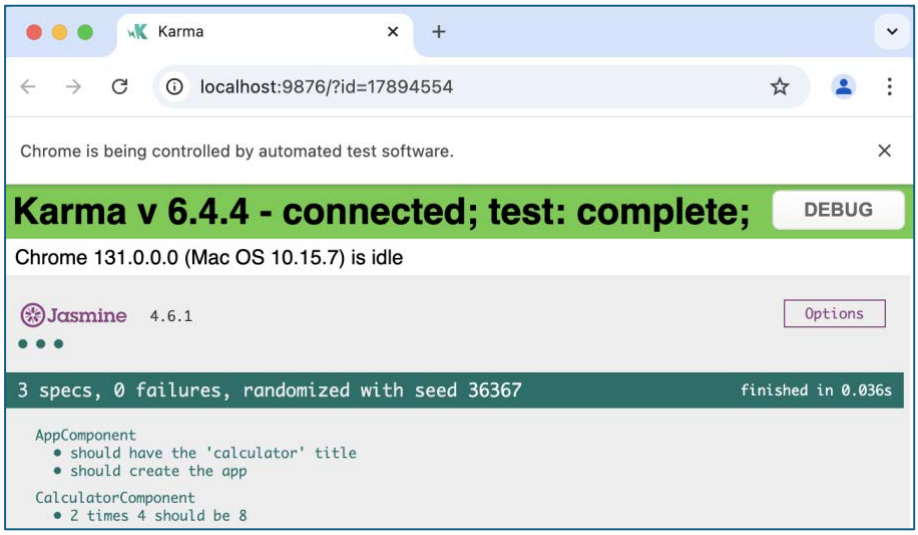


Figure 19.5 Testing the **`multiply()`** *Method*

19.1.4 Adding Tests

With a test framework in place and a first test successfully executing, we can now add additional tests to check the correct operation of the **`multiply()`** method for all combinations of input. Specifically, we want to check that if one of the numbers is negative, the result is negative, if both numbers are negative the result should be positive, and if one of the numbers is zero, the result should be zero. The specification of the new tests is shown in the code box below, while Figures 19.6 shows the sequence of operations in the browser and the result of the tests in Karma.

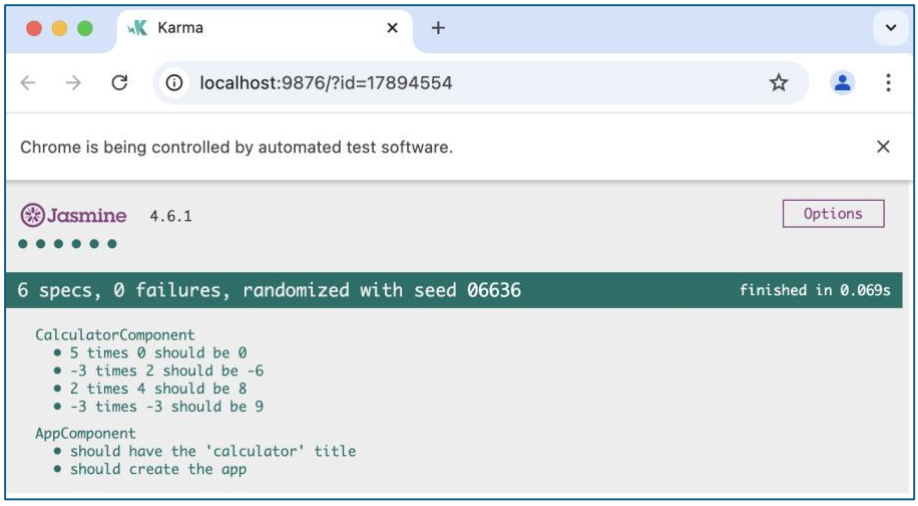


Figure 19.6 Multiplication Test Suite

File: calculator/src/app/calculator.component.spec.ts

```
...

// tests go here

  it('2 times 4 should be 8', () => {
    let result = component.multiply(2, 4);
    expect(result).toEqual(8);
  });

  it('-3 times 2 should be -6', () => {
    let result = component.multiply(-3, 2);
    expect(result).toEqual(-6);
  });

  it('-3 times -3 should be 9', () => {
    let result = component.multiply(-3, -3);
    expect(result).toEqual(9);
  });

  it('5 times 0 should be 0', () => {
    let result = component.multiply(5, 0);
    expect(result).toEqual(0);
  });

...
```

Do it now!	Add the remaining tests to the test specification as shown in the code box above. Verify that all tests run in Karma and that all pass.

Although our tests in this example are quite trivial, it is useful to also see what happens when a test fails. We can manufacture this situation by amending the last test description to expect that 5 multiplied by zero will return the result 5, as shown in the code box below.

File: calculator/src/app/calculator.component.spec.ts

```
...

  it('5 times 0 should be 5', () => {
    let result = componentmultiply(5, 0);
    expect(result).toEqual(5);
  });

...
```

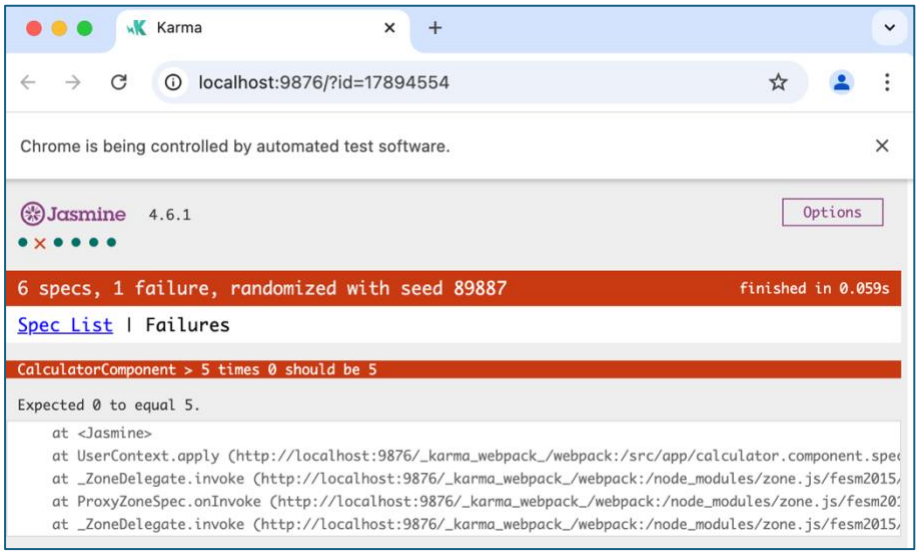


Figure 19.7 A Failing Test

Do it now!	Modify the final test so that is produces a failing result, as shown in Figiure 19.7 above.

19.2 A Custom Testing Framework

The Jasmine framework provides a convenient way of testing the functionality of our Angular applications, but we may also need to supplement it with custom tests that check the operations of specific features. In this section, we will construct a custom testing framework for the Biz Directory application, verifying the correct operation of each element of functionality in our Web Service.

The Web Service is the central element of our application and manages all communication between the front- and back-ends. It provides four functions as follows, that we will test separately

- Fetch a page of businesses
- Fetch a single business specified by ID
- Fetch the reviews of a business
- Add a new review for a business

19.2.1 The Test Component

To test our Web Service, we will create a new component called **`TestWSComponent`** that we specify by TypeScript and HTML files as described below.

File: bizFE/src/app/testWS.component.ts

```
import { Component } from '@angular/core';
import { WebService } from './web.service';

@Component({
  selector: 'testWS',
  standalone: true,
  providers: [WebService],
  templateUrl: './testWS.component.html'
})
export class TestWSComponent {

  test_output: string[] = [];

  constructor(private webService: WebService) {}

}
```

The TypeScript definition imports the WebService as a provider and creates a property **`test_output`** that will hold a series of **`string`** objects that describe the result of the tests. In the HTML template, we iterate over the **`test_output`** collection displaying the strings contained in the browser.

File: bizFE/src/app/testWS.component.html

```
<div style="margin-top: 70px">

    <h1>Test Report</h1>

    <ul>
    @for (report of test_output; track report) {
        <li>{{ report }}</li>
    }
    </ul>

</div>
```

We make the TestWS component available to the user by adding a new route to **app.routes.ts** that causes the component to be displayed when the URL **/test** is presented to the browser.

File: bizFE/src/app/app.routes.ts

```
...

import { TestWSComponent } from './testWS.component';

export const routes: Routes = [
    {
        path: '',
        component: HomeComponent
    },
    {
        path: 'businesses',
        component: BusinessesComponent
    },
    {
        path: 'businesses/:id',
        component: BusinessComponent
    },
    {
        path: 'test',
        component: TestWSComponent
    }
];
```

With the new component in place, we can verify that everything has been properly connected by running the application and visiting the new UR http://localhost:4200/test as shown in Figure 19.8. below.

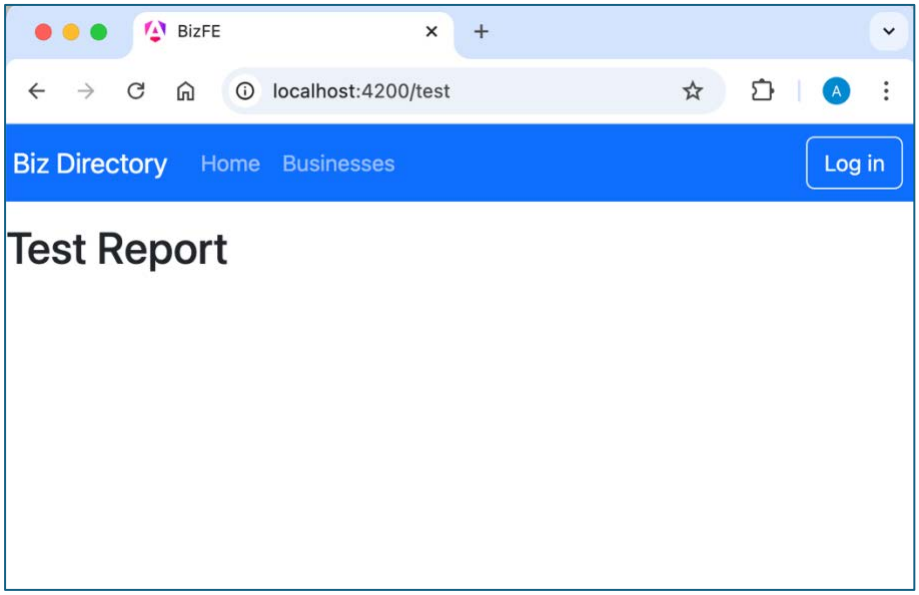


Figure 19.8 Biz Directory Test Framework

Do it now!	Create the new TestWS Component.as shown in the code boxes above and run the application. Confirm that an empty set of test reports is displayed as illustrated in Figure 19.9, above.

19.2.2 Specifying a Test

The first test is to check that the Web Service **`getBusinesses()`** method returns a page of three businesses, where three is the default page size specified in ***web.service.ts***. In this test, we will consider the method to be operating properly if the response object is an array and the length of the array is three, so we add a method to **`TestWSComponent`** to make a call tom the Web Service method and check the **`response`** object. Remember that as the Web Service method returns an Observable, we need to subscribe to it and perform our check in the subscription function.

File: bizFE/src/app/testWS.component.ts

```
...

  constructor(private webService: WebService) {}

  private testBusinessesFetched() {
    this.webService.getBusinesses(1)
      .subscribe( (response) => {
        if (Array.isArray(response) && response.length == 3)
          this.test_output.push(
               "Page of businesses fetched... PASS");
        else
          this.test_output.push(
               "Page of businesses fetched... FAIL");
    });
  }
```

With the **`testBusinessesFetched()`** method in place, we add an **`ngOnInit()`** method to the **`testWSComponent`** to call the test method when the component loads and report the result in the web browser.

File: bizFE/src/app/testWS.component.ts

```
...

    ngOnInit() {
       this.testBusinessesFetched();
    }
}
```

Running the application and visiting the test URL causes the new test to be performed and reported as shown in Figure 19.9, below.

Do it now!	Add the **`testBusinessesFetched()`** method to the TestWS Component as shown in the code boxes above. With the application running, visit the URL http://localhost:4200/test and confirm that the test to check the **`getBusinesses()`** method passes as illustrated in Figure 19.10, below.

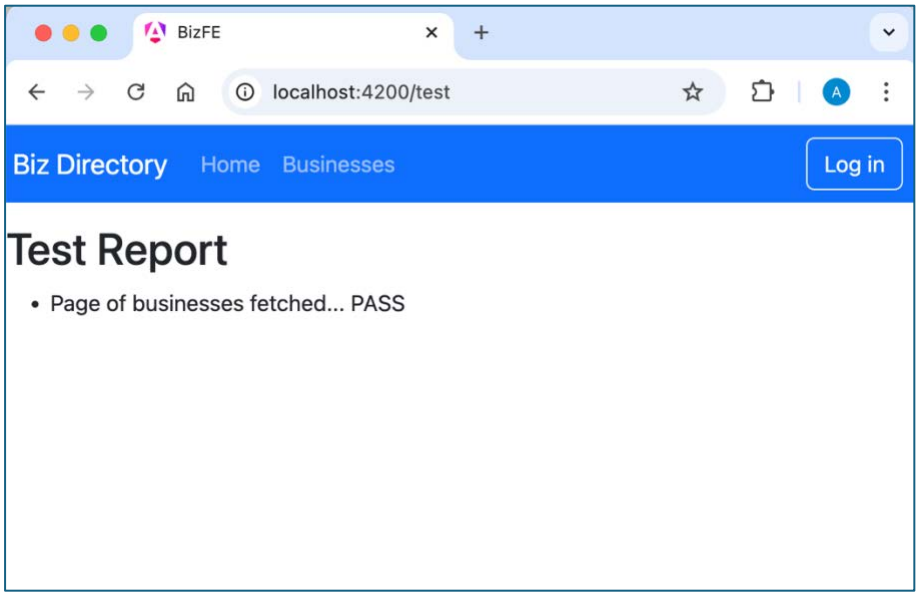


Figure 19.9 Fetching a Page of Businesses

19.2.3 Testing Other Functionality

The next test checks that the pagination in the **`getBusinesses()`** function is working by fetching the first and second pages of businesses and confirming that the first entry in each page is different. To facilitate this we add a pair of new properties to the TestWS Component called **`first_business_list`** and **`second_business_list`** and populate them by calls to **`getBusinesses(1)`** and **`getBusinesses(2)`**, respectively, where the parameter is the page number required.

The code box below presents the code for this test. First, we make a call to **`getBusinesses(1)`**, subscribing to the Observable returned. In the subscription function, we accept the response into **`first_business_list`** and make a call to **`getBusinesses(2)`**. In the subscription function for this, we accept the response into **`second_business_list`** and then check that the first entries in **`first_business_list`** and **`second_business_list`** are different, adding the appropriate response to the **`test_output`** collection. We then add a call to the new test to **`ngOnInit()`**. Figure 19.10 shows the result of adding this test to the collection.

File: bizFE/src/app/testWS.component.ts

```
...

export class TestWSComponent {

  first_business_list: any[] = [];
  second_business_list: any[] = [];

  ...

  private testPagesOfBusinessesAreDifferent() {
      this.webService.getBusinesses(1)
        .subscribe( (response) => {
           this.first_business_list = response;
           this.webService.getBusinesses(2)
             .subscribe( (response) => {
                this.second_business_list = response;
                if (this.first_business_list[0]["_id"] !=
                    this.second_business_list[0]["_id"])
                    this.test_output.push(
                       "Pages 1 and 2 are different... PASS");
                else
                    this.test_output.push(
                      "Pages 1 and 2 are different... FAIL");
           })
      });
  }

  ngOnInit() {
    this.testBusinessesFetched();
    this.testPagesOfBusinessesAreDifferent();
  }
}
```

Do it now!	Add the new test to the TestWS Component and verify that calls to **`getBusinesses()`** with different parameter values returns different results.

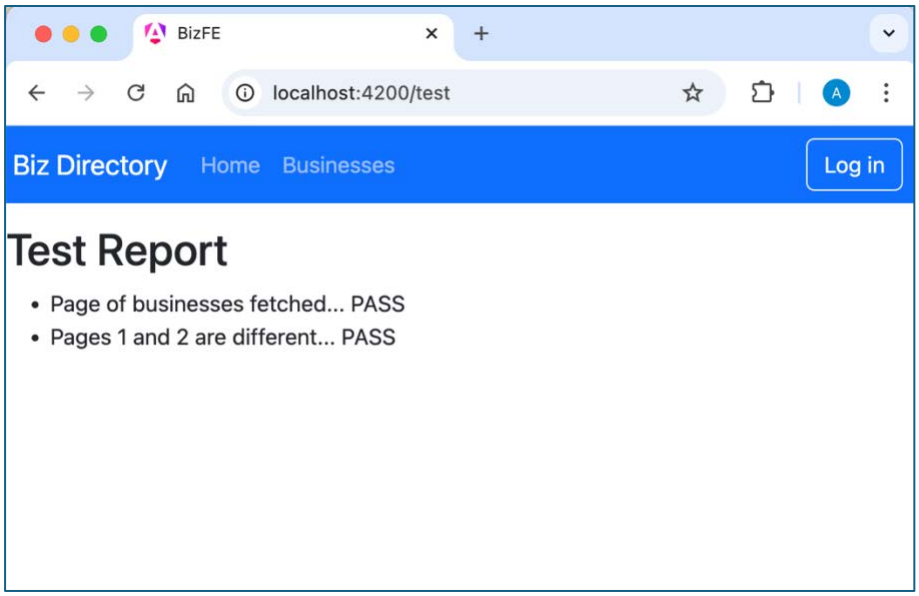


Figure 19.10 Testing that Different Pages Have Different Contents

The next test checks that providing a known **`_id`** value to a call to the **`getBusiness()`** method returns the business object expected. Here, we use the MongoDB Shell to retrieve the **`_id`** of the business with the name "Biz 0" and then make a call to retrieve the business matching the specified **`_id`**. If the business returned is that with the **`name`** attribute "Biz 0", then the test is successful.

File: bizFE/src/app/testWS.component.ts

```
      ...

      private testGetBusiness() {
      this.webService.getBusiness('66df1c8fcf0ec82461958517')
        .subscribe( (response) => {
            if (response.name == 'Biz 0')
              this.test_output.push("Fetch Biz 0 by ID... PASS");
            else
              this.test_output.push("Fetch Biz 0 by ID... FAIL");
        })
    }

    ngOnInit() {
      this.testBusinessesFetched();
      this.testPagesOfBusinessesAreDifferent();
      this.testGetBusiness();
    }

  ...
```

Next, we test that the reviews of a business are being fetched by making a call to the **`getReviews()`** function, passing the **`_id`** of a known business. Since a business does not necessarily have any reviews, we consider the test to have passed if an array is returned – even if the array is empty.

File: bizFE/src/app/testWS.component.ts

```
      ...

    private testGetReviews() {
      this.webService.getReviews('66df1c8fcf0ec82461958517')
        .subscribe( (response) => {
            if (Array.isArray(response))
              this.test_output.push(
                   "Fetch Reviews of Biz 0... PASS");
            else
              this.test_output.push(
                   "Fetch Reviews of Biz 0... FAIL");
        })
    }

    ngOnInit() {
      this.testBusinessesFetched();
      this.testPagesOfBusinessesAreDifferent();
      this.testGetBusiness();
      this.testGetReviews();
    }

  ...
```

The final Web Service function is that which adds a review to the collection for a business. To test this, we fetch the list reviews for a known business, storing the length of the reviews structure in a local variable. Then, we pass a dummy review object to the Web Service **`postReview()`** method, along with the **`_id`** of the business. In the subscription function, we again fetch the reviews for the same business and check that the number of reviews is one greater than the previous value.

File: bizFE/src/app/testWS.component.ts

```
      ...

    private testPostReview() {
      let test_review = {
        "username" : "Test User",
        "comment" : "Test Comment",
        "stars" : 5
      };
      this.webService.getReviews('66df1c8fcf0ec82461958517')
      .subscribe( (response) => {
          let numReviews = response.length;
          this.webService.postReview(
                      '66df1c8fcf0ec82461958517', test_review)
            .subscribe( (response) => {
              this.webService.getReviews(
                      '66df1c8fcf0ec82461958517')
                .subscribe( (response) => {
                  if (response.length == numReviews + 1)
                    this.test_output.push("Post review... PASS");
                  else
                    this.test_output.push("Post review... FAIL");
                });
            });
      });
    }

    ngOnInit() {
      this.testBusinessesFetched();
      this.testPagesOfBusinessesAreDifferent();
      this.testGetBusiness();
      this.testGetReviews();
      this.testPostReview();
    }

  ...
```

Adding these tests to the TestWS Component and running the application confirms that all tests are successful as illustrated in Figure 19.11, below.

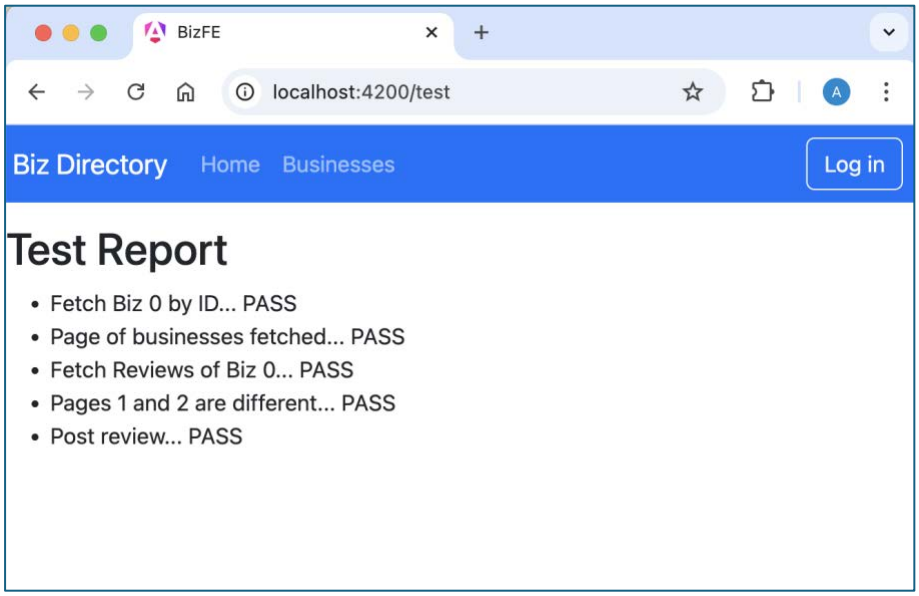


Figure 19.11 The Full Suite of Web Service Tests

Do it now!	Add the remaining tests to the TestWS Component and verify that all tests are successful.

19.3 Further Information

- https://angular.dev/guide/testing
  Angular Testing Documentation

- https://blog.logrocket.com/angular-unit-testing-tutorial/
  Angular Unit Testing Tutorial with Examples

- https://www.angularminds.com/blog/writing-your-first-angular-unit-test-step-by-step-tutorial
  Writing your First Angular Unit Test

- https://www.telerik.com/blogs/testing-angular
  Testing in Angular

- https://angular.dev/tools/libraries/creating-libraries
  How to Write Unit Tests with Jasmine and Karma

- https://www.youtube.com/watch?v=ic_Ez8PO_jc
  Angular Unit Testing Using Jasmine and Karma Tutorial (YouTube)

- https://jasmine.github.io/
  Jasmine Home Page

- https://karma-runner.github.io/latest/index.html
  Karma Test Runner Home Page

- https://www.zymr.com/blog/angular-unit-testing-with-karma-and-jasmine
  Angular Unit Testing with Karma and Jasmine

- https://dev.to/chintanonweb/mastering-angular-unit-testing-a-comprehensive-guide-with-examples-3eg9
  Mastering Angular Unit Testing: A Comprehensive Guide with Examples

COM661 Full Stack Strategies and Development

FE20. Front-end Documentation

Aims

- To summarise the role of software documentation
- To introduce the Compodoc library for generating documentation for Angular applications
- To install and prepare Compodoc
- To generate the default Compodoc documentation
- To demonstrate the specification of bespoke documentaiton using JSDoc comments
- To present JSDoc tags for providing context to documentation
- To introduce the Compodoc Documentation Coverage facility

Table of Contents

20.1 SOFTWARE DOCUMENTATION AND COMPODOC 2

20.1.1 INSTALLATION 2
20.1.2 CREATING THE DOCUMENTATION 4

20.2 SPECIFYING DOCUMENTATION 5

20.2.1 JSDOC COMMENTS 5
20.2.2 DOCUMENTATION COVERAGE 9

20.3 FURTHER INFORMATION 11

20.1 Software Documentation and Compodoc

Software documentation is a collection of written and illustrated information that explains how to use or how a piece of software works. It can be embedded within the software or provided as a separate document. Software documentation can take many forms and can include

- User documentation that provides instructions for the users of the software
- Technical documentation that provides detailed implementation information
- Process documentation that describes the steps and procedures used to develop, test and maintain the software
- Software Design Documents that outline the application's main components and interfaces
- API documentation that specifies how to invoke endpoints and how to interpret the response
- Release notes that describe the version history, releases of features, and bugs that have been found and fixed
- System documentation that describes the main requirements to use the software.

In recent years, many electronic tools and frameworks have emerged for specifying, structuring, and publishing documentation, often integrated into the environment in which the actual source code is written. One such example is **Compodoc**, which is a documentation tool for Angular applications (as well as supporting other JavaScript-based frameworks) that used a combination of code analysis and structured comments to generate comprehensive documentation that can either be published independently of hosted alongside the application. In this practical, we will introduce the Compodoc tool and investigate the documentation that it produces.

20.1.1 Installation

Compodoc is an Open-Source project led by Vincent Ogloblinsky and contributed to by a team of volunteer developers on GIthub. It can easily be installed into an Angular application by using **npm** to install the library into the application using the following command.

```
C:/BizFE> npm install @compodoc/compodoc --save
```

To run **Compodoc**, we need to create a file within the top-level project folder called ***ts.config.doc.json*** and with the following contents.

File: BizFE/tsconfig.doc.json

```
{
    "include": ["src/**/*.ts"],
    "exclude": ["src/test.ts", "src/**/*.spec.ts"]
}
```

The **`include`** and **`exclude`** attributes specify which parts of the application should be documented and which parts should be ignored. Typically, an Angular development, we specify that all TypeScript files within the **src** folder should be documented, with the exception of test files.

Next, we add an execution command to the **`scripts`** object within the application's ***package.json*** file. This can take a variety of forms and there are many command line options that can be included, but those that we choose to include here are described as follows

`-p tscomfig.doc.json`	to provide a pointer to the configuration file
`-s`	to serve the documentation from a web server that will launch at localhost:8080
`-w`	to watch the source files and automatically re-build the documentation when one of the source files changes.

The code that should be added to ***package.json*** is presented in the code box below.

File: BizFE/package.json

```
{
  "name": "biz-fe",
  "version": "0.0.0",
  "scripts": {
    ...
    "compodoc": "npx compodoc -p tsconfig.doc.json -s -w"
  }

...
```

Do it now!	Install the **compodoc** library into your Angular application using npm and provide the code presented in the code boxes above for ***tsconfig.doc.json*** and ***package.json***.

20.1.2 Creating the Documentation

With **Compodoc** installed in our application, we can run it to build the default documentation. Providing the new **`script`** entry in ***angular.json*** above means that we can run **Compodoc** from a terminal prompt by the command

```
C:/BizFE> npm run compodoc
```

This causes **Compodoc** to scan and analyse the application code and build the default documentation. By default, the documentation is created within a top-level documentation folder within our application, but specifying the **`-s`** flag in the **`script`** entry also launches a local web server that displays the documentation at http://localhost:8080.

The default application documentation is shown in Figure 20.1 below. If you navigate through the menu options presented, you can see that all Components, Services and other elements are recognised and described, and that each class, method, property, parameter and other code elements are presented. **Compodoc** is able to create all of this by simply reading the code that we provide, but the added value in documentation is where we provide our own insights about why decisions have been taken, the purpose of each code element, and other contextual information.

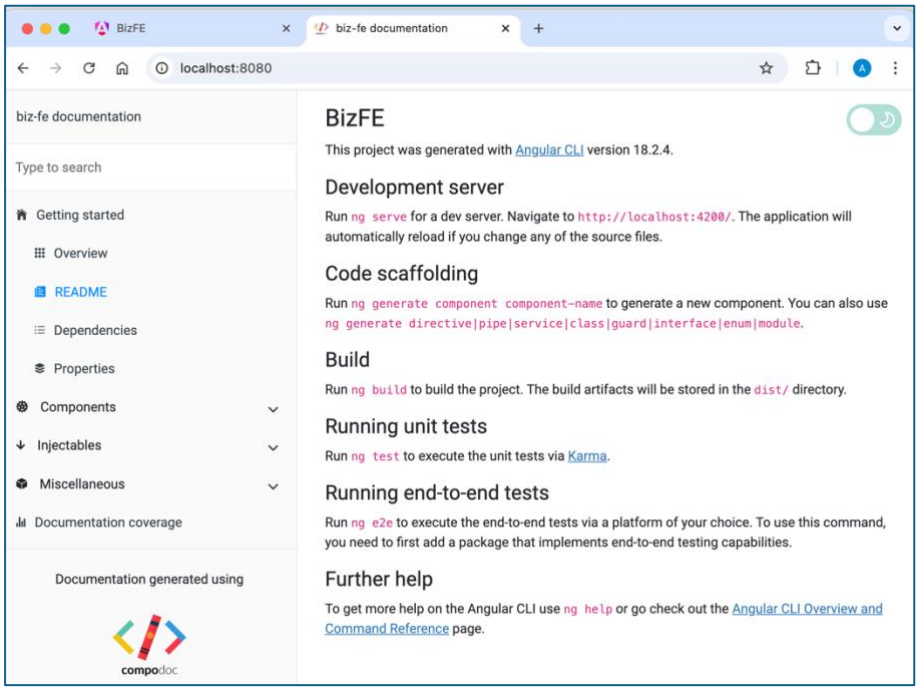


Figure 20.1 Compodoc Server Running

Do it now!	Run **Compodoc** using the command **`npm run compodoc`** at a terminal window. (Note, you can open a second terminal on the folder so that you can also run **`ng serve`** on the application at the same time. Navigate through the information provided and see how all of the various code elements are represented.

20.2 Specifying Documentation

The default documentation generated by Compodoc produces full coverage of the application code and makes all code elements available within an easy-to-navigate framework. However, as well as code structure analysis to present components, services, methods, parameters, properties and the various other code elements, Compodoc can also capture developer intent and context by using the structure and location of comments to add additional value to the documentation.

20.2.1 JSDoc Comments

JSDoc is a mark-up language used to annotate JavaScript source files. Using comments that contain JSDoc, developers can add descriptive detail about the code, providing general information as well as specifying the interface to a code element in terms of the parameters required and the return value generated.In Compodoc, comments in JSDoc format are automatically scanned and are included in the documentation produced.

JSDoc comments are provided in the following form

```
/**
*
* JSDoc Comments must begin with /** and end with */
* Each line of the comment block must begin with *
*
*/
```

In addition, JSDoc provides a range of tags that are used to identify different code elements. For example, in the JSDoc comment added to the Web Service method **`getBusinesses()`**, shown in the code box below, we use the **`@param`** tag to provide a description of the page parameter to the method and the **`@returns`** tag to provide a description of the value returned from the method.

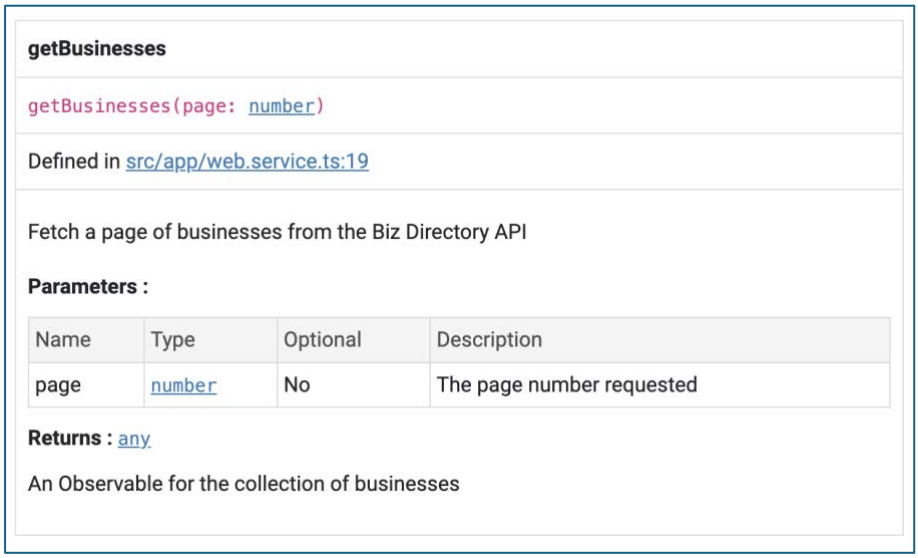
File: bizFE/src/app/web.service.ts

```
...

    /**
     * Fetch a page of businesses from the Biz Directory API
     * @param page The page number requested
     * @returns An Observable for the collection of businesses
     */
    getBusinesses(page: number) {
        return this.http.get<any>(
            'http://localhost:5000/api/v1.0/businesses?pn=' +
            page + '&ps=' + this.pageSize);
    }

...
```

The effect of the JSDoc comment can be seen by saving **`web.service.ts`** and observing the documentation re-built to reflect the new addition. Now, the comment above the **`getBusinesses()`** method is added to the documentation for the method to incorporate the additional contextual information. This is illustrated in Figure 20.2, below, which shows the revised **Compodoc** documentation for the Web Service method **`getBusinesses()`**.



getBusinesses

`getBusinesses(page: number)`

Defined in src/app/web.service.ts:19

Fetch a page of businesses from the Biz Directory API

Parameters :

Name	Type	Optional	Description
page	number	No	The page number requested

Returns : any

An Observable for the collection of businesses

Figure 20.2 Documentation Generated from JSDoc Comment

Do it now!	Run **Compodoc** using the command **`npm run compodoc`** at a terminal window. (Note, you can open a second terminal on the folder so that you can also run **`ng serve`** on the application at the same time. Navigate through the information provided and se how all of the various code elements are represented.

There are only a few formatting rules to follow when providing JSDoc comments

- Each line must contain an asterisk, and the asterisks must be vertically aligned
- Each asterisk must be followed by a space or newline (except the first and the last)
- The only characters before the asterisk on each line must be whitespace characters
- If a newline is required in the rendered documentation, then a blank line (except for the compulsory asterisk) must be included in the comment. Comment text on successive lines will be rendered on the same line of the documentation (word-wrap permitting).

In the example above, we also introduce the tags **`@param`** and **`@returns`** to describe the method parameters and return value. These are example of a range of contextual JSDoc tags that can be used in the documentation and that are understood by **Compodoc**, including those shown below. A full set can be seen in the JSDoc documentation linked from Section 20.3 in this document.

`@author`	The name of the developer
`@constructor`	Marks a method as a constructor
`@param`	Documents a method parameter. A data type indicator can also be added between **`{}`** braces such as **`@param { number } page`** for the **`page`** parameter passed to the Web Service **`getBusinesses()`** method.
`@private`	Denotes that an element is private to the class in which it is defined
`@returns`	Documents a return value
`@since`	Allows the history of an element to be describes, e.g. **`@since v2.3.1`**
`@todo`	Documents a feature that is missing or under development

Do it now!	Extend the documentation provided for the Web Service, by adding JSDoc comments for all methods and properties. The code box below provides an extract of this for you.

File: bizFE/src/app/web.service.ts

```
import { HttpClient } from "@angular/common/http";
import { Injectable } from "@angular/core";

/**
 * The Web Service provides access to the endpoints of the
 * Biz Directory API.
 */
@Injectable()
export class WebService {

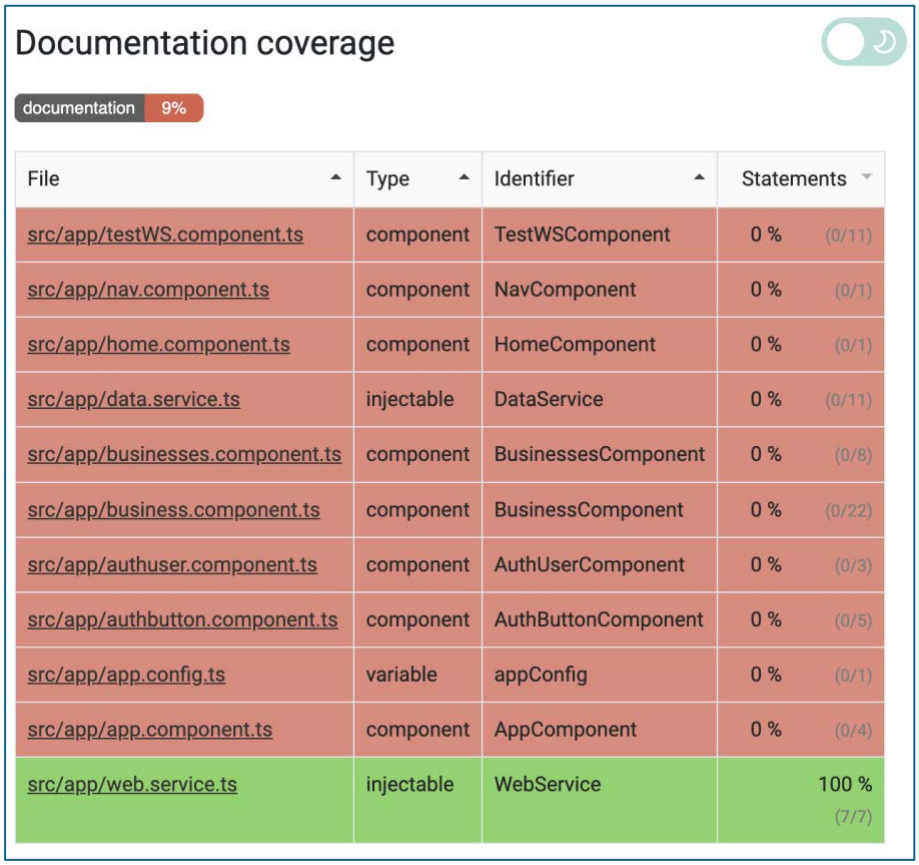
    /**
     * The default page size to be returned
     */
    pageSize: number = 3;

    /**
     * The constructor for the Web Service
     * @param http Injecting the HttpClient to the WebService
     * class
     */
    constructor(private http: HttpClient) { }

    ...
```

20.2.2 Documentation Coverage

One of the most useful features of Compodoc is the Documentation Coverage report that identifies those elements of the documentation for which good coverage has been achieved and those for which additional work remains to be done. When the documentation for the Web Service is complete, as partially shown in the code box above, the Documentation Coverage report for the BizFE application is as shown in Figure 20.3, below, where we have achieved 100% coverage for the Web Service, but not yet started the documentation of any of the other elements.



Documentation coverage

documentation 9%

File	Type	Identifier	Statements
src/app/testWS.component.ts	component	TestWSComponent	0 % (0/11)
src/app/nav.component.ts	component	NavComponent	0 % (0/1)
src/app/home.component.ts	component	HomeComponent	0 % (0/1)
src/app/data.service.ts	injectable	DataService	0 % (0/11)
src/app/businesses.component.ts	component	BusinessesComponent	0 % (0/8)
src/app/business.component.ts	component	BusinessComponent	0 % (0/22)
src/app/authuser.component.ts	component	AuthUserComponent	0 % (0/3)
src/app/authbutton.component.ts	component	AuthButtonComponent	0 % (0/5)
src/app/app.config.ts	variable	appConfig	0 % (0/1)
src/app/app.component.ts	component	AppComponent	0 % (0/4)
src/app/web.service.ts	injectable	WebService	100 % (7/7)

Figure 20.3 Documentation Coverage

Do it now!	View the Documentation Coverage report for the BizFE application. Provide new documentation as JSDoc comments for other components and verify how the coverage figure rises as documentation elements are added.

20.3 Further Information

- https://compodoc.app/
  Compodoc Home Page

- https://github.com/compodoc/compodoc
  Compodoc GitHub Repository

- https://fr.linkedin.com/in/vincentogloblinsky/en
  Vincent Ogloblinsky (Compodoc developer)

- https://medium.com/@erickzanetti/how-to-implement-compodoc-in-your-angular-project-0a2d0a05d61f
  How to Implement Compodoc in your Angular Project

- https://www.youtube.com/watch?app=desktop&v=k1EDQfkMu2c
  Angular Project Documentation with Compodoc (YouTube)

- https://www.youtube.com/watch?v=6cjaJBMmZLs
  Effortless Software Documentation with Compodoc

- https://alternativeto.net/software/compodoc/about/
  Compodoc – The Missing Documentation Tool for Angular

- https://medium.com/@martink_rsa/js-docs-a-quickstart-guide-da6ce5df4a73
  JSDocs: A Quickstart Guide

- https://compodoc.app/guides/documentation-coverage.html -:~:text=Documentation coverage is calculated only,the level of documentation coverage.
  Compodoc Documentation Coverage

- https://blog.angulartraining.com/how-to-generate-documentation-for-your-angular-porjects-c58cd4b4664b
  How to Generate Documentation for your Angular Projects